A Structural Approach to Interpretable Deep Learning

From DC Decompositions to Two-Player Games

Thesis in partial fulfillment of the requirements for the degree
Master of Science (M.Sc.)

submitted by
Jakob Zimmermann

Under the scientific supervision of
Prof. Dr. Wojciech Samek

Under the scientific guidance of
Georg Loho

Berlin, November 2026

Technische Universität Berlin
Fakultät IV – Elektrotechnik und Informatik
Fraunhofer Heinrich-Hertz-Institut
Department of Artificial Intelligence

Affidavit

Deutsch

1) Hiermit versichere ich, dass ich die vorliegende Arbeit ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen, die den benutzten Quellen und Hilfsmitteln unverändert oder inhaltlich entnommen sind, habe ich als solche kenntlich gemacht.
2) Sofern generative KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die jeweiligen Einsatzzwecke (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.
3) Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 15. Februar 2023[1] habe ich zur Kenntnis genommen.
4) Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Jakob Zimmermann
Berlin, 9. November 2026

[1] https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBl/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf

English

1) I hereby declare that I have written this thesis without the help of third parties and exclusively using the sources and aids listed. I have marked as such all passages that are taken from the sources and aids used, either unchanged or in terms of content.
2) Where generative AI tools were used, I have stated the product name, manufacturer, the software version used and the respective purposes (e.g. linguistic review and improvement of texts, systematic research). I am fully responsible for the selection, adoption and all results of the AI-generated output I used.
3) I have taken note of the Statutes for Safeguarding Good Scientific Practice at TU Berlin dated February 15, 2023[2].
4) I further declare that I have not yet submitted the work in the same or a similar form to any other examination authority.

Jakob Zimmermann
Berlin, November 9, 2026

[2] https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBl/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf

Acknowledgements

I thank Gerrit Holzbach for giving me the opportunity to work on machine learning and gain a practical perspective. I am grateful to Georg Loho for opening my eyes to the theory of deep learning and polyhedral geometry.

I thank Grégoire Montavon for taking the time to discuss the idea of DC decomposition with me and for pointing me toward the Quantus framework and the intricacies of evaluating attribution methods. I am grateful to Wojciech Samek for encouraging me to work on transformers and more modern machine learning models and paradigms, for asking me to study parameter randomization, for our conceptual discussions, and for his organizational support.

I thank Klaus Robert Müller for the interesting discussions of these results. I am also grateful to Shinichi Nakajima for the interesting discussions and for comparing my trajectory distribution view with the Normalized Relevance Measure framework and its signed-measure perspective on layer-wise relevance propagation and concept relevance propagation.

Abstract

Deutsch

Interpretierbarkeit entwickelt sich von der nachträglichen Analyse einzelner Vorhersagen zu einem Werkzeug, mit dem Modelle gezielt verbessert und gesteuert werden. Wenn Attributionen die Korrektur unerwünschter Merkmale, das Training oder die Inferenz beeinflussen, muss deshalb auch ihre Berechnung verstanden und überprüft werden. Diese Arbeit verbindet dafür zwei Perspektiven: die strukturierte Darstellung interner Beiträge und die mathematische Beschreibung ihrer Rückwärtspropagation.

Ein trainiertes neuronales Netz zu verstehen heißt, nachzuvollziehen, wie es eine Eingabe in eine Ausgabe überführt — welche gegenläufigen Einflüsse jede Vorhersage gegeneinander abwägt. Diese Arbeit untersucht Interpretierbarkeit durch *Zerlegung*: Statt ein Netz zur Interpretierbarkeit zu zwingen, legen wir interpretierbare Struktur offen, die ein trainiertes ReLU-Netz bereits enthält. Ein einziger Begriff der *Netzzerlegung* — eine Anhebung der Berechnung in ein Paar koordinierter, nichtnegativer Ströme, deren Differenz das ursprüngliche Netz exakt reproduziert — vereinheitlicht mehrere solcher Strukturen, ohne eine einzige Vorhersage zu verändern.

Die erste ist die Differenz-konvexer-Funktionen-Zerlegung (DC-Zerlegung). Da Monotonie die Interpretierbarkeit fördert, aber die Ausdrucksstärke einschränkt, trainieren wir kein monotones Modell, sondern zerlegen ein trainiertes ReLU-Netz — ein MLP oder ein CNN — in zwei monotone, konvexe Teilnetze g und h mit $f = g - h$, die positive und negative Evidenz auf getrennten Pfaden führen. Wir überwinden die exponentielle Explosion der Gewichte, die eine naive Zerlegung in der Tiefe unbrauchbar macht, und erweitern die Konstruktion über einen krümmungsbasierten Zerlegungssatz auf glatte Aktivierungsfunktionen (GELU, Swish, Tanh, Sigmoid, Softplus). Auf dieser Darstellung bauen wir die Saliency-Methoden SplitCAM und SplitLRP auf, die auf VGG-16 und ResNet-18 über ImageNet-S in allen Quantus-Metrikkategorien den Stand der Technik verbessern; zudem zeigen wir, dass das direkte Training eines Modells als Differenz zweier monotoner Netze ausgeprägte Selbstinterpretierbarkeit besitzt.

Die zweite ist die Stopp-Zerlegung, die die Gewichte nach Vorzeichen aufteilt, $W = W^+ - W^-$, und exzitatorische von inhibitorischen Beiträgen entlang der Netzpfade trennt. Diese Zerlegung ist die Brücke zu einer spieltheoretischen Deutung des Rückwärtsdurchlaufs. Aufbauend auf dem ReLU-Netz-Spiel von Gaubert und Vlassopoulos fassen wir Rückwärts-Attribution als ein Zwei-Spieler-Nullsummen-Stoppspiel auf einem erweiterten Netzgraphen auf. Gradienten und die vollständige $\alpha\beta$-LRP-Familie ergeben sich als Integrale über Spieltrajektorien unter bestimmten Gleichgewichten, sodass Attributionskarten zu Projektionen einer Verteilung über Trajektorien werden, statt das primäre Objekt zu sein. Erwünschte Eigenschaften einer Interpretation — Lokalisierungsschärfe, Robustheit gegenüber Eingangsrauschen, stabiles Attention-Routing — lassen sich als spieltheoretische Begriffe (Politik-Regularisierung, Risikoaversion, erweiterte Aktionsräume) formulieren, die sich unmittelbar in neue Rückwärtsregeln übersetzen; auf ViT-B/16 übertrifft eine solche Anpassung von $\alpha\beta$-LRP alle betrachteten Vergleichsmethoden, einschließlich transformer-spezifischer Methoden, bei der Attributionslokalisierung. Die revidierte Evaluation trennt Lokalisierung und Faithfulness bei der Validierungsauswahl und berichtet Konfidenzintervalle sowie gepaarte Tests; andere Metriken zeigen Zielkonflikte.

Ein einziger Rahmen macht so mehrere Lesarten derselben trainierten Gewichte zugleich verfügbar. Wir entwickeln zudem die tropisch-geometrische Perspektive, in der die zerlegten Ströme tropische Polynome sind, deren Newton-Polytope — *imaginative Polytope* — die erreichbaren Gradienten als

gelernte visuelle Prototypen kodieren, sowie eine Trajektorienverteilungs-Sicht, die den Rückwärtsdurchlauf mit einer Hellinger-Distanz zwischen Berechnungen ausstattet und damit vergleicht, was ein Netz berechnet, statt der Heatmap, die es projiziert.

Schlüsselwörter: *DC-Zerlegung, Interpretierbarkeit, Neuronale Netze, Spieltheorie, Tropische Geometrie*

English

Interpretability is moving from post hoc analysis of individual predictions toward improving and controlling models. When attributions guide the correction of spurious features, training, or inference, the calculation producing them must itself be understood and checked. This thesis connects two perspectives on that problem: a structured representation of internal contributions and a mathematical account of their backward propagation.

Understanding a trained neural network means accounting for how it turns an input into an output — for the competing influences that every prediction balances against one another. This thesis studies interpretability through *decomposition*: instead of constraining a network to make it interpretable, we expose interpretable structure that a trained ReLU network already contains. A single notion of a *network decomposition* — a lift of the computation into a pair of coordinated, non-negative streams whose difference reproduces the original network exactly — unifies several such structures, none of which alters a single prediction.

The first is the difference-of-convex (DC) decomposition. Monotonicity is known to aid interpretability but to restrict expressivity, so rather than training a monotone model we split a trained ReLU network — an MLP or a CNN — into two monotone, convex subnetworks g and h with $f = g - h$, carrying positive and negative evidence on separate pathways. We overcome the exponential blow-up of the weights that makes a naive split unusable at depth, and extend the construction beyond ReLU to smooth activations (GELU, Swish, tanh, sigmoid, softplus) through a curvature-based splitting theorem. On this representation we build the saliency methods SplitCAM and SplitLRP, which improve on the state of the art on VGG-16 and ResNet-18 over ImageNet-S across all Quantus metric categories; and we show that training a model directly as the difference of two monotone networks yields strong self-interpretability.

The second is the stopping decomposition, which splits the weights by sign, $W = W^+ - W^-$, separating excitatory from inhibitory contributions along the paths of the network. This decomposition is the bridge to a game-theoretic reading of the backward pass. Building on Gaubert and Vlassopoulos' ReLU Net Game, we recast backward attribution as a two-player zero-sum stopping game on an extended network graph. Gradients and the full $\alpha\beta$-LRP family arise as integrals over game trajectories under specific equilibria, so that attribution maps become projections of a distribution over trajectories rather than the primary object. Desired properties of an interpretation — localization focus, robustness to input noise, stable attention routing — can be stated as game-theoretic concepts (policy regularization, risk aversion, extended action sets) that translate directly into new backward rules; on ViT-B/16, one such adaptation of $\alpha\beta$-LRP outperforms all evaluated baselines, including transformer-specific methods, in attribution localization. The revised evaluation selects localization and faithfulness configurations independently on validation data and reports confidence intervals and paired tests; other metrics expose trade-offs.

One framework thus makes several readings of the same trained weights available at once. We develop the tropical-geometric perspective, in which the split streams are tropical polynomials whose Newton

polytopes — *imaginative polytopes* — encode the attainable gradients as learned visual prototypes; and a trajectory-distribution view that equips the backward pass with a Hellinger distance between computations, comparing what a network calculates rather than the heatmap it projects.

Keywords: *DC Decomposition, Interpretability, Neural Networks, Game Theory, Tropical Geometry*

Contents

List of Figures i

List of Tables iii

List of Symbols vi

List of Abbreviations ix

1 Introduction 1
- 1.1 Motivation and Problem Statement . . . 1
- 1.2 Contributions of This Thesis . . . 3
- 1.3 Positioning within the Literature . . . 4

2 Mathematical Foundations 7
- 2.1 Neural Network Notation and PWL Mappings . . . 7
- 2.2 Convexity and Monotonicity in Neural Networks . . . 9
- 2.3 Game Theory and Markov Decision Processes . . . 11
  - 2.3.1 The ReLU Net Game of Gaubert and Vlassopoulos . . . 13
- 2.4 Tropical Geometry and Polyhedra . . . 16
  - 2.4.1 Tropical Rational Maps and the DC Split . . . 19
  - 2.4.2 Newton Polytopes of the Split Streams . . . 21
  - 2.4.3 Imaginative Polytopes . . . 22
  - 2.4.4 DC Splitting of Smooth Activation Functions . . . 24

3 Network Decompositions and DC Formulations 27
- 3.1 Generalized Network Decomposition . . . 27
- 3.2 The Mixing Decomposition family . . . 29
- 3.3 Difference-of-Convex (DC) splitting of ReLU Networks . . . 31
- 3.4 The Stopping Decomposition . . . 34
- 3.5 Regularized Decompositions . . . 36
  - 3.5.1 The Softplus Decomposition as the entropic subcase . . . 37
- 3.6 Numerical Stabilization of Split Passes . . . 39
  - 3.6.1 Shifting the forward pass . . . 39
  - 3.6.2 Shifting the backward pass . . . 40
- 3.7 Generalization to Convolutional Neural Networks (CNNs) . . . 43

4 Game-Theoretic Framework 47
- 4.1 Related Work . . . 47
- 4.2 Backward Attribution . . . 48
- 4.3 Attribution as a Game . . . 49
  - 4.3.1 The Stopping Game . . . 50
  - 4.3.2 The Routing Game . . . 50
- 4.4 Game-Theoretic Modes . . . 51
- 4.5 Notation and Conventions . . . 52
- 4.6 Stopping Game . . . 53
  - 4.6.1 Stopping Decomposition . . . 53
  - 4.6.2 Game Definition . . . 56

4.6.3 Skip Connections and Max Pooling . . . 60
4.6.4 Discounted Occupation Measures and Gradient Representations . . . 62
4.7 Routing Game: Log-Space Formulation . . . 64
4.7.1 Exact Routing Kernel Behind $\alpha\beta$-***L**ayer-wise **R**elevance **P**ropagation* (LRP) . . . 65
4.7.2 Backward Path Integral and $\alpha\beta$-***L**ayer-wise **R**elevance **P**ropagation* (LRP) Recovery . . . 70
4.7.3 Temperature τ as a Controlled Policy Deformation . . . 74
4.8 Analytical Modes . . . 74
4.8.1 Defaults, costs and order of use . . . 75
4.8.2 Activation-Side Risk Aversion λ via ADF Variance Propagation . . . 75
4.8.3 Noisy-Observation Gate $\tilde{\Phi}$ . . . 75
4.8.4 Fan-in entropy bias λ_{ent} . . . 77
4.9 Softplus Variant of the Stopping Game . . . 77

5 Extensions: Attention and Transformers **80**
5.1 Related Work: Interpretability in Transformers . . . 80
5.2 Decomposing Multiplicative Operations . . . 81
5.3 Attention via a detached-logit oracle . . . 83
5.4 Extension: Vision Transformer Attention . . . 83
5.4.1 Attention as fixed-reference KL Value routing . . . 84
5.4.2 The reference as the policy of a QK oracle player . . . 90
5.4.3 Risk-Averse Attention via ADF . . . 91
5.4.4 Implementation details . . . 91
5.4.5 Equivariant Spatial Averaging . . . 92

6 Saliency and Attribution Methods **93**
6.1 Split-Based and Difference-of-Convex Attribution . . . 93
6.1.1 SplitCAM . . . 97
6.1.2 SplitLRP . . . 98
6.1.3 SplitGrad . . . 100
6.1.4 Self-Interpretability: Difference-of-Convex Input Neural Networks . . . 100
6.2 Game-Based Attribution: Modes and Technicalities . . . 104
6.2.1 Attribution Modes . . . 104
6.2.2 Technicalities . . . 104

7 Experiments and Evaluation **106**
7.1 Experimental Setup . . . 106
7.2 Split Methods: VGG-16 . . . 106
7.3 Split Methods: ResNet-18 . . . 107
7.4 Split-Method Stabilization Ablations . . . 108
7.4.1 Backward-Pass Stabilization of the Split Methods . . . 108
7.5 Game-Based Case Study . . . 109
7.5.1 Game-Method Ablations . . . 112

8 The Trajectory Distribution View **113**
8.1 Comparison with Normalized Relevance Measures . . . 113
8.2 Trajectory Sanity Check . . . 114
8.2.1 Trajectories change, maps need not . . . 114

8.3 Hellinger Distance and Disagreement Map . . . 115
8.3.1 Layered Markov Process Foundations . . . 115
8.3.2 Hellinger Distance from Bhattacharyya Coefficients . . . 117
8.3.3 Per-Pixel Disagreement Map . . . 121
8.3.4 Experiments . . . 126
8.4 Randomization Sanity Checks . . . 128
8.4.1 Cascading Randomization in Trajectory Space . . . 128
8.4.2 Heatmap similarity at full randomization . . . 130
8.4.3 Double-Random Empirical Baseline . . . 133
8.4.4 Per-Image Trajectory-vs-Map Independence . . . 133

9 Conclusion and Future Work **136**
9.1 Summary of Contributions . . . 136
9.2 Limitations . . . 137
9.3 Broader Impact . . . 138
9.4 Future Research Directions . . . 138

A Extended Experimental Results **140**
A.1 Full Evaluation Results for the Split Methods . . . 140
A.1.1 VGG-16 Qualitative Saliency Map Comparison . . . 140
A.1.2 Quantitative Analysis on VGG-16 . . . 141
A.1.3 Quantitative Analysis on ResNet-18 . . . 143
A.1.4 Randomization Sanity Check . . . 144
A.1.5 Computational Efficiency . . . 144
A.2 Stabilization Ablations for the Split Methods . . . 147
A.2.1 Backward-Pass Stabilization . . . 147
A.2.2 Forward-Pass Stabilization . . . 147
A.3 Evaluation Protocol . . . 151
A.3.1 Data and splits . . . 151
A.3.2 Methods . . . 151
A.3.3 Metric definitions . . . 152
A.3.4 Declared validation grids . . . 153
A.3.5 Selection . . . 154
A.3.6 Statistics . . . 155
A.3.7 Compute resources . . . 155
A.3.8 Code and correctness tests . . . 155
A.4 Additional Results . . . 156
A.4.1 Validation selections . . . 156
A.4.2 Test results with confidence intervals . . . 159
A.4.3 Trade-off sweeps . . . 162
A.4.4 Cross-Dataset Evaluation: Pascal VOC 2012 . . . 162
A.4.5 One-factor ablations . . . 164
A.4.6 Pointing game at matched granularity . . . 168
A.4.7 Gradient-fallback attention . . . 170
A.4.8 Qualitative comparison . . . 170
A.5 Formal Verification and Mathematical Audit . . . 171

B Publication Provenance and Tool Use **173**
B.1 Disclosure in the Revised Game Paper . . . 173

List of References 175

List of Figures

2.1 Imaginative Polytope of the VGG-16 dragonfly neuron 23

3.1 The network splitting procedure: a ***Re**ctified **L**inear **U**nit* (ReLU) network f is decomposed into two monotone, convex streams g and h with $f = g - h$ (Section 3.3). . . . 27

3.2 The monotone, convex split streams (top) versus a standard ***Re**ctified **L**inear **U**nit* (ReLU) layer (bottom). The negative weight part transitions positive input to negative pre-activation and vice versa, while the positive part maps positive to positive and negative to negative. 32

4.1 The backward pass as a two-player game. The Stopping Decomposition splits every weight by sign, $W = W^+ - W^-$; at each state of the extended graph the players decide (1) stop or continue and (2) where to route. Their equilibrium policies induce a distribution over backward trajectories, and the discounted occupation measure of that distribution at the input states is the attribution map. 48

4.2 Stopping Game: trajectory distribution and local stopping decisions on a toy subnetwork. *Left:* trajectory distribution visualized on the network graph. *Right:* local game around $s^{(2)}_{2,+}$. Player P^+ decides whether to stop or continue. Under continuation, transitions are to $s^{(1)}_{1,+}$ with probability 7/10, to $s^{(1)}_{2,+}$ with probability 2/10 (both turn unchanged), and to $s^{(1)}_{3,-}$ with probability 1/10 (turn switches to P^-, reflecting the negative sign of the weight). The labels $R^\pm$ are the players' expected payoffs $\tilde{Q}_\pm(\cdot, \text{cont})$ (at $s^{(2)}_{2,+}$ without the discount $\gamma^{(2)}_2 = 10$). At $s^{(1)}_{3,-}$, continuing yields expected payoffs R^-=40 and R^+=100, giving continuation advantage $R^- - R^+ = -60 < 0$; therefore P^- stops (so $V_-(s^{(1)}_{3,-}) = 0$), shown by the grayed-out half-neuron. Correspondingly, in the left panel the trajectories terminate at that state (the cemetery state is omitted). Since P^- stops in $s^{(1)}_{3,-}$, traversals to this state contribute zero continuation payoff to both players. Hence, at $s^{(2,\text{act})}_{2,+}$ the positive player has larger expected payoff under continuation than his opponent and chooses to continue. 57

4.3 Routing Game: local routing subgame around $s^{(2,\text{act})}_{2,+}$. This uses the same toy subnetwork and numerical values as Figure 4.2. At this activation state, P^+ is in turn and continues because the routed mass is positive ($V^+ = \log 9 > -\infty$ in the depicted example). The game then branches with probability 1/2 to $s^{(2,\text{lin})}_{2,+,+}$ (turn stays with P^+) and with probability 1/2 to $s^{(2,\text{lin})}_{2,-,-}$ (turn switches to P^-). In each linear state, the active player samples a predecessor according to the Gibbs policy from the immediate log-payoff and game value. Example: along $s^{(2,\text{lin})}_{2,+,+} \to s^{(1,\text{act})}_{1,+}$, the player on the positive stream state gets immediate payoff $\log W^{(2,+)}_{12} = \log 7$ and P^- gets $-\infty$. Edges with zero routed mass have payoff $\log W^{(2,-)}_{12} = \log 0 = -\infty$. For readability, the $\pm$ activation copies in layer 1 are collapsed, and the intermediate payoff $\log 2$ on the transition from position 2 to 2 is omitted. 68

6.1 Stream gradients of the inverted-input DIC model (`cnn_maxout` DICIN) on a correctly classified MNIST test digit "3", thresholded and overlaid per class logit. The g-stream gradient (red) marks factual evidence on present strokes, while the h-stream gradient (blue) marks counterfactual evidence: pixels whose addition would raise the respective class logit. Left: the original digit. 102

6.2 Second example of stream gradients of the inverted-input DIC model (`cnn_maxout` DICIN) on a correctly classified MNIST test digit "3", thresholded and overlaid per class logit. The g-stream gradient (red) marks factual evidence on present strokes, while the h-stream gradient (blue) marks counterfactual evidence: pixels whose addition would raise the respective class logit. Left: the original digit. 103
6.3 Input-gradient visualizations of the `cnn_maxout` IC model for ten MNIST digits, each with respect to its ground-truth label. Top: saliency maps; bottom: input images. As in the source figure, positive and negative gradient values above threshold 0.2 are colored red and blue, with ranking colorized using exponential decay. 103

7.1 Saliency map comparison for VGG-16 on our custom ImageNet-S test split. Each row shows the predicted class (with confidence), original image, and saliency maps from different methods. 107

8.1 Hellinger trajectory distance under cascading parameter randomization (Adebayo et al., 2018). Hellinger distance H (vertical axis, from 0.7) against the number of cascadingly re-initialized layers, output to input (horizontal axis; H = 0 before the first layer is randomized). Solid: plain H; dotted: H_{surv}, conditioned on survival to an ordinary input-layer state. Thin horizontal reference lines mark the fully-random-model values from Table 8.3 (solid: plain H; dotted: H_{surv}), in the color of the corresponding game. 114
8.2 Input-noise Hellinger trajectory distance. Solid: plain H; dotted: H_{surv}, conditioned on survival to an ordinary input-layer state. x-axis is log-scale input-noise σ; same (A,A) model on both sides, only the input differs. 127

A.1 Saliency map comparison for VGG-16 on images from the custom ImageNet-S test subset. Each row shows the predicted class (with confidence), original image, and saliency maps from different methods. Methods include SplitLRP, classical LRP, SplitCAM variants, LayerCAM, and SplitGrad. 141
A.2 Dense qualitative comparison on six ImageNet-S examples (ViT-B/16). Columns: Original; ImageNet-S pixel Mask; ***R**outing **G**ame* (RG) and ***R**outing **G**ame* (RG)+Eq (merged attention–Value rule); Gradient, SmoothGrad, Integrated Gradients, DAVE, LRP-γ, AttnLRP and TIBAV. Every method is shown at its `loc`-selected configuration, Gradient and TIBAV at their fixed configurations (values in the text). Red: positive, blue: negative attribution, each sign divided by its 99th percentile over the image and clipped. 171

List of Tables

3.1 Linear ***C**onvolutional **N**eural **N**etwork* (CNN) chunks as instances of Lemma 3.17. . . 44

4.1 Modes of the games. "Exact": the mode stays within a represented rule; "deform.": optional deformation, off (default value) at the anchor. σ^2: input variance of the *Assumed Density Filtering* (ADF) moment propagation; N: number of transformed inputs averaged by ***R**outing **G**ame* (RG)+Eq. 52

4.2 Notation. Each symbol has one meaning throughout the game and trajectory chapters, except the local attention symbols $Q_t, K_k, e_{tk}, s_{tk}, m, s, c, T_n$ and the token index t, defined in Section 5.4. 53

4.3 Modes: default (anchor) value, effect and measured cost per image. 75

7.1 Comprehensive interpretability evaluation results (VGG-16, ImageNet-S test split). The *wta* abbreviation indicates the use of the "winner-takes-it-all" max-pooling implementation; *sc* emphasizes the scaling procedure in the forward pass (Section 3.6). Best values per metric are bold with dark blue shading. The bracket notation in the layer column follows the upper indices of the split methods as introduced in Chapter 6. 108

7.2 Comprehensive interpretability evaluation results (ResNet-18, ImageNet-S test split). The triples $x.y.z$ are a ResNet-18-internal indexing of the convolutional layer at which the attribution maps are calculated. See the caption of Table 7.1 for further abbreviations and table explanation. 109

7.3 Comparison of different α parameters with two SplitGrad and SplitCAM base configurations, both in *wta* and *abs* mode, evaluated on the custom ImageNet-S test split. . 109

7.4 ViT-B/16 on the ImageNet-S test split (n=565; MaxS/AvgS on a fixed random subset of 200 test images). **Bold**, dark shading: best in column; $\approx$, light shading: not significantly different from the best (paired bootstrap / McNemar, Holm-corrected, $p \geq 0.05$); small numbers: 95% percentile-bootstrap confidence intervals over test images (Section A.3.6). One row per method, selected on the validation split by the localization rank-sum (faithfulness-selected rows and further baselines: Section A.4); the RG anchor is the exact $\alpha\beta$-***L**ayer-wise **R**elevance **P**ropagation* (LRP) representation (Theorem 4.2). t/img: seconds per image on one GPU. Config: SmoothGrad averages N samples at noise level σ relative to the input range; LRP-γ has stabilizer ε and positive-weight boost γ; AttnLRP has layer-type parameters $\gamma_{\text{conv}}, \gamma_{\text{lin}}, \varepsilon_{\text{attn}}$; KernelSHAP draws N samples for M superpixels; ***R**outing **G**ame* (RG) modes as in Table 4.1. 111

8.1 Reference baselines for the input-noise sweep, averaged over R pairs drawn from the 565-image ImageNet-S test split (clean rows: $R = 200$ sampled pairs; clean-vs-random: $R = 565$ images paired with a fresh Gaussian each; random-vs-random: $R = 200$ Gaussian pairs); entries are mean $\pm$ standard deviation over pairs. **clean-intraclass**: $\mathrm{H}(\Pi_A(x_1), \Pi_A(x_2))$ with x_1, x_2 from the same ground-truth class; **clean-interclass**: same, with distinct classes; **clean-vs-random**: one side clean, other Gaussian; **random-vs-random**: both sides Gaussian. 128

8.2 Parameter randomization sanity check (Adebayo et al., 2018). Each cell reports the average similarity between the attribution of the original pretrained model and the attribution of a fully randomized copy (every convolutional and linear layer re-initialized; normalization layers, the attention's query/key/value projection and the ViT embeddings stay pretrained). Values near zero indicate that the method detects the parameter change; the Spearman correlation is signed, so negative values occur. Bold marks the value closest to zero per metric within each architecture, among maps that are not constant; ‡: the map is constant at full randomization (similarity 0 by convention). Rows: the reported configuration of every method (Section 7.5). 132
8.3 Hellinger trajectory distance between two independently random-initialized copies of the same architecture, averaged over $R = 500$ rounds (one fresh ImageNet-S test image per round). Plain H concentrates at an architecture-dependent cemetery floor of 0.94–0.97; conditioned-survival H_{surv} saturates at unity. The VGG-16 Routing Game cell is the one case in which the plain distance also saturates at 1.000. 133
8.4 Per-image Spearman rank correlations across the 565-image ImageNet-S test split at the aligned cascading steps of the Hellinger and attribution-sanity pipelines. Left block: within-Hellinger cross-correlations of the plain ($H_{\cdot,pl}$) and survival-conditioned ($H_{\cdot,surv}$) distances under the ***R**outing **G**ame* (RG) and ***S**topping **G**ame* (SG) kernels (self-entries shown as −; trivially +1). Right block: Hellinger-vs-map cross-correlations. The Hellinger-vs-map block is uniformly $|\rho| \leq 0.19$: per-image Hellinger and per-image attribution-map similarity rank images essentially independently. 134

A.1 Comprehensive interpretability evaluation results (VGG-16; representative configurations, best per method). Abbreviations as in Table 7.1. For each method configuration the best metric values across the considered layer configurations are highlighted in bold with dark blue shading. PF^{drop} is score-drop AUC; n/r denotes a result not reported in the paper, and n/a an inapplicable entry. 142
A.2 Comprehensive interpretability evaluation results (ResNet-18; representative configurations, best per method). Abbreviations as in Table 7.2. For each method configuration the best metric values across the considered layer configurations are highlighted in bold with dark blue shading. PF^{drop} is score-drop AUC; n/r denotes a result not reported in the paper, and n/a an inapplicable entry. 143
A.3 Randomization sanity check for the split methods on VGG-16: mean similarity between attribution maps of the trained model and of a copy whose last three feedforward layers are randomly reinitialized, across the custom test subset of ImageNet-S. Lower similarity indicates stronger dependence on the learned parameters. Abbreviations as in Table 7.1. 145
A.4 Mean saliency computation time comparison (seconds per image), averaged over the custom 50-image validation subset of ImageNet-S. 146
A.5 Alpha ablation as performed in Section 7.4 across four SplitGrad and SplitCAM configurations of layers 14 and 26, respectively. The notation is similar to Table 7.1. . . 148
A.6 Scale (sc) vs. shift (sh) forward-pass stabilization for SplitLRP methods on VGG-16. 149
A.7 Alpha ablation for scale normalization across four SplitCAM configurations of layers 14 and 26. Here, in the forward pass *no* stabilization is applied. Notation and structure are similar to Table A.5. Light blue entries marked with * indicate better performance of the method compared to the default variant with scale stabilization in the forward pass. 149

A.8 Alpha ablation for scaling normalization across four SplitCAM configurations of layers 14 and 26. Here, in the forward pass shift normalization is applied. Notation and structure are similar to Table A.5. Light blue entries marked with * indicate better performance of the method compared to the default variant with scale stabilization in the forward pass. 150
A.9 Per-method validation grids (all architectures unless stated). Symbols: Section A.4. . 154
A.10 ViT-B/16: validation-selected configurations per method and criterion (`loc`: minimal rank-sum over AL, PG, TK; `faith`: over PF_5, PF_{20}, Sel; rank-sums over the configurations of one method, lower is better; the number of configurations is given after the method name). Symbols: Section A.4. 157
A.11 VGG-16: validation-selected configurations per method and criterion, as in Table A.10. 158
A.12 ResNet-50: validation-selected configurations per method and criterion, as in Table A.10. 159
A.13 ViT-B/16, ImageNet-S test split: all rows with 95% bootstrap confidence intervals. Metrics as in Table 7.4; configuration symbols and row labels: Section A.4. 159
A.14 VGG-16, ImageNet-S test split: all rows with 95% bootstrap confidence intervals, as in Table A.13. 160
A.15 ResNet-50, ImageNet-S test split: all rows with 95% bootstrap confidence intervals, as in Table A.13. 161
A.16 ViT-B/16 on the Pascal VOC 2012 test pool (2,743 samples): all rows with 95% bootstrap confidence intervals. ***Routing Game*** (RG) / ***Routing Game*** (RG)+Eq use the ImageNet-S-selected configurations without re-tuning; baselines are selected on the VOC val pool. Only localization and faithfulness metrics were evaluated; the unmeasured MaxS and AvgS columns are omitted. Metrics as in Table 7.4; configuration symbols and row labels: Section A.4. 163
A.17 ViT-B/16, ImageNet-S test split: one-factor ablations around the selected configurations. Center: the selected configuration (criterion in parentheses) around which the parameter in the Factor column is varied over the Value column, all other parameters fixed; bold: the center's own value. Symbols: Section A.4. 164
A.18 VGG-16, ImageNet-S test split: one-factor ablations around the selected configurations, as in Table A.17. 166
A.19 ResNet-50, ImageNet-S test split: one-factor ablations around the selected configurations, as in Table A.17. 167
A.20 Pointing game under the Quantus tie rule (PG, as in the main tables) and at matched granularity, ImageNet-S test split. max. set: median number of pixels attaining the map maximum; PG_{1px}: expected hit rate of one pixel drawn uniformly from that set; PG_{16px}: PG after average-pooling the map to 16×16-pixel cells. Rows: the `loc`-selected or fixed configuration of every method, plus the `faith` rows of our games. **Bold**: best per model and column. 169
A.21 ***Vision Transformer*** (ViT)-B/16, ***Routing Game*** (RG): attention rule ablation on the ImageNet-S test split (565 images), at the anchor and at the loc- and faith-selected configurations. Each cell shows the gradient fallback on top and the composite Value rule of Theorem 5.6 (gray) below; configuration symbols: Section A.4; †: paired difference significant (paired bootstrap / McNemar for PG, Holm-corrected over the table, $p < 0.05$). 170

List of Symbols

Greek Symbols

Symbol	Description	Unit
α	Backward pass shift parameter	
δ	Network gradient (sensitivity)	
δ^g, δ^h	Split stream gradients	
γ	Discount factor / LRP emphasis parameter	
π, σ	Policies of Max and Min players	
τ	Temperature / entropy regularization parameter	
ε	Numerical stability constant	

Indices

Symbol	Description
c	Channel index
i,j	Neuron indices
l	Layer index

Latin Symbols

Symbol	Description	Unit
$a^{(l)}$	Post-activation at layer l	
$a^{(l,+)}, a^{(l,-)}$	Positive and negative activations of split network	
$d^{(l)}$	Width of layer l	
f	Network function	
f_P	Support function of polytope P	
g	Positive split stream (monotone, convex)	
h	Negative split stream (monotone, convex)	
L	Number of hidden layers	
$P^l_{i\to j}$	Transition probability in game	
$Q[l]$	Auxiliary network with min and absolute weights from layer l	
Q^l	Q-function (action-value function)	
$R^{(l)}$	LRP relevance score at layer l	
$V^l_{i,\pm}$	Game value function at state (l,i,$\pm$)	
$W^{(l)}$	Weight matrix of layer l	
$W^{(l,+)}, W^{(l,-)}$	Positive and negative parts of weight matrix	
x	Network input	
$z^{(l)}$	Pre-activation at layer l	

Operators

Symbol	Description	Definition
∇	Gradient operator	$\left[\frac{\partial}{\partial x_1}, \ldots, \frac{\partial}{\partial x_n}\right]^T$
$\odot$	Hadamard (element-wise) product	
ReLU	Rectified linear unit	$\max\{0, x\}$

List of Abbreviations

Methods

ADF	Assumed Density Filtering
AL	Attribution Localization
CAM	**C**lass **A**ctivation **M**apping
CDF	Cumulative Distribution Function
CPWL	**C**ontinuous **P**iece**w**ise **L**inear
DC	**D**ifference of **C**onvex
GAE	Generic Attention Explainability
ICNN	**I**nput-**C**onvex **N**eural **N**etwork
LMP	Layered Markov Process
LRP	**L**ayer-wise **R**elevance **P**ropagation
MDP	**M**arkov **D**ecision **P**rocess
MP	Markov Process
NRM	**N**ormalized **R**elevance **M**easure
PG	Pointing Game
RG	**R**outing **G**ame
SeI	Selectivity
SG	**S**topping **G**ame
TIBAV	Transformer Interpretability Beyond Attention Visualization

Architectures

CNN	**C**onvolutional **N**eural **N**etwork
GELU	Gaussian Error Linear Unit
LLM	**L**arge **L**anguage **M**odel
MLP	**M**ulti**l**ayer **P**erceptron
ReLU	**R**ectified **L**inear **U**nit
ViT	**Vi**sion **T**ransformer

1 Introduction

1.1 Motivation and Problem Statement

Interpretability is moving from a tool for inspecting model decisions to a practical mechanism for improving and controlling models. Attribution methods now help reveal and correct spurious behavior in vision models (Anders et al., 2022; Pahde et al., 2023), identify retrieval and parametric heads in language models (Kahardipraja et al., 2025), steer decoding toward instructions or evidence (Komorowski et al., 2025), and localize circuit components through attribution patching (Hanna et al., 2024; Kramár et al., 2024; Syed et al., 2024). The central motivation of this thesis is this connection between *understanding a model* and *making it better*: once an interpretation guides training, correction, or inference, the calculation producing it becomes part of the intervention itself.

There is a concrete reason to look beyond predictive accuracy. A network can perform well on a benchmark while relying on features that fail under changed conditions. Geirhos et al. (2020) describe this as shortcut learning; Lapuschkin et al. (2019) demonstrate how attribution can uncover such "Clever Hans" strategies in trained models. Interpretability makes it possible to inspect which evidence supports a prediction, diagnose failure modes, and test whether a learned strategy matches the intended task (Mersha et al., 2024; Ribeiro et al., 2016; Samek et al., 2021). These uses matter both during model development and when assessing an existing system for deployment. Such tools have also exposed spurious "Clever Hans" behavior in anomaly detection and motivated methods for interpreting one-class models (Kauffmann et al., 2020a,b).

The step from diagnosis to improvement has several forms. Ross et al. (2017) use annotations of irrelevant inputs to constrain a model's gradients during training; Ross & Doshi-Velez (2018) study input-gradient regularization for adversarial robustness. Class Artifact Compensation suppresses identified artifact directions in learned representations (Anders et al., 2022), while Reveal to Revise organizes detection and correction into an iterative model-development cycle (Pahde et al., 2023). These examples make the motivation operational: interpretations can identify what to change, and their mathematical properties affect which changes a downstream procedure will make. Post-hoc inspection remains useful throughout this cycle.

This broader role also requires a precise account of what interpretability promises. Lipton (2018) distinguish the different objectives often grouped under the term, and Doshi-Velez & Kim (2017) argue that evaluation must match the intended application and audience. The framework of Murdoch et al. (2019) separates predictive accuracy, descriptive accuracy, and relevance to the user. In this thesis, the immediate object of study is the computation of a trained network and the backward rules used to attribute its output. Faithfulness to a specified computation, numerical stability, localization, and sensitivity to learned parameters can be investigated separately; a convincing heatmap alone does not establish that a subsequent model intervention will succeed.

Two obstacles connect the papers on which this thesis builds. The first concerns the *representation* being interpreted. Weight signs and activation gates combine opposing contributions into a single activation, obscuring how those contributions accumulate. Monotonicity has long provided useful structure for this problem (Amos et al., 2017; Sill, 1997): with non-negative weights, increasing an input cannot decrease downstream evidence. Networks that enforce non-negative weights represent parts of the learned structure without cancellation between positive and negative weighted contributions within a stream (Ayinde and Zurada, 2018; Lee and Seung, 1999). Monotonicity constraints have also been used to express domain-specific fairness requirements (Wang and Gupta, 2020); the constraint

encodes the chosen requirement rather than guaranteeing fairness on its own. Inherently interpretable models are an important option, particularly in the high-stakes settings discussed by Rudin (2019). For an already trained, unconstrained network, however, changing its weights to impose monotonicity generally changes its predictions. Hidden Monotonicity instead splits a trained ReLU network into two monotone, convex stream functions g and h such that $f = g - h$. The key contribution is the numerical stabilization of the forward and backward passes through this representation, making its internal signals accessible without retraining or altering the represented function (Zimmermann and Loho, 2026).

The second obstacle concerns the *backward calculation* applied to that representation. Gradients, Layer-wise Relevance Propagation (***L**ayer-wise **R**elevance **P**ropagation* (LRP)), Deep Taylor decomposition, and transformer-specific rules are commonly specified through different local recipes and stabilizers. Playing the Network Backward seeks a common account of those rules as weighted trajectories through the computational graph. Building on the ReLU Net Game of Gaubert & Vlassopoulos (2025), its stopping and routing games represent gradients and, at the unit-temperature routing anchor, the full $\alpha\beta$-LRP family through discounted occupation measures. Stop decisions, sign switches, and routing policies then become explicit mathematical objects that can be compared and modified.

The thesis joins these perspectives through a single notion of *network decomposition*: lift a signed computation into two coordinated streams whose difference recovers the original network. The DC and stopping decompositions use the same split of weights by sign but retain different internal information. The DC split preserves pre-activation contributions that an inactive ReLU would discard; the stopping split retains the original gate pattern and supports the trajectory representation of the backward pass. A family of mixing and regularized decompositions makes their relationship explicit. This is the common foundation for the split attribution methods, game-based modifications, tropical geometry, and trajectory diagnostics developed in the subsequent chapters.

The move toward model improvement makes existing critiques of attribution more consequential. Parameter insensitivity, input dependence, perturbation fragility, and misleading modified-gradient structure (Adebayo et al., 2018; Ghorbani et al., 2019; Kindermans et al., 2018; Nie et al., 2018) can affect any correction or steering procedure that relies on these signals. The two papers address complementary parts of this problem: the split representation exposes internal contributions and their derivative identities, while the game representation makes routing choices and distributions over calculations explicit. Numerical stabilization enables the former at depth; policy regularization, risk aversion, and extended action sets provide design choices for the latter. Hellinger diagnostics compare the resulting trajectory laws, complementing comparisons of their projected attribution maps. The experiments establish properties of these attribution methods; downstream gains from using them to correct or steer models remain a further empirical question.

Three research questions organize this combined account. *What structure can be exposed without changing the network's function?* The decomposition framework connects monotone convex streams, sign-separated paths, and the polyhedral geometry of attainable gradients. *What does a backward attribution actually calculate?* Derivative identities and game occupation measures relate local propagation rules to explicit global objects. *How can those calculations be made useful for model improvement?* Stable implementations, controlled modifications of backward rules, and complementary evaluation criteria provide the foundation on which diagnostic and intervention methods can build.

The thesis merges and extends two publications by the author: *Hidden Monotonicity: Explaining Deep Neural Networks via their DC Decomposition* (Zimmermann and Loho, 2026) and *Playing the Network Backward: A Game-Theoretic Attribution Framework* (arXiv:2605.06212v2, revised for AISTATS). Their results are organized by this shared chain of questions: representation, backward semantics,

method design, and evaluation. The original experimental protocols are retained so that the evidence for each construction remains identifiable.

1.2 Contributions of This Thesis

The contributions of this thesis are the following.

1) **A general framework of network decompositions.** Chapter 3 introduces a *network decomposition* and its *validity*: a lift of a network into a pair of non-negative streams, propagated by non-negative maps $W^{(l,+)}, W^{(l,-)}$ and pairwise activations, whose difference recovers the original computation at every layer. Within it the thesis constructs a whole family — the mixing family f^{η}_{mix} interpolating between maximum- and minimum-based pair activations, the stopping decomposition f_{stop}, and regularized decompositions f^{Ω}_{reg} whose entropy-regularized member f^{θ}_{soft} yields sigmoid-gated soft splits. The classical ***D**ifference of **C**onvex* (DC) split $f = g - h$ is the $\eta = 1$ endpoint of the mixing family rather than an isolated trick, and a splitting theorem for activations with integrable curvature extends the construction beyond ***Re**ctified **L**inear **U**nit* (ReLU) to GELU, Swish, tanh, sigmoid, and softplus networks.
2) **A numerically stable DC decomposition of ReLU networks and *Convolutional Neural Network*s (CNNs).** A naive split diverges: the two streams grow exponentially with depth while their difference stays bounded, so floating-point cancellation destroys the signal. Chapter 3 develops stabilization — affine transformations of the split activations and propagated signals — keeping both streams on the scale of the original network without altering the represented function. The construction is carried out in full for convolutional architectures, with a correctness statement for every standard operation: convolution, gated max-pooling, average pooling, batch normalization, residual connections, and dropout. The result transforms any pretrained ReLU network into two monotone convex subnetworks whose difference reproduces the original output.
3) **A game-theoretic reformulation of the backward pass.** Chapter 4 reinterprets the stopping decomposition as a two-player zero-sum stopping game on an extended network graph, adapting the ReLU Net Game of Gaubert & Vlassopoulos (2025): layers are rounds, neurons states, sign switches hand the turn to the opponent, and the network output is the game value. Backward attribution becomes a *distribution over game trajectories*: propagation rules are equilibrium policies of the players P^+ and P^-, and attribution maps discounted occupation measures of that distribution. Two games anchor the theory — the ***S**topping **G**ame* (SG), whose occupation measures recover the Jacobian, and the entropy-regularized ***R**outing **G**ame* (RG), recovering the full $\alpha\beta$-LRP family — giving the two methods a shared trajectory-space representation. These are representation theorems: the architecture alone does not determine a unique game or discount. The trajectory-space view of Chapter 8 further yields a Hellinger distance between backward *calculations* (not heatmaps), detecting the calculation collapse pixel-space sanity checks (Adebayo et al., 2018) observe only indirectly, and analytical *modes* — risk aversion, action-space augmentation, policy regularization — mapping back to principled backpropagation rules.
4) **New attribution methods and benchmark evaluations.** Chapter 6 builds three attribution methods on the split representation: SplitCAM, a LayerCAM-inspired (Jiang et al., 2021) ***C**lass **A**ctivation **M**apping* (CAM) method on the split-streams; SplitLRP, an adaptation of LRP (Bach et al., 2015) to the split-pair; and SplitGrad, a gradient method exploiting the improved signal flow. Evaluated with the Quantus suite (Hedström et al., 2023) on VGG-16 (Simonyan and Zisserman, 2015) and ResNet-18 (He et al., 2016) over ImageNet-S (Mao et al., 2022), they show competitive or improved performance compared to widely used baselines across the faithfulness, localization, and robustness metrics (Chapter 7), with different methods excelling at different

criteria and no single method dominating all metrics. The chapter also develops the inherently interpretable counterpart: difference-of-convex input neural networks trained directly as $g - h$, whose streams divide labor cleanly between present and missing features.

5) **A tropical-geometric interpretation and Imaginative Polytopes.** The tropical foundations (Section 2.4) place the decomposition in the polyhedral-geometry view of ReLU networks (Misiakos and Thrampoulidis, 2022; Zhang et al., 2018b): the split-streams g and h are the two tropical polynomials whose difference the network computes, their Newton polytopes evolving across layers by Minkowski sums and convex hulls. The thesis introduces *Imaginative Polytopes* and proves that their vertices encode the attainable gradients of the split-streams, giving them an interpretation as learned visual prototypes and counterfactual feature profiles.
6) **Extension to attention mechanisms and Transformers.** Chapter 5 extends the framework to the multiplicative operations at the heart of attention: gated splits of matrix and Hadamard products, and an oracle-player construction whose reference policy is the QK-softmax, so attention becomes a routing policy inside the backward game. This makes SplitLRP-style and game-derived attribution applicable to ***Vi**sion **T**ransformer*s (ViTs) (Dosovitskiy et al., 2021); on ViT-B/16, one selected game mode outperforms prior transformer-specific backward methods in attribution localization, with that lead carrying over to Pascal VOC without re-tuning. Pointing Game, Top-k overlap, faithfulness, and robustness reveal distinct trade-offs; the revised evaluation reports independently validation-selected localization and faithfulness configurations, confidence intervals, and paired tests.

1.3 Positioning within the Literature

This section positions the thesis along its three research questions; it is brief, since dedicated related-work discussions open the chapters where the material is developed — games and propagation rules in Section 4.1 and transformer interpretability in Section 5.1.

What structure is hidden inside a trained network? The idea that sign-constrained networks are the interpretable core of deep learning runs from monotonic networks (Daniels and Velikova, 2010; Sill, 1997) through certified and lattice-based monotone models (Gupta et al., 2016; Liu and Coogan, 2020; Nguyen et al., 2023; Runje and Shankaranarayana, 2023; Sivaraman et al., 2020) to ***I**nput-**C**onvex **N**eural **N**etwork*s (ICNNs) (Amos et al., 2017) and convex network calculus (Bach, 2017; Calafiore et al., 2020; Sivaprasad et al., 2021). DC decompositions bridge this to unconstrained networks: classical DC programming (Sriperumbudur and Lanckriet, 2009; Tao and An, 1997; Thi and Dinh, 2018) and the representation theory of piecewise-linear functions as differences of convex ones (Brandenburg et al., 2025; Griewank and Walther, 2020; Kazda and Li, 2024; Koutschan et al., 2023; Kripfganz and Schulze, 1987; Melzer, 1986; Schlüter and Darup, 2021; Zalgaller, 2000) guarantee the split exists; Awasthi et al. (2023) and Siahkamari et al. (2020) construct essentially the split used here, for training and regression, and CDiNN-type architectures make the difference structure trainable (Chen, Wang, et al., 2025; Sankaranarayanan and Rengaswamy, 2022). None confronts the numerical instability of the split or applies it to explanation, where this thesis departs. The geometric reading rests on the identification of ReLU networks with ***C**ontinuous **P**iece**w**ise **L**inear* (CPWL) maps and tropical rational functions (Balestriero and Baraniuk, 2021; Huchette et al., 2025; Koehler and Risteski, 2018; Magron and Wang, 2022; Misiakos and Thrampoulidis, 2022; Mukherjee et al., 2018; Phuong and Lampert, 2020; Zhang et al., 2018b); the Newton-polytope perspective is developed in Section 2.4.

When can explanations be trusted? The attribution toolbox spans gradient saliency (Simonyan et al., 2014), deconvolution and guided backpropagation (Springenberg et al., 2015; Zeiler and Fergus, 2014), LRP and Deep Taylor decomposition (Bach et al., 2015; Montavon et al., 2017, 2018), CAM variants (Chattopadhay et al., 2018; Jiang et al., 2021; Selvaraju et al., 2017), axiomatic methods (Shrikumar et al., 2017; Sundararajan et al., 2017), noise averaging (Smilkov et al., 2017), and perturbation- or surrogate-based explanations (Fel et al., 2023; Fong and Vedaldi, 2017; Kolek et al., 2023; Lundberg and Lee, 2017; Ribeiro et al., 2016). A wave of critiques sharpened the trust question: sanity checks reveal parameter insensitivity (Adebayo et al., 2018), explanations can be fragile or manipulable (Alvarez-Melis and Jaakkola, 2018; Dombrowski et al., 2019; Ghorbani et al., 2019), modified backward rules can produce structure unrelated to the model (Nie et al., 2018; Sixt et al., 2020), and input dependence confounds attribution (Kindermans et al., 2018). In response, evaluation frameworks such as Quantus (Hedström et al., 2023) operationalize faithfulness, localization (Zhang et al., 2018a), robustness, and complexity (Bhatt et al., 2020; Yeh et al., 2019) as measurable desiderata — the standard this thesis holds its methods to in Chapter 7. The thesis' answer is structural rather than post hoc: attribution is computed on a representation (the split-pair) in which evidence cannot cancel invisibly, or derived from a game whose trajectory distribution *is* the backward calculation, so sanity-check failures can be studied at both the trajectory and pixel levels (Chapter 8); their different responses to randomization do not by themselves establish explanation faithfulness.

How can attribution be made stable? The closest precursor to the game framework is the ReLU Net Game of Gaubert & Vlassopoulos (2025), which identifies forward evaluation with the value of a zero-sum stopping game but connects it neither to the backward pass nor to attribution. Related stochastic views of relevance flow include Excitation Backprop's absorbing Markov chain (Zhang et al., 2018a), which the SG generalizes by adding an opponent, and the ***N**ormalized **R**elevance **M**easure* (NRM) framework (Xiong et al., 2026), a signed-measure calculus over walks compared with our trajectory distributions in Section 8.1. Shapley-value methods (Lundberg and Lee, 2017) play a *different*, cooperative game over input coalitions in a black-box setting, complementary to the white-box backward game here. Stability itself has been approached via curvature reduction with softplus activations (Dombrowski et al., 2022) — recovered here as entropy regularization of the stop decision — and via risk-sensitive control (Fei et al., 2020) motivating the risk-averse game modes. On transformers, attention rollout and flow (Abnar and Zuidema, 2020), transformer-LRP (Achtibat et al., 2024; Ali et al., 2022; Chefer et al., 2021a,b), and artifact-aware constructions (Darcet et al., 2024; Wróbel et al., 2026) define the state of the art Chapter 5 engages with; Section 4.1 and Section 5.1 give the full accounts. The growing use of attribution as a control mechanism — model correction (Pahde et al., 2023), head identification (Kahardipraja et al., 2025; Voita et al., 2019), guided decoding (Komorowski et al., 2025), circuit localization (Hanna et al., 2024; Kramár et al., 2024; Syed et al., 2024) — raises the stakes: unstable attributions become failure modes of downstream procedures, not merely misleading pictures.

Outline. Chapter 2 assembles the foundations: network notation, CPWL mappings, convexity and monotonicity, ***M**arkov **D**ecision **P**rocess*s (MDPs) and zero-sum games — including the ReLU Net Game of Gaubert and Vlassopoulos — and the tropical geometry of the split, with the smooth-activation splitting theorem and the Imaginative-Polytope picture. Chapter 3 develops the decomposition framework, numerical stabilization, and the CNN extension; Chapter 4 the two-player stopping and routing games, with gradients and $\alpha\beta$-LRP recovered as equilibrium occupation measures; Chapter 5 the extension and implementation on multiplicative operations and ViTs; and Chapter 6 the split methods SplitCAM, SplitLRP, and SplitGrad, inherently interpretable difference-of-convex architectures, and

the attribution modes. Chapter 7 reports the evaluation on ImageNet-S and Pascal VOC across VGG-16, ResNet, and ViT-B/16; Chapter 8 develops the trajectory-distribution view and its Hellinger diagnostics; and Chapter 9 concludes with future directions from ***L****arge* ***L****anguage* ***M****odel*s (LLMs) to hardware-level acceleration.

2 Mathematical Foundations

This chapter fixes the notation, definitions, and background theory used throughout. Section 2.1 fixes the feedforward network notation and recalls the equivalence between ReLU networks and CPWL functions. Section 2.2 introduces the two structured classes—monotone networks and ICNNs—into which Chapter 3 splits a general network, together with the Maxout units generalizing ReLU. Section 2.3 collects the background on MDPs and zero-sum turn-based games and presents the ReLU Net Game of Gaubert and Vlassopoulos in full, so that Chapter 4 can state which parts of the framework it inherits and which are new. Section 2.4 provides the tropical-geometric vocabulary—semirings, tropical polynomials, Newton polytopes, Minkowski sums, and support functions—and, in its later subsections, develops the tropical reading of the DC split, the resulting Imaginative Polytopes (Sections 2.4.1–2.4.3), and the difference-of-convex splitting of smooth activation functions (Section 2.4.4) that lets the decompositions of Chapter 3 extend beyond ReLU.

2.1 Neural Network Notation and PWL Mappings

The standard feedforward architecture serves as the reference model throughout; convolutional and attention-based architectures are treated in Section 3.7 and Chapter 5 as structured special cases and extensions.

Definition 2.1 (Feedforward ReLU network). Let $L \in \mathbb{N}$ and let $d^{(0)}, d^{(1)}, \dots, d^{(L)} \in \mathbb{N}$ denote the *layer widths*, with input dimension $d^{(0)}$ and output dimension $d^{(L)}$. A *feedforward ReLU network* of depth L is given by weight matrices $W^{(l)} \in \mathbb{R}^{d^{(l)} \times d^{(l-1)}}$ and bias vectors $b^{(l)} \in \mathbb{R}^{d^{(l)}}$ for $l = 1, \dots, L$. On an input $x \in \mathbb{R}^{d^{(0)}}$ it computes the recursion

$$\begin{aligned} a^{(0)} &:= x, \\ z^{(l)} &:= W^{(l)} a^{(l-1)} + b^{(l)}, && l = 1, \dots, L, \\ a^{(l)} &:= \mathrm{ReLU}\big(z^{(l)}\big), && l = 1, \dots, L-1, \end{aligned} \tag{2.1}$$

where $\mathrm{ReLU}(z) := \max\{z, 0\}$ acts componentwise, and the output layer is affine, $a^{(L)} := z^{(L)}$. We call $z^{(l)}$ the *pre-activation* and $a^{(l)}$ the *(post-)activation* of layer l, and denote the *network function* by

$$f \colon \mathbb{R}^{d^{(0)}} \longrightarrow \mathbb{R}^{d^{(L)}}, \qquad f(x) := a^{(L)}. \tag{2.2}$$

Layers $1, \dots, L-1$ are the *hidden layers*. Throughout we fix the boundary convention $\mathrm{ReLU}'(0) := 0$, so that the gate indicator $\mathbf{1}\{z_i^{(l)} > 0\}$ returns 0 at $z_i^{(l)} = 0$.

When a statement concerns the layers $l, \dots, L$ only, we write $f^{(l)} \colon \mathbb{R}^{d^{(l)}} \to \mathbb{R}^{d^{(L)}}$ for the tail map, so that $f = f^{(1)} \circ \big(x \mapsto \mathrm{ReLU}(W^{(1)}x + b^{(1)})\big)$ and $f^{(L)}$ is the affine output map. A bias-free formulation is always available by appending an *intercept input* of constant value 1 and absorbing $b^{(l)}$ into a column of $W^{(l)}$; we use this in some proofs without comment.

The decompositions of Chapter 3 act on the signs of parameters and inputs, fixed by the following notation.

Definition 2.2 (Positive and negative parts). For a scalar $u \in \mathbb{R}$ we write $u^+ := \max\{u, 0\}$ and $u^- := \max\{-u, 0\}$, so that $u = u^+ - u^-$ and $|u| = u^+ + u^-$, with $u^+ u^- = 0$. For a matrix or vector W

and a sign label $\sigma \in \{+,-\}$, the *entrywise part* $[W]^\sigma$ is obtained by applying $u \mapsto u^\sigma$ to every entry. For the layer weights and biases of Definition 2.1 we abbreviate

$$W^{(l,\sigma)} := [W^{(l)}]^\sigma, \qquad b^{(l,\sigma)} := [b^{(l)}]^\sigma, \qquad \sigma \in \{+,-\}, \tag{2.3}$$

so that $W^{(l)} = W^{(l,+)} - W^{(l,-)}$ and $|W^{(l)}| = W^{(l,+)} + W^{(l,-)}$ entrywise, and both $W^{(l,\sigma)}$ are entrywise non-negative.

Remark 2.1 (Pair notation for decomposed activations). Superscript sign labels on *activations*, written $a^{(l,\sigma)}$ for $\sigma \in \{+,-\}$, are reserved for the two non-negative *streams* of a network decomposition and are defined in Section 3.1; they do *not* denote the entrywise parts $[a^{(l)}]^\sigma$ of the original activations. The distinction matters: the decomposed streams carry strictly more information than the sign pattern of $a^{(l)}$—the source of the sub-threshold signals exploited in Chapter 6.

Continuous piecewise linear functions

The function class computed by ReLU networks admits an exact geometric description.

Definition 2.3 (CPWL function). A function $f\colon \mathbb{R}^d \to \mathbb{R}^m$ is ***C**ontinuous **P**iece**w**ise **L**inear* (CPWL) if it is continuous and there exist finitely many closed polyhedra $Q_1, \dots, Q_N \subseteq \mathbb{R}^d$ with $\bigcup_{r=1}^N Q_r = \mathbb{R}^d$ such that the restriction of f to each Q_r is an affine map. The full-dimensional polyhedra on which f is affine, taken maximal with respect to inclusion, are called the *linear regions* of f.

For example, $|\cdot|\colon \mathbb{R} \to \mathbb{R}$ is CPWL with the two regions $(-\infty, 0]$ and $[0, \infty)$, and $x \mapsto \max\{x_1, \dots, x_d\}$ is CPWL with d regions.

Observation 2.1 (`Lean_1`). Every feedforward ReLU network represents a CPWL function.

Proof. Affine maps are CPWL, the componentwise ReLU is CPWL, and the composition of two CPWL maps is again CPWL: if f is affine on the polyhedra $Q_1, \dots, Q_N$ covering $\mathbb{R}^d$ and g is affine on the polyhedra $P_1, \dots, P_M$ covering the intermediate space, then $g \circ f$ is affine on each of the at most NM polyhedra $Q_r \cap f^{-1}(P_s)$ (preimages of polyhedra under affine maps are polyhedra), these cover $\mathbb{R}^d$, and continuity is preserved under composition. The claim follows by induction along the recursion (2.1). □

The converse also holds: *every* CPWL function is exactly representable by a ReLU network of bounded depth. This equivalence, due to Mukherjee et al. (2018), is the entry point for the difference-of-convex analysis of Chapter 3 and the tropical-geometric analysis of Section 2.4.1.

Theorem 2.1 (CPWL–ReLU equivalence, (Mukherjee et al., 2018), `Lean_2`). *A function $f\colon \mathbb{R}^d \to \mathbb{R}$ is CPWL if and only if it is representable by a feedforward ReLU network. Moreover, every CPWL function on $\mathbb{R}^d$ is representable by a ReLU network of depth at most $\lceil \log_2(d+1) \rceil + 1$.*

Proof. The "if" direction is Observation 2.1. For the converse we give the classical construction and refer to Mukherjee et al. (2018) for the depth bookkeeping. First, every CPWL function f admits a *difference-of-convex* representation

$$f = g - h, \qquad g = \max_{r \in [N_g]} \ell_r, \qquad h = \max_{s \in [N_h]} m_s, \tag{2.4}$$

with finitely many affine functions $\ell_r, m_s \colon \mathbb{R}^d \to \mathbb{R}$; this is the piecewise-linear DC theorem of Melzer (1986) and Kripfganz & Schulze (1987), and both g and h are convex as finite maxima of affine functions. Second, the maximum of two scalars is a two-layer ReLU computation,

$$\max\{u, v\} = \frac{u+v}{2} + \frac{|u-v|}{2} = \frac{u+v}{2} + \frac{\mathrm{ReLU}(u-v) + \mathrm{ReLU}(v-u)}{2}, \tag{2.5}$$

where the affine pre- and post-combinations are absorbed into the surrounding weight matrices. Arranging the N_g affine arguments of g in a balanced binary tree of pairwise maxima realizes g by a ReLU network of depth $\lceil \log_2 N_g \rceil + 1$, and likewise for h; a final affine layer forms the difference (2.4). This proves representability. The stated depth bound, which is uniform in the number of affine pieces and depends only on the input dimension d, follows from the refinement of Mukherjee et al. (2018): every CPWL function on $\mathbb{R}^d$ can be written as a signed sum of maxima of at most $d+1$ affine functions each, so the trees in the construction above have depth at most $\lceil \log_2(d+1) \rceil$. □

Remark 2.2 (Expressivity and linear regions). Theorem 2.1 identifies the function *class*, not the *cost* of representation: deep networks can represent CPWL functions with exponentially many linear regions in the number of neurons, an interplay of depth, width, and region counts surveyed by Huchette et al. (2025). The tropical reformulation by Zhang et al. (2018b)—ReLU networks as tropical rational maps—is taken up in Sections 2.4 and 2.4.1. The representation (2.4) also foreshadows the central theme of this thesis: the convex CPWL streams g and h of the DC split can be extracted *constructively and locally* from a trained network, the subject of Section 3.3.

2.2 Convexity and Monotonicity in Neural Networks

The decompositions of Chapter 3 split a general network into components from two structured classes with sign-constrained weights. We define both and record the load-bearing facts that they represent monotone and convex functions, respectively.

We order $\mathbb{R}^d$ coordinatewise: $x \le y$ if and only if $x_i \le y_i$ for all $i \in [d]$. A function $f \colon \mathbb{R}^d \to \mathbb{R}^m$ is *monotone* (non-decreasing) if $x \le y$ implies $f(x) \le f(y)$.

Definition 2.4 (Monotone neural network). A feedforward network with recursion (2.1) and monotone activation functions is called a *monotone neural network* if all its weight matrices are entrywise non-negative, $W^{(l)} \ge 0$ for all $l = 1, \dots, L$.

Weight-sign constraints are the oldest device for building monotone models (Daniels and Velikova, 2010; Sill, 1997) and remain the structural core of modern certified-monotone architectures (Liu and Coogan, 2020; Runje and Shankaranarayana, 2023).

Proposition 2.2 (Monotone networks represent monotone functions, `Lean_3`**).** *A monotone neural network represents a monotone function; each coordinate function is non-decreasing in every input variable.*

Proof. By induction over the layers. The input layer $a^{(0)} = x$ is monotone in x. Assume $a^{(l-1)}$ is monotone in x. An affine map with non-negative matrix preserves the coordinatewise order: if $a \le a'$, then $W^{(l)}a + b^{(l)} \le W^{(l)}a' + b^{(l)}$ entrywise, because every entry of $W^{(l)}$ is non-negative. Hence $z^{(l)}$ is monotone in x, and applying the monotone activation componentwise preserves monotonicity, so $a^{(l)}$ is monotone in x. The affine output layer is again order-preserving. □

Definition 2.5 (Input-Convex Neural Network, (Amos et al., 2017)). A feedforward network with recursion (2.1) whose activation functions are convex *and* monotone is called an ***I**nput-**C**onvex **N**eural **N**etwork* (ICNN) if all weight matrices except possibly the first are entrywise non-negative, $W^{(l)} \geq 0$ for all $l = 2, \ldots, L$, while $W^{(1)}$ is unconstrained.

Definition 2.5 covers the plain feedforward core of the architecture of Amos et al. (2017); their sign-unconstrained skip ("passthrough") connections do not affect the statement below, since they enter each pre-activation affinely in x.

Proposition 2.3 (ICNNs represent convex functions, `Lean_4`). *An ICNN represents a convex function; each coordinate of every activation $a^{(l)}$, $l \geq 1$, is a convex function of the input x.*

Proof. We use two closure properties of convex functions: (i) a non-negative linear combination of convex functions plus a constant is convex; (ii) if $\varphi\colon \mathbb{R} \to \mathbb{R}$ is convex and non-decreasing and $u\colon \mathbb{R}^d \to \mathbb{R}$ is convex, then $\varphi \circ u$ is convex, since for $\lambda \in [0,1]$,

$$\varphi\big(u(\lambda x + (1-\lambda)y)\big) \leq \varphi\big(\lambda u(x) + (1-\lambda)u(y)\big) \leq \lambda\,\varphi(u(x)) + (1-\lambda)\,\varphi(u(y)),$$

where the first inequality uses convexity of u and monotonicity of φ, and the second convexity of φ.

Now induct over layers. Each coordinate of $z^{(1)} = W^{(1)}x + b^{(1)}$ is affine, hence convex, in x; applying the convex monotone activation gives convex coordinates of $a^{(1)}$ by (ii). Assume all coordinates of $a^{(l-1)}$ are convex in x. For $l \geq 2$ the weights are non-negative, so each coordinate of $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$ is a non-negative combination of convex functions plus a constant, hence convex by (i), and applying the activation preserves convexity by (ii). The output coordinates $a^{(L)} = z^{(L)}$ are convex by the same argument with the identity activation. □

Remark 2.3 (Monotonicity and convexity combine). If, in addition, $W^{(1)} \geq 0$ in Definition 2.5, the network is simultaneously monotone and input-convex, hence non-decreasing *and* convex by Propositions 2.2 and 2.3. Both ReLU and Maxout (below) are convex and monotone, so non-negative-weight ReLU networks are of exactly this type. These are the building blocks of the DC splitting of Section 3.3: the streams g and h are computed by monotone networks acting on a signed embedding of the input, and are input-convex in the original input. The sign constraint also *limits* expressivity—a weight-constrained ReLU network represents only convex functions, precisely Proposition 2.3, which is why monotone architectures add concave counterparts of the activation (Runje and Shankaranarayana, 2023). The decompositions of this thesis turn the limitation into a feature: the difference of two such networks recovers full generality by Theorem 2.1 and (2.4).

Maxout units

Definition 2.6 (Maxout unit and Maxout network, (Goodfellow et al., 2013)). A *rank-k Maxout unit* with weights $w_1, \ldots, w_k \in \mathbb{R}^d$ and biases $\beta_1, \ldots, \beta_k \in \mathbb{R}$ computes

$$x \longmapsto \operatorname{Maxout}(x) := \max_{j \in [k]} \big(w_j^\top x + \beta_j\big). \tag{2.6}$$

A *rank-k Maxout network* is a feedforward network whose neurons are rank-k Maxout units; equivalently, layer l carries k weight matrices $W_1^{(l)}, \ldots, W_k^{(l)}$ and biases $b_1^{(l)}, \ldots, b_k^{(l)}$, and

$$a_i^{(l)} := \max_{j \in [k]} \big(W_j^{(l)} a^{(l-1)} + b_j^{(l)}\big)_i. \tag{2.7}$$

Observation 2.2 (ReLU and max-pooling as Maxout, `Lean_5`*).* The ReLU activation is the rank-2 Maxout unit with parameters $(w_1, \beta_1) = (w, \beta)$ and $(w_2, \beta_2) = (0, 0)$, since $\mathrm{ReLU}(w^\top x + \beta) = \max\{w^\top x + \beta, 0\}$. Max-pooling over a group J of neurons is the rank-$|J|$ Maxout unit whose j-th argument is the coordinate projection onto the j-th element of J. Every rank-k Maxout unit is convex in its input and non-decreasing in each pre-activation argument $w_j^\top x + \beta_j$; hence Maxout qualifies as an admissible activation in Definitions 2.4 and 2.5.

Proof. The first two claims are immediate from (2.6). Convexity holds because a pointwise maximum of affine (hence convex) functions is convex. For monotonicity in the arguments, view the unit as $(u_1, \dots, u_k) \mapsto \max_j u_j$ with $u_j = w_j^\top x + \beta_j$: if $u_j \le u'_j$ for all j, then $\max_j u_j \le \max_j u'_j$. □

Remark 2.4. Maxout networks are the natural piecewise-linear generalization of ReLU: a rank-k unit expresses a maximum of k affine functions in one neuron, matching the building blocks of the DC representation (2.4). The DC splitting of Chapter 3 is developed for ReLU networks and extended to Maxout units—and to the multiplicative interactions needed for attention—in Chapter 5, using the exchange identity $\max_{j\in[k]}(u_j - v_j) = \max_{j\in[k]}\{u_j + \sum_{i\neq j} v_i\} - \sum_{i\in[k]} v_i$ for pairs of non-negative streams.

2.3 Game Theory and Markov Decision Processes

Chapter 4 interprets the backward pass of a decomposed network as a two-player zero-sum game on the network graph. This section fixes the background: MDPs, turn-based zero-sum games, and their optimality equations. We restrict to finite state and action spaces, which suffices throughout.

Definition 2.7 (Markov Decision Process). A (finite) ***M**arkov **D**ecision **P**rocess* (MDP) (Puterman, 1994; Sutton and Barto, 2018) is a tuple $\mathscr{M} = (S, A, \mathbb{P}, R, \gamma)$ consisting of a finite state set S, a finite action set $A(s)$ for each $s \in S$, a *transition kernel* $\mathbb{P}(s' \mid s, a) \ge 0$ with $\sum_{s'} \mathbb{P}(s' \mid s, a) = 1$, a bounded *reward function* $R(s, a) \in \mathbb{R}$, and a *discount factor* $\gamma \in [0, 1)$. A *(stationary) policy* is a map π assigning to each state s a probability distribution $\pi(\cdot \mid s)$ over $A(s)$; π is *pure* (deterministic) if each $\pi(\cdot \mid s)$ is a point mass. A policy induces, for every start state, a Markov chain on S and hence a distribution over trajectories $(s_0, a_0, s_1, a_1, \dots)$; the *value function* of π is the expected discounted return

$$V^\pi(s) := \mathbb{E}^\pi\Big[\textstyle\sum_{t=0}^{\infty} \gamma^t\, R(s_t, a_t) \,\Big|\, s_0 = s\Big], \tag{2.8}$$

and the *optimal value function* is $V^\star(s) := \max_\pi V^\pi(s)$.

Fixing an enumeration of S, a pure policy π turns the kernel into a row-stochastic *transition matrix* $P^\pi \in \mathbb{R}^{S\times S}$ with $P^\pi_{ss'} = \mathbb{P}(s' \mid s, \pi(s))$ and the reward into a vector R^π, and (2.8) becomes the Neumann series $V^\pi = \sum_{t\ge 0} \gamma^t (P^\pi)^t R^\pi$. This matrix picture recurs in Chapter 4, where the (sub)stochastic matrices are built from the normalized weight parts $W^{(l,\sigma)}$ of Definition 2.2.

Proposition 2.4 (Bellman equations, `Lean_6`**).** *Let $\mathscr{M}$ be a finite MDP with $\gamma \in [0,1)$. Then:*

1) V^π is the unique solution of the linear system

$$V(s) = \sum_{a\in A(s)} \pi(a \mid s)\Big[R(s,a) + \gamma \sum_{s'} \mathbb{P}(s' \mid s, a)\, V(s')\Big], \qquad s \in S. \tag{2.9}$$

2) $V^\star$ is the unique solution of the Bellman optimality equation

$$V(s) = \max_{a \in A(s)} \Big\{ R(s,a) + \gamma \sum_{s'} \mathbb{P}(s' \mid s,a)\, V(s') \Big\}, \qquad s \in S, \tag{2.10}$$

and any pure policy $\pi^\star$ that selects, in every state, an action attaining the maximum in (2.10) is optimal: $V^{\pi^\star} = V^\star$.

Proof. Both claims are standard (Puterman, 1994; Sutton and Barto, 2018). The right-hand sides of (2.9) and (2.10) define γ-contractions on $\big(\mathbb{R}^S, \|\cdot\|_\infty\big)$, so each has a unique fixed point by the Banach fixed-point theorem; conditioning (2.8) on the first transition identifies these fixed points as V^π and $V^\star$, and any policy greedy for $V^\star$ is optimal. The acyclic games below make even the contraction argument unnecessary (Remark 2.5). □

Remark 2.5 (State-dependent structural discounts). The games of this thesis depart from Definition 2.7 in one respect: the discount is *state-dependent*, $\gamma(s) \geq 0$, and may exceed 1. Here discounts are not impatience parameters but *structural* normalizers—for neuron i in layer l the absolute weight mass

$$\gamma_i^{(l)} := \sum_j \big(W^{(l,+)} + W^{(l,-)}\big)_{ij} = \sum_j |W_{ij}^{(l)}|, \tag{2.11}$$

which restores the weight mass after row i of $|W^{(l)}|$ is normalized into transition probabilities. Contraction is then unavailable, but also unnecessary: all games are played on *layered acyclic* graphs, so every trajectory is finite and all value functions are well-defined finite sums. Backward induction gives a unique value and the existence of optimal policies; policies need not be unique when actions tie. The two-player statement follows below.

Zero-sum turn-based games

Two-player games enter because a decomposed network carries two interacting streams: a player P^+ who collects positive evidence and a player P^- who collects negative evidence. We use the following conventions throughout: $p, q, r \in \{+,-\}$ denote player, turn, or sign labels; p' denotes the opponent ($+' = -$, $-' = +$); the *sign-XOR* $p \oplus q$ is $+$ if $p = q$ and $-$ otherwise; and $\xi_p \in \{+1, -1\}$ is the numerical sign of the label p, so that $\xi_p\, \xi_q = \xi_{p \oplus q}$.

Definition 2.8 (Zero-sum turn-based game). A (finite) *zero-sum turn-based game* is a tuple $\mathscr{G} = (S_+, S_-, S_{\text{ter}}, A, \mathbb{P}, R, \gamma)$ where the state set $S = S_+ \uplus S_- \uplus S_{\text{ter}}$ is partitioned into states controlled by player P^+, states controlled by player P^-, and terminal states; the action sets $A(s)$, the transition kernel $\mathbb{P}$, and the reward R are as in Definition 2.7 on non-terminal states; $\gamma \colon S \to \mathbb{R}_{\geq 0}$ is a (possibly state-dependent) discount; and each terminal state $s \in S_{\text{ter}}$ carries a fixed one-time payoff $R(s)$. A *policy pair* $\pi = (\pi_+, \pi_-)$ consists of a policy π_p for player P^p on S_p. Play generates a trajectory as in an MDP, with the player in turn choosing the action; the expected total payoff $V^\pi(s)$, defined as in (2.8) with the discount accumulated multiplicatively along the trajectory (γ^t replaced by $\prod_{m=0}^{t-1} \gamma(s_m)$), is paid *by P^- to P^+*: player P^+ maximizes V^π and player P^- minimizes it.

Theorem 2.5 (Shapley–Bellman optimality on layered games, `Lean_7`). *Let $\mathscr{G}$ be a finite zero-sum turn-based game whose transition graph is acyclic. Then $\mathscr{G}$ has a* value*: there is a unique*

function $V^\star\colon S \to \mathbb{R}$ *satisfying the* Shapley–Bellman optimality equations

$$V^\star(s) = \begin{cases} \max\limits_{a \in A(s)} \Big\{ R(s,a) + \gamma(s) \sum\limits_{s'} \mathbb{P}(s' \mid s,a)\, V^\star(s') \Big\}, & s \in S_+, \\ \min\limits_{a \in A(s)} \Big\{ R(s,a) + \gamma(s) \sum\limits_{s'} \mathbb{P}(s' \mid s,a)\, V^\star(s') \Big\}, & s \in S_-, \\ R(s), & s \in S_{\mathrm{ter}}, \end{cases} \tag{2.12}$$

and there exist pure stationary policies $\pi^\star = (\pi_+^\star, \pi_-^\star)$*, obtained by selecting maximizing respectively minimizing actions in* (2.12)*, such that for all* $s \in S$ *and all policies* π_+, π_-*,*

$$V^{(\pi_+^\star, \pi_-)}(s) \;\geq\; V^{\pi^\star}(s) = V^\star(s) \;\geq\; V^{(\pi_+, \pi_-^\star)}(s). \tag{2.13}$$

In particular, $\pi^\star$ *is a Nash equilibrium of every subgame, i.e. a subgame perfect equilibrium.*

Proof. Since the graph is finite and acyclic, its states admit a topological order; we induct backward along it. For terminal states, (2.12) fixes $V^\star(s) = R(s)$ and (2.13) is trivial. Consider a non-terminal state $s \in S_+$ and assume $V^\star$ is uniquely defined and (2.13) holds for all states reachable from s. The bracket in (2.12) is a well-defined finite number for every action, so the maximum exists and is attained by some pure choice $\pi_+^\star(s)$; this defines $V^\star(s)$ uniquely. For the saddle-point property, fix any opponent policy π_- and let P^+ play $\pi_+^\star$. Conditioning on the first transition from s and using the induction hypothesis (the left inequality of (2.13) at the successor states) together with $\gamma(s) \geq 0$,

$$\begin{aligned} V^{(\pi_+^\star, \pi_-)}(s) &= R\big(s, \pi_+^\star(s)\big) + \gamma(s) \sum_{s'} \mathbb{P}\big(s' \mid s, \pi_+^\star(s)\big)\, V^{(\pi_+^\star, \pi_-)}(s') \\ &\geq R\big(s, \pi_+^\star(s)\big) + \gamma(s) \sum_{s'} \mathbb{P}\big(s' \mid s, \pi_+^\star(s)\big)\, V^\star(s') \;=\; V^\star(s), \end{aligned}$$

where the last equality holds because $\pi_+^\star(s)$ attains the maximum in (2.12). Similarly, if P^+ plays an arbitrary π_+ against $\pi_-^\star$, then averaging the bracket over $\pi_+(\cdot \mid s)$ is at most its maximum, so the same conditioning together with the right inequality at the successors yields $V^{(\pi_+, \pi_-^\star)}(s) \leq V^\star(s)$. The argument for $s \in S_-$ is symmetric, with max and min and the inequality directions exchanged. Evaluating both inequalities of (2.13) at $\pi = \pi^\star$ gives $V^{\pi^\star} = V^\star$. Since the argument applies verbatim to the subgame rooted at any state, $\pi^\star$ is subgame perfect. □

Remark 2.6. For general discounted stochastic games with cycles, existence of the value and of optimal stationary policies is Shapley's theorem (Shapley, 1953), proved by the contraction argument of Proposition 2.4 applied to the *Shapley operator*—the min-max analogue of the Bellman operator (2.12). Gaubert & Vlassopoulos (2025) formulate their network games in this operator language. The layered acyclic case of Theorem 2.5 is all we need, and it additionally delivers subgame perfection, on which Chapter 8 relies.

2.3.1 The ReLU Net Game of Gaubert and Vlassopoulos

Gaubert & Vlassopoulos (2025) showed that the forward pass of a ReLU network *is* the value of a zero-sum turn-based stopping game played backward on the network graph. We present their construction in full, in our notation, since Chapter 4 inherits its state space, transition kernel, and stopping structure while changing the payoff bookkeeping. Throughout, fix a network as in Definition 2.1 with scalar

output ($d^{(L)} = 1$), fix an input $x \in \mathbb{R}^{d^{(0)}}$, and assume that no row of any weight matrix vanishes, so that the discounts (2.11) are strictly positive.

Read the identity $a_i^{(l)} = \max\{0, z_i^{(l)}\}$ as a decision: a player at neuron i may *stop*, securing payoff 0, or *continue*, collecting the bias $b_i^{(l)}$ plus the weighted contributions of the previous layer. Continuing propagates the decision backward through the network; a negative weight flips the sign of its contribution, i.e. hands the turn to an adversary facing the mirrored problem. The optimal value at neuron i is exactly the activation $a_i^{(l)}$.

Definition 2.9 (ReLU Net Game, following (Gaubert and Vlassopoulos, 2025)). The *ReLU Net Game* on the network f at input x is the zero-sum turn-based game with the following components.

- **States.** For every hidden layer $l \in \{1, \dots, L-1\}$, every neuron $i \in [d^{(l)}]$, and every turn label $q \in \{+,-\}$ there is an *activation state* $s_{i,q}^{(l)}$, controlled by player P^q, so that $S_q = \{s_{i,q}^{(l)} : l \in [L-1], i \in [d^{(l)}]\}$. For every input coordinate $k \in [d^{(0)}]$ and every $q \in \{+,-\}$ there is a *terminal state* $s_{k,q}^{(0)}$, and there is one absorbing *cemetery state* $\perp$ with zero payoff. Finally, there is a single *output state* $s^{(L)}$ at the affine output neuron, at which no decision is taken.
- **Actions.** At every activation state the player in turn chooses an action from $\{\text{stop}, \text{cont}\}$. The output state has the single action cont.
- **Transitions.** Stopping moves deterministically to the cemetery, $\mathbb{P}(\perp \mid s_{i,q}^{(l)}, \text{stop}) = 1$. Continuation at $s_{i,q}^{(l)}$ moves one layer backward, to neuron j of layer $l-1$ with probability proportional to the absolute weight, keeping the turn on positive weights and passing it to the opponent on negative weights:

$$\mathbb{P}\big(s_{j,r}^{(l-1)} \mid s_{i,q}^{(l)}, \text{cont}\big) := \frac{W_{ij}^{(l,q\oplus r)}}{\gamma_i^{(l)}}, \qquad \gamma_i^{(l)} = \sum_j \big(W^{(l,+)} + W^{(l,-)}\big)_{ij}, \tag{2.14}$$

  where for $l = 1$ the successor $s_{j,r}^{(0)}$ is the corresponding terminal state. The output state transitions by the same rule (2.14) with $l = L$ and turn label $q = +$.
- **Payoffs and discounts.** Continuation at $s_{i,q}^{(l)}$ pays the signed bias $\xi_q\, b_i^{(l)}$; stopping pays 0. A terminal state pays the signed input coordinate once, $R\big(s_{k,q}^{(0)}\big) := \xi_q\, x_k$, and the game ends. The output state pays $b^{(L)}$ upon its forced continuation. The discount at every activation state and at the output state is the structural discount $\gamma\big(s_{i,q}^{(l)}\big) := \gamma_i^{(l)}$ of (2.11).

Player P^+ maximizes and P^- minimizes the expected discounted total payoff, as in Definition 2.8.

The game graph is layered and acyclic—every transition strictly decreases the layer index or ends in the cemetery—so Theorem 2.5 applies despite the discounts $\gamma_i^{(l)}$ being arbitrary positive numbers. Normalization and discounting cancel: the product of the transition probability (2.14) and the discount $\gamma_i^{(l)}$ is exactly the absolute weight $|W_{ij}^{(l)}|$, so the Shapley–Bellman recursion reproduces the affine algebra of the forward pass.

Theorem 2.6 (Forward pass as game value, (Gaubert and Vlassopoulos, 2025), `Lean_8`). *The ReLU Net Game of Definition 2.9 admits the subgame perfect equilibrium $\pi^\star = (\pi_+^\star, \pi_-^\star)$ in which both players use the same pure stopping rule,*

$$\pi_q^\star\big(s_{i,q}^{(l)}\big) = \text{cont} \quad \Longleftrightarrow \quad z_i^{(l)} > 0, \tag{2.15}$$

and the value function of the game evaluates the network:

$$V^\star\big(s_{i,q}^{(l)}\big) = \xi_q\, a_i^{(l)} \quad \text{for } l \in \{0,1,\dots,L-1\}, \qquad V^\star\big(s^{(L)}\big) = f(x), \tag{2.16}$$

where for $l = 0$ we read $a_k^{(0)} = x_k$. The equilibrium is unique up to tie-breaking at neurons with $z_i^{(l)} = 0$, where both actions are optimal; the convention (2.15) stops at ties, matching $ReLU'(0) = 0$.

Proof. By Theorem 2.5 it suffices to verify that the function defined by (2.16) satisfies the Shapley–Bellman equations (2.12), with the maximizers and minimizers given by (2.15). We induct on the layer index.

Base case $l = 0$. Terminal states carry the fixed payoff $R\big(s_{k,q}^{(0)}\big) = \xi_q\, x_k = \xi_q\, a_k^{(0)}$, which is (2.16).

Inductive step. Let $l \geq 1$ and assume (2.16) at layer $l-1$. Consider first a state $s_{i,+}^{(l)} \in S_+$. The stop action yields $0 + V^\star(\bot) = 0$. The continuation value is, by (2.14) and the induction hypothesis,

$$\begin{aligned} R\big(s_{i,+}^{(l)}, \mathrm{cont}\big) + \gamma_i^{(l)} \sum_{j,r} \frac{W_{ij}^{(l,+\oplus r)}}{\gamma_i^{(l)}}\, \xi_r\, a_j^{(l-1)} &= b_i^{(l)} + \sum_j \Big(W_{ij}^{(l,+)}\, a_j^{(l-1)} - W_{ij}^{(l,-)}\, a_j^{(l-1)} \Big) \\ &= b_i^{(l)} + \sum_j W_{ij}^{(l)}\, a_j^{(l-1)} \;=\; z_i^{(l)}, \end{aligned} \tag{2.17}$$

using $+ \oplus r = r$, $\xi_+ = 1$, $\xi_- = -1$, and $W^{(l)} = W^{(l,+)} - W^{(l,-)}$. The maximizing player P^+ therefore obtains

$$V^\star\big(s_{i,+}^{(l)}\big) = \max\big\{0, z_i^{(l)}\big\} = \mathrm{ReLU}\big(z_i^{(l)}\big) = a_i^{(l)} = \xi_+\, a_i^{(l)},$$

continuing exactly when $z_i^{(l)} > 0$. At the mirrored state $s_{i,-}^{(l)} \in S_-$ the continuation payoff is $\xi_- b_i^{(l)} = -b_i^{(l)}$, and the kernel labels satisfy $- \oplus r = r'$, so the computation (2.17) returns the continuation value

$$-b_i^{(l)} + \sum_j \Big(W_{ij}^{(l,-)}\, a_j^{(l-1)} - W_{ij}^{(l,+)}\, a_j^{(l-1)} \Big) = -z_i^{(l)}.$$

The minimizing player P^- obtains $V^\star\big(s_{i,-}^{(l)}\big) = \min\{0, -z_i^{(l)}\} = -\mathrm{ReLU}\big(z_i^{(l)}\big) = \xi_-\, a_i^{(l)}$ and prefers to continue exactly when $-z_i^{(l)} < 0$, i.e. under the same gate (2.15). At $z_i^{(l)} = 0$ both actions give value 0 for either player, which proves the uniqueness-up-to-ties claim. Finally, the output state has the single action cont, and the computation (2.17) with $l = L$ yields $V^\star\big(s^{(L)}\big) = z^{(L)} = a^{(L)} = f(x)$. □

Remark 2.7 (Softplus networks from entropy regularization). Gaubert & Vlassopoulos (2025) also treat softplus networks, by *regularizing* the stop/continue decision. Replace the hard choice at an activation state by a mixed policy $\nu \in \Delta(\{\mathrm{stop}, \mathrm{cont}\})$ rewarded with a Shannon-entropy bonus at temperature $\theta > 0$. By the Legendre–Fenchel identity for the entropy (Boyd and Vandenberghe, 2004),

$$\max_{\nu \in \Delta_n} \Big\{ \textstyle\sum_{i=1}^n \nu_i r_i + \theta\, \mathscr{H}(\nu) \Big\} = \theta \log \textstyle\sum_{i=1}^n e^{r_i/\theta}, \qquad \mathscr{H}(\nu) = -\textstyle\sum_i \nu_i \log \nu_i, \tag{2.18}$$

with the Gibbs distribution $\nu_i^\star \propto e^{r_i/\theta}$ as unique maximizer, the regularized value at a state with continuation value $z_i^{(l)}$ (computed as in (2.17)) becomes $\theta \log\big(1 + e^{z_i^{(l)}/\theta}\big)$—the softplus activation at temperature θ—and the equilibrium policy continues with the sigmoid probability $\big(1 + e^{-z_i^{(l)}/\theta}\big)^{-1}$. The hard game of Definition 2.9 is recovered in the limit $\theta \to 0$. Chapter 4 re-derives this mechanism

from the regularized decompositions of Section 3.5 and reuses it at the routing states of the RG, with the two temperatures θ (activation gates) and τ (routing) kept distinct.

Remark 2.8 (What Chapter 4 inherits and what it changes). The SG of Chapter 4 keeps the structural skeleton of Definition 2.9: per-neuron $\pm$ states, stop-versus-continue actions, the bias as continuation payoff and the input as final payoff, the transition kernel (2.14) driven by the weight parts $W^{(l,\sigma)}$, the cemetery state, and the structural discounts $\gamma_i^{(l)}$. Two changes are load-bearing for attribution. First, the original tracks only the single signed value (2.16); Chapter 4 additionally maintains a *non-negative payoff per player*, identifying the pair of player-specific values $V_p(s)$, $p \in \{+,-\}$, with the two streams of the Stopping Decomposition of Section 3.4 and recovering (2.16) as their difference. Second, the signed terminal payoffs $\xi_q\, x_k$ become the non-negative pair (x_k^+, x_k^-) of Definition 2.2, keeping every player-specific quantity non-negative and exposing the parity structure of backward trajectories. Chapter 4 further extends the graph to skip connections and max-pooling and adds risk aversion and regularized policies, none of which appear in the original game.

2.4 Tropical Geometry and Polyhedra

Tropical geometry replaces $(+, \times)$ by $(\max, +)$, turning polynomials into convex CPWL functions and polynomial multiplication into Minkowski addition of polytopes. This is the algebra governing the convex streams g and h of the DC decomposition, which Sections 2.4.1–2.4.3 exploit to read learned structure off the associated polytopes. We collect the definitions and calculus rules used there.

Definition 2.10 (Max-plus and min-plus semirings). The *max-plus semiring* is the set $\mathbb{R}_{\max} := \mathbb{R} \cup \{-\infty\}$ equipped with *tropical addition* and *tropical multiplication*,

$$u \oplus v := \max\{u, v\}, \qquad u \odot v := u + v, \tag{2.19}$$

with additive neutral element $-\infty$ and multiplicative neutral element 0. Both operations are associative and commutative, and $\odot$ distributes over $\oplus$; additive inverses do not exist, so $\mathbb{R}_{\max}$ is a semiring but not a ring. The *min-plus semiring* $\mathbb{R}_{\min} := \mathbb{R} \cup \{+\infty\}$ is defined dually with $u \oplus' v := \min\{u,v\}$; the map $u \mapsto -u$ is an isomorphism between the two, so every statement below has a min-plus counterpart.

For example, $2 \oplus 3 = 3$ and $2 \odot 3 = 5$, and the tropical square is $u^{\odot 2} = u \odot u = 2u$. More generally, tropical exponentiation is scalar multiplication, $u^{\odot \alpha} = \alpha u$, which is meaningful for any real exponent α. (The symbol $\oplus$ doubles as the sign-XOR of Section 2.3; the operands—real numbers here, sign labels there—always disambiguate.)

Definition 2.11 (Tropical polynomials and rational functions, `Lean_9`). A *tropical monomial* in d variables with exponent $\alpha \in \mathbb{R}^d$ and coefficient $c \in \mathbb{R}$ is the affine function $c \odot x_1^{\odot \alpha_1} \odot \cdots \odot x_d^{\odot \alpha_d} = c + \langle \alpha, x \rangle$. A *tropical polynomial* with finite support $\mathscr{A} \subset \mathbb{R}^d$ and coefficients $(c_\alpha)_{\alpha \in \mathscr{A}}$, $c_\alpha \in \mathbb{R}$, is the tropical sum of monomials,

$$p(x) \;=\; \bigoplus_{\alpha \in \mathscr{A}} c_\alpha \odot x^{\odot \alpha} \;=\; \max_{\alpha \in \mathscr{A}} \big(c_\alpha + \langle \alpha, x \rangle \big). \tag{2.20}$$

A *tropical rational function* is a tropical quotient of two tropical polynomials, $p \oslash q := p - q$. In classical tropical geometry (Maclagan and Sturmfels, 2015) the exponents are restricted to $\mathscr{A} \subset \mathbb{N}^d$ or $\mathbb{Z}^d$; for neural networks, real exponents (network weights) are the natural setting, and all statements below hold verbatim in this generality.

A one-dimensional example: $p(x) = 0 \oplus (0 \odot x) \oplus \big((-1) \odot x^{\odot 2}\big) = \max\{0, x, 2x-1\}$, a convex piecewise-linear function with breakpoints at $x = 0$ and $x = 1$.

Observation 2.3 (Tropical polynomials are the convex CPWL functions, `Lean_10`*).* A function $p\colon \mathbb{R}^d \to \mathbb{R}$ is a tropical polynomial (2.20) if and only if it is convex and CPWL.

Proof. A tropical polynomial is a finite maximum of affine functions, hence convex and CPWL. Conversely, let p be convex and CPWL with affine pieces $\ell_1, \dots, \ell_N$ on the linear regions $Q_1, \dots, Q_N$ of Definition 2.3. We claim $p = \max_r \ell_r$. Pointwise $p \le \max_r \ell_r$, since every x lies in some region Q_r where $p(x) = \ell_r(x)$. For the reverse inequality, fix r and a point x_0 in the interior of the full-dimensional region Q_r. Then ℓ_r agrees with p on a neighborhood of x_0, and it is a supporting affine minorant of the convex function p at x_0: for any x and sufficiently small $t \in (0,1)$, convexity gives $p\big(x_0 + t(x - x_0)\big) \le (1-t)\,p(x_0) + t\,p(x)$, while the left-hand side equals $\ell_r\big(x_0 + t(x-x_0)\big) = (1-t)\,\ell_r(x_0) + t\,\ell_r(x)$; comparing and using $p(x_0) = \ell_r(x_0)$ yields $\ell_r(x) \le p(x)$. Hence $\max_r \ell_r \le p$, and equality holds. □

Combining Observation 2.3 with the DC representation (2.4) and Theorem 2.1 identifies ReLU network functions with tropical rational functions; this is the theorem of Zhang et al. (2018b), restated and connected to the split streams g and h in Section 2.4.1. The expressivity of tropical polynomials as computed by networks is studied by Misiakos et al. (2022) and, in the real-exponent setting, by Brandenburg et al. (2024).

Newton polytopes, Minkowski sums, and support functions

The combinatorics of a tropical polynomial is captured by a polytope—the convex hull of its exponents. We refer to Ziegler (1995) for general polytope theory and write conv for the convex hull.

Definition 2.12 (Newton polytope, `Lean_11`**).** The *Newton polytope* of a tropical polynomial p with support $\mathscr{A}$ as in (2.20) is the convex hull of its exponent vectors,

$$\operatorname{Newt}(p) := \operatorname{conv}\big\{\alpha : \alpha \in \mathscr{A}\big\} \subset \mathbb{R}^d. \tag{2.21}$$

The *lifted Newton polytope* additionally records the coefficients, $\widehat{\operatorname{Newt}}(p) := \operatorname{conv}\big\{(\alpha, c_\alpha) : \alpha \in \mathscr{A}\big\} \subset \mathbb{R}^{d+1}$.

For the example $p(x) = \max\{0, x, 2x-1\}$ above, $\mathscr{A} = \{0, 1, 2\}$ and $\operatorname{Newt}(p) = [0, 2] \subset \mathbb{R}$, while $\widehat{\operatorname{Newt}}(p) = \operatorname{conv}\{(0,0), (1,0), (2,-1)\}$ is a triangle in $\mathbb{R}^2$.

Definition 2.13 (Minkowski sum). The *Minkowski sum* of two sets $P, Q \subseteq \mathbb{R}^d$ is $P + Q := \{u + v : u \in P, v \in Q\}$. For polytopes P and Q the Minkowski sum is again a polytope, and every vertex of $P + Q$ decomposes as the sum of a vertex of P and a vertex of Q (Ziegler, 1995).

For example, the Minkowski sum of two segments in general position is a parallelogram, and the Minkowski sum of k segments is a *zonotope*—the polytope class that Zhang et al. (2018b) showed to govern one-hidden-layer ReLU networks. The analytic tool linking polytopes to convex functions is the support function.

Definition 2.14 (Support function, `Lean_12`**).** The *support function* of a nonempty compact convex set $P \subset \mathbb{R}^d$ is

$$h_P\colon \mathbb{R}^d \to \mathbb{R}, \qquad h_P(x) := \max_{u \in P} \langle u, x \rangle. \tag{2.22}$$

Lemma 2.7 (Properties of support functions, Lean_13). *Let $P, Q \subset \mathbb{R}^d$ be nonempty compact convex sets. Then:*

1) *h_P is convex and positively homogeneous, i.e. $h_P(\lambda x) = \lambda h_P(x)$ for $\lambda \geq 0$; if P is a polytope with vertex set $\operatorname{vert}(P)$, then $h_P(x) = \max_{u \in \operatorname{vert}(P)} \langle u, x \rangle$ is a tropical polynomial with zero coefficients and Newton polytope P.*
2) *For any nonempty finite set $A \subset \mathbb{R}^d$ and $x \in \mathbb{R}^d$, $\max_{u \in \operatorname{conv}(A)} \langle u, x \rangle = \max_{u \in A} \langle u, x \rangle$; in particular $h_{P+Q} = h_P + h_Q$ and $h_{\operatorname{conv}(P \cup Q)} = \max\{h_P, h_Q\} = h_P \oplus h_Q$.*
3) *$P \subseteq Q$ if and only if $h_P \leq h_Q$ pointwise; in particular $h_P = h_Q$ implies $P = Q$, so a compact convex set is uniquely determined by its support function.*

Proof. 1: As a pointwise maximum of the linear functions $x \mapsto \langle u, x \rangle$, $u \in P$, the support function is convex, and homogeneity is clear from (2.22). For a polytope, the maximum of the linear objective $\langle \cdot\,, x \rangle$ over P is attained at a vertex, giving the finite-max representation; its exponent set is $\operatorname{vert}(P)$, whose convex hull is P.

2: Maximizing a linear function over a convex hull adds nothing: any convex combination satisfies $\big\langle \sum_i \lambda_i u_i, x \big\rangle = \sum_i \lambda_i \langle u_i, x \rangle \leq \max_i \langle u_i, x \rangle$, and $A \subseteq \operatorname{conv}(A)$ gives the reverse bound. For the Minkowski sum, $\max_{w \in P+Q} \langle w, x \rangle = \max_{u \in P,\, v \in Q} \big(\langle u, x \rangle + \langle v, x \rangle \big) = h_P(x) + h_Q(x)$, since the two maximizations decouple. For the union, the objective attains the same maximum over $\operatorname{conv}(P \cup Q)$ as over $P \cup Q$ by the first claim (applied with suprema over compact sets in place of finite maxima, with the same one-line proof), and $\max_{u \in P \cup Q} \langle u, x \rangle = \max\{h_P(x), h_Q(x)\}$.

3: If $P \subseteq Q$, the maximum in (2.22) over the larger set is at least as big, so $h_P \leq h_Q$. Conversely, suppose $u_0 \in P \setminus Q$. By the separating-hyperplane theorem for a point and a compact convex set (Boyd and Vandenberghe, 2004), there is $x \in \mathbb{R}^d$ with $\langle u_0, x \rangle > \max_{v \in Q} \langle v, x \rangle = h_Q(x)$, whence $h_P(x) \geq \langle u_0, x \rangle > h_Q(x)$, contradicting $h_P \leq h_Q$. The uniqueness statement follows by applying the equivalence in both directions. □

Support functions translate the tropical operations on polynomials into polytope operations on their Newton polytopes.

Proposition 2.8 (Newton polytope calculus, cf. (Zhang et al., 2018b), Lean_14). *Let p and q be tropical polynomials on $\mathbb{R}^d$. Then*

$$\operatorname{Newt}(p \odot q) = \operatorname{Newt}(p) + \operatorname{Newt}(q), \qquad \operatorname{Newt}(p \oplus q) = \operatorname{conv}\big(\operatorname{Newt}(p) \cup \operatorname{Newt}(q)\big). \tag{2.23}$$

Moreover, the lifted Newton polytope determines the polynomial function:

$$p(x) = h_{\widehat{\operatorname{Newt}(p)}}(x, 1) \qquad \text{for all } x \in \mathbb{R}^d, \tag{2.24}$$

and if all coefficients vanish, $c_\alpha \equiv 0$, then $p = h_{\operatorname{Newt}(p)}$.

Proof. We first prove (2.24). By Lemma 2.7.2 applied to the finite point set $\{(\alpha, c_\alpha) : \alpha \in \mathscr{A}\}$, maximizing the linear functional $\langle \cdot\,, (x,1) \rangle$ over its convex hull $\widehat{\operatorname{Newt}}(p)$ equals maximizing it over the points themselves:

$$h_{\widehat{\operatorname{Newt}(p)}}(x, 1) = \max_{\alpha \in \mathscr{A}} \big\langle (\alpha, c_\alpha), (x, 1) \big\rangle = \max_{\alpha \in \mathscr{A}} \big(\langle \alpha, x \rangle + c_\alpha \big) = p(x),$$

and the zero-coefficient case is the same computation one dimension lower.

For the tropical sum $p \oplus q = \max\{p, q\}$: its support is the union $\mathscr{A}_p \cup \mathscr{A}_q$ (an exponent occurring in both summands keeps the larger coefficient, which does not change the support), so

$$\mathrm{Newt}(p \oplus q) = \mathrm{conv}(\mathscr{A}_p \cup \mathscr{A}_q) = \mathrm{conv}\big(\mathrm{conv}(\mathscr{A}_p) \cup \mathrm{conv}(\mathscr{A}_q)\big) = \mathrm{conv}\big(\mathrm{Newt}(p) \cup \mathrm{Newt}(q)\big),$$

where the middle equality holds because both sides are the smallest convex set containing $\mathscr{A}_p \cup \mathscr{A}_q$.

For the tropical product $p \odot q = p + q$: expanding the product of (2.20) termwise shows that the support of $p \odot q$ is contained in the pointwise sum $\mathscr{A}_p + \mathscr{A}_q$ (tropical addition of coefficients on coinciding exponent sums never removes an exponent, in contrast to classical cancellation), so

$$\mathrm{Newt}(p \odot q) \subseteq \mathrm{conv}(\mathscr{A}_p + \mathscr{A}_q) = \mathrm{conv}(\mathscr{A}_p) + \mathrm{conv}(\mathscr{A}_q) = \mathrm{Newt}(p) + \mathrm{Newt}(q),$$

using the identity $\mathrm{conv}(A + B) = \mathrm{conv}(A) + \mathrm{conv}(B)$: the right-hand side is a convex set containing $A + B$, and conversely any sum of convex combinations $\sum_i \lambda_i u_i + \sum_j \mu_j v_j = \sum_{i,j} \lambda_i \mu_j (u_i + v_j)$ is a convex combination of points of $A + B$. For the reverse inclusion, take a vertex w of $\mathrm{Newt}(p) + \mathrm{Newt}(q)$ and a linear functional $\langle \cdot\,, x^* \rangle$ uniquely maximized over the sum at w; by Definition 2.13 we may write $w = u + v$ with $u \in \mathscr{A}_p$ maximizing $\langle \cdot\,, x^* \rangle$ over $\mathrm{Newt}(p)$ and $v \in \mathscr{A}_q$ maximizing it over $\mathrm{Newt}(q)$ (a vertex of a Newton polytope belongs to the support). Then $w = u + v$ lies in the support of $p \odot q$, so every vertex of $\mathrm{Newt}(p) + \mathrm{Newt}(q)$ lies in $\mathrm{Newt}(p \odot q)$, and taking convex hulls gives the reverse inclusion. □

Remark 2.9 (Layer transitions as polytope operations). Proposition 2.8 is the dictionary used in the remaining subsections: an affine layer with non-negative weights acts on the tropical polynomials computed by the previous layer through tropical products and powers—Minkowski sums and scalings of Newton polytopes—while ReLU and Maxout gates act through tropical sums—convex hulls of unions. The convex streams g and h of the DC decomposition—developed in full in Chapter 3—are, by Observation 2.3, tropical polynomials, so their (lifted) Newton polytopes are computable layer by layer. Section 2.4.3 studies these polytopes: by Lemma 2.7.3 they encode the split streams faithfully, and their vertices are exactly the attainable gradients of g and h—the geometric objects behind the *imaginative polytopes* of this thesis. For the broader landscape connecting polyhedral and tropical geometry with network expressivity, see (Brandenburg et al., 2024; Huchette et al., 2025; Zhang et al., 2018b).

2.4.1 Tropical Rational Maps and the DC Split

The tropical dictionary above reads the DC decomposition geometrically. Zhang et al. (2018b) showed that feedforward ReLU networks with real weights are exactly the *tropical rational maps*—componentwise differences of max-plus polynomials—so that counting linear regions becomes counting polytope vertices; the converse, that every CPWL function arises this way, is Mukherjee et al. (2018).[1] We use this correspondence in the opposite direction: rather than bounding a worst-case network, we take a *trained* network, compute its split streams g, h explicitly—the construction developed in full in Chapter 3—and read the vertices of their Newton polytopes as learned prototypes for interpreting trained networks (Zimmermann and Loho, 2026).

This subsection anticipates the difference-of-convex split developed in full in Chapter 3, presenting its geometry here because it belongs with the tropical and polyhedral foundations of this chapter. The

[1] Related max-affine perspectives include Maxout networks (Goodfellow et al., 2013) and the expressivity program on Newton polytopes (Brandenburg et al., 2024; Haase et al., 2023; Hertrich and Loho, 2025; Huchette et al., 2025; Misiakos et al., 2022).

split forward pass—formalized in Chapter 3—acts on a bias-free network as follows: the input $x \in \mathbb{R}^d$ is replaced by a pair $(a^{(0,+)}, a^{(0,-)})$ with $x = a^{(0,+)} - a^{(0,-)}$, the weights are split as $W^{(l)} = W^{(l,+)} - W^{(l,-)}$ with $W^{(l,\pm)} \geq 0$, and the hidden layers $1 \leq l < L$ iterate entrywise

$$z^{(l,+)} = W^{(l,+)}a^{(l-1,+)} + W^{(l,-)}a^{(l-1,-)}, \qquad z^{(l,-)} = W^{(l,-)}a^{(l-1,+)} + W^{(l,+)}a^{(l-1,-)}, \tag{2.25}$$

$$a^{(l,+)} = \max\{z^{(l,+)}, z^{(l,-)}\}, \qquad a^{(l,-)} = z^{(l,-)}, \tag{2.26}$$

At the final affine readout, use the same pre-activation formulas and set $a^{(L,\pm)} = z^{(L,\pm)}$ without a maximum. Thus for a scalar output the split streams $g, h\colon \mathbb{R}^{2d} \to \mathbb{R}$ map the doubled input $(a^{(0,+)}, a^{(0,-)})$ to the two final components and satisfy $f(a^{(0,+)} - a^{(0,-)}) = g - h$. The following theorem identifies this construction as a tropical rational decomposition whose components carry additional monotone structure.

Theorem 2.9 (The DC split is a monotone tropical rational decomposition, `Lean_15`). *Let $f\colon \mathbb{R}^d \to \mathbb{R}$ be a bias-free ReLU **M**ulti**l**ayer **P**erceptron (MLP) with scalar output and split streams $g, h\colon \mathbb{R}^{2d} \to \mathbb{R}$ defined by (2.25)–(2.26). Then:*

1) *every coordinate of $a^{(l,+)}$ and $a^{(l,-)}$, and in particular g and h, is a positively homogeneous* monotone *tropical polynomial in the doubled variable $(x^+, x^-) \in \mathbb{R}^{2d}$;*
2) *for every splitting $x = x^+ - x^-$,*

$$f(x) \;=\; g(x^+, x^-) \;-\; h(x^+, x^-), \tag{2.27}$$

i.e. f, precomposed with the difference map, is the tropical rational function $g \oslash h$.

Proof. (1). Induct forward. At $l = 0$ the coordinates of $a^{(0,+)} = x^+$ and $a^{(0,-)} = x^-$ are coordinate projections, i.e. monomials $\langle e_j, \cdot \rangle$ with a single non-negative exponent and zero coefficient. For the step, each coordinate of $z^{(l,\pm)}$ in (2.25) is a non-negative combination of layer-$(l-1)$ coordinates (as $W^{(l,\pm)} \geq 0$), and each coordinate of $a^{(l,+)}$ is a pointwise maximum of two of these while $a^{(l,-)} = z^{(l,-)}$. By the closure of positively homogeneous monotone tropical polynomials under non-negative combinations and maxima (Observation 2.3), the property propagates to g and h. *(2).* The identity (2.27) is not a new proof obligation here: the correctness of the split $f = g - h$ is established in Chapter 3, and the present part only restates it in tropical language—by part (1) both g and h are tropical polynomials, so their difference $g \oslash h = g - h$ is a tropical rational function. □

Monotonicity of both components—not guaranteed by generic tropical rational representations—is what confines the Newton polytopes below to the non-negative orthant, via the following elementary fact.

Theorem 2.10 (Monotone CPWL functions have non-negative gradients, `Lean_16`). *Let $u\colon \mathbb{R}^n \to \mathbb{R}$ be monotone and CPWL. Then every gradient attained by u at a point of differentiability is entrywise non-negative.*

Proof. If u is differentiable at x with $(\nabla u(x))_j < 0$, then u is affine on an ε-ball around x, and $\tilde{x} := x - \frac{\varepsilon}{2}e_j \leq x$ satisfies $u(\tilde{x}) = u(x) - \frac{\varepsilon}{2}(\nabla u(x))_j > u(x)$, contradicting monotonicity. □

2.4.2 Newton Polytopes of the Split Streams

By Observation 2.3 and Lemma 2.7, every positively homogeneous convex CPWL function u is the support function $u = h_P$ of a unique polytope $P = \mathrm{Newt}(u)$, recovered as the halfspace intersection

$$P = \big\{y \in \mathbb{R}^n \,:\, \langle y, x\rangle \le h_P(x) \text{ for all } x \in \mathbb{R}^n\big\}, \tag{2.28}$$

and the tropical operations act on P through the Minkowski calculus of Proposition 2.8. Applied to the split streams—positively homogeneous monotone tropical polynomials by Theorem 2.9—this turns the split recursion into a layer-wise propagation of Newton polytopes. Writing $P_j^{(l,\sigma)} \coloneqq \mathrm{Newt}\big(a_j^{(l,\sigma)}\big) \subseteq \mathbb{R}^{2d}$ for the jth split activation of layer l:

Corollary 2.11 (Minkowski-sum propagation across layers, `Lean_17`). *For a bias-free ReLU MLP with split streams (2.25)–(2.26), the Newton polytopes of the split activations satisfy*

$$\begin{aligned} &P_j^{(0,+)} = \{e_j\}, \qquad P_j^{(0,-)} = \{e_{d+j}\}, \qquad j \in [d], \\ &Q_i^{(l,+)} \coloneqq \bigoplus_j \Big(W_{ij}^{(l,+)} P_j^{(l-1,+)} \oplus W_{ij}^{(l,-)} P_j^{(l-1,-)}\Big), \qquad Q_i^{(l,-)} \coloneqq \bigoplus_j \Big(W_{ij}^{(l,-)} P_j^{(l-1,+)} \oplus W_{ij}^{(l,+)} P_j^{(l-1,-)}\Big), \end{aligned} \tag{2.29}$$

$$P_i^{(l,+)} = \mathrm{conv}\Big(Q_i^{(l,+)} \cup Q_i^{(l,-)}\Big), \qquad P_i^{(l,-)} = Q_i^{(l,-)}, \tag{2.30}$$

where the activation update (2.30) applies only to $l < L$; at the affine readout set $P_i^{(L,\pm)} = Q_i^{(L,\pm)}$. Here $e_1, \dots, e_{2d}$ are the standard basis of $\mathbb{R}^{2d}$ and $\oplus$ the Minkowski sum. The Newton polytopes $P_g \coloneqq \mathrm{Newt}(g)$ and $P_h \coloneqq \mathrm{Newt}(h)$ of a scalar output neuron are obtained after L steps.

Proof. The base case restates that $a_j^{(0,\pm)}$ are the coordinate functionals $\langle e_j, \cdot\rangle$, $\langle e_{d+j}, \cdot\rangle$. For the step, the ith coordinate of (2.25) is a non-negative combination whose Newton polytope is, by the Minkowski-sum and scaling identities of Proposition 2.8, the weighted sum $Q_i^{(l,\pm)}$; the entrywise maximum in (2.26) corresponds to the convex hull of the union by the same calculus, while $a^{(l,-)}$ passes its polytope through unchanged. □

For a single hidden layer this produces Minkowski sums of segments and points, i.e. *zonotopes*, recovering the shallow-network picture of Zhang et al. (2018b); in deeper networks the alternation of sums and hulls makes vertex counts grow with depth. The recursion is a structural tool, not an algorithm: the split forward pass evaluates these support functions without enumerating vertices. The link between polytope geometry and the differential behavior of its support function is the subdifferential correspondence, the engine behind Section 2.4.3.

Proposition 2.12 (Subdifferential of a support function, `Lean_18`). *Let $P \subseteq \mathbb{R}^n$ be a polytope and $x \in \mathbb{R}^n$. Then $\partial h_P(x) = F_P(x) = \{p \in P : \langle p, x\rangle = h_P(x)\}$, the maximizing face. In particular, h_P is differentiable at x if and only if $F_P(x)$ is a singleton, in which case $\nabla h_P(x) \in P$.*

Proof. "$\supseteq$": For $p \in F_P(x)$ and any y, $h_P(y) \ge \langle p, y\rangle = h_P(x) + \langle p, y - x\rangle$, the subgradient inequality. "$\subseteq$": Let $s \in \partial h_P(x)$. Testing the subgradient inequality with $y = 2x$ and $y = 0$ and using positive homogeneity gives $\langle s, x\rangle = h_P(x)$; substituting back yields $h_P(y) \ge \langle s, y\rangle$ for all y, so $s \in P$ by (2.28), hence $s \in F_P(x)$. Differentiability of a convex function is equivalent to a singleton subdifferential, whose element is the gradient. □

2.4.3 Imaginative Polytopes

We now specialize to the split streams of a trained network. Their Newton polytopes are the central geometric objects of the interpretability perspective of this thesis.

Definition 2.15 (Imaginative Polytopes). Let $g, h\colon \mathbb{R}^{2d} \to \mathbb{R}$ be the split streams of an output neuron of a bias-free ReLU network. The *Imaginative Polytopes* of that neuron are the Newton polytopes $P_g := \mathrm{Newt}(g)$ and $P_h := \mathrm{Newt}(h)$, i.e. the unique polytopes in $\mathbb{R}^{2d}$ whose support functions are g and h (Lemma 2.7).

They exist and are unique by Theorem 2.9 and the support-function bijection, and Corollary 2.11 expresses them as an alternation of weighted Minkowski sums and convex hulls. Monotonicity constrains their location.

Corollary 2.13 (Imaginative Polytopes live in the non-negative orthant, `Lean_19`**).** $P_g, P_h \subseteq \mathbb{R}^{2d}_{\geq 0}$.

Proof. By Theorem 2.9, $g = \max_i \langle a_i, \cdot \rangle$ with all $a_i \geq 0$, and $P_g = \mathrm{conv}\{a_i\}$ is a convex hull of non-negative vectors; likewise for P_h. (Equivalently, every vertex is an attainable gradient of the monotone g by Theorems 2.14 and 2.10.) □

Theorem 2.14 (Vertices encode the attainable split-stream gradients, `Lean_20`**).** *Let $u\colon \mathbb{R}^n \to \mathbb{R}$ be a positively homogeneous convex CPWL function with Newton polytope $P = \mathrm{Newt}(u)$. Then*

$$\mathrm{vert}(P) \;=\; \big\{\nabla u(\xi) \,:\, \xi \in \mathbb{R}^n,\ u \text{ differentiable at } \xi\big\}.$$

In particular, the attainable gradients of the split streams g and h are exactly the vertex sets $\mathrm{vert}(P_g)$ and $\mathrm{vert}(P_h)$ of the Imaginative Polytopes.

Proof. Here $u = h_P$, and $\partial u(\xi) = F_P(\xi)$ by Proposition 2.12. "$\supseteq$": if u is differentiable at ξ, then $F_P(\xi) = \{\nabla u(\xi)\}$ is a singleton face; a single-point face is a vertex, since $p = \frac{1}{2}(p_1 + p_2)$ with $p_i \in P$ would force $p_1, p_2 \in F_P(\xi) = \{p\}$. "$\subseteq$": let $v \in \mathrm{vert}(P)$ and $C := \mathrm{conv}(\mathrm{vert}(P) \setminus \{v\})$; then $v \notin C$, so strict separation yields ξ with $\langle v, \xi\rangle > \max_{v_i \neq v} \langle v_i, \xi\rangle$, and this persists on a ball B. By the vertex representation of the support function (Lemma 2.7), $u(\xi') = \langle v, \xi'\rangle$ on B, so u is differentiable at ξ with $\nabla u(\xi) = v$. The split-stream statement is the case $u = g, h$, admissible by Theorem 2.9. For networks with biases, homogenization gives an ambient support function, but fixing the added coordinates restricts the exposing directions. The input gradients of the restricted function are projections of vertices exposed by directions on that affine slice; not every ambient vertex need be visible there. For example, (1,0) is a vertex of $P = \mathrm{conv}\{(0,1),(2,1),(1,0)\}$, but its value x is strictly below $h_P(x,1) = \max\{1, 2x+1\}$ for every x. Thus the equality with all vertices applies in the homogeneous ambient space, while the biased network uses only the slice-visible vertices. □

Theorem 2.14 turns the support-function representation into an interpretable computational model. Writing $\xi = (x^+, x^-)$ for the doubled input,

$$g(\xi) = \max_{v \in \mathrm{vert}(P_g)} \langle v, \xi\rangle, \qquad h(\xi) = \max_{w \in \mathrm{vert}(P_h)} \langle w, \xi\rangle, \tag{2.31}$$

so each split stream scans a finite catalog of non-negative linear templates (Corollary 2.13)—for image classifiers, pairs of non-negative images—and reports the strongest response; each vertex corresponds

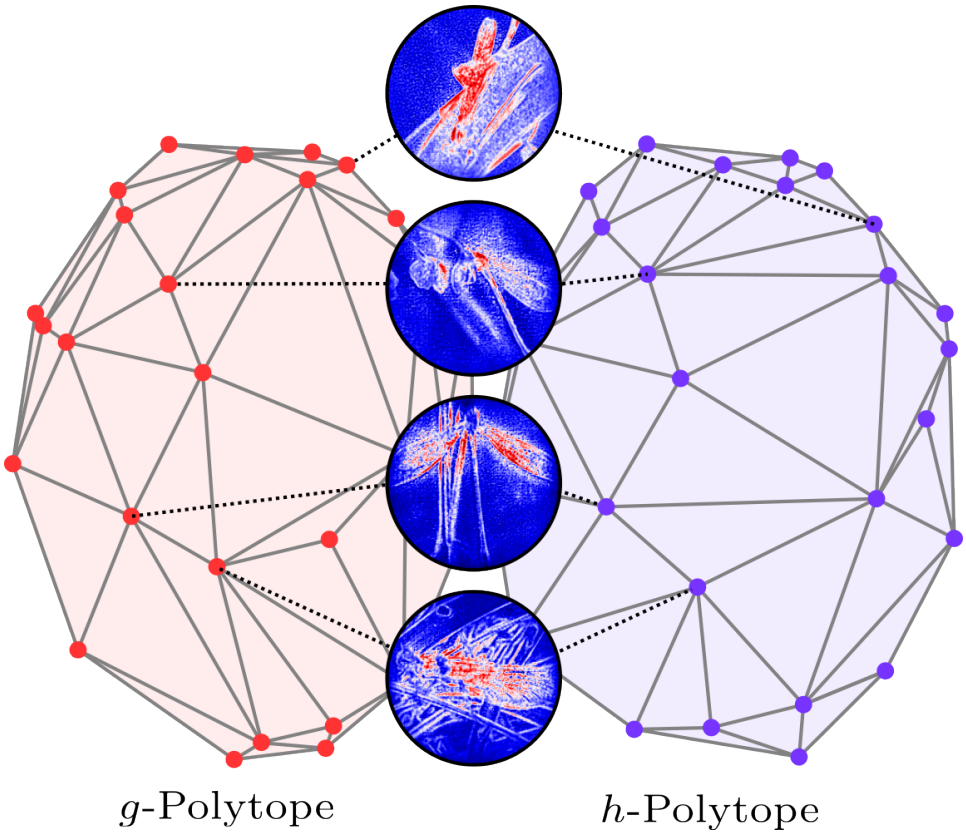


Fig. 2.1: Concept visualization of the Imaginative Polytopes associated with the "dragonfly" output neuron of VGG-16 (Simonyan and Zisserman, 2015) on four ImageNet test images (Deng et al., 2009). Shown are g-stream gradients with respect to the class logit, computed with the stabilized backward pass of Section 3.6; by Theorem 2.14, each unstabilized such gradient is a vertex of P_g, i.e. a learned visual prototype of the class. The h-stream gradients look very similar, since the two streams differ only by the gradient of the original network. Figure adapted from Zimmermann & Loho (2026).

to one gate configuration of (2.26), the linear region on which that stream has the given gradient. We therefore read $\mathrm{vert}(P_g)$ and $\mathrm{vert}(P_h)$ as the neuron's *learned prototypes*, extremal non-redundant feature detectors of which the network applies the best-matching one per input, with the logit the difference of the two strongest responses (2.27); P_g collects excitatory, P_h inhibitory evidence profiles. The same vertices double as *counterfactual profiles*: for an input with active prototype v, the non-negative gaps $\langle v - v', \xi \rangle$ over edge-neighbors v' of v measure how far ξ is from being re-imagined under another prototype, and $v - v'$ shows which evidence would have to move—answering, within the network's own geometry and without any surrogate, what minimally different perception would flip the stream's assessment. Their non-negativity also explains the stability and localization of the split-stream saliency maps studied in Chapters 6 and 7: responses aggregate non-negative evidence without the sign cancellation that erodes ∇f.

The split pair (g, h) is in general *not* minimal—the recursion accumulates common monomials in both streams—but it is computed by the network's own architecture, both components are monotone, and the split activations stay in one-to-one correspondence with the original neurons, properties that Chapters 6 and 4 exploit.[2]

[2]That every CPWL function is a difference of convex CPWL functions is classical (Kripfganz and Schulze, 1987; Melzer, 1986; Zalgaller, 2000); algorithmic and minimality aspects, including the polyhedral structure of the set of all DC

2.4.4 DC Splitting of Smooth Activation Functions

The tropical picture above rests on the piecewise-linear structure of ReLU, but the difference-of-convex splitting itself does not: the only property that matters is that the activation function has integrable curvature. The following general theory lets the decompositions of Chapter 3 extend beyond ReLU to smooth-activation networks and, conversely, exhibits ReLU networks as the piecewise-linear special case of a single construction. Throughout this subsection the symbols g and h are reused *locally* for the scalar per-activation split $g, h\colon \mathbb{R}^2 \to \mathbb{R}$ of a single unit; they no longer denote the network split streams $g, h\colon \mathbb{R}^{2d} \to \mathbb{R}$ of the preceding subsections.

Theorem 2.15 (DC–monotone splitting under integrable curvature, `Lean_21`). *Let $f\colon \mathbb{R} \to \mathbb{R}$ be differentiable with locally absolutely continuous derivative f' and $f'' \in L^1(\mathbb{R})$. Then the limit $s_- := \lim_{u\to-\infty} f'(u)$ exists and f' is bounded. Write $P := \max\{f'',0\}$ and $N := \max\{-f'',0\}$, so that $f'' = P - N$ with $P, N \geq 0$, and define*

$$C_1(x) := f(0) + \int_0^x \Big(s_- + \int_{-\infty}^t P(u)\ du \Big)\ dt, \qquad C_2(x) := \int_0^x \int_{-\infty}^t N(u)\ du\ dt, \tag{2.32}$$

as well as $M := \max\big\{\sup_{x\in\mathbb{R}}|C_1'(x)|, \sup_{x\in\mathbb{R}}|C_2'(x)|\big\} < \infty$. Then the functions

$$g(a,b) := C_1(a-b) + M(a+b), \qquad h(a,b) := C_2(a-b) + M(a+b) \tag{2.33}$$

are convex on $\mathbb{R}^2$, non-decreasing in each argument, and satisfy

$$f(a-b) = g(a,b) - h(a,b) \qquad \text{for all } (a,b) \in \mathbb{R}^2. \tag{2.34}$$

Proof. Well-definedness. For $x \leq y$ we have $|f'(y) - f'(x)| \leq \int_x^y |f''(u)|\,\mathrm{d}u$, which tends to 0 as $x, y \to -\infty$ because $f'' \in L^1(\mathbb{R})$; by the Cauchy criterion the limit s_- exists. Consequently $|f'(x)| \leq |s_-| + \|f''\|_{L^1(\mathbb{R})}$ for all x, so the boundedness of f' is automatic and need not be assumed. Since $0 \leq P, N \leq |f''|$, both P and N lie in $L^1(\mathbb{R})$, the inner integrals in (2.32) are well-defined, non-decreasing in t, and bounded by $\|P\|_{L^1(\mathbb{R})}$ and $\|N\|_{L^1(\mathbb{R})}$ respectively. Hence C_1, C_2 are well-defined with locally absolutely continuous derivatives

$$C_1'(x) = s_- + \int_{-\infty}^x P(u)\ \mathrm{d}u, \qquad C_2'(x) = \int_{-\infty}^x N(u)\ \mathrm{d}u,$$

and $|C_1'| \leq |s_-| + \|P\|_{L^1(\mathbb{R})}$, $|C_2'| \leq \|N\|_{L^1(\mathbb{R})}$, so $M < \infty$.

Exactness. For almost every x,

$$C_1'(x) - C_2'(x) = s_- + \int_{-\infty}^x \big(P(u) - N(u)\big)\,\mathrm{d}u = s_- + \int_{-\infty}^x f''(u)\ \mathrm{d}u = f'(x),$$

and $C_1(0) - C_2(0) = f(0)$, so integrating from 0 yields $C_1 - C_2 = f$ on all of $\mathbb{R}$. The linear tilt cancels upon subtraction, and (2.34) follows by evaluating at $x = a - b$.

decompositions of a fixed function, were developed by Griewank & Walther (2020) and in (Brandenburg et al., 2025; Koutschan et al., 2023).

Convexity. C_1' and C_2' are non-decreasing, being (shifted) antiderivatives of the non-negative functions P and N, so C_1 and C_2 are convex on $\mathbb{R}$. Precomposition with the linear map $(a,b) \mapsto a - b$ preserves convexity, and so does adding the linear function $M(a+b)$; hence g and h are convex on $\mathbb{R}^2$. (When f'' is continuous, this is also visible directly: the Hessian of $(a,b) \mapsto C_1(a-b)$ has eigenvalues 0 and $2C_1''(a-b) = 2P(a-b) \geq 0$.)

Monotonicity. By the choice of M,

$$\frac{\partial g}{\partial a}(a,b) = C_1'(a-b) + M \geq 0, \qquad \frac{\partial g}{\partial b}(a,b) = -C_1'(a-b) + M \geq 0,$$

and analogously for h with C_2' in place of C_1'. Hence g and h are non-decreasing in each argument. □

The construction is less arbitrary than it may first appear: the split of the curvature into P and N is canonical, and the tilt M is the smallest possible.

Proposition 2.16 (Minimality of the curvature split, `Lean_22`). *Let f be as in Theorem 2.15 and suppose $f = c_1 - c_2$ for convex functions $c_1, c_2 \colon \mathbb{R} \to \mathbb{R}$ whose second distributional derivatives are locally integrable functions. Then $c_1'' \geq P$ and $c_2'' \geq N$ almost everywhere. In particular, (C_1, C_2) is, up to a common affine summand, the unique pair of convex functions with $C_1 - C_2 = f$ of pointwise minimal curvature, and among all tilts of the form $(a,b) \mapsto \lambda(a+b)$, $\lambda \geq 0$, the constant M in Theorem 2.15 is the smallest one rendering both components of (2.33) non-decreasing in each argument.*

Proof. From $c_1'' = f'' + c_2''$ and $c_2'' \geq 0$ we get $c_1'' \geq f''$, and convexity of c_1 gives $c_1'' \geq 0$; together $c_1'' \geq \max\{f'', 0\} = P$ almost everywhere, and symmetrically $c_2'' \geq N$. If equality holds a.e. in both bounds, then $c_1 - C_1$ and $c_2 - C_2$ are affine and, since $c_1 - c_2 = f = C_1 - C_2$, they coincide. For the tilt, non-decreasingness of g in both arguments is equivalent to $\lambda \geq C_1'(x)$ and $\lambda \geq -C_1'(x)$ for all x, i.e. $\lambda \geq \sup_x |C_1'(x)|$, and analogously for h; the smallest admissible λ is exactly M. □

Corollary 2.17 (Modern activation functions, `Lean_23`). *Theorem 2.15 applies to the following activation functions, where φ and Φ denote the standard Gaussian density and distribution function and $\sigma(x) = (1+e^{-x})^{-1}$ the logistic sigmoid:*

1) GELU, *$f(x) = x\,\Phi(x)$, with $f''(x) = (2 - x^2)\,\varphi(x)$; the curvature is positive on $(-\sqrt{2}, \sqrt{2})$ and negative outside.*
2) Swish/SiLU, *$f(x) = x\,\sigma(\beta x)$ with $\beta > 0$, with $f''(x) = \beta\,\sigma'(\beta x)\big(2 + \beta x\,(1 - 2\sigma(\beta x))\big)$; the curvature is positive on a bounded interval around the origin and negative in both tails.*
3) Mish, *$f(x) = x \tanh\big(\log(1 + e^x)\big)$.*
4) Hyperbolic tangent, *$f(x) = \tanh(x)$, with $f''(x) = -2\tanh(x)\operatorname{sech}^2(x)$, so P is supported on $(-\infty, 0]$ and N on $[0, \infty)$.*
5) Logistic sigmoid, *$f = \sigma$, with $f'' = \sigma'(1 - 2\sigma)$.*
6) Softplus at temperature $\theta > 0$, *$f(x) = \theta \log(1 + e^{x/\theta})$, with $f''(x) = \sigma'(x/\theta)/\theta \geq 0$; here $N = 0$, $C_2 = 0$, and the decomposition degenerates to the statement that softplus itself is convex.*

In each case $f(0) \geq 0$, so by Remark 2.11 below the resulting pair map preserves non-negativity of the split streams.

Proof. All listed functions are smooth with the stated second derivatives, obtained by direct differentiation. In each case f'' is continuous and decays at least exponentially as $|x| \to \infty$ (with Gaussian decay for GELU): for Swish and Mish the polynomial factor x is dominated by the exponential decay

of $\sigma'(\beta x)$ and of the derivatives of $\tanh \circ \operatorname{softplus}$, respectively. Hence $f'' \in L^1(\mathbb{R})$ and f' is locally absolutely continuous, so Theorem 2.15 applies. The values $f(0) \in \{0, \frac{1}{2}, \theta \log 2\}$ are non-negative in every case. □

Remark 2.10 (Bounded variation and the piecewise-linear case). The hypothesis of Theorem 2.15 can be weakened to f' having bounded variation, i.e. f'' being a finite signed Radon measure μ; replacing $P(u)\,\mathrm{d}u$ and $N(u)\,\mathrm{d}u$ in (2.32) by the Jordan decomposition $\mu = \mu_+ - \mu_-$ leaves every step of the proof intact. This covers the piecewise-linear activations: for ReLU one has $\mu = \delta_0$, hence $C_1(x) = \max\{x, 0\}$, $C_2 = 0$, $s_- = 0$ and $M = 1$, so that

$$g(a,b) - h(a,b) = \max\{a - b, 0\} = \max\{a, b\} - b,$$

recovering the splitting identity of Section 3.3 up to the monotone tilt. Leaky ReLU corresponds to $\mu = (1 - \alpha)\,\delta_0$, and ELU to an absolutely continuous curvature $e^x \mathbb{1}_{x<0}$ plus a possible atom at the origin. Piecewise-linear and smooth activation splittings are thus instances of a single construction.

Remark 2.11 (Compatibility with valid decompositions). The pair map $(a,b) \mapsto \big(g(a,b), h(a,b)\big)$ fits the general decomposition framework of Section 3.1. Indeed, for $a, b \geq 0$ the Lipschitz bounds $|C_i(x) - C_i(0)| \leq M|x|$ and $|a - b| \leq a + b$ yield

$$g(a,b) \geq f(0) \geq 0, \qquad h(a,b) \geq 0,$$

for every activation of Corollary 2.17, so the map sends $\mathbb{R}^2_{\geq 0}$ to $\mathbb{R}^2_{\geq 0}$ and is a monotone pairwise activation function; exactness (2.34) is precisely the validity condition. For an activation with $f(0) < 0$, shifting both components by $|f(0)|$ restores non-negativity without affecting the difference.

3 Network Decompositions and DC Formulations

The structural approach of this thesis starts with a common operation: separate the positive and negative contributions of a trained network while preserving their difference. This chapter develops that operation into a general framework of network decompositions. The DC instance exposes monotone, convex stream functions for the attribution methods of Chapter 6; the stopping instance preserves the network's gates and supplies the state structure for the games of Chapter 4. Mixing and regularized decompositions connect these choices. Their shared forward invariant lets us compare what each representation retains and how its backward calculation changes, while numerical stabilization makes the constructions usable in deep networks.

3.1 Generalized Network Decomposition

An ordinary network activation is a single signed scalar that collapses two opposing flows—the contributions that *support* the prediction and those that *oppose* it—into one number, hiding their magnitudes. The decompositions of this chapter lay them bare: at every neuron and layer, the signed activation is replaced by a non-negative pair that exposes how positive and negative contributions accumulate through the network, while still recovering the original activation as their difference. Figure 3.1 illustrates the idea for the DC instance of Section 3.3.

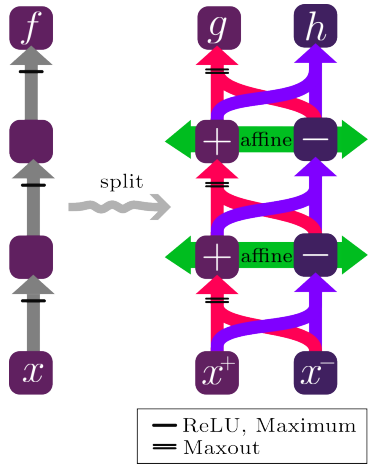


Fig. 3.1: The network splitting procedure: a ReLU network f is decomposed into two monotone, convex streams g and h with $f = g - h$ (Section 3.3).

Throughout this chapter we consider a feedforward network of L layers with weights $W^{(l)}$, biases $b^{(l)}$, and an entrywise activation $\varrho\colon \mathbb{R} \to \mathbb{R}$ (e.g. ReLU or softplus), computing the recursion

$$a^{(0)} = x, \qquad z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = \varrho_l\big(z^{(l)}\big), \qquad l = 1, \dots, L, \tag{3.1}$$

where $\varrho_l = \varrho$ for $1 \le l < L$ and $\varrho_L = \mathrm{id}$ is the ungated affine readout, with scalar output $f(x) = a^{(L)} \in \mathbb{R}$; for multi-class networks we fix the target logit and treat the rest as a single-output network. We follow the notation of Section 2.1; g and h are reserved for the split streams of Section 3.3.

A network decomposition replaces every activation $a_i^{(l)}$ by a pair of non-negative scalars, propagated through non-negative linear maps and *pairwise activation functions* that act on neuron pairs. We first

record this structural setup, then identify the decompositions that recover the original network as the difference of the decomposed activations.

Definition 3.1 (Decomposition). A *decomposition* of a feedforward network (3.1) with L layers is a calculation scheme that propagates non-negative activation pairs $\left(a_i^{(l,+)}, a_i^{(l,-)}\right) \in \mathbb{R}^2_{\geq 0}$ from input to output using only

1) **non-negative linear maps**—matrices with non-negative entries together with non-negative bias vectors—and
2) **pairwise activation functions** $\phi\colon \mathbb{R}^2_{\geq 0} \to \mathbb{R}^2_{\geq 0}$, $(z^+, z^-) \mapsto (a^+, a^-) = \phi(z^+, z^-)$, applied at every hidden neuron; the final affine readout uses the identity on the pair.

Activations are indexed by $l \in \{0, \ldots, L\}$, affine parameters by $l \in \{1, \ldots, L\}$. The layer-0 pair is any non-negative split of the input,

$$\left(a_k^{(0,+)}, a_k^{(0,-)}\right) := \left(x_k^+, x_k^-\right), \qquad x_k^+ - x_k^- = x_k, \qquad x_k^+, x_k^- \geq 0, \tag{3.2}$$

and at each affine layer l a non-negative linear map sends the layer-$(l-1)$ activation pair to the layer-l pre-activation pair $\left(z_i^{(l,+)}, z_i^{(l,-)}\right) \in \mathbb{R}^2_{\geq 0}$, to which the pairwise activation is applied for $l < L$. At $l = L$ we set $a^{(L,\pm)} = z^{(L,\pm)}$, with no activation or maximum. We call $\left(a_i^{(l,+)}, a_i^{(l,-)}\right)$ the *decomposed activation* and $\left(z_i^{(l,+)}, z_i^{(l,-)}\right)$ the *decomposed pre-activation*.

Canonical non-negative linear map. Throughout the thesis we instantiate the non-negative linear map at layer l by the entrywise positive/negative decomposition of the original network's weights and biases,

$$W^{(l,+)} := [W^{(l)}]^+, \qquad W^{(l,-)} := [W^{(l)}]^-, \qquad b^{(l,+)} := [b^{(l)}]^+, \qquad b^{(l,-)} := [b^{(l)}]^-, \tag{3.3}$$

where $[\cdot]^\pm$ are the entrywise positive and negative parts, so $W^{(l)} = W^{(l,+)} - W^{(l,-)}$ with $W^{(l,\pm)} \geq 0$. The resulting pre-activation pair at neuron i is

$$z_i^{(l,+)} := b_i^{(l,+)} + \sum_j W_{ij}^{(l,+)}\, a_j^{(l-1,+)} + \sum_j W_{ij}^{(l,-)}\, a_j^{(l-1,-)}, \tag{3.4}$$

$$z_i^{(l,-)} := b_i^{(l,-)} + \sum_j W_{ij}^{(l,+)}\, a_j^{(l-1,-)} + \sum_j W_{ij}^{(l,-)}\, a_j^{(l-1,+)}. \tag{3.5}$$

Positive weights preserve stream membership, negative weights swap it: an excitatory activation through a negative weight becomes an inhibitory contribution, and vice versa. Definition 3.1 imposes structure only; the following definition ties the pair back to the original network.

Definition 3.2 (Valid decomposition). A decomposition (Definition 3.1) is *valid* for the network (3.1) with activation function ϱ if the propagated activation pairs satisfy

$$a_i^{(l,+)} - a_i^{(l,-)} = \varrho_l\left(z_i^{(l)}\right) = a_i^{(l)} \tag{3.6}$$

for every layer $l \in \{0, \ldots, L\}$ and every neuron i, where at $l = 0$ condition (3.6) is read as $a_k^{(0,+)} - a_k^{(0,-)} = x_k$.

Validity is a layer-wise condition, but it propagates through the canonical linear map and forces the decomposition to recover the network output at the final layer.

Proposition 3.1 (Validity recovers the network, Lean_24). *Let a decomposition use the canonical non-negative linear map (3.4)–(3.5). If the validity condition (3.6) holds at layer $l-1$, then the decomposed pre-activations at layer l satisfy*

$$z_i^{(l,+)} - z_i^{(l,-)} = z_i^{(l)} \qquad \text{for every neuron } i. \tag{3.7}$$

Consequently, every valid decomposition recovers the original network output as the difference of the final decomposed activations,

$$f(x) = a^{(L,+)} - a^{(L,-)}. \tag{3.8}$$

Proof. Subtracting (3.5) from (3.4) and regrouping by predecessor j,

$$z_i^{(l,+)} - z_i^{(l,-)} = b_i^{(l,+)} - b_i^{(l,-)} + \sum_j \big(W_{ij}^{(l,+)} - W_{ij}^{(l,-)}\big)\big(a_j^{(l-1,+)} - a_j^{(l-1,-)}\big) = b_i^{(l)} + \sum_j W_{ij}^{(l)}\, a_j^{(l-1)} = z_i^{(l)},$$

where the second equality uses $W^{(l)} = W^{(l,+)} - W^{(l,-)}$, $b^{(l)} = b^{(l,+)} - b^{(l,-)}$, and the validity condition at layer $l-1$. This proves (3.7). For (3.8), validity at the final layer L gives $a^{(L,+)} - a^{(L,-)} = \varrho\big(z^{(L)}\big) = a^{(L)} = f(x)$. □

By Proposition 3.1, the pre-activation pair always carries the original pre-activation in its difference; validity therefore hinges entirely on the pairwise activation function ϕ, which must turn the pre-activation difference $z_i^{(l)} = z_i^{(l,+)} - z_i^{(l,-)}$ into the activation difference $\varrho\big(z_i^{(l)}\big) = a_i^{(l,+)} - a_i^{(l,-)}$ while keeping both components non-negative. The remainder of this chapter constructs concrete choices of ϕ: the mixing family (Section 3.2) with its convex DC endpoint (Section 3.3), the stopping decomposition (Section 3.4), and its regularized relatives (Section 3.5); the splitting of smooth activations is developed as foundational material in Section 2.4.4 and imported here via Remark 3.4.

3.2 The Mixing Decomposition family

The first family of pairwise activation functions targets ReLU networks. It exploits the fact that ReLU networks compute CPWL functions (Mukherjee et al., 2018), which admit a DC decomposition (Kripfganz and Schulze, 1987; Melzer, 1986; Zimmermann and Loho, 2026), and organizes the decompositions into a one-parameter family interpolating between two arithmetic extremes.

Definition 3.3 (Mixing decomposition function, Lean_25). For $\eta \in [0,1]$, the *mixing decomposition function* is

$$f_{\text{mix}}^{\eta}(z^+, z^-) := \big(\eta \max\{z^+, z^-\} + (1-\eta)\, z^+,\ \ \eta\, z^- + (1-\eta)\min\{z^+, z^-\}\big). \tag{3.9}$$

The decomposition obtained by applying f_{mix}^{η} at every hidden neuron and retaining the identity pair at the affine output on top of the canonical non-negative linear map (3.4)–(3.5), with the input convention (3.2), is called the *Mixing Decomposition*; when comparing decompositions we tag its activations as $a_{\text{mix}}^{(l,\pm,\eta)}$.

Both components of f^{η}_{mix} are non-negative on $\mathbb{R}^2_{\geq 0}$, being convex combinations and maxima/minima of the non-negative inputs; hence f^{η}_{mix} is a pairwise activation function (Definition 3.1). At $\eta = 1$ the pair reads $\big(\max\{z^+, z^-\}, z^-\big)$, at $\eta = 0$ it reads $\big(z^+, \min\{z^+, z^-\}\big)$, and intermediate η mixes the two linearly. The family is valid for ReLU networks for *every* mixing parameter.

Theorem 3.2 (Layer-wise recovery of the Mixing Decomposition, Lean_26). *Let the activation pairs be propagated by the canonical non-negative linear map (3.4)–(3.5), initialised by (3.2), and updated by f^{η}_{mix} for some $\eta \in [0,1]$. Then for every layer l and neuron i,*

$$a^{(l,+,\eta)}_{\text{mix},i} - a^{(l,-,\eta)}_{\text{mix},i} = \varrho_l\big(z^{(l)}_i\big) = a^{(l)}_i, \tag{3.10}$$

i.e. the Mixing Decomposition is a valid decomposition (Definition 3.2) of the ReLU network for every $\eta \in [0,1]$.

Proof. We argue by induction on the layer index l.

Base case $l = 0$. By the input convention (3.2), $a^{(0,+,\eta)}_{\text{mix},k} - a^{(0,-,\eta)}_{\text{mix},k} = x^+_k - x^-_k = x_k = a^{(0)}_k$.

Inductive step. At the final affine readout, recovery is directly Proposition 3.1. For a hidden layer, suppose (3.10) holds at layer $l-1$. By Proposition 3.1, the decomposed pre-activations at layer l satisfy $z^{(l,+)}_i - z^{(l,-)}_i = z^{(l)}_i$. Writing $u = z^{(l,+)}_i$ and $v = z^{(l,-)}_i$, the definition (3.9) gives

$$a^{(l,+,\eta)}_{\text{mix},i} - a^{(l,-,\eta)}_{\text{mix},i} = \eta\big(\max\{u,v\} - v\big) + (1-\eta)\big(u - \min\{u,v\}\big) = \text{ReLU}(u - v) = \text{ReLU}\big(z^{(l)}_i\big) = a^{(l)}_i,$$

using the identities $\max\{u,v\} - v = \max\{u - v, 0\}$ and $u - \min\{u,v\} = \max\{u - v, 0\}$. □

The proof reveals the structural difference within the family: at $\eta = 1$ the non-linearity is carried by the $+$ stream through a per-stream maximum, at $\eta = 0$ by the $-$ stream through a per-stream minimum, both realising the same difference $\text{ReLU}(u - v)$. The endpoints are not symmetric in their analytic properties, however: the $\eta = 1$ extreme produces *convex* decomposed activations at every layer.

Theorem 3.3 ($\eta = 1$ convex characterization of the Mixing Decomposition, Lean_27). *Under the input convention (3.2) with split components $x^{\pm}_k$ that are convex functions of x (e.g. the entrywise split $x^{\pm} = [x]^{\pm}$, or any affine split), the $\eta = 1$ mixing decomposed activations*

$$a^{(l,+,1)}_{\text{mix},i}(x) = \max\big\{z^{(l,+)}_i(x), z^{(l,-)}_i(x)\big\} \qquad \text{and} \qquad a^{(l,-,1)}_{\text{mix},i}(x) = z^{(l,-)}_i(x)$$

are convex in x at every hidden layer l and neuron i; at the final layer, both streams are non-negative affine combinations of the previous streams and remain convex. Consequently $a^{(l)}_i = a^{(l,+,1)}_{\text{mix},i} - a^{(l,-,1)}_{\text{mix},i}$ is a difference of convex functions, the standard DC representation of the CPWL function computed by the network (Mukherjee et al., 2018; Zimmermann and Loho, 2026).

Proof. Induction on l. At $l = 0$, $a^{(0,\pm,1)}_{\text{mix},k} = x^{\pm}_k$ is convex by assumption. At $l \geq 1$, each $z^{(l,\pm)}_i$ is a non-negative-coefficient combination of the layer-$(l-1)$ activations plus a constant, hence convex by the hypothesis; the maximum of two convex functions is convex, so both components remain convex. □

Remark 3.1 (On the $\eta = 0$ extreme). The dual DC representation writes the output as $u(x) - v(x)$ with v *concave*, and f^0_{mix} places $a^{(l,-,0)}_{\text{mix},i} = \min\big\{z^{(l,+)}_i, z^{(l,-)}_i\big\}$ in the role of v. This is concave only

when both decomposed pre-activations are concave (or affine): the minimum of two concave functions is concave, but of two convex functions is in general neither. Under the input convention (3.2), the first pre-activations are convex in x, but later $\eta = 0$ streams and pre-activations need not remain convex or concave. A canonical scalar example is $W^{(1)} = 1$, $b^{(1)} = 1$, giving the negative stream $v(x) = \min\{\max(x,0)+1, \max(-x,0)\}$. It satisfies $v(-2) = v(-1) = 1$, $v(0) = v(1) = 0$: the midpoint tests at -1 and 0 respectively exclude convexity and concavity. By Theorem 3.2, f^0_{mix} still yields a valid arithmetic decomposition, and the network-level concave representation is recoverable with an unsplit affine input convention, at the cost of the $\pm$ structure.

The mixing family thus interpolates between a convexity-carrying extreme at $\eta = 1$ and its arithmetic dual at $\eta = 0$; only the former inherits per-layer convexity. The next section develops this $\eta = 1$ endpoint into the central computational object of the thesis.

3.3 Difference-of-Convex (DC) splitting of ReLU Networks

The $\eta = 1$ endpoint underlies the stabilized split passes (Section 3.6), the CNN generalization (Section 3.7), and the attribution methods of Chapter 6. It recovers the classical DC splitting of ReLU networks from DC regression and approximation theory (Awasthi et al., 2023; Siahkamari et al., 2020; Zhang et al., 2018b), and we derive it as a corollary of Theorem 3.2. Its algebraic core is the *splitting identity*

$$\mathrm{ReLU}(a-b) = \max\{a-b, 0\} = \max\{a, b\} - b \qquad \text{for all } a, b \in \mathbb{R}, \tag{3.11}$$

which follows by adding and subtracting b inside the maximum. It rewrites the ReLU of a difference as a difference of two functions, each convex and non-decreasing in (a, b)—a Maxout unit (Goodfellow et al., 2013) and a coordinate projection.

Corollary 3.4 (DC split as the $\eta = 1$ mixing endpoint, `Lean_28`). *Set $\eta = 1$ in Definition 3.3 and drop the decomposition tag, writing $a^{(l,\pm)} := a^{(l,\pm,1)}_{\text{mix}}$. The resulting* DC split *propagates, for hidden layers $l = 1, \ldots, L-1$,*

$$a^{(l,+)} = \max\{z^{(l,+)}, z^{(l,-)}\}, \qquad a^{(l,-)} = z^{(l,-)} \tag{3.12}$$

entrywise, and sets $a^{(L,\pm)} = z^{(L,\pm)}$ at the final affine readout, with the decomposed pre-activations (3.4)–(3.5) induced by the weight split $W^{(l)} = W^{(l,+)} - W^{(l,-)}$ and the input split $x = x^+ - x^-$. It is a valid decomposition of the ReLU network, and under the input convention of Theorem 3.3 its decomposed activations are convex functions of the input.

Proof. Validity is Theorem 3.2 at $\eta = 1$, convexity Theorem 3.3. The update (3.12) is (3.9) at $\eta = 1$, and neuron-level recovery is the splitting identity (3.11) at $a = z_i^{(l,+)}$, $b = z_i^{(l,-)}$. □

The stream functions g and h. The DC split replaces each layer by a *split pair* of two monotone, non-negative *split streams* (Figure 3.2). In analogy to the tail networks $f^{(l)} \colon \mathbb{R}^{d^{(l)}} \to \mathbb{R}$ restricting the original network to layers $l+1, \ldots, L$, we denote the split streams restricted to those layers by

$$g^{(l)} \colon \mathbb{R}^{d^{(l)}} \times \mathbb{R}^{d^{(l)}} \to \mathbb{R}, \qquad h^{(l)} \colon \mathbb{R}^{d^{(l)}} \times \mathbb{R}^{d^{(l)}} \to \mathbb{R}, \tag{3.13}$$

mapping an activation pair at layer l to the $+$ and $-$ components of the decomposed output; we abbreviate $g := g^{(0)}$, $h := h^{(0)}$. On the output layer $g^{(L)}(x^+, x^-) = x^+$, $h^{(L)}(x^+, x^-) = x^-$, and for

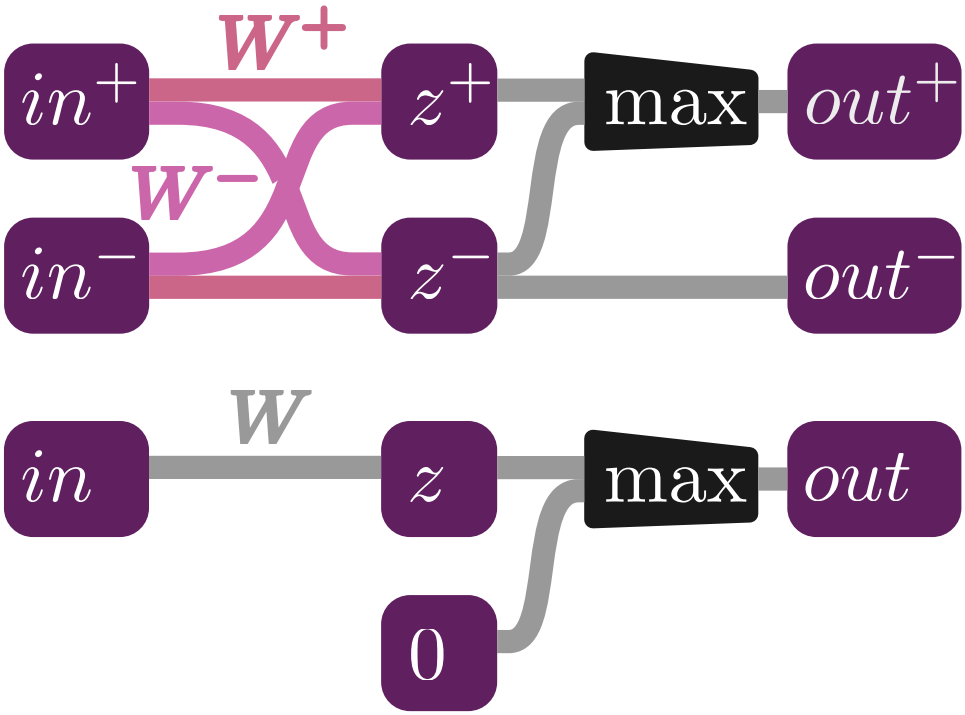


Fig. 3.2: The monotone, convex split streams (top) versus a standard ReLU layer (bottom). The negative weight part transitions positive input to negative pre-activation and vice versa, while the positive part maps positive to positive and negative to negative.

$0 \le l < L$,

$$g^{(l)}(x^+, x^-) = g^{(l+1)}\big(a^{(l+1,+)}, a^{(l+1,-)}\big), \qquad h^{(l)}(x^+, x^-) = h^{(l+1)}\big(a^{(l+1,+)}, a^{(l+1,-)}\big), \tag{3.14}$$

where the pair $\big(a^{(l+1,+)}, a^{(l+1,-)}\big)$ is computed from (x^+, x^-) by (3.4)–(3.5) and (3.12). These stream functions—not the individual layer activations—are the central objects of the DC sections of this chapter and of the saliency derivations of Chapter 6: the network is exhibited as $f = g - h$ with structured subnetworks g and h. Concretely, the split forward pass for an MLP initialises $\big(a^{(0,+)}, a^{(0,-)}\big) = (x^+, x^-)$ and, for $l = 1, \dots, L-1$, computes the pre-activation pair (3.4)–(3.5) followed by the entrywise update $a^{(l,+)} = \max\{z^{(l,+)}, z^{(l,-)}\}$, $a^{(l,-)} = z^{(l,-)}$ of (3.12). The final layer uses only the affine pair (3.4)–(3.5), so the two heads may reconstruct a negative logit.

The split streams are non-negative linear maps followed by entrywise Maxout gates and projections, i.e. monotone networks with convex, monotone activations; by the composition principles of Section 2.2 (Amos et al., 2017) this yields their two defining properties.

Proposition 3.5 (Monotonicity and convexity of the split streams, `Lean_29`). *For every $0 \le l \le L$, the stream functions $g^{(l)}$ and $h^{(l)}$ of (3.13) are convex and non-decreasing in each coordinate of the pair $(x^+, x^-) \in \mathbb{R}^{d^{(l)}} \times \mathbb{R}^{d^{(l)}}$.*

Proof. Downward induction on l. Base case: $g^{(L)}, h^{(L)}$ are coordinate projections, linear with non-negative coefficients. For the step, fix $l < L$ and assume the claim for $l+1$. The map $(x^+, x^-) \mapsto \big(z^{(l+1,+)}, z^{(l+1,-)}\big)$ of (3.4)–(3.5) is affine with non-negative coefficients, and the entrywise updates (3.12) apply a maximum or a projection, preserving convexity and monotonicity. By (3.14), $g^{(l)}, h^{(l)}$ compose

the convex, non-decreasing $g^{(l+1)}, h^{(l+1)}$ with such a map, hence are again convex and non-decreasing (Amos et al., 2017). □

The correctness statement asserts that the difference of the two split-stream outputs equals the original tail network evaluated at the difference of their inputs—at *every* layer, not only at the input.

Theorem 3.6 (Correctness of the split forward pass, `Lean_30`). *For every* $0 \le l \le L$ *and all* $x^+, x^- \in \mathbb{R}^{d^{(l)}}_{\ge 0}$,

$$f^{(l)}\big(x^+ - x^-\big) = \big(g^{(l)} - h^{(l)}\big)(x^+, x^-). \tag{3.15}$$

Proof. Downward induction on $l = L, \dots, 0$, absorbing biases into the weights via (3.3).

Base case $l = L$. $g^{(L)}(x^+, x^-) - h^{(L)}(x^+, x^-) = x^+ - x^- = f^{(L)}(x^+ - x^-)$, as $f^{(L)}$ is the identity.

Inductive step. Let $0 \le l < L$ and assume (3.15) at $l+1$. With $z^\pm := z^{(l+1,\pm)}$ from (3.4)–(3.5), Proposition 3.1 gives $z^+ - z^- = W^{(l+1)}(x^+ - x^-) + b^{(l+1)}$. Using the splitting identity (3.11) entrywise,

$$\begin{aligned} f^{(l)}\big(x^+ - x^-\big) &= f^{(l+1)}\Big(\mathrm{ReLU}\big(W^{(l+1)}(x^+ - x^-) + b^{(l+1)}\big)\Big) = f^{(l+1)}\big(\mathrm{ReLU}(z^+ - z^-)\big) \\ &= f^{(l+1)}\big(\max\{z^+, z^-\} - z^-\big) = f^{(l+1)}\big(a^{(l+1,+)} - a^{(l+1,-)}\big) \\ &= g^{(l+1)}\big(a^{(l+1,+)}, a^{(l+1,-)}\big) - h^{(l+1)}\big(a^{(l+1,+)}, a^{(l+1,-)}\big) = \big(g^{(l)} - h^{(l)}\big)(x^+, x^-), \end{aligned}$$

where the penultimate equality is the hypothesis at the pair $\big(a^{(l+1,+)}, a^{(l+1,-)}\big)$ and the last is the recursion (3.14). □

Remark 3.2 (Preservation of sub-threshold signals). The correctness theorem hides an information-theoretic asymmetry between the original and the split network. When a neuron is inactive, $z_i^{(l)} \le 0$, the original ReLU outputs $a_i^{(l)} = 0$ and discards the *magnitude* of the losing pre-activation: downstream computation is oblivious to how strongly the neuron was suppressed. The split pair reads $\big(a_i^{(l,+)}, a_i^{(l,-)}\big) = \big(z_i^{(l,-)}, z_i^{(l,-)}\big)$: the difference is zero, as validity demands, but both components retain the accumulated magnitude $z_i^{(l,-)}$ of the cancelled mass and propagate it forward. The split streams thus carry strictly more information than the original activations—sub-threshold contributions survive the gate—which the attribution methods of Chapter 6 exploit. The same effect drives the numerical growth of Section 3.6: without cancellation by subtraction, both streams accumulate the absolute mass $z_i^{(l,+)} + z_i^{(l,-)}$ instead of the signed residual.

Remark 3.3 (Flexibility of the input split). Theorem 3.6 holds for *any* non-negative representation $x = x^+ - x^-$: the entrywise split $x^\pm = [x]^\pm$ or the symmetric split $x^\pm = [x]^\pm + c$ with $c \ge 0$. This degree of freedom is exploited both for numerical stabilization and for the baseline choices of the attribution methods in Chapter 6. The half–half split $x^\pm = \pm x/2$ of Section 3.6 is *not* such a non-negative input split—its negative component is signed—but a chain-rule device for the backward pass, used only to differentiate the split streams along the unified input direction.

Remark 3.4 (Beyond ReLU: smooth activations). The difference-of-convex split is not special to ReLU. Theorem 2.15 (Section 2.4) gives a DC–monotone splitting $f(a-b) = g(a,b) - h(a,b)$ for every activation with integrable curvature — GELU, Swish, tanh, sigmoid, softplus — so the constructions of this chapter carry over to networks with smooth activations by replacing the splitting identity (3.11) with that theorem's curvature-based split.

3.4 The Stopping Decomposition

The mixing family realises the ReLU through per-stream maxima and minima: each stream applies its own non-linearity. A structurally different mechanism applies one *shared* decision to both streams—the hard gate of the original network.

Definition 3.4 (Stopping decomposition function, Lean_31). The *stopping decomposition function* is

$$f_{\text{stop}}(z^+, z^-) \coloneqq \big(\mathbf{1}\big[z^+ - z^- > 0\big]\, z^+,\ \ \mathbf{1}\big[z^+ - z^- > 0\big]\, z^-\big). \tag{3.16}$$

The decomposition obtained by applying f_{stop} at every hidden neuron and retaining the identity pair at the affine output on top of the canonical non-negative linear map (3.4)–(3.5), with the input convention (3.2), is called the *Stopping Decomposition*; when comparing decompositions we tag its activations as $a^{(l,\pm)}_{\text{stop}}$.

Both components of f_{stop} are products of an indicator and a non-negative input, so f_{stop} maps $\mathbb{R}^2_{\geq 0}$ to $\mathbb{R}^2_{\geq 0}$ and is a pairwise activation function (Definition 3.1). Its name anticipates the reinterpretation of the gate as a stop/continue decision (Chapter 4).

Theorem 3.7 (Validity of the Stopping Decomposition, Lean_32). *Let the activation pairs be propagated by the canonical non-negative linear map (3.4)–(3.5), initialised by (3.2), and updated by f_{stop}. Then for every layer l and neuron i,*

$$a^{(l,+)}_{\text{stop},i} - a^{(l,-)}_{\text{stop},i} = \varrho_l\big(z^{(l)}_i\big) = a^{(l)}_i, \tag{3.17}$$

i.e. the Stopping Decomposition is a valid decomposition (Definition 3.2) of the ReLU network.

Proof. Induction on l. The base case $l = 0$ is the input convention, and the final affine step follows directly from Proposition 3.1. For a hidden-layer step, assume (3.17) at $l-1$; Proposition 3.1 gives $z^{(l,+)}_i - z^{(l,-)}_i = z^{(l)}_i$. Since both components of (3.16) carry the *same* gate $\mathbf{1}\big[z^{(l)}_i > 0\big]$,

$$a^{(l,+)}_{\text{stop},i} - a^{(l,-)}_{\text{stop},i} = \mathbf{1}\big[z^{(l)}_i > 0\big]\big(z^{(l,+)}_i - z^{(l,-)}_i\big) = \mathbf{1}\big[z^{(l)}_i > 0\big]\, z^{(l)}_i = \mathrm{ReLU}\big(z^{(l)}_i\big) = a^{(l)}_i. \qquad \square$$

Remark 3.5 (Shared gate versus per-stream maxima). The contrast with the mixing family is sharpest at an inactive neuron, $z^{(l)}_i \leq 0$. The DC split of Corollary 3.4 outputs $\big(z^{(l,-)}_i, z^{(l,-)}_i\big)$, preserving the sub-threshold magnitude (Remark 3.2) at the price of letting cancelled mass flow onward. The Stopping Decomposition outputs $(0, 0)$: the shared hard gate annihilates *both* streams exactly where the original ReLU annihilates its activation, so decomposed mass flows only along neurons active in the original network. The mixing streams are convex (at $\eta = 1$) but blend contributions of dead neurons into every downstream pair; the stopping streams sacrifice global convexity—the gate is discontinuous—for exact support agreement with the original forward pass, while remaining as well-behaved as possible on each linear region.

Lemma 3.8 (Monotonicity of the Stopping Decomposition on linear regions, Lean_33). *On each linear region of the ReLU network—a maximal subset of input space on which all gate indicators $\mathbf{1}\big[z^{(l)}_j > 0\big]$ are constant—each stopping decomposed activation $a^{(l,\pm)}_{\text{stop},i}$ is a non-negative-linear (i.e. affine with non-negative coefficients) function of the input pair $(x^+, x^-) \in \mathbb{R}^{2d^{(0)}}_{\geq 0}$, hence monotone non-decreasing in each coordinate x^+_k and x^-_k.*

Proof. Induction on l. At $l = 0$, $a^{(0,\pm)}_{\text{stop},k} = x^{\pm}_k$ is a coordinate of (x^+, x^-). At $l \geq 1$, the map (3.4)–(3.5) has $W^{(l,\pm)}, b^{(l,\pm)} \geq 0$, so $z^{(l,\pm)}_i$ is non-negative-linear in the layer-$(l-1)$ activations and, by the hypothesis, in (x^+, x^-). On the linear region the gate $\mathbf{1}\big[z^{(l)}_i > 0\big]$ is constant, so $a^{(l,\pm)}_{\text{stop},i} = \mathbf{1}\big[z^{(l)}_i > 0\big]\, z^{(l,\pm)}_i$ stays non-negative-linear. □

Excitatory and inhibitory path isolation. Because the shared gate never mixes the two streams, the Stopping Decomposition admits an exact accounting of *which input mass reaches which stream*: each decomposed pre-activation is a sum over backward paths, and whether a path feeds the excitatory (+) or inhibitory (−) stream is determined by the parity of the number of negative weights it traverses.

Lemma 3.9 (Parity path view on the Stopping Decomposition, `Lean_34`). *Fix a ReLU network with weights $W^{(l)}$ and, for clarity, vanishing biases. For a neuron i in layer $l \geq 1$, a* backward path *to the input is a list $P = (P_l, \ldots, P_0)$ with $P_l = i$ and P_t a neuron in layer t, using weights $W^{(t)}_{P_t, P_{t-1}}$. Define the absolute weight product, parity sign, and gate factor*

$$|w|(P) \coloneqq \prod_{t=1}^{l} \big|W^{(t)}_{P_t,P_{t-1}}\big|,$$

$$\operatorname{sgn}(P) = + \iff \#\big\{t : W^{(t)}_{P_t,P_{t-1}} < 0\big\} \text{ is even},$$

$$G_{<l}(P) \coloneqq \prod_{t=1}^{l-1} \mathbf{1}\big[z^{(t)}_{P_t} > 0\big].$$

Then the decomposed pre-activations of the Stopping Decomposition admit the path-sum representation

$$z^{(l,+)}_i = \sum_{P:\, P_l = i} G_{<l}(P)\, |w|(P)\, x^{\operatorname{sgn}(P)}_{P_0}, \qquad z^{(l,-)}_i = \sum_{P:\, P_l = i} G_{<l}(P)\, |w|(P)\, x^{\operatorname{sgn}(P)'}_{P_0}, \tag{3.18}$$

where $\operatorname{sgn}(P)'$ denotes the opposite sign of $\operatorname{sgn}(P)$.

Proof. Induction on l.

Base case $l = 1$. By (3.4), $z^{(1,+)}_i = \sum_k \big(W^{(1,+)}_{ik} x^+_k + W^{(1,-)}_{ik} x^-_k\big)$. Paths with $P_1 = i$, $P_0 = k$ biject with predecessors k, each with $|w|(P) = \big|W^{(1)}_{ik}\big|$, $G_{<1}(P) = 1$, and $\operatorname{sgn}(P) = +$ iff $W^{(1)}_{ik} \geq 0$. If $W^{(1)}_{ik} > 0$, only $W^{(1,+)}_{ik}$ contributes, giving $|w|(P)\, x^+_k$ with $\operatorname{sgn}(P) = +$; if $W^{(1)}_{ik} < 0$, only $W^{(1,-)}_{ik}$ contributes, giving $|w|(P)\, x^-_k$ with $\operatorname{sgn}(P) = -$. A zero-weight edge has zero contribution regardless of its parity label. Both nonzero cases match (3.18); $z^{(1,-)}_i$ is identical with the parity swapped.

Inductive step $l \geq 2$. The recursion (3.4) takes stopping activations from layer $l-1$ as inputs,

$$z^{(l,+)}_i = \sum_j \Big(W^{(l,+)}_{ij}\, a^{(l-1,+)}_{\text{stop},j} + W^{(l,-)}_{ij}\, a^{(l-1,-)}_{\text{stop},j}\Big), \qquad a^{(l-1,\pm)}_{\text{stop},j} = \mathbf{1}\big[z^{(l-1)}_j > 0\big]\, z^{(l-1,\pm)}_j.$$

At each predecessor j, the gate indicator upgrades the gate factor $G_{<l-1}(P')$ of every sub-path P' ending at j to $G_{\leq l-1}(P')$. Every backward path P with $P_l = i$, $P_{l-1} = j$ is uniquely obtained by prepending i to such a P'; the prepended edge contributes $\big|W^{(l)}_{ij}\big|$ to $|w|(P)$, and $G_{<l}(P) = G_{\leq l-1}(P')$. If $W^{(l)}_{ij} > 0$, only $W^{(l,+)}_{ij}\, a^{(l-1,+)}_{\text{stop},j}$ contributes to $z^{(l,+)}_i$ through j and the edge preserves parity, $\operatorname{sgn}(P) = \operatorname{sgn}(P')$; the hypothesis for $z^{(l-1,+)}_j$ gives the terms of (3.18) with $x^{\operatorname{sgn}(P)}_{P_0}$. If $W^{(l)}_{ij} < 0$, only $W^{(l,-)}_{ij}\, a^{(l-1,-)}_{\text{stop},j}$

contributes and the edge flips parity; the hypothesis for $z_j^{(l-1,-)}$, whose terms carry $x_{P_0'}^{\mathrm{sgn}(P')'} = x_{P_0}^{\mathrm{sgn}(P)}$, again gives the required terms. If $W_{ij}^{(l)} = 0$, no path through j exists. Summing over j yields $z_i^{(l,+)}$; the representation of $z_i^{(l,-)}$ follows with the streams swapped. Biases are accommodated by a bias stub path terminating at each intermediate layer, subject to the same parity rule. □

Subtracting the two path sums and using $x_k^+ - x_k^- = x_k$ recovers the standard active-path-sum representation $z_i^{(l)} = \sum_P G_{<l}(P)\, w(P)\, x_{P_0}$ with signed weight products $w(P)$: the Stopping Decomposition refines it by sorting every active path into the excitatory or inhibitory stream according to its parity. Chapter 4 reinterprets this decomposition as a two-player zero-sum stopping game, in which the gate of (3.16) becomes the players' optimal stop/continue decision.

3.5 Regularized Decompositions

The hard gate of the Stopping Decomposition solves an optimization problem: for any $\Delta \in \mathbb{R}$,

$$\mathrm{ReLU}(\Delta) = \max_{\pi\in[0,1]} \pi\Delta, \qquad \mathbf{1}[\Delta > 0] \in \mathrm{argmax}_{\pi\in[0,1]}\, \pi\Delta, \tag{3.19}$$

so the indicator in (3.16) is the *unregularized* argmax over a stop ($\pi = 0$) versus continue ($\pi = 1$) decision applied to $\Delta = z^+ - z^-$. This suggests a family of decompositions: penalizing the decision variable by a convex regularizer can smooth the gate and changes the recovered activation function from ReLU to the Fenchel conjugate of the regularizer.

Definition 3.5 (Regularized decomposition function, `Lean_35`). Let $\Omega\colon [0,1] \to \mathbb{R}_{\le 0}$ be a proper, closed, convex regularizer with $\Omega \le 0$. Define the *regularized gate* and the *conjugate activation*

$$\pi_\Omega(\Delta) \in \mathrm{argmax}_{\pi\in[0,1]}\big\{\pi\Delta - \Omega(\pi)\big\}, \qquad \Omega^*(\Delta) \coloneqq \max_{\pi\in[0,1]}\big\{\pi\Delta - \Omega(\pi)\big\}, \tag{3.20}$$

where an arbitrary maximizer is fixed if several exist. The *regularized decomposition function* is

$$f_{\mathrm{reg}}^{\Omega}(z^+, z^-) \coloneqq \Big(\pi_\Omega(\Delta)\, z^+ - \Omega\big(\pi_\Omega(\Delta)\big),\ \ \pi_\Omega(\Delta)\, z^-\Big), \qquad \Delta \coloneqq z^+ - z^-. \tag{3.21}$$

The maximum in (3.20) is attained ([0,1] compact, objective upper semicontinuous), and Ω^* is the Fenchel conjugate of Ω (extended by $+\infty$ outside [0,1]). Both components of (3.21) are non-negative on $\mathbb{R}^2_{\ge 0}$, since $\pi_\Omega(\Delta) \in [0,1]$, $z^\pm \ge 0$, $-\Omega \ge 0$; hence $f_{\mathrm{reg}}^{\Omega}$ is a pairwise activation function (Definition 3.1). Every maximizer gives the same reconstructed difference $\Omega^*(\Delta)$, although the individual components may depend on the selected maximizer. For example, with $\Omega = 0$ and $z^+ = z^- = 1$, the choices $\pi = 0$ and $\pi = 1$ produce (0,0) and (1,1) respectively. We therefore retain the fixed selection in Definition 3.5. Throughout this section the untagged pair $\big(a_i^{(l,+)}, a_i^{(l,-)}\big)$ denotes the regularized-family activations produced by $f_{\mathrm{reg}}^{\Omega}$—not the DC ($\eta = 1$) activations to which Corollary 3.4 binds the same untagged symbol.

Theorem 3.10 (Validity of regularized decompositions, `Lean_36`). *Let Ω be as in Definition 3.5, and let the activation pairs be propagated by the canonical non-negative linear map (3.4)–(3.5), initialised by (3.2), and updated by $f_{\mathrm{reg}}^{\Omega}$ at hidden layers, with the identity pair at the final affine readout. Then for every hidden layer l and neuron i,*

$$a_i^{(l,+)} - a_i^{(l,-)} = \Omega^*\big(z_i^{(l)}\big), \tag{3.22}$$

At the final affine layer, the difference is $z^{(L)} = a^{(L)}$. *Thus* $f^{\Omega}_{\mathrm{reg}}$ *is a valid decomposition of the network* (3.1) *with activation function* $\varrho = \Omega^*$. *Moreover,* Ω^* *is convex, non-decreasing, and* 1*-Lipschitz, and wherever* Ω^* *is differentiable the gate is its derivative,* $\pi_\Omega(\Delta) = (\Omega^*)'(\Delta) \in [0,1]$.

Proof. Recovery. Induction on l. The base case is the input convention. Assume (3.22) at $l-1$ for the network with activation Ω^*; Proposition 3.1—whose proof uses only the difference identity—gives $\Delta_i := z_i^{(l,+)} - z_i^{(l,-)} = z_i^{(l)}$. Subtracting the components of (3.21),

$$a_i^{(l,+)} - a_i^{(l,-)} = \pi_\Omega(\Delta_i)\,\big(z_i^{(l,+)} - z_i^{(l,-)}\big) - \Omega\big(\pi_\Omega(\Delta_i)\big) = \pi_\Omega(\Delta_i)\,\Delta_i - \Omega\big(\pi_\Omega(\Delta_i)\big) = \Omega^*\big(z_i^{(l)}\big),$$

where the last step evaluates (3.20) at its maximizer (Fenchel–Young).

Properties of Ω^*. As a supremum of affine functions $\Delta \mapsto \pi\Delta - \Omega(\pi)$ with slopes $\pi \in [0,1]$, Ω^* is convex with subgradients in $[0,1]$, hence non-decreasing and 1-Lipschitz. At a point of differentiability the subdifferential is $\{(\Omega^*)'(\Delta)\}$, and by Fenchel–Young every maximizer $\pi_\Omega(\Delta)$ is a subgradient; thus $\pi_\Omega(\Delta) = (\Omega^*)'(\Delta)$. □

Corollary 3.11 (Unregularized case, `Lean_37`**).** *For* $\Omega \equiv 0$ *on* $[0,1]$, *Definition 3.5 with the maximizer* $\pi_\Omega(\Delta) = \mathbf{1}[\Delta > 0]$ *recovers the stopping decomposition function,* $f^{0}_{\mathrm{reg}} = f_{\mathrm{stop}}$, *and Theorem 3.10 specialises to Theorem 3.7 with* $\Omega^*(\Delta) = \max_{\pi\in[0,1]} \pi\Delta = ReLU(\Delta)$.

Proof. For $\Delta \neq 0$ the objective $\pi\Delta$ has the unique maximizer $\mathbf{1}[\Delta > 0]$ over $[0,1]$, and at $\Delta = 0$ every π is a maximizer, including $\mathbf{1}[0 > 0] = 0$; in all cases (3.21) with $\Omega = 0$ coincides with (3.16) and the optimal value is $\mathrm{ReLU}(\Delta)$. □

3.5.1 The Softplus Decomposition as the entropic subcase

The canonical smooth regularizer is the (negative) binary Shannon entropy at temperature $\theta > 0$,

$$\Omega_\theta(\pi) := -\theta H(\pi), \qquad H(\pi) := -\pi \log \pi - (1-\pi)\log(1-\pi), \tag{3.23}$$

which is strictly convex on $[0,1]$ and satisfies $\Omega_\theta \leq 0$.

Corollary 3.12 (Softplus Decomposition, `Lean_38`**).** *For* $\Omega = \Omega_\theta$ *as in* (3.23), *the regularized gate and conjugate activation are*

$$\pi_{\Omega_\theta}(\Delta) = \sigma'_\theta(\Delta) := \frac{1}{1 + e^{-\Delta/\theta}}, \qquad \Omega^*_\theta(\Delta) = \sigma_\theta(\Delta) := \theta \log\big(1 + e^{\Delta/\theta}\big), \tag{3.24}$$

the sigmoid gate and the softplus function at temperature θ. *The resulting decomposition function is the* softplus decomposition function

$$f^{\theta}_{\mathrm{soft}}(z^+, z^-) = \Big(\sigma'_\theta(\Delta)\, z^+ + \theta\, H\big(\sigma'_\theta(\Delta)\big),\ \ \sigma'_\theta(\Delta)\, z^-\Big), \qquad \Delta = z^+ - z^-, \tag{3.25}$$

and by Theorem 3.10 it yields a valid decomposition—the Softplus Decomposition*—of the softplus network with parameter* $\beta = 1/\theta$. *In the limit* $\theta \to 0$, $f^{\theta}_{\mathrm{soft}} \to f_{\mathrm{stop}}$ *pointwise on* $\{\Delta \neq 0\}$.

Proof. The objective $\pi\Delta + \theta H(\pi)$ is strictly concave on (0,1) with derivative $\Delta - \theta \log \frac{\pi}{1-\pi}$; the first-order condition gives the unique maximizer $\pi = \sigma'_\theta(\Delta)$. Substituting back, with $\log \sigma'_\theta(\Delta) = \Delta/\theta - \log(1 + e^{\Delta/\theta})$ and $\log(1 - \sigma'_\theta(\Delta)) = -\log(1 + e^{\Delta/\theta})$, yields the optimal value

$$\sigma'_\theta(\Delta)\,\Delta + \theta H\big(\sigma'_\theta(\Delta)\big) = \theta \log\big(1 + e^{\Delta/\theta}\big) = \sigma_\theta(\Delta), \tag{3.26}$$

the Legendre–Fenchel form of the binary log-sum-exp at temperature θ over $\{0, \Delta\}$. Then (3.25) is (3.21) with $-\Omega_\theta(\pi_{\Omega_\theta}(\Delta)) = \theta H(\sigma'_\theta(\Delta))$, and validity is Theorem 3.10 with $\Omega^*_\theta = \sigma_\theta$. For the limit, $\sigma'_\theta(\Delta) \to \mathbf{1}[\Delta > 0]$ and $\theta H(\sigma'_\theta(\Delta)) = \sigma_\theta(\Delta) - \sigma'_\theta(\Delta)\Delta \to 0$ as $\theta \to 0$ for $\Delta \neq 0$. □

The entropy bonus $\theta H(\sigma'_\theta(\Delta))$ is carried entirely by the + stream in (3.25); this convention keeps the − stream a plainly gated quantity and reappears in Chapter 4 as the active player's surplus from being allowed to mix.

Remark 3.6 (Loss of stream monotonicity). The soft gate does not inherit the well-behavedness of the hard-gated Stopping Decomposition. Consider the negative stream $a^{(l,-)} = \sigma'_\theta(\Delta)\, z^-$ with $\Delta = z^+ - z^-$: holding z^+ fixed and increasing z^- both raises the factor z^- and, through Δ, *lowers* the gate $\sigma'_\theta(\Delta)$. Differentiating, $\partial a^{(l,-)}/\partial z^- = \sigma'_\theta(\Delta)\big(1 - z^-\, \sigma''_\theta(\Delta)/\sigma'_\theta(\Delta)\big)$ turns negative once z^- exceeds $\theta/\big(1 - \sigma'_\theta(\Delta)\big)$, so $a^{(l,-)}_{\text{soft}}$ is *not* monotone non-decreasing in z^-. The per-region monotonicity of the Stopping Decomposition (Lemma 3.8) therefore does not lift to the soft-gated family; the reader should not expect the Softplus Decomposition to inherit the monotone or convex structure of the hard-gated and DC decompositions. Only the difference recovery of Theorem 3.10 survives.

Remark 3.7 (Probit regularizer and GELU). The framework is not tied to the entropy. For the probit regularizer $\Omega_\Phi(\pi) := -\phi\big(\Phi^{-1}(\pi)\big)$, with ϕ, Φ the standard Gaussian density and distribution function, one has $\frac{\mathrm{d}}{\mathrm{d}\pi}\big(-\Omega_\Phi\big)(\pi) = -\Phi^{-1}(\pi)$, so the first-order condition for (3.20) gives the gate $\pi_{\Omega_\Phi}(\Delta) = \Phi(\Delta)$ and the conjugate activation

$$\Omega^*_\Phi(\Delta) = \Delta\,\Phi(\Delta) + \phi(\Delta) = \mathbb{E}_{\varepsilon\sim\mathcal{N}(0,1)}\big[\max(\Delta + \varepsilon, 0)\big],$$

the Gaussian smoothing of the ReLU. Up to $\phi(\Delta)$, which common implementations drop, this is GELU (Hendrycks and Gimpel, 2016), and the gate $\Phi(\Delta) = \mathbb{E}_\varepsilon\big[\mathbf{1}[\Delta + \varepsilon > 0]\big]$ is the hard stopping gate averaged over a Gaussian shift of its threshold—matching the original stochastic-gate motivation for GELU. The distinction matters for exact decomposability: the conjugate activation $\Omega^*_\Phi(\Delta) = \Delta\,\Phi(\Delta) + \phi(\Delta)$ is the object recovered by f^Ω_{reg} and, being a Fenchel conjugate, is convex, non-decreasing, and 1-Lipschitz (Theorem 3.10). The *implemented* GELU $\Delta \mapsto \Delta\,\Phi(\Delta)$ that drops $\phi(\Delta)$ is non-monotone—it dips below zero for $\Delta < 0$—and is therefore *not* itself a conjugate activation Ω^*; the exactly decomposable object is the $+\phi(\Delta)$ variant, and networks using the dropped-term GELU are only approximately captured by this decomposition.

Remark 3.8 (Connection to the integrable-curvature splitting). Regularized gates produce exactly the smooth activations covered by Theorem 2.15. If Ω is strictly convex and smooth on (0,1), then Ω^* is differentiable with $(\Omega^*)' = \pi_\Omega$ increasing from 0 to 1; hence the curvature $(\Omega^*)'' \geq 0$ exists weakly and integrates to 1. Theorem 2.15 then applies with $N = 0$, $C_2 = 0$: every conjugate activation Ω^* is convex and admits the degenerate DC–monotone splitting, as recorded for the softplus in Corollary 2.17(6). Regularizing the *gate* (this section) and the curvature-based smooth split of Section 2.4.4 thus meet: the two routes yield compatible valid decompositions of the same smoothed networks, cf. Remark 2.11.

3.6 Numerical Stabilization of Split Passes

As observed in Remark 3.2, the split pair lacks cancellation by subtraction: both streams accumulate the *absolute* mass $z^{(l,+)}+z^{(l,-)}$ instead of the signed residual. In deep networks the common magnitude of the two streams therefore grows roughly geometrically, at a rate governed by products of entrywise absolute weight matrices, and quickly leaves the representable range even though the difference $a^{(l,+)}-a^{(l,-)}=a^{(l)}$ stays moderate. This section develops the affine stabilization procedures of Zimmermann & Loho (2026) for both passes: shifting the activations in the forward pass, and the local sensitivities in the backward pass. We absorb biases into the weights and write, for the product of absolute weight matrices above layer l,

$$W_{\mathrm{abs}}^{[l,\to]} := \prod_{j=L,\dots,l+1} |W^{(j)}| = \prod_{j=L,\dots,l+1} \big(W^{(j,+)}+W^{(j,-)}\big), \tag{3.27}$$

with the empty product at $l=L$ read as the identity.

3.6.1 Shifting the forward pass

The key fact is that adding a *common* shift to both streams at any layer translates both stream outputs by the same amount, so the difference—and by Theorem 3.6 the represented network value—is invariant.

Theorem 3.13 (Shift equivariance of the split streams, `Lean_39`**).** *For any $0\le l\le L$ and any $c, a^{(+)}, a^{(-)}\in\mathbb{R}^{d^{(l)}}$ with $a^{(\pm)}, a^{(\pm)}+c\ge 0$,*

$$\begin{aligned} g^{(l)}\big(a^{(+)}+c, a^{(-)}+c\big) &= g^{(l)}\big(a^{(+)}, a^{(-)}\big) + W_{\mathrm{abs}}^{[l,\to]}\, c,\\ h^{(l)}\big(a^{(+)}+c, a^{(-)}+c\big) &= h^{(l)}\big(a^{(+)}, a^{(-)}\big) + W_{\mathrm{abs}}^{[l,\to]}\, c. \end{aligned} \tag{3.28}$$

In particular, $\big(g^{(l)}-h^{(l)}\big)\big(a^{(+)}+c, a^{(-)}+c\big) = \big(g^{(l)}-h^{(l)}\big)\big(a^{(+)}, a^{(-)}\big)$.

Proof. We show the claim for $g^{(l)}$ by induction on l; $h^{(l)}$ is analogous and the difference statement follows by subtraction. The base case $l=L$ is trivial, as $W_{\mathrm{abs}}^{[L,\to]}=I$ and $g^{(L)}$ is a projection. For $0\le l<L$, the positive and negative neurons of layer $l+1$ connect to layer l by the same two matrices with $W^{(l+1,+)}$ and $W^{(l+1,-)}$ flipped (Figure 3.2), so a uniform shift of the layer-l pair produces a uniform shift of the layer-$(l+1)$ pair. For a hidden layer $l+1<L$,

$$\begin{aligned} a^{(l+1,+)}\big|_{+c} &= \max\big\{W^{(l+1,+)}a^{(+)}+W^{(l+1,-)}a^{(-)},\, W^{(l+1,-)}a^{(+)}+W^{(l+1,+)}a^{(-)}\big\} + \big|W^{(l+1)}\big|\, c,\\ a^{(l+1,-)}\big|_{+c} &= W^{(l+1,-)}a^{(+)}+W^{(l+1,+)}a^{(-)} + \big|W^{(l+1)}\big|\, c, \end{aligned}$$

since $\big(W^{(l+1,+)}+W^{(l+1,-)}\big)c = \big|W^{(l+1)}\big|c$ enters both arguments of the maximum and factors out. At the final affine layer $l+1=L$, both stream activations equal their respective pre-activations, and non-negative linearity gives the same common shift $|W^{(L)}|c$ directly, without a maximum. Applying the induction hypothesis at layer $l+1$ with the shift $\big|W^{(l+1)}\big|c$,

$$\begin{aligned} g^{(l)}\big(a^{(+)}+c, a^{(-)}+c\big) &= g^{(l+1)}\Big(a^{(l+1,+)}+\big|W^{(l+1)}\big|c, a^{(l+1,-)}+\big|W^{(l+1)}\big|c\Big)\\ &= g^{(l)}\big(a^{(+)}, a^{(-)}\big) + W_{\mathrm{abs}}^{[l,\to]}\, c. \quad \square \end{aligned}$$

Theorem 3.13 licenses *re-centering*: whenever the common magnitude of a pair explodes, we subtract a common shift without altering the represented function. The canonical choice $c^{(l)} = -\min\{a^{(l,+)}, a^{(l,-)}\}$ (entrywise) makes at least one component of every pair zero. For a fixed shift, the head Jacobians at the shifted and unshifted split coordinates agree, since the Maxout branch comparisons are unchanged. If the shift is computed from the input, it must be detached when propagating these head sensitivities: differentiating through the shift adds the common term $W_{\text{abs}}^{[l,\to]} Dc$ to both head derivatives. Their difference remains unchanged, but their individual derivatives need not. The original paper reports double-precision intermediate computations, re-centering triggered by magnitude thresholds, and optional caching of the original activations to correct the split activations on the fly. This correction targets the **forward invariant** $a^{(l,+)} - a^{(l,-)} = a^{(l)}$; the exact real-arithmetic identity does not imply zero roundoff error in a floating-point implementation. The stabilized pass thus augments each layer of the split forward pass with the entrywise re-centering step $a^{(l,\pm)} \leftarrow a^{(l,\pm)} - \min\{a^{(l,+)}, a^{(l,-)}\}$ and, when enabled, the cache correction $a^{(l,+)} \leftarrow a^{(l,-)} + a^{(l)}$.

3.6.2 Shifting the backward pass

All classical derivative statements in this subsection are restricted to points with strict gate inequalities in the relevant head. At a gate tie, a fixed branch convention defines a backpropagated selection rather than asserting classical differentiability. The gradients of g and h involve only non-negative matrices, so they blow up like the forward activations. Let

$$\delta^{(l,+,g)} := \frac{\partial g^{(l)}}{\partial a^{(l,+)}}, \qquad \delta^{(l,-,g)} := \frac{\partial g^{(l)}}{\partial a^{(l,-)}}, \qquad \delta^{(l,+,h)} := \frac{\partial h^{(l)}}{\partial a^{(l,+)}}, \qquad \delta^{(l,-,h)} := \frac{\partial h^{(l)}}{\partial a^{(l,-)}} \tag{3.29}$$

denote the *local sensitivities* of the stream heads with respect to the split activations of layer l. Theorem 3.13 pins down their pairwise structure.

Theorem 3.14 (Sensitivity complementarity, `Lean_40`**).** *For every* $0 \le l \le L$,

$$\delta^{(l,+,g)} = W_{\text{abs}}^{[l,\to]} - \delta^{(l,-,g)}, \qquad \delta^{(l,+,h)} = W_{\text{abs}}^{[l,\to]} - \delta^{(l,-,h)}. \tag{3.30}$$

Proof. For a coordinate direction e_j, $j \in \{1, \dots, d^{(l)}\}$, Theorem 3.13 with $c = \epsilon\, e_j$ gives $g^{(l)}\big(a^{(l,+)} + \epsilon e_j, a^{(l,-)}\big) = g^{(l)}\big(a^{(l,+)}, a^{(l,-)} - \epsilon e_j\big) + \epsilon\, W_{\text{abs}}^{[l,\to]} e_j$. Subtracting $g^{(l)}\big(a^{(l,+)}, a^{(l,-)}\big)$, dividing by ϵ, and letting $\epsilon \to 0$ yields $\delta_j^{(l,+,g)} = W_{\text{abs}}^{[l,\to]} e_j - \delta_j^{(l,-,g)}$. The claim for h follows analogously. □

For completeness, the four block Jacobians underlying the backward recursions are obtained by the chain rule. At a hidden layer with a strict gate pattern,

$$\begin{aligned}
\frac{\partial a^{(l+1,+)}}{\partial a^{(l,+)}} &= \frac{\partial a^{(l+1,+)}}{\partial z^{(l+1,+)}} \frac{\partial z^{(l+1,+)}}{\partial a^{(l,+)}} + \frac{\partial a^{(l+1,+)}}{\partial z^{(l+1,-)}} \frac{\partial z^{(l+1,-)}}{\partial a^{(l,+)}} \\
&= M^{(l+1,+)} W^{(l+1,+)} + M^{(l+1,-)} W^{(l+1,-)}, \\
\frac{\partial a^{(l+1,-)}}{\partial a^{(l,+)}} &= 0 \cdot W^{(l+1,+)} + W^{(l+1,-)}, \\
\frac{\partial a^{(l+1,+)}}{\partial a^{(l,-)}} &= \frac{\partial a^{(l+1,+)}}{\partial z^{(l+1,+)}} \frac{\partial z^{(l+1,+)}}{\partial a^{(l,-)}} + \frac{\partial a^{(l+1,+)}}{\partial z^{(l+1,-)}} \frac{\partial z^{(l+1,-)}}{\partial a^{(l,-)}} \\
&= M^{(l+1,+)} W^{(l+1,-)} + M^{(l+1,-)} W^{(l+1,+)}, \\
\frac{\partial a^{(l+1,-)}}{\partial a^{(l,-)}} &= 0 \cdot W^{(l+1,-)} + W^{(l+1,+)}.
\end{aligned} \tag{3.31}$$

Here $M^{(l,+)}$ and $M^{(l,-)}$ select the two complementary Maxout branches, defined below. For the final ungated affine layer, the same formulas hold with $M^{(L,+)} = I$ and $M^{(L,-)} = 0$. Biases contribute constant shifts to the pre-activations, so their derivatives with respect to the preceding activations vanish; fixed biases also preserve convexity, monotonicity, and the piecewise-affine structure.

Equation (3.30) exposes the problem: the two sensitivities of each pair sum to the exploding matrix $W_{\text{abs}}^{[l,\to]}$, so at least one is numerically large in deep networks. The stabilization subtracts at every layer an $\alpha^{(l)}$-multiple of this common bulk. Starting from $\delta_{\text{shift}}^{(L,\pm,g)} := \delta^{(L,\pm,g)} - \alpha^{(L)}\big(\delta^{(L,+,g)} + \delta^{(L,-,g)}\big)$ and analogously for h, the *stabilized backward pass* iterates for $l = L-1, \dots, 0$: first the ordinary gradient recursion on the already shifted quantities,

$$\delta_{\text{int}}^{(l,+,g)} := \delta_{\text{shift}}^{(l+1,+,g)}\big(M^{(l+1,+)}W^{(l+1,+)} + M^{(l+1,-)}W^{(l+1,-)}\big) + \delta_{\text{shift}}^{(l+1,-,g)}\, W^{(l+1,-)}, \tag{3.32}$$

$$\delta_{\text{int}}^{(l,-,g)} := \delta_{\text{shift}}^{(l+1,+,g)}\big(M^{(l+1,+)}W^{(l+1,-)} + M^{(l+1,-)}W^{(l+1,+)}\big) + \delta_{\text{shift}}^{(l+1,-,g)}\, W^{(l+1,+)}, \tag{3.33}$$

and then the uniform shift

$$\delta_{\text{shift}}^{(l,\pm,g)} := \delta_{\text{int}}^{(l,\pm,g)} - \alpha^{(l)}\big(\delta_{\text{int}}^{(l,+,g)} + \delta_{\text{int}}^{(l,-,g)}\big), \tag{3.34}$$

with the same scheme for h. Here $M^{(l,+)} := \operatorname{diag}\big(\mathbf{1}[z^{(l)} \geq 0]\big)$ and $M^{(l,-)} := \operatorname{diag}\big(\mathbf{1}[z^{(l)} < 0]\big)$ are the Maxout gate patterns, and (3.32)–(3.33) are the exact chain-rule recursions of the unshifted sensitivities, obtained by differentiating (3.4)–(3.5) and (3.12): e.g. $\partial a^{(l+1,+)}/\partial a^{(l,+)} = M^{(l+1,+)}W^{(l+1,+)} + M^{(l+1,-)}W^{(l+1,-)}$ and $\partial a^{(l+1,-)}/\partial a^{(l,+)} = W^{(l+1,-)}$. For $\alpha^{(l)} \equiv 0$ the scheme reproduces the plain split-pair sensitivities. In the implementation, the gate patterns $M^{(l,\pm)}$ are taken from the cached original activations rather than the drifted split pre-activations, and cached original gradients give small corrections enforcing the **backward invariant** $\frac{1}{2}\big[(\delta^{(l,+,g)} - \delta^{(l,-,g)}) - (\delta^{(l,+,h)} - \delta^{(l,-,h)})\big] = \partial f^{(l)}/\partial a^{(l)}$.

To state what the stabilized quantities compute, we express the split gradients with respect to a unified input. Define $\tilde{g}^{(l)}(a^{(l)}) := g^{(l)}\big(a^{(l)}/2, -a^{(l)}/2\big)$, the head on the half–half split, and analogously $\tilde{h}^{(l)}$; by the chain rule

$$\delta^{(l,g)} := \frac{\partial \tilde{g}^{(l)}}{\partial a^{(l)}} = \frac{\delta^{(l,+,g)} - \delta^{(l,-,g)}}{2}, \qquad \delta^{(l,h)} := \frac{\partial \tilde{h}^{(l)}}{\partial a^{(l)}} = \frac{\delta^{(l,+,h)} - \delta^{(l,-,h)}}{2}, \tag{3.35}$$

and write $\delta_{\text{shift}}^{(l,g)} := \big(\delta_{\text{shift}}^{(l,+,g)} - \delta_{\text{shift}}^{(l,-,g)}\big)/2$, $\delta_{\text{shift}}^{(l,h)} := \big(\delta_{\text{shift}}^{(l,+,h)} - \delta_{\text{shift}}^{(l,-,h)}\big)/2$ for their stabilized counterparts, with $\delta_{\text{shift}}^{g} := \delta_{\text{shift}}^{(0,g)}$, $\delta_{\text{shift}}^{h} := \delta_{\text{shift}}^{(0,h)}$. For the final affine readout we use the uniform-notation convention $M^{(L,+)} = I$, $M^{(L,-)} = 0$; these are algebraic placeholders, not an additional gate. Standard backpropagation gives $\delta^{(l)} := \partial f^{(l)}/\partial a^{(l)} = \prod_{j=L,\dots,l+1} W^{(j,x)}$ with gated matrices $W^{(l,x)} := M^{(l,+)}W^{(l)}$.

Theorem 3.15 (Closed formulas for the stabilized split gradients, `Lean_41`). *With $\overline{\alpha}_j := \prod_{j \leq i \leq L}\big(1 - 2\alpha^{(i)}\big)$, the stabilized backward pass (3.32)–(3.34) computes, for every $0 \leq l \leq L$,*

$$\delta_{\text{shift}}^{(l,g)} = \frac{1}{2}\Big[\delta^{(l)} - \sum_{l \leq j < L} \overline{\alpha}_{j+1}\, W_{\text{abs}}^{[j+1,\to]}\big(M^{(j+1,-)}W^{(j+1)}\big) \prod_{i=(j-1),\dots,l} W^{(i+1,x)}\Big], \tag{3.36}$$

$$\delta_{\text{shift}}^{(l,h)} = -\frac{1}{2}\Big[\delta^{(l)} + \sum_{l \leq j < L} \overline{\alpha}_{j+1}\, W_{\text{abs}}^{[j+1,\to]}\big(M^{(j+1,-)}W^{(j+1)}\big) \prod_{i=(j-1),\dots,l} W^{(i+1,x)}\Big]. \tag{3.37}$$

The $j = L-1$ correction term is zero because $M^{(L,-)} = 0$: no minimum or auxiliary Q source is inserted at the affine readout. In particular, $\delta_{\text{shift}}^{(l,g)} - \delta_{\text{shift}}^{(l,h)} = \delta^{(l)}$ for all l; for $\alpha^{(i)} \equiv 0$ (i.e. $\overline{\alpha}_j = 1$) the

formulas describe the plain split-pair gradients, and for $\alpha^{(i)} \equiv 1/2$ all correction terms vanish and the stabilized quantities are $\pm\delta^{(l)}/2$, halves of the original gradient.

Proof. We record two auxiliary identities.

Claim 1 (bulk of the shifted sensitivities). For every l,

$$\delta^{(l,+,g)}_{\text{shift}} + \delta^{(l,-,g)}_{\text{shift}} = \prod_{l\le j\le L} \big(1 - 2\alpha^{(j)}\big)\, W^{[l,\to]}_{\text{abs}} = \overline{\alpha}_l\, W^{[l,\to]}_{\text{abs}}. \tag{3.38}$$

At $l = L$, $\delta^{(L,+,g)} + \delta^{(L,-,g)} = I = W^{[L,\to]}_{\text{abs}}$, and the shift multiplies the sum by $(1 - 2\alpha^{(L)})$, since subtracting $\alpha^{(L)}$ times the sum from *each* component subtracts $2\alpha^{(L)}$ times it from the sum. For the step, adding (3.32), (3.33) and using $M^{(l+1,+)} + M^{(l+1,-)} = I$, $W^{(l+1,+)} + W^{(l+1,-)} = \big|W^{(l+1)}\big|$,

$$\delta^{(l,+,g)}_{\text{int}} + \delta^{(l,-,g)}_{\text{int}} = \big(\delta^{(l+1,+,g)}_{\text{shift}} + \delta^{(l+1,-,g)}_{\text{shift}}\big)\big|W^{(l+1)}\big| = \overline{\alpha}_{l+1}\, W^{[l+1,\to]}_{\text{abs}}\big|W^{(l+1)}\big| = \overline{\alpha}_{l+1}\, W^{[l,\to]}_{\text{abs}}$$

by induction, and the shift (3.34) multiplies this by $(1 - 2\alpha^{(l)})$, giving $\overline{\alpha}_l\, W^{[l,\to]}_{\text{abs}}$.

Claim 2 (recursion of the stabilized difference). Subtracting (3.33) from (3.32)—the shift cancels in the difference—and regrouping with $M^{(l+1,+)} - M^{(l+1,-)}$ applied to $W^{(l+1)} = W^{(l+1,+)} - W^{(l+1,-)}$,

$$\begin{aligned} 2\,\delta^{(l,g)}_{\text{shift}} &= \big(\delta^{(l+1,+,g)}_{\text{shift}} - \delta^{(l+1,-,g)}_{\text{shift}}\big) M^{(l+1,+)} W^{(l+1)} - \big(\delta^{(l+1,+,g)}_{\text{shift}} + \delta^{(l+1,-,g)}_{\text{shift}}\big) M^{(l+1,-)} W^{(l+1)} \\ &= 2\,\delta^{(l+1,g)}_{\text{shift}}\, W^{(l+1,x)} - \overline{\alpha}_{l+1}\, W^{[l+1,\to]}_{\text{abs}} M^{(l+1,-)} W^{(l+1)}, \end{aligned} \tag{3.39}$$

where the first equality uses $M^{(l+1,+)} + M^{(l+1,-)} = I$ to sort the four terms into the two brackets, the second Claim 1 at $l + 1$.

Induction. At the base $l = L$ of (3.36), the initial shift cancels in the difference, so $\delta^{(L,g)}_{\text{shift}} = \big(\delta^{(L,+,g)} - \delta^{(L,-,g)}\big)/2 = I/2 = \delta^{(L)}/2$ with empty correction sum. For the step, unfolding (3.39) with the hypothesis at $l + 1$,

$$\begin{aligned} 2\,\delta^{(l,g)}_{\text{shift}} &= \Big[\delta^{(l+1)} - \sum_{l+1\le j<L} \overline{\alpha}_{j+1} W^{[j+1,\to]}_{\text{abs}} \big(M^{(j+1,-)} W^{(j+1)}\big) \prod_{i=(j-1),\dots,l+1} W^{(i+1,x)}\Big] W^{(l+1,x)} \\ &\quad - \overline{\alpha}_{l+1}\, W^{[l+1,\to]}_{\text{abs}} M^{(l+1,-)} W^{(l+1)} \\ &= \delta^{(l)} - \sum_{l\le j<L} \overline{\alpha}_{j+1}\, W^{[j+1,\to]}_{\text{abs}} \big(M^{(j+1,-)} W^{(j+1)}\big) \prod_{i=(j-1),\dots,l} W^{(i+1,x)}, \end{aligned}$$

using $\delta^{(l+1)} W^{(l+1,x)} = \delta^{(l)}$ and absorbing the extra term as summand $j = l$. Equation (3.37) is identical up to the sign flip $\delta^{(L,h)}_{\text{shift}} = -I/2$. Subtracting (3.37) from (3.36) cancels the correction sums, giving the backward invariant $\delta^{(l,g)}_{\text{shift}} - \delta^{(l,h)}_{\text{shift}} = \delta^{(l)}$; the cases $\alpha \equiv 0,\ 1/2$ are immediate from $\overline{\alpha}_j$. □

Remark 3.9 (Interpretation of the correction terms). The correction summands in (3.36) are gradients of an auxiliary network: define $Q[j](x) := W^{[j,\to]}_{\text{abs}} \min\{z^{(j)}(x), 0\}$, with the original prefix through layer j followed by a linear absolute-weight tail without biases or activation gates. In a bias-free tail this agrees in value with subsequent minimum activations, but the explicit definition also handles arbitrary prefix biases. Then the j-th summand equals $\overline{\alpha}_{j+1}\, \partial Q[j+1]/\partial x$ (at $l = 0$), capturing the mass that switches streams at layer $j + 1$. These gradients grow rapidly toward the input, so the factors $\overline{\alpha}_j$ act as a per-layer lever that both prevents gradient explosion and regulates how strongly each layer's

stream-switching enters the stabilized gradients. In practice all $\alpha^{(l)}$ equal a common α; for $\alpha = 1/2$ the sensitivities are exact halves of the original gradient, for $\alpha = 0$ the raw split-pair gradients, and $0 < \alpha < 1/2$ is the interesting regime. These stabilized gradients are the backbone of the split saliency methods in Chapter 6.

3.7 Generalization to Convolutional Neural Networks (CNNs)

Nothing in Definitions 3.1 and 3.2 is specific to dense layers: a decomposition only requires that every component propagate non-negative pairs through non-negative linear maps and pairwise non-linearities. We extend the DC split to standard CNNs (He et al., 2016; Simonyan and Zisserman, 2015), representing every component as a *chunk module* $(u^+, u^-) \mapsto (v^+, v^-)$ and keeping linear layers and ReLU gates as separate chunks, so that architectures with several linear operations between activations remain expressible; a converter maps supported computation graphs to the split form (Paszke et al., 2019). For each chunk we verify the same three properties—non-negative pairs map to non-negative pairs, the difference invariant $v^+ - v^- = \mathrm{op}(u^+ - u^-)$ holds, and (where applicable) monotonicity and convexity of the streams are preserved (Proposition 3.5)—and chaining them yields Theorem 3.19. Two chunks carry the substance, split convolution and gated max-pooling; every other supported chunk is a non-negative linear map applied streamwise, covered by a single lemma.

Convolution. A convolutional layer with kernel K and bias b computes $u \mapsto K * u + b$, with $*$ the (multi-channel) cross-correlation. Splitting the parameters entrywise, $K = [K]^+ - [K]^-$ and $b = [b]^+ - [b]^-$, we define the split chunk

$$v^+ \coloneqq [K]^+ * u^+ + [K]^- * u^- + [b]^+, \qquad v^- \coloneqq [K]^- * u^+ + [K]^+ * u^- + [b]^-. \tag{3.40}$$

Proposition 3.16 (Correctness of split convolution, `Lean_42`). *The chunk (3.40) maps non-negative pairs to non-negative pairs, each output component is a non-negative linear (affine) function of (u^+, u^-)—hence convex and non-decreasing in each coordinate—and*

$$v^+ - v^- = K * (u^+ - u^-) + b.$$

Proof. Cross-correlation with a non-negative kernel is a linear map with non-negative coefficients, so $v^\pm \geq 0$ for $u^\pm \geq 0$ and both components are affine with non-negative coefficients in (u^+, u^-), hence convex and non-decreasing in each coordinate. By linearity of $*$,

$$v^+ - v^- = \big([K]^+ - [K]^-\big) * u^+ - \big([K]^+ - [K]^-\big) * u^- + [b]^+ - [b]^- = K * (u^+ - u^-) + b. \qquad \square$$

Convolution followed by a ReLU chunk—the entrywise Maxout gate (3.12)—is covered by the argument of Theorem 3.6, a convolution being a dense layer with a sparse, weight-shared matrix. The same entrywise weight split handles every linear chunk at once.

Lemma 3.17 (Non-negative linear chunks preserve the invariants, `Lean_43`). *Let a chunk compute an affine map $u \mapsto Au + c$ with real matrix $A = [A]^+ - [A]^-$ and bias $c = [c]^+ - [c]^-$, split streamwise as*

$$v^+ \coloneqq [A]^+ u^+ + [A]^- u^- + [c]^+, \qquad v^- \coloneqq [A]^- u^+ + [A]^+ u^- + [c]^-.$$

Then the chunk maps non-negative pairs to non-negative pairs, each component is affine with non-negative coefficients in (u^+, u^-)—hence convex and non-decreasing in each coordinate—and $v^+ - v^- = A(u^+ - u^-) + c$.

Proof. Identical to Proposition 3.16: the coefficients $[A]^\pm, [c]^\pm$ are non-negative, and subtracting the two lines gives $([A]^+ - [A]^-)(u^+ - u^-) + [c]^+ - [c]^- = A(u^+ - u^-) + c$. □

Split convolution (3.40) is the instance $A = K * (\cdot)$; the remaining linear chunks are listed in Table 3.1. Residual connections combine the identity chunk ($A = I$) with an already-split branch by streamwise addition $v^\pm = u^\pm + F^\pm(u^+, u^-)$, preserving all three invariants whenever the branch does.

Tab. 3.1: Linear CNN chunks as instances of Lemma 3.17.

Chunk	Affine map	Notes
Average pooling	$A = \frac{1}{q}\sum_{j\in[q]}(\cdot)_j$, $c = 0$	non-negative A, applied streamwise
Batch norm (eval)	$A = \mathrm{diag}(c_{\mathrm{bn}})$, $c = d$	scale c_{bn} may be signed; 1×1 diagonal conv
Dropout (train)	$A = \mathrm{diag}(m)$, $c = 0$	mask $m \in \{0, \frac{1}{1-p}\}^d$ from the original pass
Residual add	$A = I$ plus split branch	streamwise sum with branch (F^+, F^-)

Two instances deserve a comment. Evaluation-mode batch normalization has a per-channel scale $c_{\mathrm{bn}} = \gamma/\sqrt{\sigma^2 + \epsilon}$ that may carry either sign (Guillemot and Heuselé, 2020; Hui and Binder, 2019); in practice we merge it into an adjacent convolution before splitting. Training-mode batch normalization, whose statistics depend on the batch, is genuinely multiplicative and requires the techniques of Chapter 5. Training-mode dropout uses the *same* realised mask m on both streams and on $u = u^+ - u^-$, so the split network drops exactly the units the original does.

Gated max-pooling. Max-pooling over a window $[q]$ computes $\max_{j\in[q]} u_j$ per channel, which is neither linear nor a shared gate on a single pair. It is a CPWL function of the window and admits its own convexity-preserving splitting identity.

Proposition 3.18 (Correctness of gated max-pooling, `Lean_44`). *For a pooling window $[q]$, define the split chunk*

$$v^+ := \max_{j\in[q]} \Big\{ u_j^+ + \sum_{i\in[q],\, i\neq j} u_i^- \Big\}, \qquad v^- := \sum_{i\in[q]} u_i^- . \tag{3.41}$$

Then $v^\pm \geq 0$ for $u^\pm \geq 0$, both components are convex and non-decreasing in each coordinate of (u^+, u^-), and

$$v^+ - v^- = \max_{j\in[q]} \big(u_j^+ - u_j^-\big). \tag{3.42}$$

Proof. Non-negativity is clear, as every candidate and the sum are sums of non-negative inputs; both components are non-negative-coefficient linear combinations and an entrywise maximum, hence convex and non-decreasing. For (3.42), add and subtract u_j^- in each candidate: for every $j \in [q]$,

$$u_j^+ + \sum_{i\neq j} u_i^- = \big(u_j^+ - u_j^-\big) + \sum_{i\in[q]} u_i^-,$$

and since the second summand does not depend on j, taking the maximum over j and subtracting $v^- = \sum_i u_i^-$ yields $\max_j \big(u_j^+ - u_j^-\big)$. □

Identity (3.42) generalizes (3.11), the case $q = 2$ with the pair $\big(u^+ - u^-, 0\big)$. Optionally, (3.41) can be replaced by a *winner-takes-it-all* rule forwarding, in both streams, the entries where the *difference* is maximal; this preserves the difference invariant but sacrifices monotonicity and convexity, mirroring the shared-gate trade-off of Remark 3.5.

Implementation and caches. The pinned implementation uses PyTorch FX to trace a model and organize supported graph operations into decomposition chunks. A forward/backward cache accumulates inflowing activations and gradients per chunk, and a parallel cache mirrors the original model. The latter serves both analysis—layer-wise activation/gradient error—and optional on-the-fly correction, and reuses the original model's positivity patterns in place of drifted split-pair maxima in the backward pass. The current code provides configurable `float16`, `float32`, or `float64` arithmetic, with `float32` as its default; this configuration choice is distinct from the double-precision setup reported in the original paper. The graph conversion applies to the operations supported by its chunk implementations.

Alternative nonlinearities and exactness. The source implementation discussion proposes replacing alternative nonlinearities such as ELU (Clevert et al., 2016) or Swish (Ramachandran et al., 2017) by small ReLU subnetworks before applying the split procedure. Such a replacement is an approximation: a finite ReLU subnetwork is piecewise affine and cannot reproduce these non-piecewise-affine functions exactly. The resulting split preserves the approximating network, while the approximation error relative to the original activation is a separate quantity. The pinned repository contains a separate approximation utility for selected scalar operations, including Swish, but that utility is not invoked by the inspected hybrid conversion path and does not list ELU among its supported operations. The curvature construction of Section 2.4.4 instead provides exact scalar decompositions for activations satisfying its hypotheses, without replacing them by ReLU subnetworks.

The CNN split pass. The split forward pass processes the chunks of a computation graph $\mathscr{G}$ in topological order, applying to each the split rule of its type—Lemma 3.17 for the linear chunks, the Maxout gate (3.12) or max-pool split (3.41) for the non-linear ones—with optional entrywise re-centering $v^{\pm} \leftarrow v^{\pm} - \min\{v^+, v^-\}$ after each chunk. The stabilization of Section 3.6 applies unchanged, since Theorem 3.13 uses only non-negative linearity and the Maxout gate structure.

Theorem 3.19 (Correctness of the CNN split pass, `Lean_45`). *Let $\mathscr{G}$ be a feedforward computation graph composed of the chunk types above, computing the function $x \mapsto f(x)$. Then for every input split $x = x^+ - x^-$ with $x^{\pm} \geq 0$, the split pass produces non-negative pairs satisfying the difference invariant*

$$v_m^+ - v_m^- = v_m \qquad \text{for every chunk } C_m,$$

where v_m is the corresponding original activation; in particular $f(x) = v_M^+ - v_M^-$. If neither winner-takes-it-all pooling nor input-dependent re-centering is used, every $v_m^{\pm}$ is convex in (x^+, x^-) and non-decreasing in each coordinate.

Proof. Induction over the topological order of $\mathscr{G}$. At the input node the invariant is the input convention, non-negativity holds by assumption, and $x_k^{\pm}$ are affine, convex, non-decreasing. For the step, assume all predecessors of C_m satisfy the three properties. Lemma 3.17 (linear chunks), Proposition 3.18 (max-pool), and Corollary 3.4 with (3.11) (the ReLU gate) establish for every type that (i) non-negative pairs map to non-negative pairs, (ii) the output difference equals the original operation

on the input difference—by the invariant the activation v_m—and (iii) each output component is convex and non-decreasing, being built from non-negative linear combinations and entrywise maxima whose compositions preserve both properties (Amos et al., 2017). Re-centering subtracts a common shift, preserving the invariant and non-negativity (Theorem 3.13); it is omitted when convexity and coordinatewise monotonicity are required, since these properties need not survive the subtraction. Already re-centering the pair $(\max\{p,n\},n)$ gives $(\mathrm{ReLU}(p-n),0)$, whose first component decreases when n increases with p fixed. The output claim is the invariant at C_M. □

This completes the decomposition toolbox: a general validity framework, a one-parameter family with a convex DC endpoint, the curvature-based smooth-activation splitting of Section 2.4.4, a hard-gated and a regularized soft-gated decomposition, numerically stabilized forward and backward passes, and a chunk calculus for standard convolutional architectures. Chapter 4 builds the game-theoretic layer on the Stopping Decomposition, and Chapter 6 turns the split streams g and h into attribution methods.

4 Game-Theoretic Framework

The sign-separated streams of Chapter 3 expose how opposing contributions accumulate. This chapter asks how a backward rule traverses those contributions. The stopping decomposition gives a common state graph for two games whose policies represent gradients and relevance propagation. Their trajectory distributions make the backward calculation available for both method design in Chapter 6 and diagnostics in Chapter 8. Exact recovery results concern the anchor rules; myopic temperature changes and risk-sensitive policies are controlled deformations of those rules. The detailed definitions and proofs are included in this chapter.

4.1 Related Work

Ancona et al. (2018) unify several gradient-based rules as modified gradients, a map-level unification that leaves out the $\alpha\beta$ rule; we instead seek a signed trajectory-space representation of gradients and the $\alpha\beta$ rule that exposes their routing laws. Combined with the split representation, this distinguishes two choices that attribution methods make: which internal signals to retain and how to propagate them backward.

Game and trajectory views. The closest mathematical precursor is the ReLU Net Game of Gaubert & Vlassopoulos (2025), which identifies the forward evaluation of a ReLU network with the value of a backward Stopping Game but does not connect that game to the backward pass or to attribution. We reformulate the game to draw the connection to a novel Stopping Decomposition, whose inherent weight decomposition is related to the DC decomposition of neural networks—the decomposition Zimmermann & Loho (2026) use directly for attribution. Our SG and RG are different games utilizing this weight decomposition, chosen to recover backward attribution rules rather than the pure DC decomposition. The NRM framework of Xiong et al. (2026) also starts from walks through the network, assigning them normalized signed relevance. Section 8.1 compares its treatment of LRP and concept relevance propagation with our trajectory probabilities and discounted occupation measures.

Relevance propagation and stochastic explanations. LRP, Deep Taylor, and Excitation Backprop constitute the core relevance propagation family (Bach et al., 2015; Montavon et al., 2017; Zhang et al., 2018a), with later variants modifying redistribution for contrast, stability, or signed relevance (Binder et al., 2016; Gu et al., 2019; Kohlbrenner et al., 2020; Nam et al., 2020; Vukadin et al., 2024). We supply a common trajectory-space semantics for this family: SG recovers gradients, RG recovers the generalized $\alpha\beta$ rule, and its analytical modes map back to principled modifications of existing backward rules. Excitation Backprop (Zhang et al., 2018a) is an absorbing Markov chain recovering the $\alpha\beta$-LRP rule for the special case $(\alpha, \beta) = (1{,}0)$ known as the z^+-rule, which only considers the positive weights in the relevance propagation. Our game extends it with an opponent player (enabling negative-weight influence) and replaces input-dependent transition probabilities with soft Bellman equations. Deep Taylor (Montavon et al., 2017) justifies that same z^+-rule and explicitly leaves the mathematical modeling of the general $\alpha\beta$ rule as an open problem; RG closes this.

Shapley-value attribution methods. SHAP and KernelSHAP frame explanation as a cooperative game over feature coalitions (Lundberg and Lee, 2017) in a *black-box* setting; they are therefore a distinct, complementary game that explains marginal input-feature contributions relative to a baseline, whereas our backward game explains how relevance is routed through network internals in a *white-box* setting.

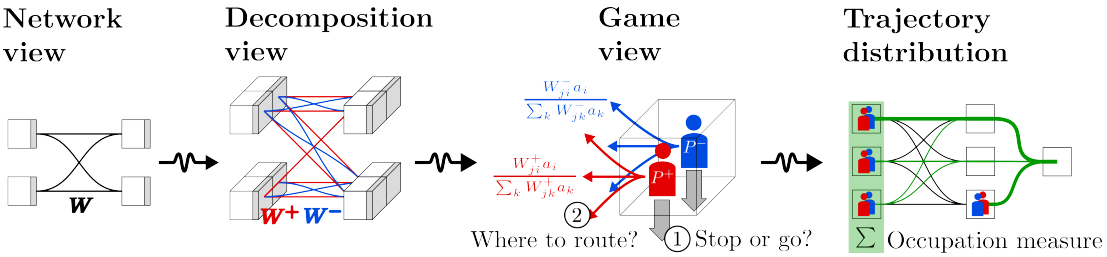


Fig. 4.1: The backward pass as a two-player game. The Stopping Decomposition splits every weight by sign, $W = W^+ - W^-$; at each state of the extended graph the players decide (1) stop or continue and (2) where to route. Their equilibrium policies induce a distribution over backward trajectories, and the discounted occupation measure of that distribution at the input states is the attribution map.

Robust explanations via Softplus. Dombrowski et al. (2022) propose Softplus replacement as a curvature-based defense against gradient-explanation manipulation. Section 4.9 shows that entropy-regularizing the activation-state stop/continue choice in SG recovers Softplus-network gradients, situating their construction within our framework.

Interpretability in Transformers and ViTs. Attention Rollout / Flow read attention as a routing graph (Abnar and Zuidema, 2020); transformer-LRP methods propagate through attention and skip connections (Achtibat et al., 2024; Ali et al., 2022; Chefer et al., 2021b); DAVE addresses ViT tokenization artifacts via conditioned operators and equivariant averaging (Wróbel et al., 2026). The conditioned-operator transformer constructions arise naturally inside our game semantics: the QK-softmax is the oracle player's reference policy, while attribution lives on the Value-routing trajectories. Hence, our method is compatible with Reynolds averaging, which in our RG+Eq mode removes the same grid artifact that DAVE targets—however, with more control via the different game modes.

4.2 Backward Attribution

For the game derivations in this and the attention and trajectory chapters, $W_{ij}^{(l)}$ denotes the edge from predecessor i to successor j. Thus the displayed adjacency matrices are the transposes of the output-by-input matrices used in the DC split recurrences; the edge weights and represented network are identical. Stopping streams are tagged by stop to distinguish them from the DC streams. Consider a ReLU network with layers $l = 1,\dots,L$, input $a^{(0)} = x$, pre-activations $z_j^{(l)} = \sum_i a_i^{(l-1)} W_{ij}^{(l)} + b_j^{(l)}$ and activations $a_j^{(l)} = \mathrm{ReLU}(z_j^{(l)})$, where $W_{ij}^{(l)}$ connects neuron i of layer $l-1$ to neuron j of layer l. We explain a scalar output $f(x) = a_u^{(L)}$. We write $[u]^+ = \max(u,0)$ and $[u]^- = \max(-u,0)$, so $u = [u]^+ - [u]^-$, and $W^{(l,\pm)} = [W^{(l)}]^\pm$ entrywise.

Layer-wise relevance propagation. LRP (Bach et al., 2015) explains $f(x)$ by redistributing it backward, layer by layer: starting from $R_u^{(L)} = f(x)$, every neuron passes its relevance $R_j^{(l)}$ on to its inputs in proportion to their contributions $a_i^{(l-1)} W_{ij}^{(l)}$ to its pre-activation, so that relevance is conserved from layer to layer up to the share absorbed by biases and stabilizers. Rules differ in how the contributions are weighted. The $\alpha\beta$-rule is parameterized by $\alpha,\beta \geq 0$, with $\alpha - \beta = 1$ giving conservation when both participating sign streams are nonempty and all bias, stabilizer, and empty-stream absorption is accounted for, and a stabilization parameter $\varepsilon \geq 0$ (added to each sign denominator, as in common implementations (Anders et al., 2023)): the relevance of neuron j is split into a positive-contribution

fraction scaled by α and a negative-contribution fraction scaled by β, each normalized by its own denominator,

$$R_i^{(l-1)} = \sum_j \Big(\alpha \, \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^+}{\Sigma_j^{(l,+)} + \varepsilon} - \beta \, \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^-}{\Sigma_j^{(l,-)} + \varepsilon} \Big) R_j^{(l)},$$
$$\Sigma_j^{(l,\pm)} = \sum_k [a_k^{(l-1)} W_{kj}^{(l)}]^\pm + [b_j^{(l)}]^\pm. \tag{4.1}$$

Relevance assigned to the bias and to ε is absorbed. $(\alpha,\beta) = (1,0)$ gives the z^+-rule of Deep Taylor decomposition (Montavon et al., 2017), which turns out to be the same rule as Excitation Backprop (Zhang et al., 2018a); other rules (LRP-ε, LRP-γ) change the redistribution fractions.

Transformers. In a Vision Transformer (ViT), LayerNorm couples the features of a token through its input-dependent mean and variance, and the attention matrix $A = \mathrm{softmax}(QK^\top/\sqrt{d})$ depends on the input, which breaks conservation (Ali et al., 2022). Transformer-specific rules therefore *condition* on the forward pass: conservative propagation (Ali et al., 2022) and DAVE (Wróbel et al., 2026) hold A and the LayerNorm statistics fixed and propagate relevance through the Value path only; AttnLRP (Achtibat et al., 2024) also propagates through the softmax and the QK product.

All these rules share one structure: a backward recursion that routes mass from each neuron to its inputs. The games below make the routing explicit.

4.3 Attribution as a Game

Setup and common state-space structure. With the notation of Section 4.2, we build a finite-horizon two-player game (Osborne and Rubinstein, 1994) played *backward* from the output: a state records a position in the backward traversal (layer, neuron, plus game-specific bookkeeping for player turn and stream sign), and at each non-terminal state the active player either stops, moving to an absorbing cemetery state †, or continues. Skip connections become random transitions to summand states; max-pooling sends players greedily to maximal-payoff states. Section 4.5 collects the bookkeeping labels (Table 4.2). The SG value $V_p(s)$ is the γ-discounted expected payoff difference $r_p - r_{p'}$ of player p and opponent p' along trajectories from s under the policies, expressed for the player p in turn at s by the Bellman recursion

$$V_p(s) = \max_{a \in \mathscr{A}_x(s)} \Big\{ r_p(s,a) - r_{p'}(s,a) \\ + \gamma(s) \sum_{s'} \mathbb{P}_x(s' \mid s,a)\, V_p(s') \Big\}, \tag{4.2}$$

whereas the **R**outing **G**ame replaces the hard maximization at routing states by an entropy-regularized maximization over mixed routing decisions, whose value is the soft game value (4.5). Any player policies induce a backward trajectory measure Π_x on trajectories $\zeta = (\zeta_0, \zeta_1, \dots)$ with a game-specific cumulative discount $d_t(\zeta)$ up to time t. The **discounted occupation measure** (Puterman, 1994)

$$\Gamma_x(v) \coloneqq \mathbb{E}_{\zeta \sim \Pi_x} \Big[\sum_t d_t(\zeta)\, \mathbf{1}\{\zeta_t = v\} \Big] \tag{4.3}$$

quantifies how much weight the trajectories going through a state have.

4.3.1 The Stopping Game

With each neuron j in layer l, we associate two states $s^{(l)}_{j,+}$ and $s^{(l)}_{j,-}$, where the sign encodes which player is in turn. The biases become the immediate payoffs for continuing; under continuation, weight magnitudes drive the environment transitions with discount $\gamma^{(l)}_j = \sum_i |W^{(l)}_{ij}|$ (positive- and negative-weight predecessors follow $\mathbb{P} \propto W^{(l,\pm)}/\gamma^{(l)}_j$, and a negative weight passes the turn to the opponent). The players' expected payoffs realize the novel Stopping Decomposition, separating positive and negative paths to each neuron activation (Lemma 4.4). The gradient is then the difference of two *non-negative* occupation measures:

Theorem 4.1 (Gradient representation, `Lean_46`**).** *The Stopping Game allows a subgame-perfect Nash equilibrium, unique up to tie-breaking at zero-activation neurons. On strict gate cells (and with unique max-pooling winners where applicable), under the stop-at-tie-breaks equilibrium the discounted occupation measure* $\Gamma_x(s^{(l)}_{i,p}) \geq 0$ *equals the output derivative with respect to the corresponding* Stopping Decomposition *activation. At ties, the corresponding identities refer to the backward quantities selected by the stated gate and winner conventions. On these cells, the gradient of* f *with respect to the input is the difference* $\Gamma_x(s^{(0)}_{i,+}) - \Gamma_x(s^{(0)}_{i,-})$.

Section 4.6 proves the theorem, works through an example, gives the full state space with skip connections and max-pooling, and compares the Stopping Decomposition with the DC decomposition.

Modeling $\alpha\beta$-LRP or attention modules via the SG would require activation-dependent transitions. The RG moves activation-dependence into the players' policies, opening a continuous family of deformations.

4.3.2 The Routing Game

In the RG we model every neuron by three game states: an activation state $s^{(l,\mathrm{act})}_{j,q}$ and two linear routing states $s^{(l,\mathrm{lin})}_{j,q,+}$ and $s^{(l,\mathrm{lin})}_{j,q,-}$, the $+$ stream state and the $-$ stream state, modeling the positive part $W^{(l,+)}$ and the negative part $W^{(l,-)}$ of the affine layer respectively. At an activation state the player in turn stops or continues to both streams with probability $\frac{1}{2}$ each; the $-$ stream passes the turn to the opponent, and discounting the game value by 2 compensates the split. At a *linear routing state* $s^{(l,\mathrm{lin})}_{j,q,\kappa}$ for neuron j in layer l on sign stream $\kappa \in \{+,-\}$ with player q in turn, the active player mixes over predecessors i, a bias node and an outside option $\perp_{\mathrm{lin}}$ under a policy $\pi_q(s^{(l,\mathrm{lin})}_{j,q,\kappa})$; the bias node and $\perp_{\mathrm{lin}}$ absorb the trajectory, with log-payoffs $\log[b^{(l)}_j]^\kappa$ and $\log \varepsilon$. Writing the game in log space turns the active player r's advantage of moving to predecessor i into an additive action–value,

$$Q_r\big(s^{(l,\mathrm{lin})}_{j,q,\kappa}, i\big) = \begin{cases} \log W^{(l,\kappa)}_{ij} + V_r(s^{(l-1,\mathrm{act})}_{i,q}), & r{=}q, l{\geq}2, \\ \log\big[x_i W^{(1)}_{ij}\big]^\kappa, & r{=}q, l{=}1, \\ -\infty, & r \neq q, \end{cases} \tag{4.4}$$

where $V_r(\cdot)$ denotes the *game value* at the successor activation state from the perspective of player r. At each linear routing subgame we regularize the players' policies with Shannon entropy (Geist et al., 2019), which yields the following equilibrium value and Gibbs best response:

$$V_q\big(s^{(l,\mathrm{lin})}_{j,q,\kappa}\big) = \tau \log \textstyle\sum_i \exp\big(Q_q(s^{(l,\mathrm{lin})}_{j,q,\kappa}, i)/\tau\big), \tag{4.5}$$

$$\pi_q(s^{(l,\mathrm{lin})}_{j,q,\kappa})(i) \propto \exp\big(Q_q(s^{(l,\mathrm{lin})}_{j,q,\kappa}, i)/\tau\big), \tag{4.6}$$

where the sums include the bias node and $\perp_{\text{lin}}$. At $\tau > 0$ the game admits an SPNE whose occupation measure at $\tau = 1$ is the $\alpha\beta$-LRP attribution:

Theorem 4.2 ($\alpha\beta$-LRP representation, Lean_47). *For $\tau > 0$, the Gibbs policies of Eq. (4.6) and stopping decisions from the corresponding continuation values yield a canonical SPNE of the entropy-regularized RG. We select stopping at every activation log tie and absorption for an empty routing stream, as specified in Remark 4.2. At the anchor $\tau = 1$, the selected gate is $\mathbf{1}[z > 0]$ and the $\alpha\beta$-LRP attribution map in (4.1) equals $R_i^{(l)} = f(x)\left(\Gamma_x(s_{i,+}^{(l)}) - \Gamma_x(s_{i,-}^{(l)})\right)$.*

Section 4.7 gives the full game definition, the proofs, and a worked example of the local routing dynamics (Figure 4.3).

In practice we use τ as a *myopic* control: each linear subgame applies the τ-scaled Gibbs policy to its own action values, but inherits game values from the soft Bellman equations at τ=1. We do not re-solve at τ downstream: that would change the forward activations, while we want trajectories that explain the network's actual forward pass. The myopic setting at $\tau \neq 1$ is a controlled temperature deformation of the occupation measure. Low τ concentrates routing on the dominant pathway.

4.4 Game-Theoretic Modes

Game-theoretic concepts like risk aversion and entropy regularization map back to novel modes of the backward rules for probing and tuning explanation properties (Table 4.1). The exact representations of Section 4.3 are the defaults; every mode other than the LRP parameters (α,β,ε) is a deformation that is off at the anchor. We use five: the temperature τ (focus), risk aversion λ at activation states and its attention-logit variant λ_{sm}, the probit gate $\tilde{\Phi}$ (gate smoothing), and equivariant averaging over N input transformations (grid artifacts).

Focus. The myopic temperature of Section 4.3.2 rescales the Gibbs policy at every linear state while keeping the τ=1 game values. Low τ concentrates routing on dominant predecessors; high τ spreads mass more uniformly, exposing broader contextual features as well as architectural priors. *Risk aversion.* *Assumed Density Filtering* (ADF; (Gast and Roth, 2018)) propagates a Gaussian mean μ and variance σ^2 for every pre-activation z from an input variance σ^2 in one additional moment pass. The risk-averse player replaces the observed z at activation states by the lower-confidence bound $\mu + \lambda\sigma$ with $\lambda < 0$, the pessimistic counterpart of confidence-bound exploration (Auer et al., 2002; Fei et al., 2020), and so attenuates routes through high-variance neurons; linear-state routing is unchanged. On attention blocks, λ_{sm} applies the same shift to the oracle's attention logits before the softmax, downweighting high-variance key tokens. *Gate smoothing.* The risk-averse stop/continue rule is the hard gate $\mathbf{1}[\mu/\sigma+\lambda > 0]$. The probit gate $\tilde{\Phi}$ lets the player instead continue with probability $\Phi(\mu/\sigma+\lambda)$, the marginal of a player who perceives the continuation payoff as $z+\eta$, $\eta \sim \mathcal{N}(0,1)$, and continues iff it is positive. At μ=z, σ=1, λ=0 this probit-regularized choice recovers the Gaussian-smoothed ReLU $\mathbb{E}[\text{ReLU}(z+\eta)]$, just as entropy regularization recovers Softplus (Section 4.9); on ReLU networks it is a smoothing deformation. Because the gate uses the ADF σ, σ^2 is active whenever $\tilde{\Phi}$ is on. *Exact modes and cost.* (α,β) weight the two streams of the occupation measure, and the LRP stabilizer ε adds an outside option $\perp_{\text{lin}}$ with log-score $\log\varepsilon$ at every linear state, leaking weak-evidence neurons into the cemetery while leaving strong-evidence ones near-unaffected (Theorem 4.10). The exact modes and τ add no cost; λ, λ_{sm} and $\tilde{\Phi}$ add one ADF pass (0.10 → 0.17 s per ViT-B/16 image); averaging multiplies the cost by N (4.93 s at N=50, 8.66 s with λ_{sm}). Section 4.8 derives the modes and gives defaults, costs and an order of use.

Tab. 4.1: Modes of the games. "Exact": the mode stays within a represented rule; "deform.": optional deformation, off (default value) at the anchor. σ^2: input variance of the ADF moment propagation; N: number of transformed inputs averaged by RG+Eq.

Mode	Effect on the game	Default	Property
(α,β)	branch discounts of the two streams (exact)	(2,1)	support / oppose
ε	outside option $\perp_{\text{lin}}$, payoff $\log \varepsilon$ (exact)	0.5	damps low-inflow neurons
τ	entropy regularization of routing (deform.)	1	sharp / diffuse
λ, σ^2	values $\to$ lower-confidence bounds (deform.)	λ=0	noise / fragility
λ_{sm}	attention logits shifted by λ_{sm} times a per-key ADF std (deform.)	0	stable attention
$\tilde{\Phi}$	noise on player observations (deform.)	off	gate smoothing
N	average over input transformations (deform.)	off	grid artifacts

4.5 Notation and Conventions

Throughout, $p, q, r \in \{+, -\}$ denote player or turn labels of the two players P^+ and P^-, and p' denotes the opponent of p ($+' = -$, $-' = +$); Table 4.2 lists all symbols, grouped as network, games, Routing Game and attention, modes and smooth activations, and Hellinger distance; each has a single meaning. We write $\delta_{p,q} = 1$ if $p = q$ and 0 otherwise, the sign operator $\xi_{p,q} = 2\delta_{p,q} - 1$ ($+1$ if $p = q$, -1 if $p \neq q$), and the XOR operator $p \oplus q = +$ if $p = q$ and $-$ if $p \neq q$; it is commutative and associative, with $p \oplus p = +$, $p \oplus + = p$, $p \oplus - = p'$, and $p' \oplus q = (p \oplus q)'$. **Boundary conventions.** We fix the selected backward value $\text{ReLU}'(0) \coloneqq 0$ (an algorithmic convention, since ReLU has no classical derivative at zero); equivalently, the indicator $\mathbf{1}[z > 0]$ used throughout returns 0 at $z = 0$. For a finite tuple $(x_1, \ldots, x_m)$, $\arg\max_k x_k \coloneqq \min\{k : x_k = \max_l x_l\}$, the smallest index attaining the maximum. We reserve the word *trajectory* for sequences of game states $\zeta = (\zeta_0, \zeta_1, \ldots)$ and use *path* for sequences of neuron indices.

Tab. 4.2: Notation. Each symbol has one meaning throughout the game and trajectory chapters, except the local attention symbols $Q_t, K_k, e_{tk}, s_{tk}, m, s, c, T_n$ and the token index t, defined in Section 5.4.

Symbol	Meaning	Symbol	Meaning
l, L	layers; input 0 ($a^{(0)} = x$), output L	$W_{ij}^{(l)}$, $b_j^{(l)}$	weight from i (layer $l-1$) to j (layer l); bias
$z_j^{(l)}$, $a_j^{(l)}$	$\sum_i a_i^{(l-1)} W_{ij}^{(l)} + b_j^{(l)}$; $\mathrm{ReLU}(z_j^{(l)})$	$f(x) = a_u^{(L)}$	scalar output (target logit u)
$[t]^{\pm}$, $x_k^{\pm}$	$\max\{\pm t, 0\}$, so $t = [t]^+ - [t]^-$; $x_k^{\pm} = [x_k]^{\pm}$	$W^{(l,\pm)}$, $b^{(l,\pm)}$	$[W^{(l)}]^{\pm}$, $[b^{(l)}]^{\pm}$ (sign split)
$z_j^{(l,\pm)}$, $a_{\mathrm{stop},j}^{(l,\pm)}$	sign split of z; Stopping Decomposition	$\nabla^2 f$	Hessian
p, q, r; p'	players / turns in $\{+,-\}$; opponent	$\kappa \in \{+,-\}$	stream sign of a linear routing state
$s_{j,q}^{(l,\mathrm{act})}$, $s_{k,q}^{(0)}$	activation state (q in turn); input terminal	$s_{j,q,\kappa}^{(l,\mathrm{lin})}$	RG linear routing state
$\dagger$	cemetery (absorbing) state	$\perp_{\mathrm{lin}}$	outside action (log-payoff $\log \varepsilon$)
$r_p(s,a)$	payoff of player p	π, $\pi^\star$	policy; equilibrium (SPNE) policy
$\tilde{V}_p(s)$, $\tilde{Q}_p(s,a)$	SG player-specific value, action value	$V_p(s)$	game value (player p minus p')
$Q_r(s,i)$	RG log-gain of routing to predecessor i	$\Sigma_j^{(l,\kappa)}$	$\sum_i [a_i^{(l-1)} W_{ij}^{(l)}]^\kappa + [b_j^{(l)}]^\kappa$
$\gamma_j^{(l)}$	SG discount $\sum_i \lvert W_{ij}^{(l)} \rvert$	γ_{O}	Oracle discount (2 add./branch, 1 max-pool)
$\gamma(s)$	per-state discount: SG $\gamma_j^{(l)}$ or γ_{O}; RG $\in \{2\alpha, 2\beta, 1\}$	ζ	trajectory $(\zeta_0, \zeta_1, \dots)$
Π_x	equilibrium trajectory law at input x	$d_t(\zeta)$	cumulative trajectory discount
$\Gamma_x(s)$	discounted occupation measure of state s	$\mathscr{H}$, $\mathscr{H}_\theta$	Shannon entropy; binary form
τ	RG temperature (entropy weight)	α, β	$\alpha\beta$-LRP weights, $\alpha - \beta = 1$
ε	LRP stabilizer	$R_i^{(l)}$	LRP relevance
A	attention matrix (held fixed)	v, W_v	Value vectors, projection (sup. "val")
n_h	number of attention heads	N	equivariant-averaging samples
λ	activation-side risk aversion ($\lambda \le 0$)	λ_{sm}	attention-logit risk aversion
μ, σ, σ^2	ADF mean, std.; input variance	λ_{ent}	fan-in entropy-bias mode
sp_θ, sp'_θ	Softplus $\theta \log(1+e^{z/\theta})$; its gate	Φ, ϕ	normal *Cumulative Distribution Function* (CDF), density
θ; $\tilde{\Phi}$	Softplus temperature; soft gate $\Phi(\mu/\sigma + \lambda)$	$\mathscr{R}_\Phi$	probit regularizer $\phi(\Phi^{-1}(\pi))$
H, $\mathrm{H}_{\mathrm{surv}}$	Hellinger distance; survival-conditioned	BC	Bhattacharyya coefficient
$T_M^{(l)}$	backward kernel of model M	$m_M^{(l)}(s)$	marginal occupation probability
$g^{(l)}(s)$	overlap mass (backward pass)	$c^{(l)}(s)$	continuation mass (forward pass)
$\mathrm{H}^2(s)$	per-terminal-state disagreement	$\psi_M^{(l)}(s)$	survival probability

4.6 Stopping Game

This section realizes the activations of the Stopping Decomposition (4.10) as player-specific game values of a backward two-player Stopping Game on an extended network graph and demonstrates that gradients of the original network can be interpreted as integrals over game trajectories.

4.6.1 Stopping Decomposition

Throughout we consider a feedforward ReLU network of L layers with input layer 0, $a^{(0)} = x$, pre-activations $z_j^{(l)} = \sum_i a_i^{(l-1)} W_{ij}^{(l)} + b_j^{(l)}$, where $W_{ij}^{(l)}$ connects neuron i in layer $l-1$ to neuron j in layer l, activations $a_j^{(l)} = \mathrm{ReLU}(z_j^{(l)})$, and a *single scalar output* $f(x) = a_u^{(L)}$ (for multi-class networks, fix the target logit u).

Canonical sign split. We use the element-wise positive/negative decomposition of the original network's weights and biases,

$$W^{(l,+)} := [W^{(l)}]^+, \qquad W^{(l,-)} := [W^{(l)}]^-, \qquad b^{(l,+)} := [b^{(l)}]^+, \qquad b^{(l,-)} := [b^{(l)}]^-, \tag{4.7}$$

which maps non-negative activation pairs $(a_i^{(l-1,+)}, a_i^{(l-1,-)})$ to the pair of pre-activations at neuron j,

$$z_j^{(l,+)} := b_j^{(l,+)} + \sum_i W_{ij}^{(l,+)} a_i^{(l-1,+)} + \sum_i W_{ij}^{(l,-)} a_i^{(l-1,-)}, \tag{4.8}$$

$$z_j^{(l,-)} := b_j^{(l,-)} + \sum_i W_{ij}^{(l,+)} a_i^{(l-1,-)} + \sum_i W_{ij}^{(l,-)} a_i^{(l-1,+)}. \tag{4.9}$$

When $a_i^{(l-1,+)} - a_i^{(l-1,-)} = a_i^{(l-1)}$ for all i, this yields the identity $z_j^{(l,+)} - z_j^{(l,-)} = b_j^{(l)} + \sum_i (W_{ij}^{(l,+)} - W_{ij}^{(l,-)})(a_i^{(l-1,+)} - a_i^{(l-1,-)}) = z_j^{(l)}$, so the layer-$l$ pre-activation pair already encodes the original network's pre-activation through its difference.

The **Stopping decomposition function** $f_{\text{stop}} \colon \mathbb{R}^2_{\geq 0} \to \mathbb{R}^2_{\geq 0}$ is

$$f_{\text{stop}}(z^+, z^-) \;:=\; \big(\mathbf{1}\big[z^+ - z^- > 0\big]\, z^+,\; \mathbf{1}\big[z^+ - z^- > 0\big]\, z^-\big). \tag{4.10}$$

Applied at every neuron and propagated by (4.8)–(4.9), it defines the **Stopping Decomposition** activations $\big(a_{\text{stop},j}^{(l,+)}, a_{\text{stop},j}^{(l,-)}\big) = f_{\text{stop}}\big(z_j^{(l,+)}, z_j^{(l,-)}\big)$, with the input-layer initialization

$$a_{\text{stop},k}^{(0,\pm)} = x_k^{\pm}. \tag{4.11}$$

Theorem 4.3 (Layer-wise Recovery of the Original Activations, `Lean_48`**).** *For every layer l and neuron j,*

$$a_{\text{stop},j}^{(l,+)} - a_{\text{stop},j}^{(l,-)} = a_j^{(l)} = ReLU\big(z_j^{(l)}\big). \tag{4.12}$$

Proof. Induction on l. At $l = 0$, $x_k^+ - x_k^- = x_k = a_k^{(0)}$ by (4.11). At layer $l \geq 1$ the identity above gives $z_j^{(l,+)} - z_j^{(l,-)} = z_j^{(l)}$, hence $a_{\text{stop},j}^{(l,+)} - a_{\text{stop},j}^{(l,-)} = \mathbf{1}\{z_j^{(l)} > 0\}\big(z_j^{(l,+)} - z_j^{(l,-)}\big) = \text{ReLU}(z_j^{(l)}) = a_j^{(l)}$. □

ReLU networks compute continuous piecewise linear functions (Mukherjee et al., 2018), which admit a DC decomposition (Zimmermann and Loho, 2026); it uses the same sign split (4.8)–(4.9) with the pairwise activation $(\max\{z^+, z^-\}, z^-)$ in place of f_{stop} and yields components that are convex in x. The Stopping Decomposition instead gates both streams by the same indicator, so its components are not convex, but on each linear region they are non-negative affine, hence monotone, in (x^+, x^-).

Parity path decomposition of the split streams. We can decompose the Stopping Decomposition activations into small contributions coming from individual paths through the network: each non-negative decomposed activation at neuron j is the sum, over all backward paths from j to the input, of the absolute weight product along the path times either the positive part $x_{P_0}^+$ or the negative part $x_{P_0}^-$ of the input pixel where the path terminates, with the choice between $x_{P_0}^+$ and $x_{P_0}^-$ determined by the parity of the number of negative weights traversed.

Lemma 4.4 (Path view on the Stopping Decomposition, Lean_49**).** *Fix a feedforward ReLU network with weights $W^{(l)}$ (biases ignored; see remark at end of proof). For neuron j in layer $l \geq 1$, call a* backward path *from j to the input layer a list of neuron indices*

$$P = (P_l, P_{l-1}, \dots, P_0), \qquad P_l = j, \tag{4.13}$$

with P_t indexing a neuron in layer t. The edges of P use the weights $W^{(t)}_{P_{t-1},P_t}$ for $t = 1, \dots, l$. Define

$$w(P) \coloneqq \prod_{t=1}^{l} W^{(t)}_{P_{t-1},P_t}, \qquad |w|(P) \coloneqq \prod_{t=1}^{l} \big|W^{(t)}_{P_{t-1},P_t}\big|, \tag{4.14}$$

the parity sign

$$\operatorname{sgn}(P) \in \{+,-\}, \qquad \operatorname{sgn}(P) = + \iff \#\big\{t : W^{(t)}_{P_{t-1},P_t} < 0\big\} \text{ is even}, \tag{4.15}$$

and the gate factor *(1 iff every gate on P at intermediate layers $1, \dots, l-1$ is open)*

$$G_{<l}(P) \coloneqq \prod_{t=1}^{l-1} \mathbf{1}\big[z^{(t)}_{P_t} > 0\big]. \tag{4.16}$$

Then the decomposed pre-activations (4.8)–(4.9) admit the path-sum representation

$$z^{(l,+)}_j = \sum_{P:\, P_l = j} G_{<l}(P)\, |w|(P)\, x^{\operatorname{sgn}(P)}_{P_0}, \tag{4.17}$$

$$z^{(l,-)}_j = \sum_{P:\, P_l = j} G_{<l}(P)\, |w|(P)\, x^{\operatorname{sgn}(P)'}_{P_0}. \tag{4.18}$$

Proof. We prove (4.17)–(4.18) by induction on l.

Base case ($l = 1$): By the input convention (4.11), $a^{(0,+)}_{\text{stop},k} = x^+_k$ and $a^{(0,-)}_{\text{stop},k} = x^-_k$. Using (4.8) with biases omitted,

$$z^{(1,+)}_j = \sum_k \big(W^{(1,+)}_{kj} x^+_k + W^{(1,-)}_{kj} x^-_k\big). \tag{4.19}$$

Backward paths with $P_1 = j$ and $P_0 = k$ are in bijection with predecessors k; each has $|w|(P) = |W^{(1)}_{kj}|$, $\operatorname{sgn}(P) = +$ iff $W^{(1)}_{kj} > 0$, and $G_{<1}(P) = 1$ (empty product). Splitting by the sign of $W^{(1)}_{kj}$:

- $W^{(1)}_{kj} > 0$: $W^{(1,+)}_{kj} = |W^{(1)}_{kj}|$ contributes $|W^{(1)}_{kj}| x^+_k$ with $\operatorname{sgn}(P) = +$, matching $|w|(P)\, x^{\operatorname{sgn}(P)}_{P_0}$.
- $W^{(1)}_{kj} < 0$: $W^{(1,-)}_{kj} = |W^{(1)}_{kj}|$ contributes $|W^{(1)}_{kj}| x^-_k$ with $\operatorname{sgn}(P) = -$, again matching.

The argument for $z^{(1,-)}_j$ is identical with the parity swapped, establishing (4.18).

Inductive step ($l \geq 2$): The decomposition recursion (4.8) takes *Stopping Decomposition* activations from layer $l-1$ as inputs,

$$z^{(l,+)}_j = \sum_i \big(W^{(l,+)}_{ij}\, a^{(l-1,+)}_{\text{stop},i} + W^{(l,-)}_{ij}\, a^{(l-1,-)}_{\text{stop},i}\big), \tag{4.20}$$

and $a_{\text{stop},i}^{(l-1,\pm)} = \mathbf{1}[z_i^{(l-1)} > 0]\, z_i^{(l-1,\pm)}$. Hence, at each predecessor i, the gate indicator $\mathbf{1}[z_i^{(l-1)} > 0]$ multiplies the inductive representation of $z_i^{(l-1,\pm)}$, upgrading $G_{<l-1}(P')$ to $G_{\le l-1}(P')$ for every sub-path P' ending at i.

Every backward path P with $P_l = j$ and $P_{l-1} = i$ is uniquely obtained by prepending j to a sub-path $P' = (P'_{l-1}, \dots, P'_0)$ with $P'_{l-1} = i$; the prepended edge uses $W_{ij}^{(l)}$, contributing $|W_{ij}^{(l)}|$ to $|w|(P)$ and either preserving or flipping parity depending on $\operatorname{sgn}(W_{ij}^{(l)})$. Crucially, the new gate factor $G_{<l}(P)$ for P equals $G_{\le l-1}(P')$: up to layer $l-1$ the gates are those along P', and there is no gate at layer l in the $G_{<l}$ product.

Case $W_{ij}^{(l)} > 0$. Then $W_{ij}^{(l,+)} = |W_{ij}^{(l)}|$ and $W_{ij}^{(l,-)} = 0$. Only $W_{ij}^{(l,+)}\, a_{\text{stop},i}^{(l-1,+)}$ contributes, and the new edge preserves parity, $\operatorname{sgn}(P) = \operatorname{sgn}(P')$. Using the induction hypothesis under the updated gate factor,

$$\begin{aligned} W_{ij}^{(l,+)}\, a_{\text{stop},i}^{(l-1,+)} &= |W_{ij}^{(l)}| \sum_{P' \colon P'_{l-1}=i} G_{\le l-1}(P')\, |w|(P')\, x_{P'_0}^{\operatorname{sgn}(P')} \\ &= \sum_{P \colon P_l=j,\, P_{l-1}=i} G_{<l}(P)\, |w|(P)\, x_{P_0}^{\operatorname{sgn}(P)}. \end{aligned} \tag{4.21}$$

Case $W_{ij}^{(l)} < 0$. Then $W_{ij}^{(l,+)} = 0$ and $W_{ij}^{(l,-)} = |W_{ij}^{(l)}|$. Only $W_{ij}^{(l,-)}\, a_{\text{stop},i}^{(l-1,-)}$ contributes, and the new edge flips parity, $\operatorname{sgn}(P) = \operatorname{sgn}(P')'$, so $\operatorname{sgn}(P') = \operatorname{sgn}(P)'$, and the induction hypothesis for $z_i^{(l-1,-)}$ (with $x_{P'_0}^{\operatorname{sgn}(P')'} = x_{P_0}^{\operatorname{sgn}(P)}$) gives the same path sum as in the previous case. *Case* $W_{ij}^{(l)} = 0$. Both terms vanish and no path through i is prepended.

Summing over i yields (4.17). The same argument with positive/negative decomposed activations swapped gives (4.18).

Signed recovery. Using $x_k^+ - x_k^- = x_k$ and $\operatorname{sgn}(P) \in \{+,-\}$,

$$z_j^{(l,+)} - z_j^{(l,-)} = \sum_P G_{<l}(P)\, |w|(P) \left(x_{P_0}^{\operatorname{sgn}(P)} - x_{P_0}^{\operatorname{sgn}(P)'}\right) = \sum_P G_{<l}(P)\, w(P)\, x_{P_0}, \tag{4.22}$$

which is the standard active-path-sum representation of $z_j^{(l)}$ (paths through an inactive intermediate neuron contribute zero).

Biases. Adjoining a bias "stub path" at each intermediate layer l'—starting from (l,j), terminating at the bias node of neuron k' at layer l', with weight product taken over the $l - l'$ edges above and an appropriate gate factor $G_{<l}$ restricted to those edges—handles the bias terms by the same parity rule; the argument above goes through verbatim after absorbing these stub contributions into the path sums. □

4.6.2 Game Definition

Relation to the network games of Gaubert & Vlassopoulos (2025). The Stopping Game adapts the game of Gaubert and Vlassopoulos to the Stopping Decomposition. We keep their state-space structure —per-neuron ± states, the bias as the continuation and network input as final payoff, transitions driven by the positive and negative parts of the weights, a cemetery state, and a stop-versus-continue action. Two changes are load-bearing for attribution: (i) the original formulation tracks only one game value, which equals $\pm a_j^{(l)}$; we additionally maintain the non-negative payoff per player, and the original

game value is recovered as their difference. (ii) We replace the signed terminal values $\pm x_k$ by the non-negative pair x_k^+, x_k^-, which keeps every player-specific payoff non-negative and exposes the parity decomposition of Lemma 4.4.

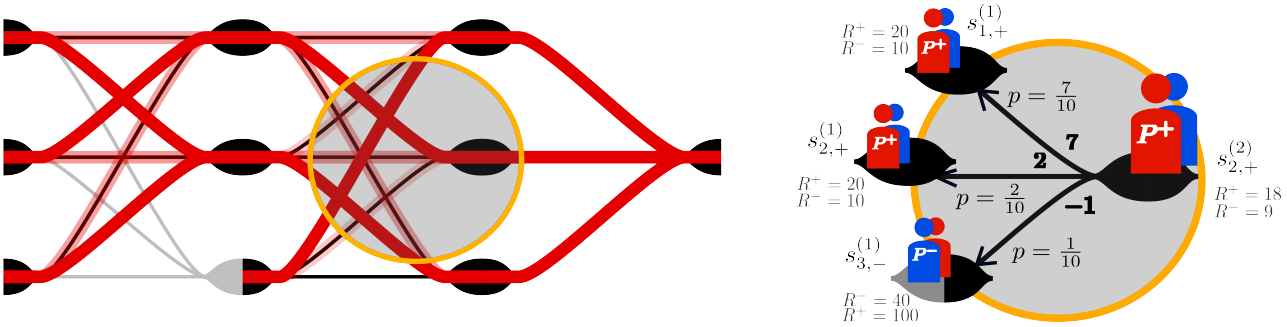


Fig. 4.2: Stopping Game: trajectory distribution and local stopping decisions on a toy subnetwork. *Left:* trajectory distribution visualized on the network graph. *Right:* local game around $s_{2,+}^{(2)}$. Player P^+ decides whether to stop or continue. Under continuation, transitions are to $s_{1,+}^{(1)}$ with probability 7/10, to $s_{2,+}^{(1)}$ with probability 2/10 (both turn unchanged), and to $s_{3,-}^{(1)}$ with probability 1/10 (turn switches to P^-, reflecting the negative sign of the weight). The labels $R^\pm$ are the players' expected payoffs $\tilde{Q}_\pm(\cdot, \text{cont})$ (at $s_{2,+}^{(2)}$ without the discount $\gamma_2^{(2)} = 10$). At $s_{3,-}^{(1)}$, continuing yields expected payoffs R^-=40 and R^+=100, giving continuation advantage $R^- - R^+ = -60 < 0$; therefore P^- stops (so $V_-(s_{3,-}^{(1)}) = 0$), shown by the grayed-out half-neuron. Correspondingly, in the left panel the trajectories terminate at that state (the cemetery state is omitted). Since P^- stops in $s_{3,-}^{(1)}$, traversals to this state contribute zero continuation payoff to both players. Hence, at $s_{2,+}^{(2,\text{act})}$ the positive player has larger expected payoff under continuation than his opponent and chooses to continue.

Definition 4.1 (Stopping Game State Space). The game is played on the following states:

- **Activation States:** $s_{j,q}^{(l,\text{act})}$ where layer $l \in \{1, \dots, L\}$ and player $q \in \{+,-\}$ is in turn at neuron j.
- **Terminal States:** $s_{k,q}^{(0)}$ where k is an input dimension and $q \in \{+,-\}$ is the final turn label.
- **Cemetery State:** †, an absorbing state reached upon stopping.

We adopt the convention that, when a transition kernel or recursion at layer l references the predecessor state $s_{i,r}^{(l-1,\text{act})}$, the symbol denotes the activation state at layer $l-1 \geq 1$ as defined above and, in the boundary case $l = 1$, is interpreted as the terminal state $s_{i,r}^{(0)}$.

Definition 4.2 (Payoffs, Transitions, and Value Recursion). Fix an input x.

Payoffs. The player-specific *payoff* r_p at activation states is paid out depending on the action chosen by the active player, and paid out at terminal and cemetery states without prerequisites:

- Cemetery: $r_p(\dagger) := 0$.
- Active state: $r_p(s_{j,q}^{(l,\text{act})}, \text{stop}) := 0$ (stopping pays nothing) and $r_p(s_{j,q}^{(l,\text{act})}, \text{cont}) := b_j^{(l,p\oplus q)}$ (continuation pays a positive bias to the active player q—covered by the case $p \oplus q = +$—and the absolute value of a negative bias to its opponent q'—covered by the case $p \oplus q = -$).
- Terminal: $r_p(s_{k,q}^{(0)}) := x_k^{p\oplus q}$ (at terminal states the payoff depends on state only—the game ends here. A positive input is paid to the active player q—covered by the case $p \oplus q = +$—whereas the absolute value of a negative input is paid to its opponent q'—covered by the case $p \oplus q = -$).

Transition kernel. Stopping at state $s_{j,p}^{(l,\mathrm{act})}$ deterministically sends the game to the cemetery; continuation transports backward by one layer with transition probabilities depending on the fraction of the respective absolute network weight compared to the sum of absolute incoming weights at neuron j of layer l. In case of a positive weight, the active player p stays in turn, in case of a negative weight, the turn switches to its opponent p'.

$$\mathbb{P}\big(\dagger \,\big|\, s_{j,q}^{(l,\mathrm{act})}, \mathrm{stop}\big) \coloneqq 1, \tag{4.23}$$

$$\mathbb{P}\big(s_{i,r}^{(l-1,\mathrm{act})} \,\big|\, s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) \coloneqq \frac{W_{ij}^{(l,q\oplus r)}}{\gamma_j^{(l)}}, \qquad \gamma_j^{(l)} \coloneqq \sum_i \big(W_{ij}^{(l,+)} + W_{ij}^{(l,-)}\big) = \sum_i \big|W_{ij}^{(l)}\big|, \tag{4.24}$$

where $\gamma_j^{(l)}$ is the local structural discount that restores unit row-sum (if $\gamma_j^{(l)} = 0$, continuation leads to $\dagger$). For understanding the idea of the occupation measure, we remark that those $\gamma_j^{(l)}$ are going to weight the trajectories in the occupation measure.

Policy. A pure, i.e., deterministic policy $\pi = (\pi_+, \pi_-)$ assigns each active state $s_{j,q}^{(l,\mathrm{act})}$ a binary stop/continue decision by the player in turn,

$$\pi_q\big(s_{j,q}^{(l,\mathrm{act})}\big) \in \{0,1\}, \qquad 1 = \mathrm{cont},\ 0 = \mathrm{stop}. \tag{4.25}$$

Player-specific action and game values. For each policy π, the player-specific **action value** $\tilde{Q}_p^\pi(s,a)$ is the expected discounted payoff for player p starting at s with action a and following π thereafter, defined by the standard Bellman equation

$$\tilde{Q}_p^\pi(s,a) \coloneqq r_p(s,a) + \gamma(s) \sum_{s'} \mathbb{P}\big(s' \,\big|\, s,a\big)\, \tilde{V}_p^\pi(s'), \tag{4.26}$$

where $\tilde{V}_p^\pi$ is the player-specific game value defined below. We use tildes to reserve V_p^π for the advantage further down, which already carries a subtracted $\tilde{V}_{p'}^\pi$. At active states (4.26) specializes, using (4.23)–(4.24), to

$$\tilde{Q}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{stop}\big) = 0 + 1 \cdot \tilde{V}_p^\pi(\dagger) = 0 \tag{4.27}$$

$$\tilde{Q}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) = b_j^{(l,p\oplus q)} + \gamma_j^{(l)} \sum_{s'} \mathbb{P}\big(s' \,\big|\, s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big)\, \tilde{V}_p^\pi(s') \tag{4.28}$$

$$= b_j^{(l,p\oplus q)} + \sum_{i,r} W_{ij}^{(l,q\oplus r)}\, \tilde{V}_p^\pi\big(s_{i,r}^{(l-1,\mathrm{act})}\big). \tag{4.29}$$

The **player-specific game value** $\tilde{V}_p^\pi(s)$ is the action value averaged by the policy. At terminal and cemetery states it equals the local payoff there:

$$\tilde{V}_p^\pi\big(s_{k,q}^{(0)}\big) \coloneqq x_k^{p\oplus q}, \qquad \tilde{V}_p^\pi(\dagger) \coloneqq 0. \tag{4.30}$$

At active states,

$$\tilde{V}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}\big) \coloneqq \pi_q\big(s_{j,q}^{(l,\mathrm{act})}\big)\, \tilde{Q}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) + \big(1 - \pi_q\big(s_{j,q}^{(l,\mathrm{act})}\big)\big)\, \tilde{Q}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{stop}\big). \tag{4.31}$$

Since π_q is binary, (4.31) selects either $\tilde{Q}_p^\pi(\cdot, \mathrm{cont})$ or $\tilde{Q}_p^\pi(\cdot, \mathrm{stop}) = 0$.

Game value. The quantity the player in turn maximizes at each active state is the advantage of player p at s,

$$V_p^{\pi}(s) \coloneqq \tilde{V}_p^{\pi}(s) - \tilde{V}_{p'}^{\pi}(s). \tag{4.32}$$

Figure 4.2 shows a worked example of the induced trajectory distribution and the local stop/continue logic on a small toy network.

Theorem 4.5 (Forward Equivalence for Stopping Game, `Lean_50`). *The game admits a Subgame Perfect Nash Equilibrium (SPNE) where both players follow the same pure strategy:*

$$\pi_q^{\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) = \mathbf{1}\big[z_j^{(l)} > 0\big]. \tag{4.33}$$

Under this equilibrium, the player-specific game values and the game values satisfy

$$\tilde{V}_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) = a_{\mathrm{stop},j}^{(l,p\oplus q)}, \qquad V_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) = \xi_{p,q}\, a_j^{(l)}. \tag{4.34}$$

Remark 4.1 (Uniqueness of the SPNE up to tie-breaking). It is easy to see in the following proof that the SPNE (4.33) is unique up to tie-breaking at states where continuing yields zero advantage $\tilde{Q}_q^{\pi^\star}(\cdot,\mathrm{cont}) - \tilde{Q}_{q'}^{\pi^\star}(\cdot,\mathrm{cont}) = z_j^{(l)} = 0$—representing neurons of pre-activation zero. The pinned Stopping-Game attribution code selects continuation at exact zero in its hard backward mask, while the Routing Game and Hellinger diagnostic use strict positivity. These choices give the same original forward activation at a tie but can give different selected backward quantities there. The theorem here uses stopping at ties. We refer to the policy (4.33) by **stop-at-tie-breaks** equilibrium.

Proof. We proceed by backward induction on the layer index l.

Base Case ($l = 0$): By (4.30), $\tilde{V}_p^{\pi^\star}(s_{k,q}^{(0)}) = x_k^{p\oplus q} = a_{\mathrm{stop},k}^{(0,p\oplus q)}$ (input convention (4.11)). The game value is

$$V_p^{\pi^\star}\big(s_{k,q}^{(0)}\big) = x_k^{p\oplus q} - x_k^{p'\oplus q} = \xi_{p,q}\, x_k, \tag{4.35}$$

using $x_k^+ - x_k^- = x_k$ and $x_k^- - x_k^+ = -x_k$.

Inductive Step: Assume the subgames starting at states up to layer $l-1$ admit the claimed SPNE and (4.34) holds at layer $l-1$. Consider an active state $s_{j,q}^{(l,\mathrm{act})}$. The active player q chooses $a \in \{\mathrm{stop}, \mathrm{cont}\}$ to maximize the advantage $\tilde{Q}_q^{\pi^\star}(s_{j,q}^{(l,\mathrm{act})}, a) - \tilde{Q}_{q'}^{\pi^\star}(s_{j,q}^{(l,\mathrm{act})}, a)$. For $a = \mathrm{stop}$ this advantage is 0 by (4.27); for $a = \mathrm{cont}$, using (4.28),

$$\tilde{Q}_q^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) - \tilde{Q}_{q'}^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) = \big(b_j^{(l,q\oplus q)} - b_j^{(l,q'\oplus q)}\big) + \sum_{i,r} W_{ij}^{(l,q\oplus r)}\, V_q^{\pi^\star}\big(s_{i,r}^{(l-1,\mathrm{act})}\big) \tag{4.36}$$

$$= \big(b_j^{(l,+)} - b_j^{(l,-)}\big) + \sum_{i,r} W_{ij}^{(l,q\oplus r)}\, V_q^{\pi^\star}\big(s_{i,r}^{(l-1,\mathrm{act})}\big). \tag{4.37}$$

By the inductive hypothesis $V_q^{\pi^\star}(s_{i,r}^{(l-1,\mathrm{act})}) = \xi_{q,r}\, a_i^{(l-1)}$, with $\xi_{q,q} = +1$ and $\xi_{q,q'} = -1$,

$$\tilde{Q}_q^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) - \tilde{Q}_{q'}^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) = b_j^{(l)} + \sum_i \big(W_{ij}^{(l,q\oplus q)}\, \xi_{q,q} + W_{ij}^{(l,q\oplus q')}\, \xi_{q,q'}\big) a_i^{(l-1)} \tag{4.38}$$

$$= b_j^{(l)} + \sum_i W_{ij}^{(l)}\, a_i^{(l-1)} = z_j^{(l)}. \tag{4.39}$$

Thus, player q prefers *continue* in case $z_j^{(l)} > 0$ and prefers *stop* in case $z_j^{(l)} < 0$, and is indifferent at $z_j^{(l)} = 0$, where we tie-break in favor of *stop*. The rule $\pi_q^\star(s_{j,q}^{(l,\mathrm{act})}) = \mathbf{1}[z_j^{(l)} > 0]$ together with the player policies of earlier layers thus constitutes an SPNE of the subgame starting at state $s_{j,q}^{(l,\mathrm{act})}$.

To verify (4.34), substitute $\pi_q^\star$ into (4.31) and use the inductive hypothesis $\tilde{V}_p^{\pi^\star}(s_{i,r}^{(l-1,\mathrm{act})}) = a_{\mathrm{stop},i}^{(l-1,p\oplus r)}$ in (4.28):

$$\tilde{V}_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) = \mathbf{1}\big[z_j^{(l)} > 0\big]\, \tilde{Q}_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) \tag{4.40}$$

$$= \mathbf{1}\big[z_j^{(l)} > 0\big] \left[b_j^{(l,p\oplus q)} + \sum_i \big(W_{ij}^{(l,+)}\, a_{\mathrm{stop},i}^{(l-1,p\oplus q)} + W_{ij}^{(l,-)}\, a_{\mathrm{stop},i}^{(l-1,p\oplus q')}\big) \right]. \tag{4.41}$$

If $p = q$, the bracket is $b_j^{(l,+)} + \sum_i \big(W_{ij}^{(l,+)}\, a_{\mathrm{stop},i}^{(l-1,+)} + W_{ij}^{(l,-)}\, a_{\mathrm{stop},i}^{(l-1,-)}\big) = z_j^{(l,+)}$. If $p = q'$, the bracket is $b_j^{(l,-)} + \sum_i \big(W_{ij}^{(l,+)}\, a_{\mathrm{stop},i}^{(l-1,-)} + W_{ij}^{(l,-)}\, a_{\mathrm{stop},i}^{(l-1,+)}\big) = z_j^{(l,-)}$. In both cases, $\tilde{V}_p^{\pi^\star}(s_{j,q}^{(l,\mathrm{act})}) = \mathbf{1}[z_j^{(l)} > 0]\, z_j^{(l,p\oplus q)} = a_{\mathrm{stop},j}^{(l,p\oplus q)}$. Finally,

$$V_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) = a_{\mathrm{stop},j}^{(l,p\oplus q)} - a_{\mathrm{stop},j}^{(l,(p\oplus q)')} = \xi_{p,q}\, a_j^{(l)}. \quad \square \tag{4.42}$$

$\square$

4.6.3 Skip Connections and Max Pooling

Architectures such as ResNets and CNNs with pooling contain structural operators beyond single-parent affine maps. To handle these natively in the Stopping Game, we add an **Addition Oracle** for skip connections and a deterministic **Max-Pooling Decision State** for pooling, mirroring the analogous constructions in the Routing Game (Section 4.7.1).

Definition 4.3 (Addition Oracle). For a residual node $z = x + y$, the *addition state* $s_{z,q}^{\mathrm{add}}$ has active player $q \in \{+,-\}$ and operand states $s_{x,q}, s_{y,q}$ at the two summands. The transition kernel is the uniform Oracle

$$\mathbb{P}\big(s_{x,q} \,\big|\, s_{z,q}^{\mathrm{add}}\big) := \tfrac{1}{2}, \qquad \mathbb{P}\big(s_{y,q} \,\big|\, s_{z,q}^{\mathrm{add}}\big) := \tfrac{1}{2}, \tag{4.43}$$

with player q retained at the chosen operand. The immediate payoff is $r_p(s_{z,q}^{\mathrm{add}}) := 0$ for both $p \in \{+,-\}$, and the structural discount is $\gamma_{\mathrm{O}} := 2$.

Proposition 4.6 (Forward Equivalence for Addition, `Lean_51`). *The player-specific state value at the addition state equals the sum of the operand state values,*

$$\tilde{V}_p^{\pi}\big(s_{z,q}^{\mathrm{add}}\big) \;=\; \tilde{V}_p^{\pi}(s_{x,q}) + \tilde{V}_p^{\pi}(s_{y,q}). \tag{4.44}$$

Consequently, if the operand states recover the Stopping-Decomposition activations—$\tilde{V}_p^{\pi^\star}(s_{x,q}) = x^{(p\oplus q)}$ *and* $\tilde{V}_p^{\pi^\star}(s_{y,q}) = y^{(p\oplus q)}$*—then* $\tilde{V}_p^{\pi^\star}(s_{z,q}^{\mathrm{add}}) = z^{(p\oplus q)}$*, preserving the forward equivalence of Theorem 4.5 across the residual sum.*

Proof. Apply the Bellman recursion (4.26)–(4.31) at $s_{z,q}^{\mathrm{add}}$ with $r_p = 0$, the uniform Oracle kernel, and $\gamma_{\mathrm{O}} = 2$ from Definition 4.3:

$$\tilde{V}_p^{\pi}\big(s_{z,q}^{\mathrm{add}}\big) \;=\; 0 + \gamma_{\mathrm{O}} \cdot \Big(\tfrac{1}{2}\,\tilde{V}_p^{\pi}(s_{x,q}) + \tfrac{1}{2}\,\tilde{V}_p^{\pi}(s_{y,q})\Big) \;=\; \tilde{V}_p^{\pi}(s_{x,q}) + \tilde{V}_p^{\pi}(s_{y,q}), \tag{4.45}$$

which is the first identity. Substituting the operand assumption and using $z^{(p\oplus q)} = x^{(p\oplus q)} + y^{(p\oplus q)}$ from the additive convention (4.8)–(4.9) of Section 4.6.1 yields the second. □

On the uniform $1/2$ split. The $(\frac{1}{2}, \frac{1}{2})$ Oracle with $\gamma_{\mathrm{O}} = 2$ is not forced by the architecture; it is a convention, the simplest choice inside a one-parameter family of conservation-preserving Oracles $(\rho, 1-\rho)$ with matching discounts $(1/\rho, 1/(1-\rho))$ attached to the branch taken (with the single state discount γ_{O} of (4.26) and d_t, $\rho\gamma_{\mathrm{O}} = (1-\rho)\gamma_{\mathrm{O}} = 1$ forces $\rho = \frac{1}{2}$). The uniform split is the maximum-entropy choice—the one that imposes *no prior* on which summand carries more responsibility. In the Stopping Game, Proposition 4.6 and Theorems 4.5 and 4.8 hold for any such split with branch discounts, since the proofs only use that routing probability times Oracle discount equals 1 on each branch; only the trajectory law Π_x (and with it the Hellinger distance of Section 8.3) depends on the split. This invariance does not extend to the Routing Game (Section 4.7.1), where the split changes the relevance assigned to each summand.

Max pooling is simpler, because the forward operator already selects a single predecessor.

Definition 4.4 (Max-Pooling Decision State). For a pooled scalar output $z = \max\{x_1, \dots, x_m\}$, the *pooling state* $s_{z,q}^{\max}$ has active player $q \in \{+,-\}$ and admissible action set $\mathscr{A}(s_{z,q}^{\max}) = \{1, \dots, m\}$ indexing the operand states $s_{x_1,q}, \dots, s_{x_m,q}$. Under action k, the transition is deterministic,

$$\mathbb{P}\big(s_{x_k,q} \,\big|\, s_{z,q}^{\max}, k\big) \coloneqq 1, \tag{4.46}$$

with player q retained at the chosen operand. The immediate payoff is $r_p(s_{z,q}^{\max}, k) \coloneqq 0$ for both $p \in \{+,-\}$, and the structural discount is $\gamma_{\mathrm{O}} \coloneqq 1$.

Tie-breaking at the pooling state. Whenever multiple operands x_k attain the maximum, the active player is indifferent over the corresponding actions, and any pure best response over the tied indices yields the same forward value but a different equilibrium policy and hence a different occupation measure on the operand states. The *stop-at-tie-breaks equilibrium*, fixed by the boundary conventions of Section 4.5, resolves this by the convention $k^\star \coloneqq \min\{k : x_k = \max_l x_l\}$, i.e., the active player picks the smallest index among the operands attaining the maximal payoff. All gradient and occupation-measure statements below are stated for this equilibrium.

Proposition 4.7 (Forward Equivalence for Max Pooling, `Lean_52`**).** *Assume the operand states recover the Stopping-Decomposition activations,* $\tilde{V}_p^{\pi^\star}(s_{x_k,q}) = x_k^{(p\oplus q)}$ *for all k, with non-negative operand activations $x_k \geq 0$. Then at the SPNE of the subgame rooted at* $s_{z,q}^{\max}$*,*

$$\tilde{V}_p^{\pi^\star}\big(s_{z,q}^{\max}\big) \;=\; z^{(p\oplus q)}, \tag{4.47}$$

so the pooling state inherits the same Stopping-Decomposition identity as its winning operand and the forward equivalence of Theorem 4.5 extends across the pooling node.

Proof. The active player q chooses the operand that maximizes its game value $V_q^{\pi^\star} = \tilde{V}_q^{\pi^\star} - \tilde{V}_{q'}^{\pi^\star}$. By the operand assumption, $V_q^{\pi^\star}(s_{x_k,q}) = \xi_{q,q}\, x_k = x_k$ for every k, so the maximizer is $k^\star \in \arg\max_k x_k$ (ties resolved by the smallest index). The Bellman recursion (4.26)–(4.31) with $r_p = 0$, the deterministic transition, and $\gamma_{\mathrm{O}} = 1$ from Definition 4.4 gives

$$\tilde{V}_p^{\pi^\star}\big(s_{z,q}^{\max}\big) \;=\; 0 + 1 \cdot \tilde{V}_p^{\pi^\star}(s_{x_{k^\star},q}) \;=\; x_{k^\star}^{(p\oplus q)}. \tag{4.48}$$

Since $z = x_{k^\star}$, the right-hand side equals $z^{(p\oplus q)}$. □

4.6.4 Discounted Occupation Measures and Gradient Representations

To connect game trajectories with neural network gradients, we define the **discounted occupation measure**. This measure quantifies the expected influence of a neuron on the game's outcome, weighted by the discount factors accumulated along all possible game trajectories (including those created by skip connections and max-pooling nodes).

Definition 4.5 (Game trajectory and trajectory law, `Lean_53`). Fix an input x and a root state s. A *game trajectory* rooted at s is a finite sequence of game states

$$\zeta \;=\; (\zeta_0, \zeta_1, \ldots, \zeta_T), \qquad \zeta_0 = s, \tag{4.49}$$

that follows the admissible transitions of the game (Definition 4.2 together with the addition/max-pooling Oracle splits of Definitions 4.3–4.4) until it terminates either at a layer-0 terminal state $\zeta_T = s_{k,r}^{(0)}$ (reaching an input dimension k) or at the cemetery state $\zeta_T = \dagger$ (reached the first time a player chooses *stop*). The set of all such trajectories is the trajectory *ground set* $\mathscr{T}_s$. Composing the transition kernel (4.23)–(4.24), the Oracle splits, and a (possibly stochastic) policy π at every activation state defines a Markov kernel on game states, which together with the deterministic start at s induces a probability distribution $\Pi_x^{\pi}(\cdot \mid s)$ on $\mathscr{T}_s$. Trajectories that pass through closed gates—neurons j with $\mathbf{1}[z_j^{(l)} > 0] = 0$—are admissible elements of $\mathscr{T}_s$, but under the equilibrium policy $\pi^\star$ the player at the activation state stops there, so any trajectory continuing past a closed gate carries probability zero. We write $\Pi_x(\cdot \mid s) \coloneqq \Pi_x^{\pi^\star}(\cdot \mid s)$ for the equilibrium *trajectory law* and $\Pi_x \coloneqq \Pi_x(\cdot \mid s_0)$ for the law rooted at the output state $s_0 \coloneqq s_{u,+}^{(L,\mathrm{act})}$.

Definition 4.6 (Discounted Occupation Measure, `Lean_54`). Let $\Pi_x(\cdot \mid s)$ be the equilibrium trajectory law of Definition 4.5. Define the cumulative discount

$$d_t(\zeta) \coloneqq \prod_{m=0}^{t-1} \gamma(\zeta_m), \qquad d_0(\zeta) = 1, \tag{4.50}$$

where $\gamma(\zeta_m) = \gamma_{\mathrm{O}} = 2$ for addition Oracle states, $\gamma(\zeta_m) = \gamma_{\mathrm{O}} = 1$ for max-pooling states, and $\gamma(\zeta_m) = \gamma_j^{(l)}$ for continuation at activation states $\zeta_m = s_{j,q}^{(l,\mathrm{act})}$ (per-neuron weight-magnitude discount). The *discounted occupation measure* of state v for trajectories rooted at s is

$$\Gamma_x(v \mid s) \coloneqq \mathbb{E}_{\zeta\sim\Pi_x(\cdot|s)}\left[\sum_t d_t(\zeta)\, \mathbf{1}\{\zeta_t = v\}\right], \qquad \Gamma_x(v) \coloneqq \Gamma_x(v \mid s_0). \tag{4.51}$$

Theorem 4.8 (Activation–Occupation Correspondence, Lean_55). *Fix a neuron state $s = s^{(m,\mathrm{act})}_{j,q}$ and a layer $l \le m$ (for $l = 0$, $s^{(0,\mathrm{act})}_{i,\cdot}$ denotes the terminal state $s^{(0)}_{i,\cdot}$). Under the stop-at-tie-breaks equilibrium fixed by the boundary conventions of Section 4.5 (uniquely fixing the SPNE at ReLU gates with $z = 0$ and at max-pooling states with multiple maximizers), the discounted occupation measures exactly represent, in a multi-trajectory ReLU network (e.g., a ResNet or a CNN with max pooling), the local Stopping-Decomposition and scalar activation derivatives on open cells with nonzero ReLU pre-activations and unique max-pooling winners. At ties, the same formulas represent the backward quantities computed using the selected gates and winners; they do not assert classical differentiability of the individual stopping streams, which can be discontinuous there:*

1) ***Stopping-Decomposition Gradient:***

$$\frac{\partial a^{(m,p\oplus q)}_{\mathrm{stop},j}}{\partial a^{(l,k)}_{\mathrm{stop},i}} \;=\; \Gamma_x\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \;\big|\; s\big). \tag{4.52}$$

2) ***Network Activation Gradient.*** *The ordinary-network gradient of $a^{(m)}_j$ with respect to the scalar activation $a^{(l)}_i$ is*

$$\frac{\partial a^{(m)}_j}{\partial a^{(l)}_i} \;=\; \xi_{q,+}\big(\Gamma_x(s^{(l,\mathrm{act})}_{i,+} \mid s) - \Gamma_x(s^{(l,\mathrm{act})}_{i,-} \mid s)\big). \tag{4.53}$$

Proof. We establish Part 1 by backward induction on the layer gap $m - l$ and derive Part 2 from it. Since x is fixed, we write $\Gamma(v \mid s) \coloneqq \Gamma_x(v \mid s)$.

Part 2. By Theorem 4.5, $a^{(l)}_i = a^{(l,+)}_{\mathrm{stop},i} - a^{(l,-)}_{\mathrm{stop},i}$ and $a^{(m)}_j = a^{(m,+)}_{\mathrm{stop},j} - a^{(m,-)}_{\mathrm{stop},j}$. Differentiating $a^{(m)}_j$ along the $+$ direction (perturb $a^{(l,+)}_{\mathrm{stop},i}$, keep $a^{(l,-)}_{\mathrm{stop},i}$ fixed) and applying Part 1 at $k = +$,

$$\frac{\partial a^{(m)}_j}{\partial a^{(l)}_i} = \frac{\partial a^{(m,+)}_{\mathrm{stop},j}}{\partial a^{(l,+)}_{\mathrm{stop},i}} - \frac{\partial a^{(m,-)}_{\mathrm{stop},j}}{\partial a^{(l,+)}_{\mathrm{stop},i}} = \Gamma\big(s^{(l,\mathrm{act})}_{i,q} \;\big|\; s\big) - \Gamma\big(s^{(l,\mathrm{act})}_{i,q'} \;\big|\; s\big) = \xi_{q,+}\big(\Gamma(s^{(l,\mathrm{act})}_{i,+} \mid s) - \Gamma(s^{(l,\mathrm{act})}_{i,-} \mid s)\big), \tag{4.54}$$

using $q \oplus + = q$ and $q' \oplus + = q'$ in the second equality.

Part 1. The decomposition recursion of Theorem 4.5 writes the upper-layer Stopping-Decomposition activation as

$$a^{(m,p\oplus q)}_{\mathrm{stop},j} = \mathbf{1}\big[z^{(m)}_j > 0\big]\left[b^{(m,p\oplus q)}_j + \sum_h \Big(W^{(m,+)}_{hj}\, a^{(m-1,p\oplus q)}_{\mathrm{stop},h} + W^{(m,-)}_{hj}\, a^{(m-1,p\oplus q')}_{\mathrm{stop},h}\Big)\right]. \tag{4.55}$$

Base case ($m = l$). If $s = s^{(l,\mathrm{act})}_{i,r}$ already lies at the target neuron and decomposed activation, then

$$\frac{\partial a^{(l,p\oplus r)}_{\mathrm{stop},i}}{\partial a^{(l,k)}_{\mathrm{stop},i}} = \delta_{p\oplus r,\,k} = \mathbf{1}\{r = p \oplus k\} = \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \;\big|\; s^{(l,\mathrm{act})}_{i,r}\big). \tag{4.56}$$

Inductive step (linear routing, $m > l$). Differentiating (4.55) with respect to $a^{(l,k)}_{\mathrm{stop},i}$ (the bias $b^{(m,p\oplus q)}_j$ does not depend on lower-layer activations) and substituting the inductive hypothesis at layer $m-1$,

$$\frac{\partial a^{(m,p\oplus q)}_{\mathrm{stop},j}}{\partial a^{(l,k)}_{\mathrm{stop},i}} = \mathbf{1}\big[z^{(m)}_j > 0\big] \sum_h \Big(W^{(m,+)}_{hj}\, \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s^{(m-1,\mathrm{act})}_{h,q}\big) + W^{(m,-)}_{hj}\, \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s^{(m-1,\mathrm{act})}_{h,q'}\big)\Big). \quad (4.57)$$

On the game side, the one-step occupation recursion at $s = s^{(m,\mathrm{act})}_{j,q}$ uses transition probabilities $W^{(m,q\oplus r)}_{hj}/\gamma^{(m)}_j$ together with the matching cumulative-discount factor $\gamma^{(m)}_j$ and the equilibrium gate $\mathbf{1}[z^{(m)}_j > 0]$:

$$\Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s\big) = \mathbf{1}\big[z^{(m)}_j > 0\big] \sum_{h,r} W^{(m,q\oplus r)}_{hj}\, \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s^{(m-1,\mathrm{act})}_{h,r}\big). \quad (4.58)$$

The two right-hand sides agree, completing the inductive step at linear routing layers.

Addition-node step. For an addition node $z = x + y$, the additivity of the Stopping Decomposition from Section 4.6.1 gives $a^{(p\oplus q)}_{\mathrm{stop},z} = a^{(p\oplus q)}_{\mathrm{stop},x} + a^{(p\oplus q)}_{\mathrm{stop},y}$, so by the inductive hypothesis,

$$\frac{\partial a^{(p\oplus q)}_{\mathrm{stop},z}}{\partial a^{(l,k)}_{\mathrm{stop},i}} = \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s_{x,q}\big) + \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s_{y,q}\big). \quad (4.59)$$

The Oracle split of Definition 4.3 sends mass $\frac{1}{2}$ to each branch and applies the discount $\gamma_{\mathrm{O}} = 2$, so the occupation measure satisfies the matching identity

$$\Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s^{\mathrm{add}}_{z,q}\big) = 2 \cdot \tfrac{1}{2}\, \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s_{x,q}\big) + 2 \cdot \tfrac{1}{2}\, \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s_{y,q}\big). \quad (4.60)$$

Max-pooling step. For a max-pooling node $z = \max\{x_1, \ldots, x_m\}$ with winner $k^\star$, Proposition 4.7 gives $a^{(p\oplus q)}_{\mathrm{stop},z} = a^{(p\oplus q)}_{\mathrm{stop},x_{k^\star}}$, and the inductive hypothesis yields

$$\frac{\partial a^{(p\oplus q)}_{\mathrm{stop},z}}{\partial a^{(l,k)}_{\mathrm{stop},i}} = \Gamma\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s_{x_{k^\star},q}\big). \quad (4.61)$$

The deterministic transition with unit discount in Definition 4.4 gives the matching identity $\Gamma(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s^{\max}_{z,q}) = \Gamma(s^{(l,\mathrm{act})}_{i,p\oplus k} \mid s_{x_{k^\star},q})$. Hence, the correspondence also holds across skip connections and max-pooling nodes, completing the induction. □

4.7 Routing Game: Log-Space Formulation

This section makes the two-player payoff structure of the Routing Game explicit. Its global network interpretation uses non-negative activation outputs; the GELU discussion later concerns local gates conditioned on a fixed forward pass. The recursion used in Section 4.3.2 is the logarithmic representation of an entropy-regularized two-player game, and admits a Gibbs subgame-perfect equilibrium for every $\tau > 0$.

4.7.1 Exact Routing Kernel Behind $\alpha\beta$-LRP

The linear routing subgames are entropy-regularized one-step maximization problems whose closed form is given by the Legendre–Fenchel identity for Shannon entropy $\mathscr{H}(\pi) \coloneqq -\sum_i \pi(i)\log\pi(i)$ (Boyd and Vandenberghe, 2004): for every $\tau > 0$, $n \in \mathbb{N}$, and payoff vector $r \in \mathbb{R}^n$,

$$\max_{\pi\in\Delta_n}\Big\{\textstyle\sum_i \pi(i)\, r_i + \tau\,\mathscr{H}(\pi)\Big\} = \tau\log\textstyle\sum_i \exp(r_i/\tau), \qquad \pi^\star(i) \propto \exp(r_i/\tau). \tag{4.62}$$

We write

$$\ell(z) \coloneqq \begin{cases}\log z, & z > 0,\\ -\infty, & z \le 0,\end{cases} \qquad \exp(-\infty) \coloneqq 0, \tag{4.63}$$

so that $\exp(\ell(x)) = \mathrm{ReLU}(x)$ for any $x \in \mathbb{R}$; we use this fact for notational convenience. Zero or negative routed mass thus receives the stopping value $-\infty$. At every activation state we explicitly choose *stop whenever the two log-space action values tie*. Hence continuing is selected exactly when its log value is strictly greater than $-\infty$, equivalently when the signed routed mass is positive. The ReLU-gating itself is fully modeled by the player policies at states modeling the ReLU gates—just like in the Stopping Game.

State Space and Game Mechanics. To correctly model the interaction between player turns and stream signs, we define a structured state space:

- **Activation States:** $\mathscr{S}^{(l)}_{act} = \{s^{(l,act)}_{j,p}\}$, where $p \in \{+,-\}$ is the player in turn.
- **Linear Routing States:** $\mathscr{S}^{(l)}_{lin} = \{s^{(l,lin)}_{j,q,\kappa}\}$, where q is the player turn and $\kappa \in \{+,-\}$ is the stream sign.

At $s^{(l,act)}_{j,p}$, the active player p chooses between *stop* and *continue*. Under the *continue* action, the environment's transition kernel is uniform over the two sign branches,

$$\mathbb{P}\big(s^{(l,lin)}_{j,p,+} \,\big|\, s^{(l,act)}_{j,p}, \text{continue}\big) = \tfrac{1}{2}, \qquad \mathbb{P}\big(s^{(l,lin)}_{j,p',-} \,\big|\, s^{(l,act)}_{j,p}, \text{continue}\big) = \tfrac{1}{2}, \tag{4.64}$$

where the first branch keeps player p in control evaluating $\kappa = +$ weights and the second switches the turn to opponent p' evaluating $\kappa = -$ weights. To compensate for the $1/2$ branching, the environment applies an Oracle structural discount of $\gamma_{\mathrm{O}} = 2$ (distinct from the per-neuron weight-magnitude discount $\gamma^{(l)}_j$ used in the Stopping Game).

Definition 4.7 (Entropy-Regularized Two-Player Routing Game). Fix an input x and a stabilizer $\varepsilon \ge 0$. At each activation state $s^{(l,act)}_{j,p}$, the player in turn chooses *stop* or *continue* with similar mechanics as in the Stopping Game (Section 4.6) reflecting the open-closed state of a ReLU gate: stopping sends the game to the cemetery † giving immediate payoff $\ell(0) = -\infty$—and continuation transitions backward with structural discount $\gamma = 2$ and the uniform $1/2$ sign-branch kernel given above. Here happens the critical step: gains of the opponent on the negative stream state $s^{(l,\mathrm{lin})}_{j,p',-}$ shall be counted as losses of the active player p. At each linear routing state $s^{(l,lin)}_{j,q,\kappa}$, the player in turn instead chooses a mixed action $\nu \in \Delta(\mathrm{Pred}^{(l)}_j \cup \{\mathrm{bias}, \bot_{\mathrm{lin}}\})$ over the predecessors of neuron j augmented by a terminal *bias node* with immediate payoff $\ell([b^{(l)}_j]^\kappa)$ and a fictitious *outside option* $\bot_{\mathrm{lin}}$ which routes the game's mass to the cemetery with immediate payoff $\ell(\varepsilon)$; trajectories entering either

are absorbed. Let $\mathscr{H}(\nu) \coloneqq -\sum_a \nu(a)\log\nu(a)$ be Shannon entropy over this augmented action set. At $\varepsilon = 0$ the outside option carries $\ell(0) = -\infty$ and is dominated by every positive-mass predecessor, so the augmented game reduces to the Routing Game over $\mathrm{Pred}_j^{(l)} \cup \{\mathrm{bias}\}$.

An admissible profile chooses a pure stop/continue action at each activation state and a probability vector at each linear routing state. Under any profile, evaluate the displayed recursions using those choices, without taking maxima: use the chosen activation action and the chosen routing vector's expected log score plus entropy. A zero-probability term contributes zero even when its score is $-\infty$; positive probability on a $-\infty$ score makes the routing value $-\infty$.

Throughout, $\mathbf{V}(\cdot) = (V_p(\cdot), V_{p'}(\cdot))^\top$ denotes the game's backward-induction player-specific log-value function, defined jointly at linear routing states by (4.69) and at activation states by (4.72) below. The recursion is well-posed because the layered state graph is a finite DAG: the $l = 1$ case of (4.65) needs no lower-layer $\mathbf{V}$, and every subsequent layer uses only $\mathbf{V}$ at strictly lower layers.

For a **linear routing state** at $l \geq 2$ the active player continuing to the ith neuron's activation state receives an immediate payoff of the logarithm of the respective weight summed with the respective log-value function of the next state, whereas in the first layer the final payoff is paid out to the active player. The inactive player always obtains an immediate payoff of $\ell(0) = -\infty$. Let us denote by $Q_r(s,a)$ player r's gain for the game value in case action a actually happens, whereas $\bar{Q}_r$ denotes the mixture of the differences between the payoffs of the opponents including the entropy regularization term given the mixed strategy chosen by the active player.

$$Q_r\big(s_{j,q,\kappa}^{(l,lin)}, i\big) \coloneqq \begin{cases} \ell\big(W_{ij}^{(l,\kappa)}\big) + V_r\big(s_{i,q}^{(l-1,act)}\big), & r = q,\ l \geq 2, \\ \ell\big([x_i W_{ij}^{(1)}]^\kappa\big), & r = q,\ l = 1, \\ -\infty, & r = q'. \end{cases} \tag{4.65}$$

The outside option $\perp_{\mathrm{lin}}$ and the bias node have log-scores $Q_r(s_{j,q,\kappa}^{(l,\mathrm{lin})}, \perp_{\mathrm{lin}}) \coloneqq \ell(\varepsilon)$ and $Q_r(s_{j,q,\kappa}^{(l,\mathrm{lin})}, \mathrm{bias}) \coloneqq \ell([b_j^{(l)}]^\kappa)$ for $r = q$ and $-\infty$ for $r = q'$; they route the ε-fraction and the bias share of the mass to absorbing terminals instead of to any predecessor. If player q uses mixed action ν at this state, the induced *individual routed masses* are

$$\begin{aligned} \bar{Q}_r\big(s_{j,q,\kappa}^{(l,lin)}; \nu\big) \coloneqq \sum_a \nu(a)\, \ell\left(\exp\left(Q_r\big(s_{j,q,\kappa}^{(l,lin)}, a\big)\right) - \exp\left(Q_{r'}\big(s_{j,q,\kappa}^{(l,lin)}, a\big)\right)\right) \\ + \tau\mathscr{H}(\nu), \qquad r \in \{q, q'\}, \end{aligned} \tag{4.66}$$

which, as $\exp(-\infty) = 0$, evaluates to

$$\bar{Q}_q\big(s_{j,q,\kappa}^{(l,lin)}; \nu\big) = \sum_a \nu(a)\, Q_q\big(s_{j,q,\kappa}^{(l,lin)}, a\big) + \tau\mathscr{H}(\nu) \tag{4.67}$$

$$\bar{Q}_{q'}\big(s_{j,q,\kappa}^{(l,lin)}; \nu\big) = -\infty. \tag{4.68}$$

If $W_{ij}^{(l,\kappa)} = 0$ for every predecessor i and $[b_j^{(l)}]^\kappa = 0$ (i.e. the stream-κ part of the fan-in to neuron j is identically zero), then $Q_q(s_{j,q,\kappa}^{(l,\mathrm{lin})}, i) = -\infty$ for all i as well and the value (4.69) equals $\ell(\varepsilon)$. If $\varepsilon = 0$ and every action scores $-\infty$ (e.g. $\Sigma_j^{(l,\kappa)} = 0$ at $\tau = 1$), every ν is a best response; we fix $\nu = \delta_{\perp_{\mathrm{lin}}}$ (absorption), matching the convention $0/0 \coloneqq 0$ in (4.91). We define the corresponding equilibrium

linear-game values by

$$V_q^\star\big(s_{j,q,\kappa}^{(l,lin)}\big) \coloneqq \max_{\nu\in\Delta(\mathrm{Pred}_j^{(l)}\cup\{\mathrm{bias},\perp_{\mathrm{lin}}\})} \bar{Q}_q\big(s_{j,q,\kappa}^{(l,lin)};\nu\big)$$
$$= \max_{\nu\in\Delta(\mathrm{Pred}_j^{(l)}\cup\{\mathrm{bias},\perp_{\mathrm{lin}}\})} \left\{\sum_a \nu(a)\, Q_q\big(s_{j,q,\kappa}^{(l,lin)},a\big) + \tau\mathscr{H}(\nu)\right\}$$
$$= \tau\cdot\ell\left(\varepsilon^{\frac{1}{\tau}} + \big([b_j^{(l)}]^\kappa\big)^{\frac{1}{\tau}} + \sum_i \exp\Big(Q_q(s_{j,q,\kappa}^{(l,lin)},i)\Big)^{\frac{1}{\tau}}\right), \tag{4.69}$$
$$V_{q'}^\star\big(s_{j,q,\kappa}^{(l,lin)}\big) \coloneqq -\infty, \tag{4.70}$$

where we used (4.62). At $\tau = 1$ the associated Gibbs policy assigns the outside option the probability $\varepsilon/(\varepsilon + \sum_{a\neq\perp_{\mathrm{lin}}} \exp Q_q(\cdots,a))$ and distributes the remaining mass over the predecessors and the bias node in proportion to $\exp Q_q$.

For an **activation state**, continuing exposes both sign branches. Let $s_+ \coloneqq s_{j,p,+}^{(l,\mathrm{lin})}$ and $s_- \coloneqq s_{j,p',-}^{(l,\mathrm{lin})}$. If player p chooses *continue* at $s_{j,p}^{(l,act)}$, the game transitions to s_+ or to s_- with equal probability, while *stop* sends the game to the cemetery with game value $V_r(\dagger) = \ell(0) = -\infty$. Taking the expectation over the uniform sign-branch kernel ($\frac{1}{2}$ each) and respecting the discount $\gamma_O = 2$ interpreted in logarithmic space, the continuation value becomes

$$Q_r\big(s_{j,p}^{(l,act)},\mathrm{continue}\big) \coloneqq \ell\left(\sum_{s'\in\{s_+,s_-\}} \mathbb{P}(s' \mid s_{j,p}^{(l,act)})\cdot\gamma_{\mathrm{O}}\cdot\Big(\exp V_r(s') - \exp V_{r'}(s')\Big)\right)$$
$$= \ell\Big(\xi_{r,p}\Big(\exp\left(V_p\left(s_+\right)\right) - \exp\left(V_{p'}\left(s_-\right)\right)\Big)\Big). \tag{4.71}$$

Stopping yields $-\infty$ for both players. We select *stop at every activation-state tie in log space*, so the selected policy and active-player value are

$$\pi_p^\star(\mathrm{continue} \mid s_{j,p}^{(l,\mathrm{act})}) \coloneqq \mathbf{1}\Big[Q_p(s_{j,p}^{(l,\mathrm{act})},\mathrm{continue}) > -\infty\Big], \tag{4.72}$$
$$V_p(s_{j,p}^{(l,\mathrm{act})}) \coloneqq \max\{-\infty, Q_p(s_{j,p}^{(l,\mathrm{act})},\mathrm{continue})\}. \tag{4.73}$$

Writing $D_p = \exp V_p(s_+) - \exp V_{p'}(s_-)$, this selects continuing exactly when $D_p > 0$. When $D_p \leq 0$, both actions have log value $-\infty$ and the convention selects stopping. If continuing is selected, the opponent's continuation value is $-\infty$; if stopping is selected, both players receive $-\infty$. Thus in either case

$$V_{p'}(s_{j,p}^{(l,\mathrm{act})}) = -\infty. \tag{4.74}$$

Figure 4.3 shows a worked example of the local routing subgame.

In the following let us denote, with $a^{(0)} = x$,

$$\Sigma_j^{(l,\kappa)} \coloneqq \textstyle\sum_i [a_i^{(l-1)} W_{ij}^{(l)}]^\kappa + [b_j^{(l)}]^\kappa, \tag{4.75}$$

so that $z_j^{(l)} = \Sigma_j^{(l,+)} - \Sigma_j^{(l,-)}$; for $l \geq 2$, $a^{(l-1)} \geq 0$ gives $[a_i^{(l-1)} W_{ij}^{(l)}]^\kappa = a_i^{(l-1)} W_{ij}^{(l,\kappa)}$.

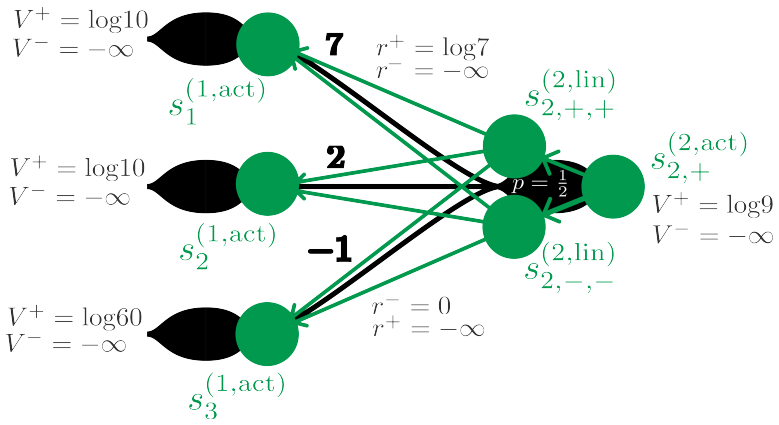


Fig. 4.3: Routing Game: local routing subgame around $s_{2,+}^{(2,\mathrm{act})}$. This uses the same toy subnetwork and numerical values as Figure 4.2. At this activation state, P^+ is in turn and continues because the routed mass is positive ($V^+ = \log 9 > -\infty$ in the depicted example). The game then branches with probability 1/2 to $s_{2,+,+}^{(2,\mathrm{lin})}$ (turn stays with P^+) and with probability 1/2 to $s_{2,-,-}^{(2,\mathrm{lin})}$ (turn switches to P^-). In each linear state, the active player samples a predecessor according to the Gibbs policy from the immediate log-payoff and game value. Example: along $s_{2,+,+}^{(2,\mathrm{lin})} \to s_{1,+}^{(1,\mathrm{act})}$, the player on the positive stream state gets immediate payoff $\log W_{12}^{(2,+)} = \log 7$ and P^- gets $-\infty$. Edges with zero routed mass have payoff $\log W_{12}^{(2,-)} = \log 0 = -\infty$. For readability, the $\pm$ activation copies in layer 1 are collapsed, and the intermediate payoff $\log 2$ on the transition from position 2 to 2 is omitted.

Theorem 4.9 (SPNE and Forward Equivalence of the Routing Game, `Lean_56`).

(i) *SPNE existence for every $\tau > 0$ and $\varepsilon \geq 0$. For every $\tau > 0$ and every $\varepsilon \geq 0$, the entropy-regularized two-player Routing Gameof Definition 4.7 (with linear-state entropy weight $\tau\mathscr{H}$ and outside-option stabilizer ε) admits a subgame-perfect Nash equilibrium (see (4.80)–(4.81) in the proof for the explicit form of the equilibrium policy). We write $\cdot^\star$ for equilibrium quantities throughout.*

(ii) *Forward equivalence at the anchor $\tau = 1$, $\varepsilon \geq 0$. At $\tau = 1$ and $\varepsilon \geq 0$, the equilibrium quantities recover the network forward pass: for every layer l, neuron j, player labels p,q,r, and stream sign $\kappa \in \{+,-\}$,*

$$V_r\big(s_{j,q,\kappa}^{(l,lin)}\big) = \ell\big(\delta_{r,q}\,(\Sigma_j^{(l,\kappa)} + \varepsilon)\big), \tag{4.76}$$

$$Q_r\left(s_{j,p}^{(l,act)}, \mathit{continue}\right) = \ell\left(\xi_{r,p} \cdot z_j^{(l)}\right), \tag{4.77}$$

$$V_r\big(s_{j,p}^{(l,act)}\big) = \ell\big(\delta_{r,p}\ \cdot a_j^{(l)}\big), \tag{4.78}$$

$$\pi_p^\star\big(s_{j,p}^{(l,act)}\big) = \mathbf{1}[z_j^{(l)} > 0]. \tag{4.79}$$

Remark 4.2 (Uniqueness). The linear-state Gibbs best response (4.80) is the unique maximizer of the strictly concave entropy-regularized objective whenever some action has a finite score (otherwise Definition 4.7 fixes $\perp_{\mathrm{lin}}$). At activation states the active player is indifferent whenever the signed continuation mass $\exp V_p(s_+) - \exp V_{p'}(s_-)$ is nonpositive ($z_j^{(l)} \leq 0$ at $\tau = 1$), because both log-space action values are then $-\infty$. Thus the unconstrained best-response set can contain several policies. The theorem specifies the tie-breaker $\pi_p^\star = \mathbf{1}[Q_p(\cdot,\mathrm{continue}) > -\infty]$, which at $\tau = 1$ reduces to $\mathbf{1}[z_j^{(l)} > 0]$. Just like in the SG (Section 4.6), we refer to this SPNE as the **stop-at-tie-breaks** equilibrium. This canonical choice also selects the absorbing outside action whenever every routing action has zero mass;

without these conventions, both activation log ties and empty routing streams permit multiple optimal choices.

Proof. **SPNE existence for every $\tau > 0$.** The layered state graph is a finite DAG: linear routing states at layer l transition to activation states at layer $l-1$ (or to terminal input positions at $l=1$), and activation states at layer l transition to linear routing states at the same layer l or to the cemetery $\dagger$. We construct the equilibrium by backward induction on the layer index l.

At layer $l = 1$ the linear routing subgames have no unresolved game values, so the action values Q_q in (4.65) are fully specified by the input x. At each linear state the active player solves $\max_{\nu \in \Delta(\mathrm{Pred}_j^{(1)})} \{\sum_i \nu(i)\, Q_q(s_{j,q,\kappa}^{(1,lin)}, i) + \tau\, \mathscr{H}(\nu)\}$, which is strictly concave on the simplex face supported on finite-score actions whenever that face is nonempty. If every score is $-\infty$, every distribution is optimal and we select the absorbing outside action as above. Identity (4.62) yields the unique Gibbs best response in the nonempty finite-score case, and simplifying $\exp \circ \ell$ in (4.65) gives the explicit form

$$\pi_q^\star\big(i \,\big|\, s_{j,q,\kappa}^{(l,lin)}\big) \propto \begin{cases} \big(W_{ij}^{(l,\kappa)}\big)^{1/\tau} \exp\big(V_q(s_{i,q}^{(l-1,act)})/\tau\big), & l \geq 2, \\ \big([x_i W_{ij}^{(1)}]^\kappa\big)^{1/\tau}, & l = 1, \end{cases} \tag{4.80}$$

with zero mass on routes of zero or wrong sign and closed-form value $V_q(s_{j,q,\kappa}^{(1,lin)}) = \tau \log \sum_i \exp(Q_q(s_{j,q,\kappa}^{(1,lin)}, i)/\tau)$. With these values in hand, the activation-state subgame at layer $l = 1$ is a binary choice between stop (log-payoff $\ell(0) = -\infty$) and continue (4.71); comparing log-space action values and choosing stopping at every tie, the active player's best response is the pure stop/continue strategy

$$\pi_p^\star\big(s_{j,p}^{(l,act)}\big) = \mathbf{1}\big[Q_p(s_{j,p}^{(l,act)}, \text{continue}) > -\infty\big]. \tag{4.81}$$

Assume the equilibrium has been constructed at all layers $\leq l-1$, yielding definite values $V_r(s_{i,q}^{(l-1,act)})$. At layer l the linear-state action values Q_q in (4.65) are then specified, and the same strict-concavity argument gives the Gibbs best response (4.80) with the analogous log-sum-exp value. The layer-l activation-state best response is again the binary hard comparison (4.81). Because each subgame's best response is taken to be optimal given the game values from lower layers, the composed strategy profile is subgame-perfect. This proves existence of an SPNE for every $\tau > 0$. The argument is unchanged under the bias-node and outside-option augmentation of Definition 4.7: the linear-state action set gains the extra elements bias and $\perp_{\mathrm{lin}}$ with constant log-scores $\ell([b_j^{(l)}]^\kappa)$ and $\log \varepsilon$ (finite or $-\infty$), the strict-concavity / Legendre–Fenchel step goes through verbatim on the augmented simplex, and the closed-form value picks up the extra terms $([b_j^{(l)}]^\kappa)^{1/\tau}$ and $\varepsilon^{1/\tau}$ inside the log-sum-exp. Hence, an SPNE exists for every $(\tau, \varepsilon) \in \mathbb{R}_{>0} \times \mathbb{R}_{\geq 0}$.

Forward equivalence at $\tau = 1$. We now specialize to $\tau = 1$ and show by induction on l that the equilibrium quantities recover the network forward pass.

Base case ($l = 1$): Identical to the inductive step below, with the fused-input action values $Q_q(s_{j,q,\kappa}^{(1,lin)}, i) = \ell([x_i W_{ij}^{(1)}]^\kappa)$ of (4.65) in place of $\ell(a_i^{(l-1)} W_{ij}^{(l,\kappa)})$.

Inductive step ($l \geq 2$): Assume $V_q(s_{i,q}^{(l-1,act)}) = \ell(a_i^{(l-1)})$ for all predecessors i and both player labels q. Since the assumed non-negative activation functions satisfy $a_i^{(l-1)} \geq 0$, the linear routing-state action values become

$$Q_q\big(s_{j,q,\kappa}^{(l,lin)}, i\big) = \ell\big(W_{ij}^{(l,\kappa)}\big) + \ell\big(a_i^{(l-1)}\big) = \ell\big(a_i^{(l-1)} W_{ij}^{(l,\kappa)}\big). \tag{4.82}$$

Applying (4.62) to the linear routing subgame gives the Gibbs best response (4.80) and

$$V_q\big(s^{(l,lin)}_{j,q,\kappa}\big) = \ell\left(\sum_a \exp\big(Q_q(s^{(l,lin)}_{j,q,\kappa}, a)\big)\right) = \ell\left(\sum_i a_i^{(l-1)} W_{ij}^{(l,\kappa)} + [b_j^{(l)}]^\kappa + \varepsilon\right) = \ell(\Sigma_j^{(l,\kappa)} + \varepsilon), \quad (4.83)$$

At the activation state the active player's continuation advantage is again $z_j^{(l)}$

$$Q_p\big(s^{(l,act)}_{j,p}, \text{continue}\big) = \ell\left(\Sigma_j^{(l,+)} - \Sigma_j^{(l,-)}\right) = \ell(z_j^{(l)}), \quad (4.84)$$

whereas for the opponent

$$Q_{p'}\left(s^{(l,\text{act})}_{j,p}, \text{continue}\right) = \ell\left(\Sigma_j^{(l,-)} - \Sigma_j^{(l,+)}\right) = \ell(-z_j^{(l)}), \quad (4.85)$$

again confirming (4.77). Thus, the best response is (4.81). Finally,

$$V_r\big(s^{(l,act)}_{j,p}\big) = \ell\big(\delta_{r,p}\, \mathbf{1}[z_j^{(l)} > 0]\, z_j^{(l)}\big) = \ell\big(\delta_{r,p}\, a_j^{(l)}\big), \quad (4.86)$$

proving (4.78). Since each subgame has an optimal action chosen by the player in turn, the resulting strategy profile is subgame-perfect. This completes the induction. □

Log-Space Addition Subgame, Skip Connections, and Max Pooling

Residual additions $z = x + y$ and max-pooling nodes $z = \max_k x_k$ enter the RG through the addition state $s^{\text{add}}_{z,p}$ (uniform Oracle to the operand states, player p retained, $\gamma_{\text{O}} = 2$) and the pooling state $s^{\max}_{z,p}$ (deterministic choice of an operand, player p retained, $\gamma_{\text{O}} = 1$, ties broken by the smallest index) of Section 4.6.3, with the values written in log space:

$$V_r(s^{\text{add}}_{z,p}) = \ell\big(\exp V_r(s_{x,p}) + \exp V_r(s_{y,p})\big), \qquad V_p(s^{\max}_{z,p}) = \max_k V_p(s_{x_k,p}). \quad (4.87)$$

If the operand states recover their (non-negative) activations, $V_p(s_{x,p}) = \ell(a_x)$ etc., then $V_p(s^{\text{add}}_{z,p}) = \ell(a_x + a_y) = \ell(a_z)$ and $V_p(s^{\max}_{z,p}) = \ell(a_{x_{k^\star}}) = \ell(a_z)$, so forward equivalence extends across both nodes. Two things differ from the SG. First, $\gamma_{\text{O}} = 2$ enters only the values: the trajectory discount (4.89) of the addition transition is 1, so each operand receives half of the addition-state occupation (Theorem 4.10, item 2). Second, the relevance split therefore equals the Oracle probabilities: a $(\rho, 1-\rho)$ Oracle with value discounts $(1/\rho, 1/(1-\rho))$ leaves all forward values unchanged but splits the relevance as $(\rho, 1-\rho)$, so the uniform split is a convention of the RG rule. The implementation passes the full relevance to both operands (trajectory discount 2); since every backward trajectory of a ResNet or ViT that reaches the input crosses each of its B addition nodes exactly once, this rescales the input occupation by the constant 2^B and leaves normalized maps unchanged.

4.7.2 Backward Path Integral and $\alpha\beta$-LRP Recovery

We now show that the projected discounted occupation measure induced by the SPNE on the layered routing graph realizes the $\alpha\beta$-LRP-ε relevance vector of (4.1). To keep the game side and the attribution side strictly separate, we use distinct symbols throughout this subsection:

– the *game-side* object is the discounted occupation measure $\Gamma_x(\,\cdot\,)$ on the states of the routing graph (Definition 4.6), now driven by the SPNE trajectory law Π_x rooted at $s^{(L,\text{act})}_{u,+}$ with unit initial mass: $\Gamma_x(s^{(L,\text{act})}_{u,+}) = 1$ and $\Gamma_x(s^{(L,\text{act})}_{j,p}) = 0$ for every other output activation state. With

non-negative branch discounts (cf. (4.89)) Γ_x is itself non-negative; we then form the per-neuron difference of the two player copies,

$$\Gamma_j^{(l)} \;\coloneqq\; \Gamma_x\big(s_{j,+}^{(l,\mathrm{act})}\big) \;-\; \Gamma_x\big(s_{j,-}^{(l,\mathrm{act})}\big). \tag{4.88}$$

– the *attribution-side* object is the $\alpha\beta$-LRP-ε relevance vector $R^{(l)}$ (the $\alpha\beta$ rule of Bach et al. (2015) with the stabilization convention adopted here, see below), a deterministic backward DP on the network's linear layers seeded by a chosen output mass $R_u^{(L)}$ and recursing via (4.91) below.

The recovery theorem (Theorem 4.10) shows that, under the SPNE policy and the following trajectory discount (4.89), the per-neuron difference $\Gamma_j^{(l)}$ satisfies the same recursion as $R_j^{(l)}$, hence equals $R_j^{(l)}/R_u^{(L)}$.

Trajectory discount. We track sign-dependent branch discounts governed by the stream-balance mode (α,β) $(\alpha-\beta=1)$. Let $\zeta=(\zeta_0,\zeta_1,\dots)$ be a backward trajectory on linear routing, activation, addition or max-pooling states. Define the per-state discount γ as a non-negative function of the entered state,

$$\gamma(s) \coloneqq \begin{cases} 2\alpha, & s = s_{j,q,+}^{(l,lin)} \text{ for some } j,q,l, \\ 2\beta, & s = s_{j,q,-}^{(l,lin)} \text{ for some } j,q,l, \\ 1, & \text{otherwise (activation, addition Oracle, max-pool).} \end{cases} \tag{4.89}$$

The cumulative discount along the trajectory up to time t is the product

$$d_t(\zeta) \coloneqq \prod_{m=1}^{t} \gamma(\zeta_m), \qquad d_0(\zeta)=1 \text{ (empty product).} \tag{4.90}$$

The signed minus of the $\alpha\beta$-LRP recursion is produced by the difference (4.88) between the two player copies.

Remark 4.3 (Choice of starting player). Rooting Π_x at $s_{u,-}^{(L,\mathrm{act})}$ instead of $s_{u,+}^{(L,\mathrm{act})}$ swaps the two output-layer occupations and therefore negates $\Gamma_j^{(l)}$ at every layer. We fix the canonical convention that the game starts at the +-state.

Remark 4.4 (Equivalent signed-discount sum formulation). The same recovery theorem can be phrased with the signed trajectory discount $\gamma(s_{j,q,+}^{(l,\mathrm{lin})})=2\alpha$, $\gamma(s_{j,q,-}^{(l,\mathrm{lin})})=-2\beta$ and the per-neuron *sum* $\Gamma_j^{(l)} = \Gamma_x(s_{j,+}^{(l,\mathrm{act})}) + \Gamma_x(s_{j,-}^{(l,\mathrm{act})})$ in place of (4.88). We have chosen the non-negative-discount version because it is easier to comprehend—Γ_x remains a genuine non-negative occupation measure, and the LRP sign is generated by an explicit algebraic difference rather than absorbed into the trajectory weights.

Attribution-side: $\alpha\beta$-LRP-ε relevance. Independently of the game, the $\alpha\beta$-LRP-ε relevance vector $R^{(l)}$ is defined recursively from $R_j^{(L)} = R_u^{(L)}\,\delta_{j,u}$ at the output by

$$R_i^{(l-1)} \;=\; \sum_j \left(\alpha \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^+}{\varepsilon + \Sigma_j^{(l,+)}} - \beta \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^-}{\varepsilon + \Sigma_j^{(l,-)}} \right) R_j^{(l)}, \tag{4.91}$$

where $\Sigma_j^{(l,\pm)}$ is (4.75): the bias enters the denominators as a terminal bias node, and the relevance routed to it (and to the outside option) is absorbed, as in the implementation. Recursion (4.91) is the $\alpha\beta$ rule of Bach et al. (2015) with the stabilization convention adopted throughout this paper: the stabilizer ε of their LRP-ε rule is added separately to each sign denominator. Bach et al. (2015) present the two rules as alternative constructions and do not state this combined form; it is the convention of common LRP implementations (Anders et al., 2023), and Theorem 4.10 is exact for this form. At $\varepsilon = 0$ it is the original $\alpha\beta$ rule.

Theorem 4.10 (Recovery of Multi-Path $\alpha\beta$-LRP, `Lean_57`). *For every $\varepsilon \geq 0$ and every input with $f(x) = a_u^{(L)} > 0$ (if $f(x) = 0$ the root player stops and $R^{(l)} = f(x)\Gamma^{(l)} = 0$ trivially), under the stop-at-tie-breaks equilibrium (Remark 4.2; ties at max-pooling states broken by the smallest index) at the anchor temperature $\tau = 1$ and stabilizer ε, with the cumulative trajectory discount (4.90) and the occupation measure $\Gamma_j^{(l)}$ from (4.88):*

1) ***Linear Map.*** *The per-neuron difference $\Gamma_j^{(l)}$ of (4.88) satisfies*

$$\Gamma_i^{(l-1)} \;=\; \sum_j \left(\alpha \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^+}{\varepsilon + \Sigma_j^{(l,+)}} - \beta \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^-}{\varepsilon + \Sigma_j^{(l,-)}} \right) \Gamma_j^{(l)}. \tag{4.92}$$

   Because (4.92) and (4.91) share their computational step, $\Gamma_j^{(l)} \cdot R_u^{(L)} = R_j^{(l)}$ at every layer; in particular $R_j^{(l)} = f(x)\,\Gamma_j^{(l)}$ for the seed $R_u^{(L)} = f(x)$. The same formula covers the fused input layer with $a^{(0)} = x$. At $\varepsilon = 0$ this reduces to the unstabilized $\alpha\beta$-LRP redistribution (not conservative: bias nodes absorb relevance); at $\varepsilon > 0$ it is the $\alpha\beta$ rule with the stabilization convention adopted here, i.e. (4.1) with ε added to each sign denominator separately.

2) ***Residual Addition.*** *Let $z = x_{\mathrm{op}} + y_{\mathrm{op}}$ be an addition node with addition state $s_{z,p}^{\mathrm{add}}$ and operand states $s_{x_{\mathrm{op}},p}, s_{y_{\mathrm{op}},p}$ as in Section 4.7.1. For every player label $p \in \{+,-\}$, with the edge occupation $\Gamma_x(s \to s') := \mathbb{E}_{\zeta\sim\Pi_x}\big[\sum_t d_{t+1}(\zeta)\mathbf{1}\{\zeta_t = s, \zeta_{t+1} = s'\}\big]$ (an operand's total occupation adds the edges from its other parents),*

$$\Gamma_x\big(s_{z,p}^{\mathrm{add}} \to s_{x_{\mathrm{op}},p}\big) \;=\; \tfrac{1}{2}\,\Gamma_x\big(s_{z,p}^{\mathrm{add}}\big), \qquad \Gamma_x\big(s_{z,p}^{\mathrm{add}} \to s_{y_{\mathrm{op}},p}\big) \;=\; \tfrac{1}{2}\,\Gamma_x\big(s_{z,p}^{\mathrm{add}}\big), \tag{4.93}$$

   so the operand pair carries the full addition-state mass with no duplication.

3) ***Max Pooling.*** *For a pooled output $z = \max\{x_1, \ldots, x_m\}$ with winner $k^\star$ and pooling state $s_{z,p}^{\max}$ (Section 4.7.1), the deterministic value-maximizing transition concentrates all mass on the winner: for every player label $p \in \{+,-\}$,*

$$\Gamma_x\big(s_{z,p}^{\max} \to s_{x_{k^\star},p}\big) \;=\; \Gamma_x\big(s_{z,p}^{\max}\big), \qquad \Gamma_x\big(s_{z,p}^{\max} \to s_{x_k,p}\big) \;=\; 0 \text{ for } k \neq k^\star. \tag{4.94}$$

Proof. We prove $R_u^{(L)} \cdot \Gamma_j^{(l)} = R_j^{(l)}$ at every layer l by backward induction on l.

Base case ($l = L$): The trajectory law Π_x has unit mass at $s_{u,+}^{(L,\mathrm{act})}$ by definition, so $\Gamma_x(s_{u,+}^{(L,\mathrm{act})}) = 1$ and $\Gamma_x(s_{j,p}^{(L,\mathrm{act})}) = 0$ for every $(j,p) \neq (u,+)$. By (4.88),

$$\Gamma_j^{(L)} \;=\; \Gamma_x\big(s_{j,+}^{(L,\mathrm{act})}\big) - \Gamma_x\big(s_{j,-}^{(L,\mathrm{act})}\big) \;=\; \delta_{j,u}, \tag{4.95}$$

i.e. the difference is 1 at the chosen output neuron u and 0 elsewhere. Multiplying by the output mass gives $R_u^{(L)} \cdot \Gamma_j^{(L)} = R_u^{(L)}\,\delta_{j,u} = R_j^{(L)}$, matching the LRP root specification of (4.91).

Inductive step ($l \Rightarrow l-1$): Assume $R_u^{(L)} \cdot \Gamma_j^{(l)} = R_j^{(l)}$ for every neuron j in layer l. From the SPNE construction in Theorem 4.9 together with the activation-game value reduction $V_p(s_{j,p}^{(l,\mathrm{act})}) = \ell(a_j^{(l)})$ (which holds for every $\varepsilon \geq 0$ because the additive ε-shifts on the two sign branches cancel in the continuation advantage), the linear-state Gibbs best response on the alive support is

$$\pi^\star_{j,\kappa}(\perp_{\mathrm{lin}}) = \frac{\varepsilon}{\varepsilon + \Sigma_j^{(l,\kappa)}}, \qquad \pi^\star_{j,\kappa}(\mathrm{bias}) = \frac{[b_j^{(l)}]^\kappa}{\varepsilon + \Sigma_j^{(l,\kappa)}}, \qquad \pi^\star_{j,\kappa}(i) = \frac{a_i^{(l-1)} W_{ij}^{(l,\kappa)}}{\varepsilon + \Sigma_j^{(l,\kappa)}}. \tag{4.96}$$

A trajectory reaches the layer-$(l-1)$ activation state $s_{i,r}^{(l-1,\mathrm{act})}$ from a layer-l activation state through one of two routes. By the activation-state continuation kernel of Definition 4.7, departing from $s_{j,p}^{(l,\mathrm{act})}$ (the player continues: at $l = L$ since $f(x) > 0$; at $l < L$, $\Gamma_x(s_{j,\cdot}^{(l,\mathrm{act})}) = 0$ unless $a_j^{(l)} > 0$ by (4.96)) the trajectory takes the $+$ branch to $s_{j,p,+}^{(l,\mathrm{lin})}$ with probability $\frac{1}{2}$ (player p retains the turn), and the $-$ branch to $s_{j,p',-}^{(l,\mathrm{lin})}$ with probability $\frac{1}{2}$ (turn switches to opponent p'); from each linear state the policy (4.96) routes to predecessor i with the player label of that linear state. Combining the two branch discount factors $2\alpha, 2\beta$ from (4.89) with these $\frac{1}{2}$-probabilities, the trajectory contributions to $\Gamma_x(s_{i,r}^{(l-1,\mathrm{act})})$ are

$$\begin{aligned}
\Gamma_x\big(s_{i,r}^{(l-1,\mathrm{act})}\big) &= \sum_j \Big(\tfrac{1}{2}(2\alpha)\, \pi^\star_{j,+}(i)\, \Gamma_x\big(s_{j,r}^{(l,\mathrm{act})}\big) \;+\; \tfrac{1}{2}(2\beta)\, \pi^\star_{j,-}(i)\, \Gamma_x\big(s_{j,r'}^{(l,\mathrm{act})}\big) \Big) \\
&= \sum_j \Big(\alpha\, \pi^\star_{j,+}(i)\, \Gamma_x\big(s_{j,r}^{(l,\mathrm{act})}\big) \;+\; \beta\, \pi^\star_{j,-}(i)\, \Gamma_x\big(s_{j,r'}^{(l,\mathrm{act})}\big) \Big),
\end{aligned} \tag{4.97}$$

where $r' \in \{+,-\}$ denotes the opposite player label and the absorbing actions bias and $\perp_{\mathrm{lin}}$ contribute no mass to any predecessor.

Differencing (4.97) over $r \in \{+,-\}$ (i.e. $r{=}+$ minus $r{=}-$) flips the role of r and r' between the two terms inside the bracket. Using $\Gamma_x(s_{j,r}^{(l,\mathrm{act})}) - \Gamma_x(s_{j,r'}^{(l,\mathrm{act})}) = +\Gamma_j^{(l)}$ when $r{=}+$, $r'{=}-$, and the same expression with the opposite sign when $r{=}-$, $r'{=}+$, the per-neuron difference at layer $l-1$ satisfies

$$\begin{aligned}
\Gamma_i^{(l-1)} &= \Gamma_x\big(s_{i,+}^{(l-1,\mathrm{act})}\big) - \Gamma_x\big(s_{i,-}^{(l-1,\mathrm{act})}\big) \\
&= \sum_j \Big(\alpha\, \pi^\star_{j,+}(i) \left[\Gamma_x(s_{j,+}^{(l,\mathrm{act})}) - \Gamma_x(s_{j,-}^{(l,\mathrm{act})})\right] \;+\; \beta\, \pi^\star_{j,-}(i) \left[\Gamma_x(s_{j,-}^{(l,\mathrm{act})}) - \Gamma_x(s_{j,+}^{(l,\mathrm{act})})\right] \Big) \\
&= \sum_j \big(\alpha\, \pi^\star_{j,+}(i) - \beta\, \pi^\star_{j,-}(i) \big)\, \Gamma_j^{(l)} \\
&= \sum_j \left(\alpha \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^+}{\varepsilon + \Sigma_j^{(l,+)}} - \beta \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^-}{\varepsilon + \Sigma_j^{(l,-)}} \right) \Gamma_j^{(l)},
\end{aligned} \tag{4.98}$$

where the third equality regroups the two square brackets as $+\Gamma_j^{(l)}$ and $-\Gamma_j^{(l)}$ respectively, and the fourth substitutes (4.96) together with $a_i^{(l-1)} W_{ij}^{(l,\kappa)} = [a_i^{(l-1)} W_{ij}^{(l)}]^\kappa$ for non-negative $a_i^{(l-1)}$. Multiplying both sides by $R_u^{(L)}$ and using the inductive hypothesis $R_u^{(L)} \cdot \Gamma_j^{(l)} = R_j^{(l)}$,

$$R_u^{(L)} \cdot \Gamma_i^{(l-1)} \;=\; \sum_j \left(\alpha \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^+}{\varepsilon + \Sigma_j^{(l,+)}} - \beta \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^-}{\varepsilon + \Sigma_j^{(l,-)}} \right) R_j^{(l)} \;=\; R_i^{(l-1)}. \tag{4.99}$$

This closes the induction and proves $R_u^{(L)} \cdot \Gamma_j^{(l)} = R_j^{(l)}$ for every layer l and neuron j.

Input layer ($l = 1$): The same calculation applies with the fused first-layer masses $[x_i W_{ij}^{(1)}]^\kappa$ in place of $a_i^{(l-1)} W_{ij}^{(l,\kappa)}$, yielding the identical formula with $a^{(0)} = x$.

Addition node: Let $z = x_{\text{op}} + y_{\text{op}}$ be an addition node. By Section 4.7.1, from $s_{z,p}^{\text{add}}$ the Oracle transitions to $s_{x_{\text{op}},p}$ and $s_{y_{\text{op}},p}$ with probability $\frac{1}{2}$ each, retaining player label p. Under the trajectory-discount convention (4.89)–(4.90), addition Oracle transitions carry trajectory-discount factor 1. Hence

$$\Gamma_x\big(s_{z,p}^{\text{add}} \to s_{x_{\text{op}},p}\big) \;=\; \tfrac{1}{2}\,\Gamma_x\big(s_{z,p}^{\text{add}}\big), \qquad \Gamma_x\big(s_{z,p}^{\text{add}} \to s_{y_{\text{op}},p}\big) \;=\; \tfrac{1}{2}\,\Gamma_x\big(s_{z,p}^{\text{add}}\big), \tag{4.100}$$

proving Item 2 of Theorem 4.10 for both player labels.

Max-pooling node: Let $z = \max\{x_1, \ldots, x_m\}$ with winner $k^\star$ and pooling state $s_{z,p}^{\max}$. By Section 4.7.1, the active player has the unique value-maximizing action $x_{k^\star}$ at $s_{z,p}^{\max}$, so the transition to $s_{x_{k^\star},p}$ is deterministic, retains player label p, and (4.89) assigns unit trajectory factor. Therefore $\Gamma_x(s_{x_{k^\star},p}) = \Gamma_x(s_{z,p}^{\max})$ and $\Gamma_x(s_{x_k,p}) = 0$ for $k \neq k^\star$, proving Item 3. □

4.7.3 Temperature τ as a Controlled Policy Deformation

Theorem 4.9(i) establishes an SPNE for every $\tau > 0$ when the entropy weight τ is used throughout the backward recursion, but exact $\alpha\beta$-LRP recovery (Theorem 4.10) holds only at $\tau = 1$. In practice, we deform τ only *locally* at each linear subgame's in-round Gibbs readout,

$$\pi_{j,\kappa}^{(\tau)\star}(i) \propto \exp\big(Q_q(s_{j,q,\kappa}^{(l,lin)}, i)/\tau\big), \tag{4.101}$$

while keeping the game values $V(s_{i,q}^{(l-1,act)})$ at their τ=1 equilibrium—i.e. at the network activations recovered by Theorem 4.9(ii). The players are therefore *unaware* of the τ-deformation (a *myopic* policy): their decisions rest on the Bellman equations at temperature 1 (the network forward pass), and only the local routing readout is perturbed. Under non-negative predecessor activations this gives $\pi_{j,\kappa}^{(\tau)\star}(i) \propto (a_i^{(l-1)} W_{ij}^{(l,\kappa)})^{1/\tau}$, so $\tau < 1$ sharpens and $\tau > 1$ flattens the route distribution. This controlled deformation is used in the experiments; it carries no exactness claim at $\tau \neq 1$. The implementation raises activations, weights and bias-node masses to the power $1/\tau$ but keeps the outside-option weight at ε rather than $\varepsilon^{1/\tau}$. On ResNet-50 it passes relevance through the global average pool uniformly (not in proportion to the activations) and merges batch normalization into the preceding convolution with an ε-stabilized rule, so the ResNet-50 anchor is $\alpha\beta$-LRP up to these two operations; VGG-16 (whose average pool is the identity at 224×224) and ViT-B/16 are not affected.

4.8 Analytical Modes

This section collects the formal derivations for the analytical modes that the game framework exposes on top of the $\alpha\beta$-LRP anchor: the local temperature deformation τ on the linear-state Gibbs readout (Section 4.7.3), the activation-side risk-aversion λ via ADF variance propagation, the noisy-observation gate $\tilde{\Phi}$ on the player's stop/continue comparison, and the fan-in entropy bias λ_{ent}. Each mode maps a concept in game space (focus, risk aversion) to an explicit modification of the backward propagation rule; none carries an exactness claim, and each recovers the anchor at its default ($\tau = 1$, $\lambda = 0$, $\tilde{\Phi}$ off, $\lambda_{\text{ent}} = 0$). The mode-by-mode empirical sweeps live in Section A.4.3.

4.8.1 Defaults, costs and order of use

Table 4.3 lists every mode with its default, its effect and its measured cost (median seconds per image, one A100 GPU, batch size 1; Section A.3.7). At the defaults, SG is the gradient and RG is $\alpha\beta$-LRP. A practical order of use: (i) start at the anchor; (ii) vary τ first, since it is free and moves routing between focused and diffuse maps (Table A.17); (iii) add risk aversion (λ on CNNs, λ_{sm} on ViTs, with the input variance σ^2) when the maps should discount routes through high-variance units, at the price of one ADF pass; (iv) use equivariant averaging on ViTs only when token-grid artifacts matter, since it multiplies the cost by N. The validation selections of Section A.4.1 show which combination the protocol picks per model and criterion.

Tab. 4.3: Modes: default (anchor) value, effect and measured cost per image.

Mode	Default	Effect	Cost (s/image)
(α,β), ε	(2,1), 0.5	LRP stream weights and stabilizer (exact)	none
τ	1	sharper ($\tau < 1$) or more diffuse ($\tau > 1$) routing	none (VGG-16 0.035 → 0.037, ResNet-50 0.089 → 0.094)
λ, σ^2	λ=0	attenuates routes through high-variance units	one ADF pass (ResNet-50 0.089 → 0.140)
λ_{sm}, σ^2	0	downweights high-variance keys in attention routing	one ADF pass (ViT-B/16 0.097 → 0.172)
$\tilde{\Phi}$	off	smooth stop/continue gate	needs the ADF σ (one ADF pass)
N	off	averages over input transformations (token-grid artifacts)	$\times N$ (ViT-B/16, N=50: 4.93; with λ_{sm}: 8.66)

4.8.2 Activation-Side Risk Aversion λ via ADF Variance Propagation

The activation-side risk-aversion mode replaces the deterministic activation $a_i^{(l)} = \text{ReLU}(z_i^{(l)})$ that the player observes by a mean–variance confidence bound $V_\lambda = \mu + \lambda\sigma$ ($\lambda \leq 0$) computed from an ADF variance propagation pass (Gast and Roth, 2018). We propagate the exact mean $\mathbb{E}[\text{ReLU}(X)]$ and the variance approximation $\sigma^2\Phi(\mu/\sigma)$ of $\text{Var}[\text{ReLU}(X)]$ for $X \sim \mathcal{N}(\mu, \sigma^2)$ (Gast and Roth, 2018).

The propagated variance is initialized at the input by the scale σ^2 at every input neuron. Negative λ shifts the player's observed game value at stopping layers (not linear layers!) by a lower-confidence amount, attenuating routes through high-variance neurons; this is the activation-side analog of confidence-bound exploration in bandit / RL settings (Auer et al., 2002; Fei et al., 2020). The softmax-shift variant λ_{sm} on the attention-logit oracle (ViT) is treated separately in Section 5.4.3.

4.8.3 Noisy-Observation Gate $\tilde{\Phi}$

The noisy-observation mode replaces the player's pure stop/continue policy at every activation state by a *mixed* policy. At $s_{i,q}^{(l,\text{act})}$, the player continues with probability

$$\tilde{\Phi}(z_i^{(l)}) \;:=\; \Phi\big(\mu_i^{(l)}/\sigma_i^{(l)} + \lambda\big) \tag{4.102}$$

and stops with the complementary probability $1 - \tilde{\Phi}(z_i^{(l)})$, where Φ is the standard normal *Cumulative Distribution Function* (CDF) and $\mu_i^{(l)}$, $\sigma_i^{(l)}$ are the ADF mean and standard deviation of $z_i^{(l)}$ (Section 4.8.2). The analysis below takes $\mu_i^{(l)} = z_i^{(l)}$, $\sigma_i^{(l)} = 1$ and $\lambda = 0$; the general case follows with

$z \mapsto \mu + \lambda\sigma$ and $\mathscr{R}_\Phi \mapsto \sigma\mathscr{R}_\Phi$. The mixed policy (4.102) arises as the unique solution of a regularized maximization that the player can solve directly. Define the probit regularizer

$$\mathscr{R}_\Phi(\pi) \;:=\; \phi\big(\Phi^{-1}(\pi)\big), \qquad \pi \in [0,1], \tag{4.103}$$

where $\phi(y) = (2\pi)^{-1/2}\, e^{-y^2/2}$ is the standard normal probability density and Φ^{-1} is the quantile function (inverse of the standard normal CDF).

Lemma 4.11 (Probit dual of the soft gate). *For every $z \in \mathbb{R}$, the regularized maximization*

$$\max_{\pi\in[0,1]} \big\{\, \pi z \;+\; \mathscr{R}_\Phi(\pi) \,\big\} \tag{4.104}$$

admits the unique maximizer $\pi^\star = \Phi(z)$ and optimum value

$$z\,\Phi(z) + \phi(z) \;=\; \mathbb{E}_{\eta\sim\mathscr{N}(0,1)}\big[\max(z+\eta, 0)\big]. \tag{4.105}$$

Proof. Using $\phi'(y) = -y\,\phi(y)$ and the inverse-function derivative $(\Phi^{-1})'(\pi) = 1/\phi(\Phi^{-1}(\pi))$,

$$\frac{\mathrm{d}\mathscr{R}_\Phi}{\mathrm{d}\pi} \;=\; \phi'\big(\Phi^{-1}(\pi)\big)\cdot\frac{1}{\phi(\Phi^{-1}(\pi))} \;=\; -\Phi^{-1}(\pi). \tag{4.106}$$

The first-order condition $z - \Phi^{-1}(\pi) = 0$ yields $\pi^\star = \Phi(z)$; substituting back and using $\phi(\Phi^{-1}(\Phi(z))) = \phi(z)$ gives the left-hand side of (4.105). The right-hand identity is the standard Gaussian smoothing $\mathbb{E}[\max(z+\eta,0)] = z\Phi(z) + \phi(z)$. □

Remark 4.5 (Optimal policy under perceived payoff noise). The optimum $\pi^\star = \Phi(z_i^{(l)})$ admits an equivalent interpretation as the marginal of an optimal pure rule: the player perceives the continuation payoff at $s_{i,q}^{(l)}$ as the noisy quantity $z_i^{(l)} + \eta$ with $\eta \sim \mathscr{N}(0,1)$ independent across states, and applies the noiseless-optimal pure rule *"continue iff the perceived payoff is strictly positive"* on this perception. Marginalizing the rule over η at fixed $z_i^{(l)}$ recovers the continue probability,

$$\mathbb{P}_{\eta\sim\mathscr{N}(0,1)}\big(z_i^{(l)} + \eta > 0\big) \;=\; \Phi(z_i^{(l)}). \tag{4.107}$$

The realized continuation payoff is the deterministic $z_i^{(l)}$ from the actual network forward pass: the noise η lives in the player's perception of the payoff while the environment pays out $z_i^{(l)}$ on continuation, so the expected backward Bellman comparison at $s_{i,q}^{(l)}$ is the soft mass $\Phi(z_i^{(l)}) \cdot z_i^{(l)}$ in place of the deterministic $\mathbf{1}[z_i^{(l)} > 0] \cdot z_i^{(l)}$.

The regularizer $\mathscr{R}_\Phi$ is symmetric around $\pi = \frac{1}{2}$ and peaks there at $\phi(0) = 1/\sqrt{2\pi}$ (where $\Phi^{-1}(\frac{1}{2}) = 0$), tapering to 0 at the boundary $\pi \in \{0,1\}$. It rewards *indecisiveness* — exactly the role Shannon entropy $\mathscr{H}(\pi)$ plays in the Softplus variant of §4.9, where the entropy bonus is the active player's surplus from being allowed to mix. The optimum $\pi^\star = \Phi(z) = \mathbb{E}_{\eta\sim\mathscr{N}(0,1)}[\,\mathbf{1}(z+\eta > 0)\,]$ is the hard ReLU gate averaged over a Gaussian shift of its threshold, aligning with the original stochastic-gate motivation for GELU (Hendrycks and Gimpel, 2016). The optimal value (4.105) is the Gaussian-smoothed ReLU $z\Phi(z) + \phi(z)$, whose derivative is Φ; it is not $\mathrm{GELU}(z) = z\Phi(z)$, which is not convex and hence not the conjugate of any regularizer.

On ReLU networks the forward pass still computes $\mathrm{ReLU}(z_i^{(l)})$ exactly, so the downstream game values inherited from Theorem 4.5 are unchanged: the player evaluates the soft gate $\tilde{\Phi}$ on the *current* layer

and reuses the original network activations for everything below. There, $\tilde{\Phi}$ is an optional smoothing mode (used in the VGG-16 SG row) without an exactness claim.

GELU networks. On a GELU network, $\tilde{\Phi}$ multiplies the occupation-measure recursion by $\Phi(z_i^{(l)})$ at every layer, which is the chain rule of the gate-conditioned map $a_i^{(l)} = \Phi(\bar{z}_i^{(l)})\, z_i^{(l)}$ with the gate argument $\bar{z}_i^{(l)}$ held at its forward value, the conditioning used for ViTs (Section 5.4.4). It is not the GELU gradient, which needs $\mathrm{GELU}'(z) = \Phi(z) + z\phi(z)$; the substitution $\mathbf{1}[z > 0] \to \Phi(z)$ is exact only for the Gaussian-smoothed-ReLU network, as the entropy-regularized gate is for Softplus (Section 4.9).

4.8.4 Fan-in entropy bias λ_{ent}

The fan-in entropy mode lowers the bias, i.e. the continuation payoff (SG) or bias-node payoff (RG), of neurons with diffuse fan-in: $b_j^{(l)} \mapsto b_j^{(l)} - \lambda_{\mathrm{ent}} \tilde{\mathscr{H}}_j^{(l)}$ with $\tilde{\mathscr{H}}_j^{(l)} = -\sum_i p_{ij} \log p_{ij} / \log n_j$, $p_{ij} = |W_{ij}^{(l)}| / \gamma_j^{(l)}$ and n_j the fan-in of neuron j. It depends on the weights only (negligible cost) and changes the forward pass of the explained network.

4.9 Softplus Variant of the Stopping Game

We introduce the entropy regularization that Gaubert & Vlassopoulos (2025) apply at activation states to model networks with Softplus instead of ReLU activations and, similarly to the ReLU case, adapt it to the Stopping Decomposition of Section 4.6.1: the player-specific game values agree with the activations of a Softplus Decomposition, and the gradient of the Softplus network is recovered by a linear combination of occupation measures.

Modified rule: entropy-regularized activation-state policy. The technical tool throughout this section, and re-used by the Routing Game in Section 4.7 and the attention block of Section 5.4, is the standard Legendre–Fenchel identity for Shannon entropy: for every $\theta > 0$, $n \in \mathbb{N}$, and payoff vector $r \in \mathbb{R}^n$,

$$\max_{\pi \in \Delta_n} \left\{ \sum_{i=1}^{n} \pi(i)\, r_i + \theta\, \mathscr{H}(\pi) \right\} = \theta \, \log \sum_{i=1}^{n} \exp\!\left(\frac{r_i}{\theta}\right), \qquad \pi^\star(i) \propto \exp\!\left(\frac{r_i}{\theta}\right), \tag{4.108}$$

with $\mathscr{H}(\pi) := -\sum_i \pi(i) \log \pi(i)$; the unique maximizer is the Gibbs measure at inverse temperature $1/\theta$. Identity (4.108) is the Fenchel conjugate of the (scaled) negative entropy (Boyd and Vandenberghe, 2004).

Only the active player's stop/continue choice changes. At every activation state $s = s_{j,q}^{(l,\mathrm{act})}$, replace the binary maximization (4.31) by an entropy-regularized maximization over mixed policies $\pi_q(s) \in \Delta(\{\mathrm{stop},\mathrm{cont}\})$ at temperature θ. With $\tilde{Q}_q^\pi(s,\mathrm{stop}) = \tilde{Q}_{q'}^\pi(s,\mathrm{stop}) = 0$ (cf. (4.27)) and $\tilde{Q}_p^\pi(s,\mathrm{cont})$ as in (4.28), the soft Bellman recursion for the active-player advantage is

$$V_q^\pi(s) := \max_{\nu \in \Delta(\{\mathrm{stop},\mathrm{cont}\})} \Big\{ \textstyle\sum_a \nu(a) \big(\tilde{Q}_q^\pi(s,a) - \tilde{Q}_{q'}^\pi(s,a) \big) + \theta\, \mathscr{H}(\nu) \Big\}. \tag{4.109}$$

Applying (4.108) to the binary action set {stop,cont} with payoffs 0 and $\tilde{Q}^{\pi}_{q}(s,\mathrm{cont}) - \tilde{Q}^{\pi}_{q'}(s,\mathrm{cont})$ immediately yields the unique optimizer as the Gibbs policy at inverse temperature $1/\theta$ and the closed-form soft advantage,

$$\pi^{\star}_{q}\big(\mathrm{cont} \,\big|\, s\big) = \mathrm{sp}'_{\theta}\big(\tilde{Q}^{\pi}_{q}(s,\mathrm{cont}) - \tilde{Q}^{\pi}_{q'}(s,\mathrm{cont})\big), \qquad V^{\pi}_{q}(s) = \mathrm{sp}_{\theta}\big(\tilde{Q}^{\pi}_{q}(s,\mathrm{cont}) - \tilde{Q}^{\pi}_{q'}(s,\mathrm{cont})\big), \tag{4.110}$$

with $\mathrm{sp}_{\theta}(z) = \theta \log(1+e^{z/\theta})$ and $\mathrm{sp}'_{\theta}(z) = 1/(1+e^{-z/\theta})$. The transition kernel (4.23)–(4.24), the payoffs r_p, the Bellman equations (4.26)–(4.28) for $\tilde{Q}^{\pi}_{p}$ and the value recursion (4.31) for $\tilde{V}^{\pi}_{p}$ are unchanged, except that π is no longer pure and the entropy bonus $\theta\mathscr{H}(\pi_q(s))$ is credited to the active player q.

Theorem 4.12 (Softplus forward equivalence, `Lean_58`). *The entropy-regularized Stopping Game of (4.109) admits an SPNE whose activation-state policy is the Gibbs policy (4.110); it is unique on networks without max-pooling nodes, since (4.109) is strictly concave (ties at max-pooling states are resolved as in Section 4.5). Under this equilibrium, at every activation state $s^{(l,\mathrm{act})}_{j,q}$,*

$$\tilde{V}^{\pi^{\star}}_{p}\big(s^{(l,\mathrm{act})}_{j,q}\big) \;=\; a^{(l,p\oplus q,\,\theta)}_{\mathrm{soft},j}, \qquad V^{\pi^{\star}}_{q}\big(s^{(l,\mathrm{act})}_{j,q}\big) \;=\; \mathrm{sp}_{\theta}\big(z^{(l)}_{j}\big), \tag{4.111}$$

where $a^{(l,\pm,\theta)}_{\mathrm{soft},j}$ are the Softplus-Decomposition activations of the Softplus network with activation sp_θ, propagated by (4.8)–(4.9) from $a^{(0,\pm,\theta)}_{\mathrm{soft},k} = x^{\pm}_{k}$ with the decomposition function

$$f^{\theta}_{\mathrm{soft}}(z^{+},z^{-}) \;\coloneqq\; \big(\,\mathrm{sp}'_{\theta}(z^{+}-z^{-})\,z^{+} + \theta\,\mathscr{H}_{\theta}(z^{+}-z^{-}),\;\mathrm{sp}'_{\theta}(z^{+}-z^{-})\,z^{-}\,\big), \tag{4.112}$$

and $\mathscr{H}_{\theta}(z) \coloneqq \mathscr{H}(\mathrm{sp}'_{\theta}(z))$ the binary entropy of the Gibbs weight.

Proof. By backward induction in l, mirroring the proof of Theorem 4.5, with the same base case $\tilde{V}^{\pi^{\star}}_{p}(s^{(0)}_{k,q}) = x^{p\oplus q}_{k} = a^{(0,p\oplus q,\theta)}_{\mathrm{soft},k}$. For $l \geq 1$, assume (4.111) holds at layer $l-1$, so the decomposed activations obey $a^{(l-1,+,\theta)}_{\mathrm{soft},i} - a^{(l-1,-,\theta)}_{\mathrm{soft},i} = a^{(l-1)}_{i}$. Substituting into (4.28), the Q-difference of (4.110) computes to $b^{(l)}_{j} + \sum_{i,r} W^{(l,q\oplus r)}_{ij}\,\xi_{q,r}\,a^{(l-1)}_{i} = z^{(l)}_{j}$, using $\sum_{r} W^{(l,q\oplus r)}_{ij}\xi_{q,r} = W^{(l)}_{ij}$ exactly as in the proof of Theorem 4.5. Hence, the Gibbs policy is $\pi^{\star}_{q}(\mathrm{cont} \mid \cdot) = \mathrm{sp}'_{\theta}(z^{(l)}_{j})$, and (4.110) gives $V^{\pi^{\star}}_{q}(s^{(l,\mathrm{act})}_{j,q}) = \mathrm{sp}_{\theta}(z^{(l)}_{j})$, the second identity of (4.111).

For the first identity, for the active player ($p = q$, so $p \oplus q = +$), the entropy-regularized optimization (4.109) attributes both the routed expectation and the entropy bonus to player q: $\tilde{V}^{\pi^{\star}}_{q}(s^{(l,\mathrm{act})}_{j,q}) = \mathrm{sp}'_{\theta}(z^{(l)}_{j})\,z^{(l,+)}_{j} + \theta\,\mathscr{H}_{\theta}(z^{(l)}_{j}) = a^{(l,+,\theta)}_{\mathrm{soft},j}$, where $\tilde{Q}^{\pi^{\star}}_{q}(\cdot,\mathrm{cont}) = z^{(l,+)}_{j}$ follows from the inductive hypothesis via (4.8). For the opponent ($p = q'$, so $p \oplus q = -$), the entropy bonus is absent (it is the active player's surplus from being allowed to mix) and only the routed expectation contributes: $\tilde{V}^{\pi^{\star}}_{q'}(s^{(l,\mathrm{act})}_{j,q}) = \mathrm{sp}'_{\theta}(z^{(l)}_{j})\,z^{(l,-)}_{j} = a^{(l,-,\theta)}_{\mathrm{soft},j}$ by (4.9). The two identities together close the induction. □

Gradient recovery on Softplus networks. The discounted occupation measure of Definition 4.6 is unchanged in form, but the trajectory law Π^{θ}_{x} is now generated by the Gibbs policies (4.110) in place of the deterministic stop/continue rule.

Theorem 4.13 (Softplus gradient recovery, `Lean_59`). *Under the SPNE of Theorem 4.12 on the Softplus network with activation sp_θ, the discounted occupation difference recovers the Softplus-network*

gradient exactly as in Theorem 4.8:

$$\frac{\partial a_j^{(m)}}{\partial a_i^{(l)}} \;=\; \xi_{q,+}\Big(\Gamma_x\big(s_{i,+}^{(l,\mathrm{act})} \mid s\big) - \Gamma_x\big(s_{i,-}^{(l,\mathrm{act})} \mid s\big)\Big), \qquad s = s_{j,q}^{(m,\mathrm{act})}, \tag{4.113}$$

with Γ_x now driven by Π_x^θ.

Proof. Write $D(s_{h,r}^{(m',\mathrm{act})}) \coloneqq \Gamma_x(s_{i,r}^{(l,\mathrm{act})} \mid s_{h,r}^{(m',\mathrm{act})}) - \Gamma_x(s_{i,r'}^{(l,\mathrm{act})} \mid s_{h,r}^{(m',\mathrm{act})})$, the right-hand side above. The one-step occupation recursion in the proof of Theorem 4.8 holds with the gate $\mathbf{1}[z_j^{(m)} > 0]$ replaced by the Gibbs continuation probability $\mathrm{sp}'_\theta(z_j^{(m)})$; taking its difference over the two targets gives $D(s_{j,q}^{(m,\mathrm{act})}) = \mathrm{sp}'_\theta(z_j^{(m)}) \sum_h \big(W_{hj}^{(m,+)} D(s_{h,q}^{(m-1,\mathrm{act})}) - W_{hj}^{(m,-)} D(s_{h,q'}^{(m-1,\mathrm{act})})\big)$. By induction on $m-l$, $D(s_{h,r}^{(m-1,\mathrm{act})}) = \partial a_h^{(m-1)}/\partial a_i^{(l)}$ for both r, so the right-hand side is $\mathrm{sp}'_\theta(z_j^{(m)}) \sum_h W_{hj}^{(m)}\, \partial a_h^{(m-1)}/\partial a_i^{(l)}$, the chain rule at a Softplus gate; the base case $D(s_{i,r}^{(l,\mathrm{act})}) = 1$ and the addition and max-pooling steps are unchanged. Unlike Part 1 of Theorem 4.8, the individual occupation measures are not derivatives of the decomposed activations (4.112), because the soft gate depends on them; only their difference is. □

Probit analog and GELU. Replacing $\theta\mathscr{H}$ in (4.109) by the probit regularizer $\mathscr{R}_\Phi(\pi) = \phi(\Phi^{-1}(\pi))$ gives, by Lemma 4.11, the continuation probability $\Phi(z)$ and the value $z\Phi(z)+\phi(z) = \mathbb{E}_{\eta\sim\mathcal{N}(0,1)}[\mathrm{ReLU}(z+\eta)]$, so the proofs above carry over verbatim with sp_θ replaced by this Gaussian-smoothed ReLU, sp'_θ by its derivative Φ and $\theta\mathscr{H}_\theta(z)$ by $\phi(z)$: the probit-regularized game recovers Gaussian-smoothed-ReLU networks and their exact gradient. GELU, $z\Phi(z)$, is *not* covered: it is not convex, so it is not the value $\max_\pi\{\pi z + \mathscr{R}(\pi)\}$ (a Fenchel conjugate) of any regularized stop/continue choice. Substituting the gate $\Phi(z)$ into Theorem 4.8 on a GELU network yields the gradient of the gate-conditioned map $a = \Phi(z^{\mathrm{fwd}})\, z$, not the GELU gradient $\Phi(z) + z\phi(z)$ (Section 4.8.3).

Relation to the curvature-based defense of Dombrowski et al. Dombrowski et al. (2022) train Softplus networks from scratch as a defense against gradient-explanation manipulation. Their argument bounds the Frobenius norm of the network Hessian: replacing ReLU, whose distributional second derivative $\delta(z)$ leaves $\|\nabla^2 f\|_F$ unbounded, by sp_θ with $|\mathrm{sp}''_\theta(z)| \le 1/(4\theta)$ yields a uniform bound on $\|\nabla^2 f\|_F$, and the gradient theorem then bounds the maximal change of the gradient explanation under input perturbations. Theorems 4.12–4.13 place this Softplus replacement inside our framework as a single change in the game: entropy-regularizing the activation-state stop/continue policy at temperature θ. Whether the Hellinger response of the resulting trajectory law (Section 8.3) yields bounds sharper than, or differently targeted from, the Lipschitz bounds of Dombrowski et al. (2022) is left for future work.

5 Extensions: Attention and Transformers

This is an extensions-and-outlook chapter with lighter rigor than the frameworks of Chapters 3 and 4 just developed: it carries the decomposition and routing-game machinery over to multiplicative layers and Vision Transformers (ViTs), with the weight on how the resulting backward pass is actually implemented.

Two features separate a Transformer (Vaswani et al., 2017) from the architectures treated so far. First, attention multiplies *activations with activations*: the score matrix $QK^\top$ and the mixing product AV are bilinear in input-dependent quantities, so the fixed-weight linear algebra of Chapter 3 and the routing states of Chapter 4 no longer apply directly. Second, softmax and layer normalization are smooth, *normalizing* non-linearities that couple whole token rows, unlike the pointwise gates of Theorem 2.15. The chapter addresses both once per framework: Section 5.2 extends the DC decomposition to multiplicative components, and Section 5.4 extends the RG by reading the softmax row as a policy. Section 5.4.4 then turns both into a concrete backward pass on ViT-B/16 — the implementation that instantiates the split-stream saliency methods developed next in Chapter 6 and evaluated in Chapter 7.

5.1 Related Work: Interpretability in Transformers

Attribution research on Transformers began at the attention weights themselves: since the rows of an attention matrix are distributions over tokens, early analyses of head roles and pruning read them directly as explanations (Voita et al., 2019). Abnar & Zuidema (2020) objected that single-layer weights ignore propagation through depth, and proposed *attention rollout* — multiplying the residual-corrected attention matrices across layers — and *attention flow*, a max-flow computation on the layered attention graph. Both commit to a routing view: attention matrices are transition kernels of a walk on tokens, and an explanation states where that walk deposits its mass. What they omit is everything outside the attention weights — value magnitudes, the MLPs, the sign structure of the projections — so rollout can assign high relevance to tokens whose values contribute nothing.

A second line adapts LRP (Bach et al., 2015) to the Transformer. Chefer et al. (2021a) combined LRP-style relevance with attention gradients to propagate class-specific relevance through self-attention, later generalized to co-attention and encoder–decoder architectures (Chefer et al., 2021b). Ali et al. (2022) made the structural point that shapes this chapter: naive gradient propagation through softmax and layer normalization *violates conservation*, since both route relevance through data-dependent normalizers, whereas propagating through a locally linearized map with those outputs and statistics held constant ("detached") restores it. AttnLRP (Achtibat et al., 2024) systematized this into dedicated rules for the bilinear products and non-linearities, scaling to LLMs, with concept-level (Achtibat et al., 2023) and sign-aware (Vukadin et al., 2024) extensions. On the vision side, DAVE (Wróbel et al., 2026) traced grid artifacts to the 16×16 patch tokenization of ViTs (Dosovitskiy et al., 2021) — related to the artifact tokens of Darcet et al. (2024) — and countered them with *conditioned* (detached) operators and attribution averaging over random spatial transformations. At LLM scale, gradient approximations of activation patching (Kramár et al., 2024; Syed et al., 2024) localize circuits against interventions (Hanna et al., 2024), and attribution increasingly acts as a control signal for head identification and guided decoding (Kahardipraja et al., 2025; Komorowski et al., 2025).

The two lines pull in opposite directions: rollout has clean routing semantics but ignores the network outside the attention weights, while transformer-LRP covers the whole network yet justifies detachment operationally — softmax and LayerNorm are detached *because* conservation then holds — rather than

from a model of the backward pass. This chapter closes the gap from both ends: Section 5.2 keeps the split streams intact across matrix and Hadamard products (at the price of leaving the convex world for maxima of posynomials), and Section 5.4 shows the detached softmax row to be the equilibrium policy of an entropy-regularized oracle player, recovering the conservative attention-block rules of Ali et al. (2022) as occupation measures of the RG — the routing intuition of Abnar & Zuidema (2020), made exact at the attention blocks.

5.2 Decomposing Multiplicative Operations

The decompositions of Chapter 3 treat networks whose only non-linearities are pointwise gates and maxima between *fixed* linear maps. Gating layers, squeeze-and-excitation blocks, and attention add a new primitive: two activation streams multiplied entrywise (Hadamard product) or contracted against each other (matrix product). This section extends the split-stream construction to such components, following Zimmermann & Loho (2026); results are stated at the level of the split rules and the main theorem, with proof sketches for the inductive constructions.

The gated product rule. Suppose two scalar quantities are carried in split form, $a = a^+ - a^-$ and $b = b^+ - b^-$ with all four streams non-negative, as the valid decompositions of Section 3.1 guarantee. Their product again splits into non-negative parts by expanding and sorting the four cross terms by sign:

$$(a^+ - a^-)(b^+ - b^-) \;=\; \underbrace{\left(a^+b^+ + a^-b^-\right)}_{=:\,(ab)^+} \;-\; \underbrace{\left(a^+b^- + a^-b^+\right)}_{=:\,(ab)^-}. \tag{5.1}$$

Both bracketed terms are non-negative and non-decreasing in each incoming stream, so multiplication maps monotone stream pairs to monotone stream pairs — the analogue of routing $W^{(l,+)}$ and $W^{(l,-)}$ into separate channels in Section 3.3. Applied entrywise, (5.1) splits Hadamard layers; applied to every scalar contraction, it splits matrix products of two split-stream tensors. What is *not* preserved is convexity: products of convex functions need not be convex, so the split streams need not stay convex in the input. The correct replacement for "convex CPWL function" is "maximum of finitely many posynomials", a *posynomial* being a polynomial with non-negative coefficients: posynomials are closed under addition, multiplication, and positive scaling, and are monotone on the non-negative orthant.

Maxout multiplication networks. To state the splitting theorem in its natural generality we enlarge the architecture class. A rank-k Maxout unit (Goodfellow et al., 2013) computes the maximum of k affine forms and reduces to ReLU for $k = 2$ with one form fixed to zero.

Definition 5.1 (Maxout multiplication network). A *rank-k Maxout multiplication network* is a finite directed acyclic graph of *input neurons*, *addition neurons*, and *multiplication neurons*. An addition neuron v carries weight vectors $w^{uv} \in \mathbb{R}^k$ on its incoming arcs and computes the rank-k Maxout activation $a^v = \max_{i\in[k]} \sum_u w_i^{uv} a^u$; a multiplication neuron carries natural exponents $w^{uv} \in \mathbb{N}$ and computes the monomial $a^v = \prod_u (a^u)^{w^{uv}}$. The network is called *monotone* if all weights are non-negative, and *input-convex* if all weights are non-negative except possibly those on arcs leaving input neurons.

Linear and convolutional layers, biases (via a constant input neuron), maxima and max-pooling, residual connections, Hadamard products, and matrix products are all expressible here; restricting exponents to natural numbers avoids divisions and fractional powers of possibly negative quantities. The two structural adjectives of Definition 5.1 pin down what such networks compute.

Proposition 5.1 (Representation by maxima of posynomials, Lean_65). *A monotone rank-k Maxout multiplication network with non-negative intermediate quantities represents the maximum of finitely many posynomials in its non-negative input variables, and is monotone on the non-negative orthant. This representation does not imply ordinary convexity: the positive-coefficient monomial xy is a counterexample. Allowing signed input-incident affine maps likewise supplies no general input-convexity guarantee in the presence of multiplication.*

Proof sketch. Induct along a topological order on the non-negative orthant. Coordinate projections are posynomials. Non-negative scaling, addition and maxima preserve finite maxima of posynomials. Products do so when the factors are non-negative, by distributing the product over the finite maxima. These are precisely the stated network operations. The failure of convexity is witnessed by $F(x,y) = xy$: $F(1,1) = 1$ exceeds $\frac{1}{2}F(2,0) + \frac{1}{2}F(0,2) = 0$. □

The splitting theorem. The main result transfers the split of Section 3.3 to this class. Two local identities drive it. Maxout units are split by the maximum-shift identity

$$\max_{j\in[k]}\left(a_j^+ - a_j^-\right) \;=\; \max_{j\in[k]}\left\{a_j^+ + \sum_{i\neq j} a_i^-\right\} \;-\; \sum_{i\in[k]} a_i^-, \tag{5.2}$$

which generalizes both the ReLU splitting identity $\mathrm{ReLU}(a-b) = \max\{a,b\} - b$ of Section 3.3 and the gated max-pooling rule of Section 3.7; multiplication neurons are split by the multinomial generalization of (5.1), expanding $\prod_u (a^{u,+} - a^{u,-})^{w^{uv}}$ and sorting the resulting monomials by the parity of the number of $(-)$-stream factors.

Theorem 5.2 (Monotone split of multiplicative ReLU networks, Lean_66). *Let N be a (residual) ReLU network containing linear, convolutional, maximum, matrix-product, and Hadamard-product components. Then, on non-negative split-input variables, N has two* monotone *rank-2 Maxout multiplication networks N^+ and N^- satisfying*

$$N(x) = N^+(x^+,x^-) - N^-(x^+,x^-), \qquad x = x^+ - x^-, \quad x^\pm \geq 0,$$

Each component is monotone as a function of its non-negative split inputs and is representable there by a finite maximum of posynomials. This gives a difference-of-monotone representation; it does not establish a difference-of-convex representation in the original inputs. If the number of expanded monomials per multiplication neuron is uniformly bounded, the sizes of the split networks are linear in the size of N.

Proof sketch. Process the neurons in topological order, replacing each by a coordinated pair (v^+, v^-) preserving the computed function; the construction extends the extended-formulation argument of Hertrich & Loho (2025). *Input neurons* are supplied as non-negative split variables x^+,x^- (for instance $\mathrm{ReLU}(x)$,$\mathrm{ReLU}(-x)$) with output weights sign-flipped toward the $(-)$-copy; keeping inputs unsplit does not, in general, yield a convex variant once multiplication is present. *Addition (Maxout) neurons* split their weight vectors entrywise into $[w^{uv}]^+ - [w^{uv}]^-$ and apply (5.2): the maximum term defines v^+, the subtracted sum v^-, both with non-negative weights. *Multiplication neurons* apply the sign-sorted multinomial expansion; one auxiliary multiplication neuron per summand feeds collecting addition neurons v^+ (even parity) and v^- (odd parity), all non-negative. Each rewrite is an identity, so correctness telescopes to $N = N^+ - N^-$; all weights are non-negative, hence both components are monotone in the split variables, with the posynomial representation from Proposition 5.1. The

multinomial summand count is the source of growth; bounded fan-in alone does not control it if exponents are unbounded. A uniform bound on the expanded monomial count is sufficient. □

Theorem 5.2 extends the algebraic split to multiplicative layers. The two streams retain non-negativity and monotonicity in the split inputs, while their difference preserves the original computation. Convexity is not preserved by arbitrary products, so the DC interpretation and convex-geometric guarantees of Chapter 3 require additional hypotheses in this setting.

Remark 5.1 (Smooth normalizers: softmax and sigmoid gates). Theorem 5.2 covers the piecewise-polynomial skeleton of an attention block — projections, the bilinear products $QK^\top$ and AV, residual additions — but not the softmax, which is smooth and divides by the partition sum. Three treatments are available. *(i) CPWL approximation:* replacing exp and the division by CPWL surrogates on a compact range turns the block into a Maxout multiplication network, at a size cost proportional to the resolution. *(ii) Streamwise smooth splitting:* the scalar non-linearities (sigmoid gates, exp of bounded scores, GELU in the MLP blocks (Hendrycks and Gimpel, 2016)) have integrable curvature on bounded domains, so Theorem 2.15 splits them pointwise into monotone pairs that enter the product rule (5.1). *(iii) Detachment:* the softmax output A is treated as a forward-pass constant, so the block becomes the fixed-coefficient linear map $X \mapsto AV(X)$ with no multiplicative machinery. Option (iii) is what transformer-LRP practice converged to (Achtibat et al., 2024; Ali et al., 2022; Wróbel et al., 2026); the next section shows it is not an approximation but an exact statement about *whose* decision the softmax is.

5.3 Attention via a detached-logit oracle

Two ViT subtleties break conservative transport: LayerNorm's input-dependent statistics and the data-dependent QK-softmax. LayerNorm statistics are detached so the layer becomes a pointwise linear map (Ali et al., 2022; Wróbel et al., 2026); the QK-softmax block is replaced by an *oracle game* emitting a detached reference policy, the softmax row $A_{t\cdot}$, with occupation-measure trajectories flowing only through the Value route. Like conservative propagation (Ali et al., 2022), this explains the network conditioned on its realized attention pattern.

Theorem 5.3 (Attention, informal). *With reference policy $A_{t\cdot}$, the KL-regularized Value-routing subgame has an SPNE whose occupation measure at the block input equals $\alpha\beta$-LRP on the conditioned effective map $x \mapsto \sum_k A_{tk}\, x_k W_v + b_v$ (Theorem 5.6).*

Our implementation applies this rule to the attention product and the Value projection as one composite step; a full gradient fallback through the attention block degrades localization (AL 0.624 vs. 0.711 at the localization-selected configuration). Equivariant Reynolds averaging optionally suppresses tokenization-grid artifacts (Wróbel et al., 2026) (RG+Eq). Full KL formulation, oracle embedding, occupation-measure theorem and implementation are in Section 5.4.

5.4 Extension: Vision Transformer Attention

This section develops the full state-space formulation for attention modules in the RG. The development proceeds in two stages. §5.4.1 extends the backward walk of Theorem 4.10 through the attention block by treating the softmax row $A_{t,\cdot}$ as a fixed reference policy on the keys, derives the resulting Value-routing equilibrium from the Legendre–Fenchel transform of the KL divergence, and proves $\alpha\beta$-LRP recovery for the conditioned effective map $X \mapsto O$ in the occupation-measure notation Γ of

Theorem 4.10. §5.4.2 then identifies the fixed reference $A_{t,\cdot}$ with the policy of an auxiliary entropy-regularized QK oracle player, closing the loop with the rest of the game-theoretic framework. The risk-averse attention variant is in §5.4.3; forward-pass detachments and gradient-hook conventions used by all ViT experiments are collected in §5.4.4; the equivariant spatial averaging is in §5.4.5.

5.4.1 Attention as fixed-reference KL Value routing

Target rule: $\alpha\beta$-LRP with detached logits

We first state the rule that the Routing-Game state-space construction below is designed to recover. We focus on a single attention head; the multi-head extension carries an additional head index h and the recovery is per-head. Note that ViT-B/16 implements the Value projection as a single $D \times D$ linear $v = XW_v$ followed by a reshape into n_h heads (the per-head matrix $W_v^{(h)} \in \mathbb{R}^{D\times d_h}$ is the head-h column block of the full W_v), so one input embedding $X_{k,e}$ feeds Value entries of every head and the per-head R^X contributions sum at $X_{k,e}$ to give the total. The per-head description below applies unchanged. We omit the ε stabilizer inside the attention block — the surrounding ReLU recursion of §4.7.2 carries its own ε on every linear-routing state and that mechanism transfers to the $Av{+}W_v$ composite without modification, but tracking it through the tensor indexing here only obscures the structure.

Forward pass. The Value projection is applied per token: for each $k \in \{1,\dots,S\}$,

$$v_{k,d} \;=\; \sum_e W_{v,e,d}\, X_{k,e} + b_{v,d}, \qquad d \in \{1,\dots,d_h\}, \tag{5.3}$$

where the same matrix $W_v \in \mathbb{R}^{D\times d_h}$ and bias $b_v \in \mathbb{R}^{d_h}$ are applied independently to every token. The detached attention matrix $A \in \mathbb{R}^{S\times S}_{\geq 0}$ (with $A_{t,k} \geq 0$ and $\sum_k A_{t,k} = 1$ from softmax) then mixes Values across tokens *independently per feature dimension* d: for each output token $t \in \{1,\dots,S\}$ and feature d,

$$O_{t,d} \;=\; \sum_k A_{t,k}\, v_{k,d} \;=\; \sum_{k,e} A_{t,k}\, W_{v,e,d}\, X_{k,e} + b_{v,d}. \tag{5.4}$$

Composite linear map and κ-stream split. Holding A, the LayerNorm statistics and the GELU gates at their forward values (§5.4.4) is a modeling assumption about the explanation question—given the realized attention policy, through which Value routes does relevance flow?—and not a theorem about relevance through Q and K. Our attention extension is therefore a Value-only attribution conditioned on the realized attention matrix, resembling the conservative propagation of Ali et al. (2022) (CP-LRP), which holds the attention matrix constant, and DAVE's conditioned operators (Wróbel et al., 2026). It differs from AttnLRP (Achtibat et al., 2024), which also propagates relevance through the attention-generating operations (the softmax and the bilinear QK product) and so attributes the Query and Key paths. With A so conditioned, the attention block reduces to the affine *conditioned effective map* $X \mapsto O$ with composite weights $W^{\text{eff}}_{(k,e),\,(t,d)} = A_{t,k}\, W_{v,e,d}$ and bias $b_{v,d}$. Since $A_{t,k} \geq 0$, the sign of the composite weight is the sign of $W_{v,e,d}$:

$$\big[\, W^{\text{eff}}_{(k,e),\,(t,d)} \,\big]^{\kappa} \;=\; A_{t,k}\, W^{\kappa}_{v,e,d}, \qquad \kappa \in \{+,-\}. \tag{5.5}$$

For signed inputs X (LayerNorm outputs), the per-token κ-stream split of the Value tensor (as in the fused input layer of the RG, with the bias as a bias node) and the per-output-feature stream sum are

$$\tilde{v}^{\kappa}_{k,d} \coloneqq \sum_{e} [X_{k,e} W_{v,e,d}]^{\kappa} + [b_{v,d}]^{\kappa}, \qquad Z^{\kappa}_{t,d} \coloneqq \sum_{k} A_{t,k}\, \tilde{v}^{\kappa}_{k,d} = \sum_{k,e} A_{t,k}\, [X_{k,e} W_{v,e,d}]^{\kappa} + [b_{v,d}]^{\kappa}. \quad (5.6)$$

Note that $\tilde{v}^{\kappa}_{k,d}$ is the κ-stream split inherited from W_v and the sign of X (not the standard scalar positive part $[v_{k,d}]^{+} = \max(0, v_{k,d})$); the two coincide only when $v_{k,d}$ has a single-signed predecessor decomposition.

Target rule. Let $R^{O}_{t,d}$ denote the relevance arriving at the output entry $O_{t,d}$ from the surrounding LRP recursion. We define the target rule as the $\alpha\beta$-LRP rule (4.1) on the composite map $X \mapsto O$, written in two stages that carry the κ-stream split $\tilde{v}^{\kappa}_{k,d}$ inherited from W_v via (5.5) through the Value entries. It is not sequential (layer-wise) $\alpha\beta$-LRP on $X \to v \to O$, which splits by the sign of the netted Value entry $v_{k,d}$ and applies (α,β) at both stages; the two rules differ whenever a Value entry has mixed-sign predecessors. The sequential variant is the alternative that this paper does not use: the implementation (Section 5.4.4) and every ViT result reported here apply the composite rule stated below, and the numerical identity between the implementation and (5.9) is checked by the tests of Section A.3.8.

AV mixing $O \to v$. Applied to $O_{t,d} = \sum_k A_{t,k}\, v_{k,d}$, the rule gives the relevance at the Value entry $v_{k,d}$ as the weighted difference of two per-stream contributions:

$$R^{\mathrm{val}}_{k,d} = \alpha\, R^{\mathrm{val},+}_{k,d} - \beta\, R^{\mathrm{val},-}_{k,d}, \qquad R^{\mathrm{val},\kappa}_{k,d} \coloneqq \sum_{t} \frac{A_{t,k}\, \tilde{v}^{\kappa}_{k,d}}{Z^{\kappa}_{t,d}}\, R^{O}_{t,d}, \quad \kappa \in \{+,-\}. \quad (5.7)$$

Value projection $v \to X$. Applied to $v_{k,d} = \sum_e W_{v,e,d}\, X_{k,e} + b_{v,d}$, the rule routes each κ-stream contribution $R^{\mathrm{val},\kappa}_{k,d}$ to the input dim e via the matching κ-stream ratio $[X_{k,e}W_{v,e,d}]^{\kappa}/\tilde{v}^{\kappa}_{k,d}$ (the bias share is absorbed):

$$R^{X}_{k,e} = \sum_{d} \left(\alpha\, \frac{[X_{k,e}W_{v,e,d}]^{+}}{\tilde{v}^{+}_{k,d}}\, R^{\mathrm{val},+}_{k,d} - \beta\, \frac{[X_{k,e}W_{v,e,d}]^{-}}{\tilde{v}^{-}_{k,d}}\, R^{\mathrm{val},-}_{k,d} \right). \quad (5.8)$$

Aggregate. Substituting (5.7) into (5.8) and canceling $\tilde{v}^{\kappa}_{k,d}$ in numerator and denominator expresses $R^{X}_{k,e}$ directly in terms of $R^{O}_{t,d}$:

$$R^{X}_{k,e} = \sum_{t,d} \left(\alpha\, \frac{A_{t,k}\, [X_{k,e}W_{v,e,d}]^{+}}{Z^{+}_{t,d}} - \beta\, \frac{A_{t,k}\, [X_{k,e}W_{v,e,d}]^{-}}{Z^{-}_{t,d}} \right) R^{O}_{t,d}, \quad (5.9)$$

which coincides with what the standard $\alpha\beta$-LRP rule produces directly on the conditioned effective map $X \mapsto O$ with effective weight $W^{\mathrm{eff}}_{(k,e),(t,d)} = A_{t,k}\, W_{v,e,d}$ and bias $b_{v,d}$.

State space

We embed the attention block into the routing-graph state space of §4.7 using the tuple-superscript notation $s_{\bullet}^{(\mathrm{att},\,\mathrm{Side},\,\mathrm{type})}$ throughout, where Side $\in \{\mathrm{O}, \mathrm{val}, \mathrm{X}\}$ identifies the activation slot inside the attention block and type $\in \{\mathrm{act}, \mathrm{lin}\}$ matches state types of §4.7. The state indices encode the tensor structure of the AV mixing and the Value projection, with $t,k \in \{1,\ldots,S\}$ ranging over output/key

tokens (sequence axis), $d \in \{1,\dots,d_h\}$ over the within-head feature axis of the focused head (i.e. the head index h is fixed and d runs over that head's d_h feature dimensions), and $e \in \{1,\dots,D\}$ over the per-token input embedding dimensions.

States for the composite Av+W_v block. The explanatory walk through the attention block visits three state types beyond the surrounding ReLU-network's; all are parametrized by the player label in turn.

- **Output activation states** $s^{(\mathrm{att},\mathrm{O},\mathrm{act})}_{t,d,p}$ at the head's output entry $O_{t,d}$, indexed by output token $t \in \{1,\dots,S\}$, within-head feature dim $d \in \{1,\dots,d_h\}$, and player $p \in \{+,-\}$. These are the entry points of the explanatory walk: the surrounding linear-layer recursion of §4.7.2 delivers occupation mass to $\{s^{(\mathrm{att},\mathrm{O},\mathrm{act})}_{t,d,p}\}_{t,d,p}$ from the layer behind the head.
- **Output sign-branch (linear) states** $s^{(\mathrm{att},\mathrm{O},\mathrm{lin})}_{(t,d),p,\kappa}$, indexed by (t,d,p,κ). These mirror the linear routing states $s^{(l,\mathrm{lin})}_{j,q,\kappa}$ of Definition 4.7 for the AV mixing at output entry (t,d): *this is where the QK logits enter the routing* via the reference $\bar{\pi}_t = A_{t,\cdot}$. The active player picks a key token k by a mixed action $\pi^\star_{t,d,\kappa} \in \Delta(\{1,\dots,S\})$ that combines the AV reference $\bar{\pi}_t$ with the κ-stream Value mass $\tilde{v}^\kappa_{k,d}$.
- **Value-projection linear states** $s^{(\mathrm{att},\mathrm{val},\mathrm{lin})}_{(k,d),p,\kappa}$, indexed by key token k, within-head feature dim d, player p, and sign κ. These are the linear-routing states of the per-token Value projection: the player picks an input dimension e (or the bias node) and the game value satisfies $V_p\big(s^{(\mathrm{att},\mathrm{val},\mathrm{lin})}_{(k,d),p,\kappa}\big) = \ell(\tilde{v}^\kappa_{k,d})$ with $\tilde{v}^\kappa_{k,d}$ from (5.6).
- **Input activation states** $s^{(\mathrm{att},\mathrm{X},\mathrm{act})}_{(k,e),p}$ at the input embedding entry $X_{k,e}$, indexed by token–embedding-dim pair (k,e) and player p. X lives upstream of the Value projection and carries the input-embedding axis e; the within-head feature axis d enters first at the Value projection's output downstream. They are the exit points of the attention block: the surrounding network's recursion resumes from the per-player occupation on $\{s^{(\mathrm{att},\mathrm{X},\mathrm{act})}_{(k,e),p}\}_{(k,e),p}$.

Transitions. The trajectory through the attention block has three steps; all transitions inside the block carry trajectory discount 1 except the sign-oracle split, which uses the same $\{2\alpha, 2\beta\}$ as a ReLU activation state of Definition 4.7. There is only *one* sign-oracle split per output entry (t,d), since $A_{t,k} \geq 0$ absorbs the AV mixing into the W_v sign split.

1) *Sign-oracle split (output activation → output sign-branch, fixed (t,d)).* At $s^{(\mathrm{att},\mathrm{O},\mathrm{act})}_{t,d,p}$ an unobserved Oracle transitions uniformly to $s^{(\mathrm{att},\mathrm{O},\mathrm{lin})}_{(t,d),p,+}$ (player p retains the turn, trajectory discount 2α) or to $s^{(\mathrm{att},\mathrm{O},\mathrm{lin})}_{(t,d),p',-}$ (turn switches to opponent p', discount 2β), each with probability $\frac{1}{2}$. The feature index d is preserved.
2) *Value-routing policy (output sign-branch → Value-projection linear).* At $s^{(\mathrm{att},\mathrm{O},\mathrm{lin})}_{(t,d),p,\kappa}$ the active player picks a key token k by the mixed action

$$\pi^\star_{t,d,\kappa}(k) \;=\; \frac{A_{t,k}\,\tilde{v}^\kappa_{k,d}}{Z^\kappa_{t,d}}, \tag{5.10}$$

derived in §5.4.1 as the equilibrium of a KL-regularized log-payoff problem against the reference $\bar{\pi}_t = A_{t,\cdot}$. The trajectory transitions to $s^{(\mathrm{att},\mathrm{val},\mathrm{lin})}_{(k,d),p,\kappa}$ *with the player label, the sign κ, and the within-head feature dim d preserved*; the only thing changing is the sequence-axis index from output token t to key token k. The trajectory-discount on this transition is 1.

3) *Value-projection routing (Value-projection linear → input activation).* At $s^{(\mathrm{att},\,\mathrm{val},\,\mathrm{lin})}_{(k,d),p,\kappa}$ the active player picks an input dim e by the standard linear-state Gibbs policy of Definition 4.7 on the κ-stream masses $[X_{k,e}W_{v,e,d}]^\kappa$ (the bias node receives $[b_{v,d}]^\kappa/\tilde{v}^\kappa_{k,d}$ and is absorbed):

$$\pi^\star_{\mathrm{val},k,d,\kappa}(e) \;=\; \frac{[X_{k,e}W_{v,e,d}]^\kappa}{\tilde{v}^\kappa_{k,d}}. \tag{5.11}$$

The trajectory transitions to $s^{(\mathrm{att},\,\mathrm{X},\,\mathrm{act})}_{(k,e),p}$ with the player label preserved and trajectory-discount factor 1.

The discounted occupation measure $\Gamma_x(\,\cdot\,)$ of §4.7.2 is evaluated at all four state types along the explanatory walk. The recovery theorem of §5.4.1 then compares the per-input-neuron player-difference of Γ_x at the input activation states with the target rule (5.9).

Legendre–Fenchel transform of the KL divergence

The Value-routing equilibrium below is a direct corollary of the convex-conjugate identity for the KL divergence; we record it as a stand-alone Lemma to make the inheritance explicit.

Lemma 5.4 (Legendre–Fenchel transform of KL divergence, `Lean_67`). *Let $\mathscr{X}$ be a finite set, $\bar\pi \in \Delta(\mathscr{X})$ a reference distribution with full support on $\mathscr{X}$, and $r : \mathscr{X} \to \mathbb{R}$ a real-valued payoff. The KL-regularized linear maximization*

$$F(r;\bar\pi) \;\coloneqq\; \max_{\pi\in\Delta(\mathscr{X})} \Big\{ \mathbb{E}_\pi[r] \;-\; \mathrm{KL}(\pi \,\|\, \bar\pi) \Big\} \tag{5.12}$$

has the following optimizer and optimum:

$$\pi^\star_r(x) \;=\; \frac{\bar\pi(x)\,\exp\big(r(x)\big)}{Z(r;\bar\pi)}, \qquad F(r;\bar\pi) \;=\; \log Z(r;\bar\pi) \;=\; \log \sum_{x\in\mathscr{X}} \bar\pi(x)\,\exp\big(r(x)\big). \tag{5.13}$$

Two specializations are central below. When $r(x) = \ell(u(x))$ for some non-negative $u : \mathscr{X} \to \mathbb{R}_{\ge 0}$ (so $\exp(r(x)) = u(x)$ with the convention $\exp(-\infty) \coloneqq 0$), the value collapses to the $\bar\pi$-weighted linear sum and the optimizer becomes the $\bar\pi$-tilted normalization of u:

$$F\big(\ell(u);\,\bar\pi\big) \;=\; \ell\Big(\sum_x \bar\pi(x)\,u(x)\Big), \qquad \pi^\star_{\ell(u)}(x) \;=\; \frac{\bar\pi(x)\,u(x)}{\sum_{x'} \bar\pi(x')\,u(x')}. \tag{5.14}$$

When $\bar\pi$ is uniform on $\mathscr{X}$, $\mathrm{KL}(\pi \,\|\, \mathrm{Unif}_{\mathscr{X}}) = \log|\mathscr{X}| - \mathscr{H}(\pi)$, so subtracting the KL penalty in (5.12) flips into a $+\mathscr{H}$ entropy bonus:

$$\mathbb{E}_\pi[r] \;-\; \mathrm{KL}\big(\pi \,\|\, \mathrm{Unif}_{\mathscr{X}}\big) \;=\; \mathbb{E}_\pi[r] \;+\; \mathscr{H}(\pi) \;-\; \log|\mathscr{X}|. \tag{5.15}$$

Up to the additive constant $-\log|\mathscr{X}|$, (5.12) therefore reduces to the entropy-regularized maximization of (4.62), with value $\log\sum_x \exp(r(x))$ and optimizer $\pi^\star(x) \propto \exp(r(x))$.

Proof. This is the Gibbs variational principle (Boyd and Vandenberghe, 2004): for every π, $\mathbb{E}_\pi[r] - \mathrm{KL}(\pi \,\|\, \bar\pi) = \log Z(r;\bar\pi) - \mathrm{KL}(\pi \,\|\, \pi^\star_r)$, which is maximal, with value $\log Z(r;\bar\pi)$, exactly at $\pi = \pi^\star_r$. □

Value-routing equilibrium

Fix an output entry (t,d), a sign $\kappa \in \{+,-\}$, and the reference $\bar{\pi}_t = A_{t,\cdot}$ from §5.4.1 (the *same* $\bar{\pi}_t$ for every d, since the AV mixing reuses one row across all feature dims). At the output sign-branch state $s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}$ the active player p solves the KL-regularized log-payoff maximization

$$V_p\big(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}\big) \;:=\; \max_{\pi \in \Delta(\{1,\dots,S\})} \left\{ \sum_{k=1}^{S} \pi(k)\, \ell\big(\tilde{v}^{\kappa}_{k,d}\big) \;-\; \mathrm{KL}\big(\pi \,\|\, \bar{\pi}_t\big) \right\}, \tag{5.16}$$

where $\tilde{v}^{\kappa}_{k,d} \geq 0$ is the κ-stream split of the Value entry $v_{k,d}$ from (5.6) and ℓ is the extended logarithm of §4.7. The payoff $\ell(\tilde{v}^{\kappa}_{k,d})$ on action k is the active-player value of the predecessor Value-projection linear state, $V_p(s^{(\mathrm{att},\,\mathrm{val},\,\mathrm{lin})}_{(k,d),p,\kappa}) = \ell(\tilde{v}^{\kappa}_{k,d})$, so combining with the AV log-weight $\ell(A_{t,k})$ gives composite payoff $\ell(A_{t,k}\,\tilde{v}^{\kappa}_{k,d})$ at the (t,d)-output. Define the per-output-entry $\bar{\pi}_t$-weighted Value mass

$$Z^{\kappa}_{t,d} \;:=\; \sum_{m=1}^{S} \bar{\pi}_t(m)\, \tilde{v}^{\kappa}_{m,d}. \tag{5.17}$$

The dependence on d enters only through $\tilde{v}^{\kappa}_{m,d}$; the AV reference $\bar{\pi}_t$ is shared across all feature dims.

Proposition 5.5 (Value-side optimal policy, `Lean_68`). *If $Z^{\kappa}_{t,d} > 0$, the unique optimizer of (5.16) is*

$$\pi^{\star}_{t,d,\kappa}(k) \;=\; \frac{\bar{\pi}_t(k)\, \tilde{v}^{\kappa}_{k,d}}{Z^{\kappa}_{t,d}}, \tag{5.18}$$

with active-player value $V_p(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}) = \ell(Z^{\kappa}_{t,d})$ and inactive-player value $V_{p'}(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}) = -\infty$ as in Definition 4.7. If $Z^{\kappa}_{t,d} = 0$, no key contributes mass on stream κ and $V_p(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}) = -\infty$.

Proof. Apply the logit specialization (5.14) of Lemma 5.4 with $\mathscr{X} = \{1,\dots,S\}$, $\bar{\pi} = \bar{\pi}_t$, and $u(k) = \tilde{v}^{\kappa}_{k,d}$, so that the payoff $r(k) = \ell(\tilde{v}^{\kappa}_{k,d})$ has $\exp r(k) = \tilde{v}^{\kappa}_{k,d}$. Equation (5.14) immediately gives the optimizer $\pi^{\star}_{t,d,\kappa}$ of (5.18) and value $\ell(Z^{\kappa}_{t,d})$ when $Z^{\kappa}_{t,d} > 0$. The inactive-player value $-\infty$ is the Routing-Game convention of Definition 4.7, inherited from the $r = q'$ branch of (4.65). If $Z^{\kappa}_{t,d} = 0$ then $\bar{\pi}_t(k)\,\tilde{v}^{\kappa}_{k,d} = 0$ for every k and the supremum of (5.16) over the simplex is $-\infty$, consistent with $\ell(0) = -\infty$. □

Specialization to the softmax reference. Setting $\bar{\pi}_t = A_{t,\cdot}$ in (5.18) yields the closed-form per-(t,d,κ) policies used in the RG's backward walk through the attention block,

$$\pi^{\star}_{t,d,+}(k) \;=\; \frac{A_{t,k}\,\tilde{v}^{+}_{k,d}}{Z^{+}_{t,d}}, \qquad \pi^{\star}_{t,d,-}(k) \;=\; \frac{A_{t,k}\,\tilde{v}^{-}_{k,d}}{Z^{-}_{t,d}}, \tag{5.19}$$

with $Z^{\kappa}_{t,d} = \sum_m A_{t,m}\,\tilde{v}^{\kappa}_{m,d}$ from (5.17). The numerical specification $\bar{\pi}_t = A_{t,\cdot}$ does not require the oracle interpretation of §5.4.2; it is enough that the cached softmax row is treated as a fixed input to (5.16).

$\alpha\beta$-LRP recovery for the attention block

We now show that, under the trajectory through the four state types of §5.4.1 and the trajectory discount of (4.89), the player-difference of Γ_x at the *input activation states* (the predecessors of the Value-projection linear states, where the W_v weights actually fire) recovers exactly the target rule (5.9) of §5.4.1. We keep the game side and the attribution side strictly separate:

- the *game-side* object is the discounted occupation measure $\Gamma_x(\,\cdot\,)$ of §4.7.2 evaluated at the four attention-block state types of §5.4.1. Following the per-neuron difference convention (4.88) of the surrounding Routing-Game recursion, we form the per-output-entry difference $\Gamma^O_{t,d}$ at the entry point (defined in (5.22) below) and the per-input-neuron difference at the exit

$$\Gamma^X_{k,e} \;\coloneqq\; \Gamma_x\big(s^{(\mathrm{att},\,\mathrm{X},\,\mathrm{act})}_{(k,e),+}\big) - \Gamma_x\big(s^{(\mathrm{att},\,\mathrm{X},\,\mathrm{act})}_{(k,e),-}\big). \tag{5.20}$$

- the *attribution-side* object is the standard $\alpha\beta$-LRP relevance vector $R^X_{k,e}$ given by the input-level target rule (5.9), seeded by $R^O_{t,d}$ from the surrounding linear-layer recursion above the head.

Theorem 5.6 ($\alpha\beta$-LRP recovery for attention, `Lean_69`). *For signed inputs X, under the SPNE of §5.4.1 with reference $\bar\pi_t = A_{t,\cdot}$ and the trajectory discount of (4.89) restricted to the (sole) sign-oracle split at $s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{act})}$, the per-input-neuron player-difference $\Gamma^X_{k,e}$ from (5.20) satisfies*

$$\Gamma^X_{k,e} \;=\; \sum_{t,d}\left(\alpha\,\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^+}{Z^+_{t,d}} \;-\; \beta\,\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^-}{Z^-_{t,d}}\right)\Gamma^O_{t,d}, \tag{5.21}$$

where the per-output-entry input difference is

$$\Gamma^O_{t,d} \;\coloneqq\; \Gamma_x\big(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{act})}_{t,d,+}\big) \;-\; \Gamma_x\big(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{act})}_{t,d,-}\big), \tag{5.22}$$

with $\Gamma^O_{t,d} = R^O_{t,d}$ by inductive hypothesis from the surrounding recursion, for the seed $R^{(L)}_u = 1$ (for the seed $f(x)$ every relevance equals $f(x)$ times the corresponding Γ, as in Theorem 4.10). The right-hand side is exactly the target rule (5.9), hence $\Gamma^X_{k,e} = R^X_{k,e}$. Aggregating over e and the bias node at fixed (k,d) collects the relevance routed through the Value entry $v_{k,d}$:

$$\Gamma^{\mathrm{val}}_{k,d} \;=\; \sum_{t}\left(\alpha\,\frac{A_{t,k}\,\tilde v^+_{k,d}}{Z^+_{t,d}} \;-\; \beta\,\frac{A_{t,k}\,\tilde v^-_{k,d}}{Z^-_{t,d}}\right)\Gamma^O_{t,d} \;=\; R^{\mathrm{val}}_{k,d}, \tag{5.23}$$

recovering the per-Value-entry formula (5.7).

Proof. The statement is block-local: by hypothesis the surrounding recursion delivers the output-side per-entry differences $\Gamma^O_{t,d} = R^O_{t,d}$ at every output position (t,d).

Per-state recursion at the input activation states. Fix an input embedding (k,e) and a player label $r \in \{+,-\}$. A trajectory reaches $s^{(\mathrm{att},\,\mathrm{X},\,\mathrm{act})}_{(k,e),r}$ from an output activation $s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{act})}_{t,d,p}$ in three stages, with all transitions inside the attention block carrying their discounts and player-label switches as in the kernel of §5.4.1. The path goes through one specific within-head feature dim $d \in \{1,\dots,d_h\}$ (an index along the focused head's d_h-wide feature axis): the sign-oracle is per (t,d), the Value-routing policy $\pi^\star_{t,d,\kappa}$ depends on d via $\tilde v^\kappa_{k,d}$, and the Value-projection routing $\pi^\star_{\mathrm{val},k,d,\kappa}$ also depends on d. The d index is dropped at the final transition into X, where the input embedding $X_{k,e}$ has indices (k,e) and the within-head feature axis d enters downstream at the Value projection's output.

– $\kappa = +$ *branch (per* (t,d)*).* Sign-oracle takes $s^{(\mathrm{O,act})}_{t,d,p} \to s^{(\mathrm{O,lin})}_{(t,d),p,+}$ with probability $\frac{1}{2}$ and trajectory discount 2α (player retained). The Value-routing policy $\pi^{\star}_{t,d,+}(k)$ takes the trajectory to $s^{(\mathrm{val,lin})}_{(k,d),p,+}$ (player, sign, d preserved, discount 1). The Value-projection routing $\pi^{\star}_{\mathrm{val},k,d,+}(e)$ takes the trajectory to $s^{(\mathrm{X,act})}_{(k,e),p}$ (player preserved, d dropped, discount 1). The composite per-(t,d) contribution to $\Gamma_x(s^{(\mathrm{X,act})}_{(k,e),r})$ is

$$\tfrac{1}{2}(2\alpha)\,\pi^{\star}_{t,d,+}(k)\,\pi^{\star}_{\mathrm{val},k,d,+}(e)\,\Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r}\big) \;=\; \alpha\,\frac{A_{t,k}\,\tilde{v}^{+}_{k,d}}{Z^{+}_{t,d}}\,\frac{[X_{k,e}W_{v,e,d}]^{+}}{\tilde{v}^{+}_{k,d}}\,\Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r}\big) \tag{5.24}$$

$$\;=\; \alpha\,\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{+}}{Z^{+}_{t,d}}\,\Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r}\big), \tag{5.25}$$

where the $\tilde{v}^{+}_{k,d}$ in numerator and denominator cancel.

– $\kappa = -$ *branch (per* (t,d)*).* Sign-oracle takes $s^{(\mathrm{O,act})}_{t,d,p} \to s^{(\mathrm{O,lin})}_{(t,d),p',-}$ with probability $\frac{1}{2}$, discount 2β, turn switched to opponent p'; the rest of the path preserves p', ending at $s^{(\mathrm{X,act})}_{(k,e),p'}$. The contribution to $\Gamma_x(s^{(\mathrm{X,act})}_{(k,e),r})$ comes from sources with player $p = r'$ (so the switch yields r):

$$\tfrac{1}{2}(2\beta)\,\pi^{\star}_{t,d,-}(k)\,\pi^{\star}_{\mathrm{val},k,d,-}(e)\,\Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r'}\big) \;=\; \beta\,\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{-}}{Z^{-}_{t,d}}\,\Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r'}\big). \tag{5.26}$$

Summing over (t,d) and over the two branches,

$$\Gamma_x\big(s^{(\mathrm{att,\,X,\,act})}_{(k,e),r}\big) \;=\; \sum_{t,d}\Big[\alpha\,\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{+}}{Z^{+}_{t,d}}\,\Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r}\big) \;+\; \beta\,\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{-}}{Z^{-}_{t,d}}\,\Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r'}\big)\Big], \tag{5.27}$$

where the $\frac{1}{2}$ from the sign-oracle splits cancels against the 2 in $2\alpha, 2\beta$.

Player difference. Differencing (5.27) for $r{=}+$ minus $r{=}-$ uses $\Gamma_x(s^{(\mathrm{O,act})}_{t,d,+}) - \Gamma_x(s^{(\mathrm{O,act})}_{t,d,-}) = \Gamma^{O}_{t,d}$ in the $+$ branch (player kept) and the opposite $-\Gamma^{O}_{t,d}$ in the $-$ branch (player switched):

$$\Gamma^{X}_{k,e} \;=\; \alpha\sum_{t,d}\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{+}}{Z^{+}_{t,d}}\,\Gamma^{O}_{t,d} \;-\; \beta\sum_{t,d}\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{-}}{Z^{-}_{t,d}}\,\Gamma^{O}_{t,d}, \tag{5.28}$$

which is (5.21).

Target-rule identification. The right-hand side matches (5.9) verbatim. Since $\Gamma^{O}_{t,d} = R^{O}_{t,d}$, $\Gamma^{X}_{k,e} = R^{X}_{k,e}$. Aggregating over e and the bias node at fixed (k,d) uses $\sum_e [X_{k,e}W_{v,e,d}]^{\kappa} + [b_{v,d}]^{\kappa} = \tilde{v}^{\kappa}_{k,d}$, giving (5.23), which matches (5.7). □

5.4.2 The reference as the policy of a QK oracle player

§5.4.1 treated the softmax row $A_{t,\cdot}$ as a fixed reference policy. It is itself the entropy-regularized best response of an auxiliary *oracle* player whose payoff is the detached query–key logit $e_{tk} = Q_t K_k^{\top}/\sqrt{d_h}$:

by (4.62) at temperature 1,

$$\begin{aligned} A_{t,\cdot} \;&=\; \operatorname{argmax}_{\pi\in\Delta(\{1,\dots,S\})}\Big\{\textstyle\sum_k \pi(k)\, e_{tk} + \mathscr{H}(\pi)\Big\}, \\ \max_{\pi}\Big\{\textstyle\sum_k \pi(k)\, e_{tk} + \mathscr{H}(\pi)\Big\} \;&=\; \log \textstyle\sum_k \exp(e_{tk}). \end{aligned} \tag{5.29}$$

The oracle acts in the forward pass only and hands $\bar{\pi}_t = A_{t,\cdot}$ to the Value-routing player; its states carry no occupation mass, so no relevance flows through Q or K (§5.4.4).

5.4.3 Risk-Averse Attention via ADF

When ADF variance propagation is active, let s_{tk} denote the uncertainty score attached to attending from query token t to key token k (in the current implementation, $s_{tk} = \hat{\sigma}_k$, the square root of the feature-averaged ADF variance of token k's query projection; it depends only on the key token and is a proxy for, not the standard deviation of, e_{tk}). Risk-averse attention shifts the QK-logits in the direction of reduced uncertainty,

$$\tilde{e}_{tk} = e_{tk} + \lambda_{\mathrm{sm}}\, s_{tk}, \qquad \lambda_{\mathrm{sm}} \le 0, \tag{5.30}$$

yielding the risk-modified policy

$$\tilde{\pi}_t(k) = \frac{\exp(\tilde{e}_{tk})}{\sum_{k'} \exp(\tilde{e}_{tk'})}. \tag{5.31}$$

The sign convention $\lambda_{\mathrm{sm}} \le 0$ is used throughout the paper and in all tables: more-negative values penalize high-uncertainty keys more strongly, so the shift always redirects relevance toward tokens whose QK-interactions are stable under input noise. The shift is computed in the AV-backward hook from cached pre-softmax logits, so the forward pass and model predictions remain unchanged.

The naming *softmax-shift risk aversion* is a deliberate analog of the activation-side risk aversion λ but acts in a different direction: λ is risk-averse on the value side (it routes through stable activations by lowering mean$-|\lambda|\sigma$), whereas $\lambda_{\mathrm{sm}} < 0$ exponentially tilts the routing policy, $\tilde{\pi}_t(k) \propto A_{t,k} \exp(\lambda_{\mathrm{sm}} s_{tk})$, toward *low*-uncertainty keys: $\sum_k \tilde{\pi}_t(k)\, s_{tk}$ is non-decreasing in λ_{sm}, while the entropy of $\tilde{\pi}_t$ may rise or fall. We retain the "risk-averse" label because what is being avoided is uncertainty in the routed pathway, but the reader should note that the shift acts on the routing policy, not on the player's utility curvature.

5.4.4 Implementation details

The forward and backward passes of the RG on ViT-B/16 share four operator-level conventions that together preserve LRP conservation through the non-linearities of the ViT and align the explanatory walk with the state-space construction of §5.4.1. We refer to this collectively as the DAVE-style *conditioned forward pass* (Wróbel et al., 2026), augmented with the QK gradient detachment that the QK oracle of §5.4.2 requires.

LayerNorm detachment. Standard backpropagation through $\mathrm{LN}(x) = w_{\mathrm{LN}} \odot (x - m(x))/s(x) + b_{\mathrm{LN}}$ routes gradients through the data-dependent statistics $m(x), s(x)$, which couple the feature coordinates of each token (LayerNorm acts per token; it does not mix tokens) and violate LRP conservation (Ali et al., 2022). Following the detached-operator construction of Wróbel et al. (2026), we compute

m, s from a detached copy $c = x.\text{detach}()$, so the backward pass sees only the pointwise map $x \mapsto w_{\text{LN}} \odot (x - m(c))/s(c) + b_{\text{LN}}$ and conservation is preserved.

GELU gate detachment. GELU gates are conditioned analogously: the gate is implemented as $x \cdot \Phi(c)$ with $c = x.\text{detach}()$ and Φ the standard normal CDF, so the backward pass treats the gate as a pointwise scaling by the (constant) activation $\Phi(c)$.

Softmax detachment. Softmax outputs are fully detached. This matches the QK oracle interpretation of §5.4.2: the cached softmax row $A_{t,\cdot}$ is the equilibrium policy of an auxiliary entropy-regularized problem, supplied as a fixed input to the Value-routing player; no trajectory of the explanatory walk enters Q or K.

QK gradient detachment. The softmax and QK-matmul backward hooks return zero gradients, ensuring that the QK pathway is used *only* for computing the detached kernel $A_{t,\cdot}$ and that no relevance flows through Q or K. The QK oracle therefore supplies the KL reference, and all explanatory mass through the attention block flows through the Value tensor. The implemented Value rule is the composite rule of Theorem 5.6: each Value entry keeps the stream masses $\tilde{v}^{\kappa}_{k,d}$ of its Value-projection products, the attention step routes stream κ in proportion to $A_{t,k}\tilde{v}^{\kappa}_{k,d}$, and the Value projection routes stream κ only to the contributions $[X_{k,e}W_{v,e,d}]^{\kappa}$ of the same sign, so the discounts (α,β) act once per block, as in (5.9). This composite rule is the only attention–Value rule in the released code (option `av_rule='merged'`); the sequential variant of §5.4.1 was not used for any reported number. The identity tests of Section A.3.8 check the implementation against an independent reference of (5.9).

Together the four conventions realize the detached-operator semantics of Wróbel et al. (2026) for the RG backward pass, so that its explanatory mass flows only through the linear effective transformation. They implement the explanation question of §5.4.1 (Value-only attribution conditioned on the realized attention matrix, as in CP-LRP (Ali et al., 2022)) and are not the rules of AttnLRP (Achtibat et al., 2024), which propagates relevance through the softmax and the QK product as well.

5.4.5 Equivariant Spatial Averaging

The 16×16 patch tokenization breaks translation equivariance. Following DAVE's transform-averaging strategy (Wróbel et al., 2026), RG+Eq computes explanations on transformed inputs, maps each explanation back to the original image coordinates, and averages the aligned maps (a Monte-Carlo Reynolds average),

$$\mathscr{M}_N(x) \;=\; \frac{1}{N}\sum_{n=1}^{N} T_n^{-1}\big[\mathscr{M}(T_n x)\big], \tag{5.32}$$

where $\mathscr{M}$ is the RG attribution map and the T_n are i.i.d. random rotations (uniform in $\pm 10°$), translations (up to $\pm 5\%$ of the image extent per axis) and horizontal flips (probability 0.5), with $N = 50$ and no input noise. The conditioned forward pass of §5.4.4 separates operator variation from the effective transformation, in the same spirit as DAVE's conditioned-operator argument (Wróbel et al., 2026). The transforms do not form a group, so the average is an optional smoothing mode without an exactness claim.

6 Saliency and Attribution Methods

This chapter introduces the novel interpretability methods enabled by the split-stream representations. The guiding objects throughout are the stream functions g and h with $f = g - h$ constructed in Section 3.3: every method below reads the two streams *separately* instead of only their difference. Section 6.1 develops the *split-based* route — attributing the decisions of a pretrained network through its DC decomposition, yielding SplitCAM, SplitLRP and SplitGrad (Zimmermann and Loho, 2026), together with the complementary by-design route of difference-of-convex input neural networks (DCINNs), in which the difference of two constrained networks is *trained* directly and turns out to be self-interpretable. Section 6.2 turns to the *game-based* route of Chapter 4: rather than re-deriving the stopping and routing games, it collects the practical *modes* — the tunable knobs that turn the game equilibria into concrete attribution maps — and the *technicalities* of equivariant averaging that raise attribution quality.

6.1 Split-Based and Difference-of-Convex Attribution

This section develops the split-based methods on a shared conceptual basis, then instantiates SplitCAM, SplitLRP and SplitGrad on a pretrained network and closes with the by-design DCINNs. Post-hoc attribution methods assign to every input feature a score quantifying its influence on a fixed network output. The established families include gradient saliency (Simonyan et al., 2014), deconvolution and guided backpropagation (Springenberg et al., 2015; Zeiler and Fergus, 2014), LRP and deep Taylor decomposition (Bach et al., 2015; Montavon et al., 2017), class-activation mapping (Selvaraju et al., 2017; Zhou et al., 2016), axiomatic methods such as Integrated Gradients and DeepLIFT (Shrikumar et al., 2017; Sundararajan et al., 2017), noise-averaged gradients (Smilkov et al., 2017), and perturbation- or surrogate-based explanations (Fel et al., 2023; Fong and Vedaldi, 2017; Kolek et al., 2023; Lundberg and Lee, 2017; Ribeiro et al., 2016). All of them ultimately probe the same computational graph: the original network with its ReLU gates. This section identifies a structural information loss that this graph inflicts on every backward-pass method, and shows that the split-pair representation of Section 3.3 removes it.

Throughout this chapter we consider a feedforward ReLU network with layers $1, \dots, L$, weight matrices $W^{(l)}$, pre-activations $z^{(l)} = W^{(l)} a^{(l-1)}$ and hidden activations $a^{(l)} = \mathrm{ReLU}(z^{(l)})$ for $l < L$, with the ungated readout $a^{(L)} = z^{(L)}$, attributing a scalar output (a fixed class logit), so that $f^{(l)} \colon \mathbb{R}^{d^{(l)}} \to \mathbb{R}$ denotes the function computed by layers $l+1, \dots, L$ from the layer-l activations; biases are absorbed as in Section 3.3. The split-pair consists of the stream functions $g^{(l)}, h^{(l)} \colon \mathbb{R}^{d^{(l)}} \times \mathbb{R}^{d^{(l)}} \to \mathbb{R}$ operating on the split activations $a^{(l,+)}, a^{(l,-)}$, with the correctness guarantee

$$f^{(l)}\big(a^+ - a^-\big) = g^{(l)}(a^+, a^-) - h^{(l)}(a^+, a^-) \qquad \text{for all } a^+, a^- \in \mathbb{R}^{d^{(l)}}, \tag{6.1}$$

and we write $g \coloneqq g^{(0)}$, $h \coloneqq h^{(0)}$ with the balanced input split $a^{(0,\pm)} = \pm x/2$. All derivatives in this chapter are treated as row vectors (Jacobians of scalar functions), and statements involving derivatives hold at every differentiability point, hence Lebesgue-almost everywhere, since all functions involved are CPWL.

A ReLU gate transmits its pre-activation if it is positive and emits an exact zero otherwise. In the forward pass the *magnitude* of a negative pre-activation is annihilated; in the backward pass the local derivative of the gate is zero, so no gradient traverses the neuron either. We call this the *subthreshold signal loss*: all information carried by inactive neurons is invisible both to the network

output and to every attribution method that propagates through the original graph. The consequence for gradient-based attribution is not merely attenuation but exact regional blindness.

Proposition 6.1 (Regional blindness of the input gradient, `Lean_60`). *Let f be a ReLU network as above, let x_0 be an input at which no pre-activation is exactly zero, and let i be an input coordinate such that every path from input neuron i to the output neuron passes through at least one neuron that is inactive at x_0. Then there is an open neighborhood $U \ni x_0$ with*

$$\frac{\partial f}{\partial x_i} \equiv 0 \quad \text{on } U,$$

even if f depends on x_i elsewhere.

Proof. Write $M^{(l,+)} := \operatorname{diag}\big(\mathbf{1}\{z^{(l)} > 0\}\big)$ and $M^{(l,-)} := \operatorname{diag}\big(\mathbf{1}\{z^{(l)} \le 0\}\big)$ for the activation masks of the hidden ReLU layers $1 \le l < L$, and $W^{(l,x)} := M^{(l,+)} W^{(l)}$ for the corresponding masked weight matrices. The readout layer L produces the class logit through a purely affine map with *no* ReLU gate — the logit may be negative — so it carries no activation mask, and we set $W^{(L,x)} := W^{(L)}$; this unmasked factor is also the base case $\partial f / \partial a^{(L-1)} = W^{(L)}$ of the chain-rule iteration below. Since no hidden pre-activation vanishes at x_0 and pre-activations depend continuously on the input, all masks are locally constant: there is an open $U \ni x_0$ on which the activation pattern is fixed and f is affine. On U, iterating the chain rule through the layers gives

$$\frac{\partial f}{\partial x} = W^{(L,x)}\, W^{(L-1,x)} \cdots W^{(1,x)}, \tag{6.2}$$

and expanding the matrix product, the i-th entry is the sum, over all paths from input neuron i through the hidden layers to the output, of the product of the traversed weights multiplied by the indicator that every *hidden* neuron on the path is active (the ungated output neuron carries no such indicator). By assumption every such path contains an inactive hidden neuron, so every summand vanishes, and the expression is constant on U. □

The phenomenon is not an artifact of adversarial constructions; it occurs in the smallest possible network.

Remark 6.1 (The canonical counterexample). Following Sundararajan et al. (2017), consider the one-dimensional ReLU network

$$f(x) = 1 - \mathrm{ReLU}(1 - x), \tag{6.3}$$

and compare the input $x = 2$ against the baseline $x' = 0$. The prediction changes, $f(2) - f(0) = 1 - 0 = 1$, so the single feature is plainly relevant; yet f is constant on $x \ge 1$, so the gradient at $x = 2$ — and with it Input×Gradient and every purely local backward rule — assigns exactly zero attribution. The single hidden neuron is inactive at $x = 2$, and Proposition 6.1 applies with $U = (1, \infty)$.

The failure mode above was axiomatized by Sundararajan et al. (2017) as a minimal requirement on attribution methods.

Definition 6.1 (Sensitivity). An attribution method A, mapping an input x and a baseline x' to feature scores $A_i(x, x') \in \mathbb{R}$, satisfies *Sensitivity* if for every pair (x, x') that differs in exactly one feature i and satisfies $f(x) \neq f(x')$, the attribution of that feature is non-zero, $A_i(x, x') \neq 0$.

Plain gradients violate Sensitivity by Remark 6.1. Integrated Gradients restores the axiom by integrating the gradient along the segment from x' to x, at the cost of many forward–backward passes and a baseline choice that the explanation inherits. The split-pair offers a structurally different resolution: instead of repairing the attribution rule, it repairs the *representation* on which the rule operates, so that the subthreshold signal is never destroyed in the first place.

Recall from Section 3.3 how the split-pair processes a ReLU gate: the pre-activation pair $z^{(l,+)}, z^{(l,-)}$ is mapped to

$$a^{(l,+)} = \max\{z^{(l,+)}, z^{(l,-)}\}, \qquad a^{(l,-)} = z^{(l,-)}, \tag{6.4}$$

a rank-2 Maxout unit in the g-stream and the identity in the h-stream. The difference reproduces $\mathrm{ReLU}(z^{(l)})$ by the splitting identity, but inactivity does not force the individual streams to vanish: when the original neuron is inactive, both $a^{(l,+)}$ and $a^{(l,-)}$ equal $z^{(l,-)}$ and still record the full magnitude of the suppressed signal. The subthreshold information is thus not annihilated but relocated — from the value of the difference into the common level of the two streams. The same happens in the backward pass, which we now make precise.

Definition 6.2 (Local sensitivities). For $s \in \{g, h\}$, $\sigma \in \{+, -\}$ and $0 \le l \le L$, the *local sensitivities* of the split-pair at layer l are the row vectors

$$\delta^{(l,\sigma,s)} := \frac{\partial s^{(l)}}{\partial a^{(l,\sigma)}} \in \mathbb{R}^{1\times d^{(l)}},$$

the derivatives of the stream outputs with respect to the positive and negative split activations of layer l. We further abbreviate the original-network sensitivity by $\delta^{(l)} := \partial f^{(l)}/\partial a^{(l)}$ and define the *absolute weight product*

$$W_{\mathrm{abs}}^{[l]} := |W^{(L)}|\,|W^{(L-1)}|\cdots|W^{(l+1)}| \in \mathbb{R}^{1\times d^{(l)}}, \qquad W_{\mathrm{abs}}^{[L]} := \mathbf{1}^\top,$$

where $|\cdot|$ is taken entrywise. Since the attributed output is scalar, $W_{\mathrm{abs}}^{[l]}$ is a row vector, and its i-th entry sums the absolute weight products over *all* paths from neuron i of layer l to the output — active or not.

The four local sensitivities obey coupled backward recursions — the split analogue of standard backpropagation and the computational core of every method in this chapter.

Lemma 6.2 (Split backward recursions, `Lean_61`). *With the activation masks $M^{(l,\pm)}$ of Proposition 6.1, extended by $M^{(L,+)} = I$ and $M^{(L,-)} = 0$ for the affine readout, for every $0 \le l < L$ and $s \in \{g, h\}$,*

$$\delta^{(l,+,s)} = \delta^{(l+1,+,s)}\big(M^{(l+1,+)}W^{(l+1,+)} + M^{(l+1,-)}W^{(l+1,-)}\big) + \delta^{(l+1,-,s)}\,W^{(l+1,-)}, \tag{6.5}$$

$$\delta^{(l,-,s)} = \delta^{(l+1,+,s)}\big(M^{(l+1,+)}W^{(l+1,-)} + M^{(l+1,-)}W^{(l+1,+)}\big) + \delta^{(l+1,-,s)}\,W^{(l+1,+)}, \tag{6.6}$$

with base case $\delta^{(L,+,g)} = \delta^{(L,-,h)} = \mathbf{1}^\top$ and $\delta^{(L,-,g)} = \delta^{(L,+,h)} = 0$.

These are the chain-rule recursions of the split forward equations of Section 3.3, derived in Section 3.6.2; the mask $M^{(l+1,+)}$ appears because the maximum in (6.4) is attained by $z^{(l+1,+)}$ exactly when the original neuron is active. The recursions immediately yield the central structural fact of this chapter: the *total* sensitivity mass carried by the two streams does not depend on the activation pattern at all.

Proposition 6.3 (Sensitivity mass conservation, Lean_62). *For every $0 \le l \le L$ and both streams $s \in \{g, h\}$,*

$$\delta^{(l,+,s)} + \delta^{(l,-,s)} = W_{\text{abs}}^{[l]}. \tag{6.7}$$

Proof. Induction on $l = L, \dots, 0$. The base case is $\mathbf{1}^\top + 0 = W_{\text{abs}}^{[L]}$. For the step, add (6.5) and (6.6): the mask terms combine via $M^{(l+1,+)} + M^{(l+1,-)} = I$ into

$$\delta^{(l,+,s)} + \delta^{(l,-,s)} = \big(\delta^{(l+1,+,s)} + \delta^{(l+1,-,s)}\big)\big(W^{(l+1,+)} + W^{(l+1,-)}\big) = \big(\delta^{(l+1,+,s)} + \delta^{(l+1,-,s)}\big)|W^{(l+1)}|,$$

since $W^{(l+1,+)} + W^{(l+1,-)} = |W^{(l+1)}|$ for the entrywise weight split. The induction hypothesis turns the right-hand side into $W_{\text{abs}}^{[l+1]}|W^{(l+1)}| = W_{\text{abs}}^{[l]}$. □

The relation between the split sensitivities and the original gradient is a one-line consequence of the correctness guarantee.

Corollary 6.4 (Backward invariant, Lean_63). *For every $0 \le l \le L$,*

$$\delta^{(l,+,g)} - \delta^{(l,+,h)} = \delta^{(l)} = -\big(\delta^{(l,-,g)} - \delta^{(l,-,h)}\big), \quad \textit{hence} \quad \big(\delta^{(l,+,g)} - \delta^{(l,-,g)}\big) - \big(\delta^{(l,+,h)} - \delta^{(l,-,h)}\big) = 2\,\delta^{(l)}. \tag{6.8}$$

Proof. Differentiate the correctness identity (6.1) with respect to a^+ and a^-, respectively, and subtract the two resulting identities. □

Together, Proposition 6.3 and Corollary 6.4 express a conservation law: the difference of the stream sensitivities reproduces the original gradient exactly, while their sum is pinned to the input-independent quantity $W_{\text{abs}}^{[l]}$. Where Proposition 6.1 shows the original gradient collapsing to zero, the two combined stream gradients coincide: writing $\delta^{(l,s)} := \frac{1}{2}\big(\delta^{(l,+,s)} - \delta^{(l,-,s)}\big)$ for the combined gradient of stream s, Corollary 6.4 gives $\delta^{(l,g)} - \delta^{(l,h)} = \delta^{(l)}$, so the vanishing of $\delta^{(l)}$ is precisely the equality $\delta^{(l,g)} = \delta^{(l,h)}$ of the two combined stream gradients — their difference, the network gradient, is zero. The common value may itself be zero. The individual sensitivities $\delta^{(l,\pm,s)}$ need not equal $\frac{1}{2}W_{\text{abs}}^{[l]}$; their deviation from that pattern-independent half-bulk records precisely the routing decisions taken at the ReLU gates, including the inactive ones. For a feature connected to the output by a path of nonzero weights, the sensitivity-mass identity guarantees that at least one of the two non-negative partial sensitivities of each head is positive. It does not guarantee that a difference of partials, a combined gradient, or an attribution map is nonzero. For example, with $f(x) = \mathrm{ReLU}(\mathrm{ReLU}(x) + \mathrm{ReLU}(-x) - 10)$ and the balanced input split, both heads are locally equal to 10 near $x = 1$ and both combined gradients vanish.

Remark 6.2 (The counterexample, revisited). Split the network (6.3) with the balanced input split $a^{(0,\pm)} = \pm x/2$. A short computation along Section 3.3 gives, for $x > 1$,

$$g(x) = \frac{x}{2} + 1, \qquad h(x) = \frac{x}{2}, \qquad g(x) - h(x) = 1 = f(x),$$

so at $x = 2$ both stream gradients equal $\frac{1}{2} \neq 0$ while $f'(2) = 0$: the split representation *sees* that the feature matters — consistent with $f(2) \neq f(0)$ — and encodes the vanishing of f' as the cancellation $\frac{1}{2} - \frac{1}{2}$. For this example and the baseline $x' = 0$, the pair of head gradients therefore supplies a nonzero signal even though the output gradient vanishes. This example does not establish the sensitivity axiom of Definition 6.1 for a general attribution rule or baseline. The attribution methods of the following

sections differ in how they turn this pair-valued signal into a scalar map; the shifted combinations of Section 6.1.3 keep the two streams from cancelling exactly, and in this example assign the feature a strictly positive attribution for every shift parameter $\alpha < \frac{1}{2}$.

There is also a conceptual reading of these results. Networks with non-negative weights implement a computation in which every neuron can only *add* evidence, never cancel it — a scheme long associated with interpretability and with biologically plausible processing (Ayinde and Zurada, 2018; Dayan and Abbott, 2001; Lee and Seung, 1999). The split-pair equips an arbitrary pretrained network with exactly this structure, at the price of doubling the width; the remainder of this chapter harvests the improved signal flow.

6.1.1 SplitCAM

CAM methods explain CNN decisions by combining spatial activation maps of a convolutional layer with class-specific weighting coefficients. The original CAM (Zhou et al., 2016) requires a global-average-pooling head; Grad-CAM (Selvaraju et al., 2017) generalizes it to arbitrary architectures by weighting each channel with the spatially pooled gradient of the class logit, and Grad-CAM++ (Chattopadhay et al., 2018) refines the pooling weights. LayerCAM (Jiang et al., 2021) drops the pooling altogether and weights every spatial location individually: for a layer l with channel activation maps $a_c^{(l)}$ and class logit y,

$$\mathrm{LayerCAM}^{(l)} = \mathrm{ReLU}\Big(\sum_c \frac{\partial y}{\partial a_c^{(l)}} \odot a_c^{(l)}\Big), \tag{6.9}$$

where $\odot$ denotes the elementwise product and the sum runs over the channels. This makes LayerCAM applicable to early layers with high spatial resolution, but it inherits both failure modes of Section 6.1: inactive neurons contribute neither activation nor gradient, and the outer ReLU discards all negative evidence.

SplitCAM transplants the LayerCAM formula onto the split-pair, replacing activations and gradients of the original network by their split-stream counterparts. Since the raw local sensitivities of deep split-pairs are numerically enormous — they are sums of *absolute* path products, cf. Proposition 6.3 — SplitCAM operates on the *stabilized* sensitivities of Section 3.6.2.

Definition 6.3 (Shifted local sensitivities, `Lean_64`). Fix shift parameters $\alpha^{(l)} \in [0, \frac{1}{2}]$. For $s \in \{g, h\}$ define, backward from layer L, intermediate sensitivities $\delta_{\mathrm{int}}^{(l,\pm,s)}$ by evaluating the recursions (6.5)–(6.6) (respectively the base case of Lemma 6.2 at $l = L$) with the already-shifted sensitivities of the subsequent layer, and set

$$\delta_{\mathrm{shift}}^{(l,\pm,s)} := \delta_{\mathrm{int}}^{(l,\pm,s)} - \alpha^{(l)}\big(\delta_{\mathrm{int}}^{(l,+,s)} + \delta_{\mathrm{int}}^{(l,-,s)}\big), \qquad \delta_{\mathrm{shift}}^{(l,s)} := \frac{\delta_{\mathrm{shift}}^{(l,+,s)} - \delta_{\mathrm{shift}}^{(l,-,s)}}{2}. \tag{6.10}$$

Both signs receive the *same* shift, so the informative differences are preserved layer by layer, while by Proposition 6.3 the subtracted direction is the pattern-independent bulk $\delta_{\mathrm{int}}^{(l,+,s)} + \delta_{\mathrm{int}}^{(l,-,s)} = \big[\prod_{i=l+1}^{L}(1 - 2\alpha^{(i)})\big] W_{\mathrm{abs}}^{[l]}$ (the empty product at $l = L$ is 1); the factor $\frac{1}{2}$ in the combined sensitivity $\delta_{\mathrm{shift}}^{(l,s)}$ matches the balanced input split. Unless stated otherwise all $\alpha^{(l)} \equiv \alpha$. For $\alpha = 0$ the shifted sensitivities are the raw ones; for $\alpha = \frac{1}{2}$ they collapse onto multiples of the original gradient (Chapter 3, Theorem 3.15), so the informative regime is $0 < \alpha < \frac{1}{2}$, where the split signal survives with a controlled, layer-dependent damping.

The SplitCAM attribution.

Definition 6.4 (SplitCAM). For a layer l of the decomposed model, a stream $s \in \{g, h\}$ and a sign $\sigma \in \{+, -\}$, the SplitCAM attribution maps are

$$\mathrm{SplitCAM}^{(l,\sigma,s)} := \sum_c \delta^{(l,\sigma,s)}_{\mathrm{shift}}\big|_c \odot a_c^{(l,\sigma)}, \tag{6.11}$$

$$\mathrm{SplitCAM}^{(l,s)} := \sum_c \delta^{(l,s)}_{\mathrm{shift}}\big|_c \odot a_c^{(l)}, \tag{6.12}$$

where $a_c^{(l,\sigma)}$ denotes the channel-c activation map of the σ-stream, $a_c^{(l)} = a_c^{(l,+)} - a_c^{(l,-)}$ the reconstructed original activation map, and $\cdot|_c$ the restriction of a sensitivity tensor to channel c.

Two deliberate deviations from (6.9) deserve comment. First, SplitCAM *omits the outer ReLU*: on the split-pair, evidence against the class is carried by a dedicated non-negative stream rather than by sign flips, so clipping would discard exactly the counter-evidence that the decomposition makes legible. Second, the gradients entering the formula are the shifted sensitivities rather than raw derivatives; by (6.10) this changes each map only by multiples of the pattern-independent vector $W^{[l]}_{\mathrm{abs}}$, but keeps all quantities in a numerically meaningful range.

Both passes run under the numerical stabilization of Section 3.6: the forward pass scales the split-activation pairs and re-imposes the forward invariant against the cached original activation, and the backward pass applies the shifting of Definition 6.3 with routing masks read from the cached original activations, so that the backward invariant (6.8) holds against the cached original gradient.

Layer choice and resolution. Like LayerCAM, SplitCAM is defined at every layer, and the resulting map is bilinearly upsampled to the input resolution. Empirically the method peaks in the deep convolutional blocks — for VGG-16 (Simonyan and Zisserman, 2015) at layer 26 — where it concentrates on entire classified objects noticeably better than LayerCAM at the same layer; the full layer-ablation study and the quantitative comparison on the Quantus benchmark (Hedström et al., 2023) for VGG-16 and ResNet-18 (He et al., 2016) are reported in Chapter 7.

6.1.2 SplitLRP

LRP (Bach et al., 2015) explains a prediction by propagating a relevance budget backward through the network, layer by layer, distributing the relevance of each neuron onto its inputs in proportion to their contributions; see Montavon et al. (2018) for an overview and Kohlbrenner et al. (2020) for best practices. Among the many propagation rules, the γ-rule of Montavon et al. (2019) emphasizes positive contributions: for an affine layer $z_i^{(l+1)} = \sum_j W_{ij}^{(l+1)} a_j^{(l)} + b_i^{(l+1)}$, one defines the γ-adapted contributions

$$\tilde{z}_{ij}^{(l+1)} := a_j^{(l)} \Big(W_{ij}^{(l+1)} + \gamma\, [W_{ij}^{(l+1)}]^+ \Big), \qquad \gamma \geq 0, \tag{6.13}$$

and redistributes the relevances $R_i^{(l+1)}$ onto the inputs as

$$R_j^{(l)} = \sum_i \frac{\tilde{z}_{ij}^{(l+1)}}{\sum_{j'} \tilde{z}_{ij'}^{(l+1)} + \varepsilon\, \mathrm{sign}\big(\sum_{j'} \tilde{z}_{ij'}^{(l+1)}\big)}\, R_i^{(l+1)}, \tag{6.14}$$

with a small stabilizer $\varepsilon > 0$. The choice $\gamma = 0$ recovers the classical ε-rule. Activation functions are transparent to LRP: the rule acts on the affine layers only.

Relevance propagation on the split-pair. *SplitLRP* runs LRP on the decomposed model, treating the positive and negative streams of each layer as two groups of neurons within one unified network. Accordingly, each layer carries *two* relevance tensors $R^{(l,+)}$ and $R^{(l,-)}$, propagated jointly through the four weight blocks built from $W^{(l,+)}$ and $W^{(l,-)}$ that couple the streams. The propagation is initialized at the class-specific positive output $a^{(L,+)}$ of the g-stream. Since *all* weights of the decomposed model are non-negative, the γ-rule loses its purpose — there are no negative contributions to de-emphasize — and we use (6.14) with $\gamma = 0$ and $\varepsilon = 10^{-6}$, i.e. the ε-rule, on the split-pair. Because the Maxout gates (6.4) are activation functions, they require no special treatment in the relevance pass.

The following conventions complete the specification.

- *Forward pass.* The split activations are computed with the same scaling stabilization and forward-invariant correction of Section 3.6 as SplitCAM.
- *Normalization.* No backward stabilization is needed: the denominator of (6.14) renormalizes relevance in every layer, so the total relevance carried by $R^{(l,+)}$ and $R^{(l,-)}$ together remains equal — up to the ε-stabilizer, cf. Remark 6.3 — to the initial budget $a^{(L,+)}$ throughout the backward pass, and this conservation replaces the shifting of Definition 6.3.
- *Pooling and normalization layers.* Relevance passes through max-pooling by the winner-takes-it-all scheme, assigning the full relevance to the location attaining the maximum; batch-normalization layers are merged into the adjacent linear or convolutional layers beforehand; residual additions split relevance evenly across branches.

Definition 6.5 (Combined SplitLRP map). The SplitLRP attribution at layer l is the pair $\big(R^{(l,+)}, R^{(l,-)}\big)$; the *combined* map is

$$R^{(l,\mathrm{comb})} := R^{(l,+)} - R^{(l,-)}. \tag{6.15}$$

The combination is motivated by the semantics that the decomposition imposes: relevance arriving in the positive stream traversed an even number of sign inversions and constitutes evidence *supporting* the prediction, while relevance in the negative stream constitutes evidence *against* it, so (6.15) aggregates support and opposition into a single signed saliency map.

Remark 6.3 (A probabilistic reading). On the original network the ε-rule mixes signed contributions, and the resulting backward transport is a *signed* measure on paths — the source of the interpretability caveats discussed in Chapter 8. On the split-pair all contributions $\tilde{z}_{ij}^{(l+1)}$ are products of non-negative activations and non-negative weights, so up to the stabilizer the LRP propagation matrices are genuinely stochastic: SplitLRP is a Markov chain on the doubled computational graph, and the pair $\big(R^{(l,+)}, R^{(l,-)}\big)$ is an honest, unsigned relevance measure. This is the same mechanism — doubling the state space to absorb the weight signs — by which the stopping game of Chapter 4 turns backward relevance flow into probability.

Remark 6.4 (Sensitivity to learned parameters). The parameter-randomization sanity check of Adebayo et al. (2018) differentiates the three split methods. SplitCAM and SplitGrad react strongly to reinitialized late layers — more strongly than all considered baselines — confirming that the shifted sensitivities track the learned decision process. SplitLRP is markedly less sensitive: on the split-pair the ε-rule essentially reduces to a rescaled Input×Gradient, and the layer-wise renormalization in (6.14) suppresses the influence of late-layer weights on the propagated relevance, a known limitation of purely conservative rules (Sixt et al., 2020). The quantitative results, together with the caveats of Binder et al. (2023) on randomization-based checks, are reported in Chapter 7.

6.1.3 SplitGrad

The third method dispenses with activation weighting and relevance budgets altogether and uses the stabilized stream gradients themselves as the attribution signal. The stream functions consume a *pair* of inputs, so we fix the balanced embedding $a^{(0,\pm)} = \pm x/2$ and write $\tilde{g}^{(l)}(a) \coloneqq g^{(l)}\big(\frac{a}{2}, -\frac{a}{2}\big)$, $\tilde{h}^{(l)}$ analogously. By the chain rule (Section 3.6.2) the unified stream gradient $\partial \tilde{s}^{(l)}/\partial a$ equals the combined sensitivity $\big(\delta^{(l,+,s)} - \delta^{(l,-,s)}\big)/2$, which justifies the factor $\frac{1}{2}$ in (6.10): the combined shifted sensitivities $\delta^{(l,g)}_{\text{shift}}, \delta^{(l,h)}_{\text{shift}}$ are stabilized gradients of the convex functions $\tilde{g}^{(l)}, \tilde{h}^{(l)}$, and their difference always reproduces the exact original gradient, since the common shift in (6.10) cancels sign-wise.

Definition 6.6 (SplitGrad). For a layer l and stream $s \in \{g, h\}$, the SplitGrad attribution map is the channel mean of the combined shifted sensitivity,

$$\text{SplitGrad}^{(l,s)} \coloneqq \frac{1}{C^{(l)}} \sum_{c=1}^{C^{(l)}} \delta^{(l,s)}_{\text{shift}}\big|_c, \tag{6.16}$$

where $C^{(l)}$ is the number of channels of layer l; the default configuration reads the g-stream, with forward and backward stabilization exactly as in Section 6.1.1.

What SplitGrad measures. Chapter 3 computes these stabilized gradients in closed form: by Theorem 3.15, the input-level stream gradients have original-gradient terms $\delta/2$ and $-\delta/2$, respectively, where $\delta \coloneqq \partial f/\partial x$, and share a subtracted, geometrically damped sum of *subthreshold probe-gradients* — one per layer, each carrying the signal routed through the *inactive* neurons whose loss causes the blindness of Proposition 6.1. Both streams contain this probe sum with the same sign, so it cancels in their difference, which reproduces δ exactly. The probe sum can itself vanish; the decomposition makes subthreshold contributions available without guaranteeing a nonzero attribution for every input. SplitGrad therefore recovers the feature the plain gradient misses: in the counterexample of Remark 6.2, $\delta = 0$ while the probe term is non-zero, so the feature is attributed correctly for every $\alpha < \frac{1}{2}$. The layer-wise damping factors $\overline{\alpha}_j$ of Theorem 3.15 both counteract the rapid growth of the probe-gradient norms toward the input — a failure otherwise reminiscent of shattered gradients (Balduzzi et al., 2017) — and let α interpolate between fidelity to the original gradient ($\alpha = \frac{1}{2}$) and full exposure of the split information ($\alpha = 0$); the sweep over α is ablated in Chapter 7, where values around $\alpha \approx 0.4$ perform best across the Quantus metrics.

The three methods share one pipeline: a stabilized split forward pass caching the original activations, a stabilized backward pass — relevance-based for SplitLRP (initialized at the class-specific $a^{(L,+)}$ and propagated by the ε-rule), sensitivity-based otherwise (the shifted recursions of (6.10)) — and a method-specific readout, with the resulting map bilinearly upsampled to the input resolution. The splitting is performed once per model and is negligible compared to a forward pass; the per-image cost of SplitCAM and SplitGrad is comparable to classical CAM methods and far below perturbation-based explainers, cf. Chapter 7.

6.1.4 Self-Interpretability: Difference-of-Convex Input Neural Networks

The methods of Sections 6.1.1–6.1.3 explain a *pretrained* network by imposing the split structure post hoc. This raises a converse question: if the difference-of-two-streams representation is what makes explanations legible, what happens when the difference structure is not derived but *trained* — when the hypothesis class itself is the set of differences of two constrained networks? This section answers

the question with DC input neural networks (DCINNs) and their monotone relatives, and finds that the trained streams disentangle their explanatory roles into factual and counterfactual evidence, at a modest and architecture-dependent cost in accuracy.

Architecture and training. Recall from Section 2.2 that an ICNN (Amos et al., 2017) is a network with convex, monotone activation functions and non-negative weights in all layers except possibly the first, and that such a network represents a convex function; a network with monotone activations and *all* weights non-negative represents a monotone function. Throughout this section we view ICNNs as monotone networks acting on an affine transformation of the input, embodied by the unconstrained first layer.

Definition 6.7 (DCINN and difference-of-monotone models). Fix a base architecture. A *DCINN* (in short: DIC model) is the function

$$f_{\mathrm{DIC}} \coloneqq g - h,$$

where g and h are two ICNNs of the base architecture with independent parameters, trained *jointly* by standard empirical risk minimization on the difference. The *DM model* is defined analogously with both streams monotone networks. The *inverted-input* variants DICIN and DMIN feed the h-stream the complemented input $\mathbf{1} - x$ (for inputs normalized to $[0,1]^d$) instead of x.

The construction mirrors the split-pair — two non-negative streams whose difference is the model — but with a crucial change of direction: nothing is split; both streams are free to learn any convex (respectively monotone) function, and the difference structure is a training-time inductive bias rather than a post hoc reparameterization. Closely related architectures appeared as convex difference neural networks (CDiNN) in optimal control (Sankaranarayanan and Rengaswamy, 2022), with recurrent extensions for safety-critical model predictive control (Chen, Wang, et al., 2025); the interpretability perspective developed here is complementary to that line of work. The inverted-input variant is motivated by monotonicity itself: a monotone h-stream can only accumulate evidence from the *presence* of input mass, so handing it $\mathbf{1} - x$ lets it accumulate evidence from what is *missing* in the input.

Three implementation choices matter in practice.

- *Activations.* Both streams use rank-2 Maxout units (Goodfellow et al., 2013), which are convex and monotone as required, and which consistently outperformed ReLU streams in the difference setting. This aligns the trained models exactly with the hypothesis class of Section 3.3: the split-streams obtained there *are* rank-2 Maxout networks with non-negative weights, so training DIC and DM models searches the very class into which the splitting maps.
- *Non-negativity.* Constrained weights are parameterized as $p(\tilde{W})$ with free parameters $\tilde{W}$ and a positivity transform p; among softplus, exponential and squaring, $p = \mathrm{ReLU}$ with absolute-value-of-Gaussian initialization gave the best test accuracy. The transform is also applied to biases and to the scale and shift parameters of batch-normalization layers.
- *Regularization.* Since non-negative weights preclude cancellation, activations can grow quickly; a small initialization scale and a weight decay of 10^{-3} keep the streams stable during training.

The empirical study uses two MNIST base architectures: `cnn_maxout`, a CNN with four convolutional layers (32, 64, 128 and 128 channels with batch normalization) followed by three fully connected layers, roughly 1.2 million parameters in total, and `mlp_maxout`, a wide four-layer MLP ($784 \to 800 \to 800 \to 800 \to 10$) with roughly 4.2 million parameters — each in eight variants: unconstrained (None),

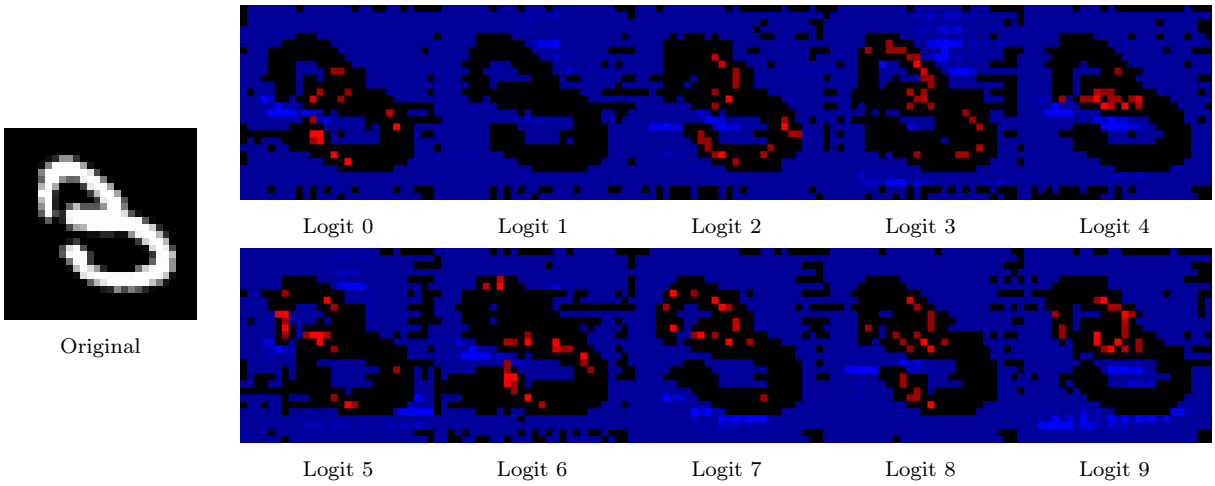


Fig. 6.1: Stream gradients of the inverted-input DIC model (`cnn_maxout` DICIN) on a correctly classified MNIST test digit "3", thresholded and overlaid per class logit. The g-stream gradient (red) marks factual evidence on present strokes, while the h-stream gradient (blue) marks counterfactual evidence: pixels whose addition would raise the respective class logit. Left: the original digit.

single-stream constrained (IC, Mon), and difference-of-two models (DIC, DM, DICIN, DMIN, and an unconstrained difference baseline).

Counterfactual feature attribution. The payoff of the by-design route is that the *plain input gradients* of the trained streams are already meaningful explanations — no shifting, no relevance rules, no probe networks. For the inverted-input DIC model, the two gradients answer complementary questions:

- the g-stream gradient $\nabla_x g(x)$ is a *factual* explanation: since g is convex and monotone beyond its first layer, its gradient indicates which *present* features increase the class logit;
- the h-stream gradient $(\nabla h)(\mathbf{1} - x)$ — the gradient of h evaluated at the complemented input $\mathbf{1} - x$, *not* the input-space derivative $\nabla_x h(\mathbf{1} - x) = -(\nabla h)(\mathbf{1} - x)$, which merely flips its sign by the chain rule — is a *counterfactual* explanation: a large entry indicates that *adding* mass at a currently empty location would increase the class logit, i.e. it marks the missing features that the class would require.

Figure 6.1 overlays both gradients, per class logit, for a test digit "3", and the separation of concerns is clearly visible: for the ground-truth logit the g-gradient traces the strokes most accurately, while for the competing classes "6", "8" and "9" the counterfactual h-gradients concentrate densely *between the arches* of the "3" — exactly the missing pixels that would close the loops. A second correctly classified "3" in Figure 6.2 shows the same qualitative pattern. The plain DIC model without input inversion shows the same effect in a single signed map: its gradients are mostly positive on the strokes and negative on the background. A *single* constrained model (`cnn_maxout` IC) does not show the same class-dependent directional variation: the difference structure, not the convexity constraint alone, produces the disentanglement in these examples.

Qualitative comparison with a single input-convex model. Figure 6.3 reproduces the source paper's ten-class input-gradient comparison for the single `cnn_maxout` IC model. Its gradients mostly mark positive evidence on digit strokes and negative evidence on the background. The maps are informative about the input, but they show little class-dependent directional variation. By contrast, the DICIN

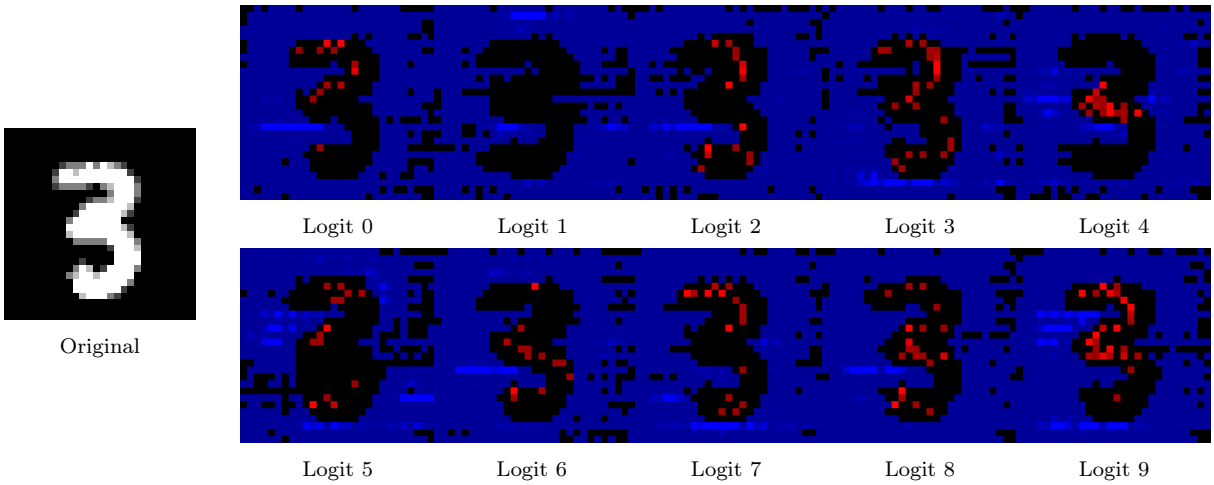


Fig. 6.2: Second example of stream gradients of the inverted-input DIC model (`cnn_maxout` DICIN) on a correctly classified MNIST test digit "3", thresholded and overlaid per class logit. The g-stream gradient (red) marks factual evidence on present strokes, while the h-stream gradient (blue) marks counterfactual evidence: pixels whose addition would raise the respective class logit. Left: the original digit.

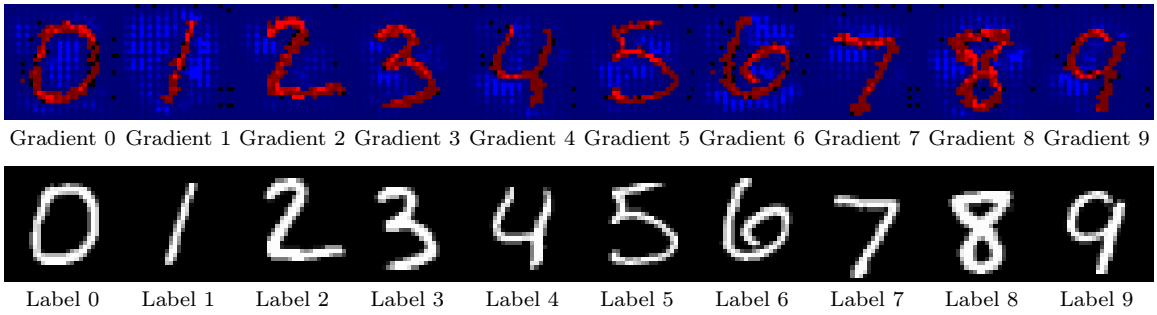


Fig. 6.3: Input-gradient visualizations of the `cnn_maxout` IC model for ten MNIST digits, each with respect to its ground-truth label. Top: saliency maps; bottom: input images. As in the source figure, positive and negative gradient values above threshold 0.2 are colored red and blue, with ranking colorized using exponential decay.

maps in Figures 6.1 and 6.2 show different evidence patterns for competing class logits: the h-stream highlights missing strokes that would support those classes. These visual examples motivate the factual/counterfactual reading of the two streams; they do not establish that this separation holds for all inputs or models.

Accuracy trade-offs. Self-interpretability by architectural constraint traditionally costs expressivity (Liu and Coogan, 2020; Sill, 1997), but for the difference-of-two construction the cost is small on the convolutional base model and even negative on the MLP, where every difference-of-two variant outperforms the unconstrained baseline; the full accuracy table across ten seeds and all eight variants is reported in Chapter 7.

Remark 6.5 (Post-hoc and by-design, reconciled). The two routes of this chapter probe the same structural hypothesis — that difference-of-monotone representations are inherently more legible — from opposite ends. The MNIST study answers the two underlying questions affirmatively: constrained monotone and input-convex networks are self-interpretable, and differences of two such networks can

be trained directly to competitive accuracy, with gradients that separate factual from counterfactual evidence more cleanly than the split-pairs of the pretrained VGG-16 and ResNet-18 models, for which such a disentanglement of concerns is observed only partially and has not been established rigorously. Transferring the by-design disentanglement to large-scale architectures — or explaining why splitting a pretrained model cannot fully reproduce it — remains an open direction, taken up again in Chapter 9.

6.2 Game-Based Attribution: Modes and Technicalities

The split-based methods above read a fixed DC decomposition. The complementary route of Chapter 4 attributes through the equilibrium trajectory distribution of the SG and RG: the per-neuron difference of the two players' discounted occupation measures is the attribution map, and at the anchor temperature $\tau = 1$ this difference recovers the $\alpha\beta$-LRP family exactly (Theorem 4.2), just as the plain gradient arises as a difference of occupation measures. Those equivalences — the stopping and routing games, the recovery of gradients and of the $\alpha\beta$-LRP rule — are *defined and proven* in Chapter 4 and are not restated here. This section collects only what turns those equilibria into concrete, tunable attribution maps: the *modes* that deform the games, and the input-side *technicalities* that raise attribution quality.

6.2.1 Attribution Modes

Chapter 4 deforms the equilibrium trajectory distribution through a small set of orthogonal knobs — the routing and stopping temperatures τ, θ, the policy-entropy bonus, the risk parameter $\lambda \leq 0$, the stream balance (α, β), and the LRP stabilizer $\epsilon \geq 0$ — and establishes an *exact dictionary* (Section 4.4) relating each knob to the backward rule it deforms. We do not restate that dictionary here. What matters for attribution is only the operational role of the knobs: each is a control that turns the abstract game equilibrium into a concrete, tunable attribution map, and together they are the parameters swept in the evaluation of Chapter 7.

Two notational cautions specific to this chapter are worth flagging, since two symbols carry a different meaning here than in the split-based methods above. First, the stream balance (α, β) — the $\alpha\beta$-LRP discount pair, with $\alpha - \beta = 1$ and the theorem-driven anchor $(\alpha, \beta) = (2, 1)$ that reweights support against opposition — is unrelated to the backward-shift parameter $\alpha \in [0, \frac{1}{2}]$ of Section 6.1; wherever a bare "α" appears in this section it denotes the LRP stream weight. Second, the routing temperature τ (which recovers $\alpha\beta$-LRP exactly only at the anchor $\tau = 1$, Theorem 4.2, and is applied myopically so the recovered forward pass is unchanged) is distinct from the spatial input transforms τ_t introduced in the Technicalities below.

6.2.2 Technicalities

A second, orthogonal lever improves the maps on the input side without touching the game: averaging attributions over input transformations, and the detached-operator (DAVE-style) reading of the resulting gradient decomposition. Both matter most on the ViT, where the 16×16 patch tokenization breaks translation equivariance and introduces grid artifacts (Dosovitskiy et al., 2021; Wróbel et al., 2026).

Equivariant Reynolds averaging. Following DAVE's transform-averaging strategy (Wróbel et al., 2026), an attribution is computed on N inputs transformed by random spatial maps $\tau_1, \ldots, \tau_N$ — small rotations, translations, and horizontal flips (distinct from the routing temperature τ of the games) — each explanation is mapped back to the original image coordinates by the inverse transform τ_t^{-1}, and the aligned maps are averaged. This classical Reynolds-operator construction cancels architecture-specific, transform-dependent noise while preserving the image-dependent structure common to all transforms; the sample count N trades compute for smoothness and is ablated in Chapter 7.

Connection to DAVE-style gradient decomposition. The averaging pairs naturally with a *conditioned forward pass* (Wróbel et al., 2026): LayerNorm statistics, the GELU gate, and the softmax row are detached, so the backward pass sees only the linear effective transformation and LRP conservation is preserved through the non-linearities. The resulting attribution takes the transform-averaged input×gradient form $\frac{1}{N}\sum_t \tau_t^{-1}(W_t) \odot X$, where W_t is the conditioned backward signal on the t-th transformed input. The game-based maps of Chapter 5 inherit exactly these conditioned operators, so equivariant averaging applies to them as a drop-in quality technique.

7 Experiments and Evaluation

This chapter evaluates the two design choices developed throughout the thesis: changing the representation available to attribution through the split streams, and changing the backward routing rule through the game policies. The common questions concern localization, faithfulness, robustness, and the numerical or computational cost of obtaining useful attribution signals. The split-method experiments and the game-based case study provide complementary evidence for these questions. Their original checkpoints, sample exclusions, metric conventions, and selection protocols are retained, and numerical comparisons are made within each campaign. The results are inherited from the papers; no experiment is re-run for this thesis.

7.1 Experimental Setup

The split campaign evaluates pretrained VGG-16 and ResNet-18 on a custom 50-class ImageNet-S subset at 224×224 resolution. A 50-image validation split (one image per class) selects layer–formula combinations; the remaining 566 images form its test split. Baseline attributions use Captum, with Zennit for γ-LRP and separate Grad-CAM++ and LayerCAM implementations. For each method, localization-selected and Pixel-Flipping-selected configurations are retained. Full settings and results are given in Appendix A.1.

The split tables report Pixel Flipping as area under the score-drop curve ($\mathrm{PF}_5^{\mathrm{drop}}$, $\mathrm{PF}_{20}^{\mathrm{drop}}$; $\uparrow$), perturbing 112 pixels per step, and Maximum Sensitivity with 25 perturbations. The revised game campaign reports area under the remaining-logit curve (PF_5, PF_{20}; $\downarrow$), using 1,568 features per step, and sensitivity with 50 perturbations on a common 200-image test subset. Selectivity, Attribution Localization and Pointing Game are measured using Quantus. VGG-16 is shared, while ResNet-18 belongs to the split campaign and ResNet-50 to the game campaign; their scores are not interchangeable. Throughout the tables, AL denotes Attribution Localization, PG Pointing Game, Sel Selectivity, MaxS Maximum Sensitivity, and AvgS Average Sensitivity. Method abbreviations are GCAM (Grad-CAM), GCAM++ (Grad-CAM++), LCAM (LayerCAM), IntGrad (Integrated Gradients), and SmGrad (SmoothGrad); Guided GCAM denotes Guided Grad-CAM. Dark blue shading and bold type mark the best values within the comparison stated in each caption. The notation n/r means that the source paper did not report a result; n/a means that the entry does not apply.

7.2 Split Methods: VGG-16

Qualitative analysis. Figure 7.1 qualitatively compares saliency maps from our split methods with their baseline counterparts at the best-performing layers. Mid-layer SplitCAM shares key visual characteristics with classical γ-LRP from earlier layers. Compared to standard LayerCAM on the same layer it focuses more weight to all object pixels, while still marking the key object features as especially important. For $\alpha \ll 0.5$, SplitGrad yields less focused yet finely aligned gradients that closely follow image details. We attribute this to the Maxout activation retaining pre-activation information, unlike ReLU, which discards it in inactive neurons. Overall, mid-layer SplitCAM offers an effective balance between interpretability and focus (low entropy). Further qualitative examples are collected in Appendix A.1.

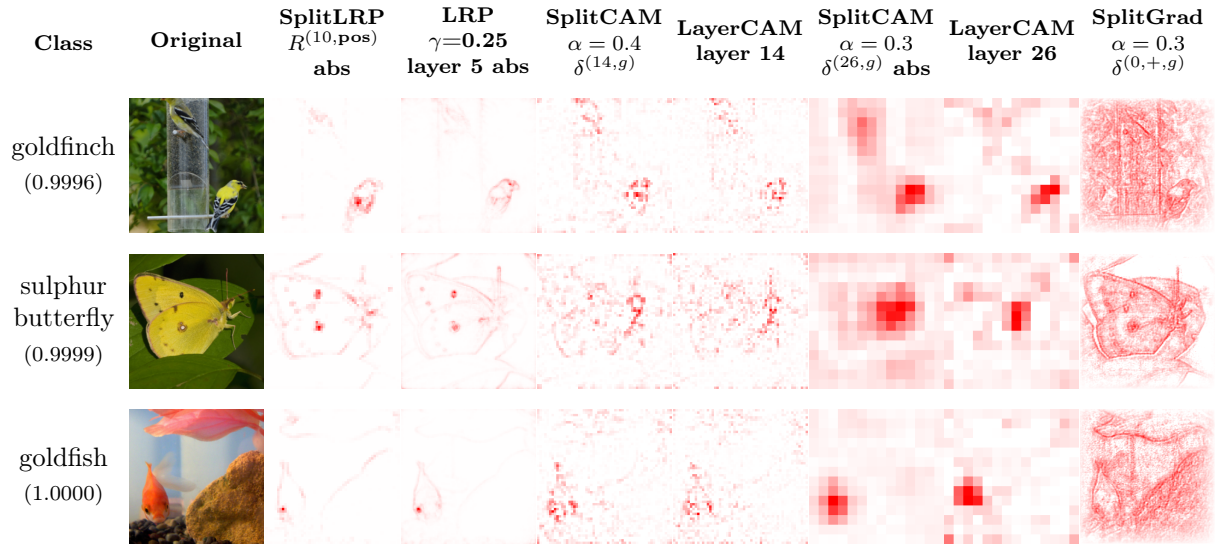


Fig. 7.1: Saliency map comparison for VGG-16 on our custom ImageNet-S test split. Each row shows the predicted class (with confidence), original image, and saliency maps from different methods.

Quantitative analysis. Table 7.1 and Table 7.2 report a comprehensive comparison of attribution methods on VGG-16 and ResNet-18 across five key Quantus metrics: Selectivity, Attribution Localization, Pointing Game accuracy, Pixel Flipping (score-drop AUC, $\text{PF}_5^{\text{drop}}$ and $\text{PF}_{20}^{\text{drop}}$) and Maximum Sensitivity. The *Abs* column specifies whether absolute saliency values were used prior to evaluation in Quantus. A full table and explanation of both VGG-16 and ResNet-18 evaluation results with all selected layers and other parameter configurations is found in Appendix A.1.

Overall, the considered backpropagation-based attribution methods heavily depend on the selected network layer, also see Appendix A.2 for extensive ablation.

On VGG-16, the split methods demonstrate superior performance compared to all considered baseline methods across the Quantus metric categories. For example, SplitCAM achieves the highest Pointing Game score **0.938** with $\alpha = 0.4$ overshadowing the best baseline Guided Backprop in layer 5 with 0.887 and LayerCAM in layer 2 with 0.855. SplitCAM outperforms classical methods on Selectivity, with comparable Pixel Flipping score-drop AUC performance as the most faithful baseline LayerCAM. SplitLRP slightly outperforms classical γ-LRP in localization while staying competitive in faithfulness.

7.3 Split Methods: ResNet-18

Table 7.2 reports the comparison of attribution methods on ResNet-18 across the same five Quantus metrics, with the *Abs* column again specifying whether absolute saliency values were used prior to evaluation; the full evaluation tables are given in Appendix A.1. On ResNet-18, the split methods take the best Selectivity (**3.503** for SplitCAM against 3.690 for the strongest baseline LayerCAM) and edge the top localization scores, reaching 0.972 Pointing Game and **0.673** Attribution Localization versus LayerCAM's 0.971 and 0.664, while staying competitive with the remaining baselines across the considered metrics.

Tab. 7.1: Comprehensive interpretability evaluation results (VGG-16, ImageNet-S test split). The *wta* abbreviation indicates the use of the "winner-takes-it-all" max-pooling implementation; *sc* emphasizes the scaling procedure in the forward pass (Section 3.6). Best values per metric are bold with dark blue shading. The bracket notation in the layer column follows the upper indices of the split methods as introduced in Chapter 6.

Method	Layer	Abs	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
Split Methods								
SplitCAM (sc, $\alpha = 0.4$)	(12, g)	n	4.720	**0.651**	0.912	0.213	1.794	0.389
SplitCAM (sc, $\alpha = 0.4$)	(14, g)	n	**2.727**	0.563	0.836	0.300	2.404	1.012
SplitCAM (sc, $\alpha = 0.4$, wta)	(26, +, g)	y	4.711	0.635	**0.938**	0.165	1.590	0.456
SplitGrad ($\alpha = 0.4$, wta)	(12, +, g)	n	4.735	0.635	0.883	0.186	1.756	0.651
SplitGrad ($\alpha = 0.4$, wta)	(14, +, g)	y	3.287	0.540	0.818	0.249	2.186	0.857
SplitLRP (sc, wta)	(26, pos)	y	5.168	0.588	0.871	0.151	1.473	**0.282**
Baselines (With Layer Optimization)								
Guided Backprop	2	n	3.276	0.632	0.883	0.281	2.302	0.571
Guided Backprop	5	n	3.448	0.614	0.887	0.240	2.091	0.375
LCAM	14	n/a	2.792	0.576	0.798	**0.317**	**2.474**	2.656
LCAM	2	n/a	3.128	0.580	0.855	0.300	2.191	5.508
LRP $\gamma = 0.25$	0	n	3.189	0.615	0.680	0.218	1.918	0.519
LRP $\gamma = 0.25$	14	n	3.593	0.577	0.611	0.279	2.255	0.491
Baselines								
DeepLIFT	0	y	4.447	0.551	0.726	0.281	1.945	1.053
Feature Ablation	0	y	6.029	0.479	0.624	0.123	1.250	0.859
GradientSHAP	0	y	4.808	0.552	0.783	0.261	1.862	1.142
GCAM++	30	n	5.955	0.564	0.827	0.130	1.245	0.401
IntGrad	0	y	4.838	0.554	0.797	0.262	1.851	1.069
Occlusion	0	y	5.214	0.499	0.716	0.130	1.217	0.699
Random Baseline	0	y	3.570	0.425	0.458	0.203	1.585	n/r

7.4 Split-Method Stabilization Ablations

7.4.1 Backward-Pass Stabilization of the Split Methods

Table 7.3 presents the performance of two selected SplitGrad and SplitCAM configurations across different shifting parameters α, evaluated on the custom ImageNet-S test split. As introduced in Section 3.6, the α value determines how strongly the backward pass is stabilized: $\alpha = 0$ results in no stabilization, and $\alpha = 0.5$ results in local sensitivities resembling scaled versions of the original network gradient. Performance on all considered metrics increases for α below 0.5 but diminishes for very small α values, with the exception of Maximum Sensitivity, which is best at $\alpha = 0$. This demonstrates that both the splitting itself and the backward-pass stabilization boost the explanations. The full α grid, including the comparison between the convex and the winner-takes-it-all max-pooling modes, is reported in Appendix A.2 (Table A.5).

Forward-pass stabilization. The forward stabilization of Section 3.6 is not optional: without it, the split-stream activations of late VGG-16 layers explode to the order of 10^{18}, producing numerical errors of order 10^9, and SplitLRP degenerates to nearly uniform, semantically meaningless maps.

Tab. 7.2: Comprehensive interpretability evaluation results (ResNet-18, ImageNet-S test split). The triples $x.y.z$ are a ResNet-18-internal indexing of the convolutional layer at which the attribution maps are calculated. See the caption of Table 7.1 for further abbreviations and table explanation.

Method	Layer	Abs	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
SplitCAM Methods								
SplitCAM (sc, $\alpha = 0.3$, wta)	(1.0.2, g)	n	**3.503**	0.487	0.747	0.168	1.395	1.277
SplitCAM (sc, $\alpha = 0.3$, wta)	(4.1.2, g)	n	5.654	**0.673**	**0.972**	0.081	0.880	0.233
SplitGrad ($\alpha = 0.3$, wta)	(1.0.2, g)	y	4.449	0.513	0.777	0.149	1.266	0.813
SplitGrad ($\alpha = 0.3$, wta)	(4.1.2, g)	y	5.663	0.649	0.972	0.082	0.882	0.219
SplitLRP (sc, wta)	(conv1, pos)	y	6.313	0.506	0.587	0.056	0.697	0.755
Baselines (With Layer Optimization)								
GCAM	4.1.2	n	6.070	0.554	0.945	0.076	0.829	**0.131**
LCAM	2.0.2	n	3.690	0.549	0.865	0.144	1.428	0.913
LCAM	4.1.1	n	5.661	0.664	0.971	0.083	0.890	0.227
LRP $\gamma = 0.25$	conv1	y	3.550	0.656	0.852	**0.213**	1.737	0.521
Baselines								
Guided Backprop	conv1	n	3.517	0.621	0.867	0.212	**1.757**	0.459
GradientSHAP	conv1	y	5.269	0.527	0.779	0.166	1.227	1.074
IntGrad	conv1	y	5.265	0.528	0.777	0.167	1.226	0.994
Feature Ablation	conv1	n	4.655	0.136	0.684	0.188	1.388	1.077
Occlusion	conv1	n	5.300	0.179	0.726	0.130	1.030	0.785
Deconvolution	conv1	n	3.934	0.484	0.666	0.147	1.203	0.751
Random Baseline	n/a	y	4.071	0.425	0.440	0.147	1.067	n/r

Tab. 7.3: Comparison of different α parameters with two SplitGrad and SplitCAM base configurations, both in *wta* and *abs* mode, evaluated on the custom ImageNet-S test split.

Method	**Metric**	α .50	α .45	α .40	α .35	α .30	α .00
SplitGrad (26, g)	PG↑	0.898	0.906	0.947	**0.956**	0.889	0.401
	AL↑	0.543	0.548	0.605	**0.617**	0.535	0.449
	MaxS↓	0.616	0.607	0.440	0.238	0.334	**0.028**
	PF_{20}^{drop} ↑	1.527	1.534	**1.581**	1.448	1.317	1.247
SplitCAM (26, +, g)	PG↑	0.868	0.868	**0.938**	0.933	0.613	0.855
	AL↑	0.550	0.554	0.635	**0.663**	0.403	0.513
	MaxS↓	0.691	0.682	0.457	0.265	0.251	**0.120**
	PF_{20}^{drop} ↑	1.607	**1.609**	1.585	1.499	0.853	1.507

Comparing the two stabilization variants, scaling clearly dominates shifting for SplitLRP: for early-layer configurations, replacing scaling with shifting degrades every metric, while for late-layer configurations, scaling retains markedly better Pointing Game scores and robustness (Appendix Table A.6; the corresponding SplitCAM grids without stabilization and with shift stabilization are Tables A.7 and A.8).

7.5 Game-Based Case Study

Setup. We evaluate RG and SG against standard backward baselines (Section A.3.2) on VGG-16 (Simonyan and Zisserman, 2015), ResNet-50 (He et al., 2016), and ViT-B/16 (Dosovitskiy et al.,

2021) with torchvision ImageNet-1K checkpoints (Deng et al., 2009; Paszke et al., 2019; Russakovsky et al., 2015), on a 50-class ImageNet-S subset (Gao et al., 2023) at 224×224 (50-image validation split, 565-image test split), and on Pascal VOC 2012 (Everingham et al., 2009). ImageNet-S provides the pixel masks required by the localization metrics; ImageNet boxes over-credit mass anywhere inside the box. All metrics are computed through Quantus (Hedström et al., 2023) (definitions in Section A.3.3): localization (*Attribution Localization* (AL), *Pointing Game* (PG), Top-k Intersection (TK); ↑), faithfulness (Pixel Flipping area under the remaining-logit curve over the first 5% and 20% of flips (PF_5, PF_{20}), *Selectivity* (Sel); ↓), robustness (Maximum Sensitivity (MaxS), Average Sensitivity (AvgS); ↓). Baselines run in their reference implementations (lxt, zennit, captum, pytorch-grad-cam, and the authors' code for TIBAV and GAE) at the authors' settings, plus declared grids (Achtibat et al., 2024; Anders et al., 2023; Chefer et al., 2021a,b; Gildenblat and contributors, 2021; Kokhlikyan et al., 2020). RG analytical modes are reported alongside each configuration: temperature τ, risk aversion λ (softmax-shift variant λ_{sm} on ViT attention logits), LRP stabilizer ε, equivariant-Reynolds sample count N.

Protocol. Methods are evaluated under a two-stage protocol that treats baselines and our games alike. Stage 1 runs a grid search on the validation split (one image per class) for every method with tunable parameters, including DAVE (Section A.3.4, Table A.9; 2,021 configurations in total). Stage 2 picks, for each method, the *loc* configuration with the best localization rank-sum over $\{\text{AL}, \text{PG}, \text{TK}\}$ and, independently, the *faith* configuration with the best faithfulness rank-sum over $\{\text{PF}_5, \text{PF}_{20}, \text{Sel}\}$, and evaluates both once on the test split, with 95% percentile-bootstrap confidence intervals (Efron and Tibshirani, 1993) and Holm-corrected (Holm, 1979) paired tests against the best row of each metric (Section A.3.6); for RG / RG+Eq we additionally show the $\alpha\beta$-LRP anchor $(\alpha,\beta,\varepsilon,\tau)=(2,1,0.5,1)$ as a theorem-driven reference (Theorem 4.2). Localization is a natural target for modes designed to control routing focus, and object annotations provide external ground truth: Kohlbrenner et al. (2020) compare LRP rule composites by the share of relevance inside ground-truth bounding boxes, and we use pixel masks. The main table reports the loc row of every method; since mask agreement is not faithfulness, the faith rows of all methods are reported in Section A.4.

Tab. 7.4: ViT-B/16 on the ImageNet-S test split (n=565; MaxS/AvgS on a fixed random subset of 200 test images). **Bold**, dark shading: best in column; ≈, light shading: not significantly different from the best (paired bootstrap / McNemar, Holm-corrected, $p \geq 0.05$); small numbers: 95% percentile-bootstrap confidence intervals over test images (Section A.3.6). One row per method, selected on the validation split by the localization rank-sum (faithfulness-selected rows and further baselines: Section A.4); the RG anchor is the exact $\alpha\beta$-LRP representation (Theorem 4.2). t/img: seconds per image on one GPU. Config: SmoothGrad averages N samples at noise level σ relative to the input range; LRP-γ has stabilizer ε and positive-weight boost γ; AttnLRP has layer-type parameters $\gamma_{\text{conv}}, \gamma_{\text{lin}}, \varepsilon_{\text{attn}}$; KernelSHAP draws N samples for M superpixels; RG modes as in Table 4.1.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓	t/img
Gradient	fixed	0.411 [0.389,0.432]	0.361 [0.322,0.402]	0.386 [0.359,0.413]	8.45 [8.33,8.56]	8.08 [7.94,8.22]	6.033 [5.905,6.154]	1.290 [1.233,1.351]	0.922 [0.893,0.953]	**0.02**
SmGrad	N=50,σ/range=0.2	0.517 [0.498,0.536]	0.706 [0.667,0.743]	0.675 [0.654,0.694]	8.10 [7.96,8.24]	7.02 [6.83,7.20]	5.029 [4.894,5.163]	0.597 [0.569,0.630]	0.468 [0.451,0.485]	1.12
LRP-γ	ε=0.1,γ=2	0.644 [0.628,0.660]	0.873 [0.844,0.899]	0.816 [0.800,0.832]	7.93 [7.78,8.08]	6.64 [6.44,6.83]	**4.451** [4.324,4.572]	0.300 [0.287,0.314]	0.218 [0.210,0.226]	0.08
AttnLRP	γ_{conv}=0.25,γ_{lin}=0.05,$\varepsilon_{\text{attn}}$=0.25	0.644 [0.628,0.660]	0.855 [0.825,0.883]	0.809 [0.794,0.825]	**7.77** [7.61,7.93]	**6.43** [6.23,6.62]	4.582 [4.452,4.706]	0.456 [0.430,0.484]	0.342 [0.324,0.364]	0.10
TIBAV	fixed	0.615 [0.597,0.634]	0.878 [0.851,0.904]	0.782 [0.759,0.805]	8.13 [8.01,8.24]	7.60 [7.46,7.74]	4.906 [4.774,5.033]	1.062 [1.001,1.126]	0.512 [0.486,0.540]	0.15
GAE	fixed	0.533 [0.514,0.551]	0.743 [0.708,0.779]	0.649 [0.623,0.677]	8.13 [8.01,8.25]	7.62 [7.47,7.76]	5.098 [4.961,5.231]	0.525 [0.494,0.558]	0.368 [0.346,0.391]	0.03
KernelSHAP	N=2M+2048, 80 superpixels	0.565 [0.548,0.583]	**0.961** [0.945,0.977]	0.823≈ [0.799,0.848]	8.29 [8.18,8.40]	7.50 [7.35,7.65]	5.140 [5.012,5.261]	0.897 [0.868,0.928]	0.650 [0.632,0.667]	6.71
RG anchor	τ=1	0.509 [0.490,0.528]	0.724 [0.685,0.759]	0.699 [0.678,0.720]	8.19 [8.05,8.31]	6.97 [6.78,7.15]	4.854 [4.732,4.972]	**0.166** [0.161,0.172]	0.120 [0.117,0.124]	0.10
RG	τ=0.4,λ_{sm}=−10,σ^2=5,ε=0.1	**0.711** [0.692,0.730]	0.825 [0.793,0.855]	0.788 [0.768,0.808]	8.27 [8.14,8.39]	7.48 [7.31,7.64]	5.109 [4.987,5.226]	0.729 [0.685,0.777]	0.494 [0.471,0.518]	0.17
RG+Eq anchor	τ=1,N=50	0.520 [0.501,0.539]	0.761 [0.726,0.796]	0.752 [0.729,0.775]	8.41 [8.29,8.52]	7.72 [7.56,7.87]	4.864 [4.742,4.981]	0.170≈ [0.165,0.175]	**0.115** [0.112,0.118]	4.93
RG+Eq	τ=0.5,λ_{sm}=−10,σ^2=2,N=50	0.698 [0.680,0.717]	0.855 [0.827,0.883]	**0.836** [0.815,0.857]	8.39 [8.28,8.50]	7.80 [7.65,7.94]	5.010 [4.889,5.127]	0.441 [0.421,0.463]	0.301 [0.290,0.313]	8.66

ViT-B/16 (Table 7.4). Under the localization-selected protocol, the RG family beats every baseline on attribution localization, including the prior transformer-specific backward methods AttnLRP (Achtibat et al., 2024), DAVE (Wróbel et al., 2026), TIBAV (Chefer et al., 2021b) and GAE (Chefer et al., 2021a): the selected mode reaches AL 0.711 against 0.644 for the best baseline (LRP-γ), and RG+Eq 0.698, both significantly ahead of every baseline row after Holm correction. RG+Eq also has the highest TK (0.836, not significantly above KernelSHAP at 0.823). The pointing game is led by KernelSHAP (0.961), whose maps are constant on superpixels: a hit needs only one pixel of the maximal region inside the mask. Scoring a single pixel drawn from that region instead, KernelSHAP falls below RG+Eq, which is then not significantly below the best baseline (LRP-γ; Section A.4.6). Faithfulness favors the LRP rules: AttnLRP is best on PF_5 and PF_{20} and LRP-γ on Sel (PF_{20} 6.43 versus 7.48 for RG), and the faithfulness-selected RG row narrows but does not close the gap (PF_{20} 6.78). The two anchors are the most robust rows (MaxS 0.166 and 0.170 versus 0.208 for AR), whereas the localization-selected mode reaches MaxS 0.729: sharper routing trades robustness and faithfulness for localization. ADF adds modest overhead (RG: 0.17 s vs. 0.10 s base), still below sampling baselines like SmoothGrad or IntGrad; Reynolds averaging is the only expensive variant (8.66 s), exceeding even the perturbation method KernelSHAP (6.71 s).

Higher τ spreads attribution into context. The localization-optimal modes are the sharpest ones: they place mass on the few patches that align with the segmentation mask. As τ increases, the same equilibrium spreads routing over more predecessors; the object remains in focus, but the heatmap also highlights environmental features the network used. These broader maps can be visually more informative, but on ViT-B/16 attribution localization worsens monotonically with τ around the localization-selected setting (AL 0.723 at τ=0.3 down to 0.461 at τ=1), while the faithfulness side

is non-monotonic. High τ is the clearest example of the gap between metric-optimal and perception-optimal explanations within the same game.

Cross-dataset transfer (Pascal VOC). A natural worry with validation-tuned modes is that the chosen τ, λ_{sm} and σ^2 are overfitted to the ImageNet-S 50-class subset. We test transfer on a per-image disjoint split of Pascal VOC 2012 trainval (2,743 test (image, foreground-class) samples with pixel-accurate segmentation masks): RG and RG+Eq use the *same* mode parameters as Table 7.4 without any VOC-side re-tuning, while every baseline gets a fresh per-method validation grid on the new pool. This tests whether the selected modes capture model-specific rather than dataset-specific structure: the localization lead carries over (AL 0.459 for RG and 0.456 for RG+Eq versus 0.413 for TIBAV, both significant against every baseline row), while TK and PG are led by TIBAV and the perturbation methods, and faithfulness by the LRP rules (Section A.4.4).

VGG-16 and ResNet-50 (Section A.4). On the CNNs, where LRP-type rules are already strong, the localization-selected RG rows also lead on AL, clearly on VGG-16 (0.685 versus 0.613 for LRP-ε) and narrowly but significantly on ResNet-50 (0.719 versus 0.711 for LRP-γ). Pixel flipping favors the LRP rules (LRP-ε on VGG-16, LRP-γ on ResNet-50), while the CAM methods and the perturbation methods lead TK and PG. On VGG-16 the risk-averse SG mode is the most robust method (MaxS 0.278 versus 0.314 for GCAM++).

7.5.1 Game-Method Ablations

The revised one-factor ablations in Section A.4.5 vary one mode around each validation-selected configuration and evaluate the resulting rows on the 565-image test split. They are diagnostic results and do not feed back into selection. Section A.4.6 controls the pointing-game comparison for map granularity, and Section A.4.7 compares the composite Value-routing attention rule with a gradient fallback. These analyses expose trade-offs between mask agreement, perturbation faithfulness and robustness rather than a uniform improvement of all desiderata.

8 The Trajectory Distribution View

The attribution methods of Chapter 6 turn internal signals and backward routing choices into maps, and Chapter 7 evaluates those maps. For the intervention-oriented motivation of this thesis, we also need to examine the calculation producing them. The equilibrium policies of Chapter 4 induce distributions over backward trajectories, making such a comparison possible before projection onto input pixels. This chapter studies their Hellinger distance, its exact per-input-state disagreement map, and the distinction between complete and survival-conditioned laws. It then evaluates these diagnostics under input noise and parameter randomization.

8.1 Comparison with Normalized Relevance Measures

The NRM framework of Xiong et al. (2026), developed by Ping Xiong, Thomas Schnake, Grégoire Montavon, Klaus-Robert Müller, and Shinichi Nakajima, provides a closely related view of the backward calculation. Its basic object is a normalized signed measure on complete neuron walks. Relevance assigned to selected neurons or concepts is obtained by summing over the walks passing through them; conditional relevance is a ratio of joint and marginal relevance when the conditioning relevance is nonzero. With suitable normalized propagation rules, this includes ordinary LRP, concept relevance, and Concept Relevance Propagation (CRP) (Achtibat et al., 2023). In particular, their CRP quantity is the joint relevance of an input neuron, an intermediate concept neuron, and the target output, rather than automatically a conditional probability.

Both views therefore retain paths through the network before reducing them to an attribution map. Their factorization supports sum-product computation and queries involving neurons in several layers. The distinction is what a path carries. In NRM, walk relevance may be negative even though its total is one. Here, the game policies define a non-negative probability law Π on the expanded state graph, including the player label and termination. Gradients and relevance scores are recovered from signed differences of discounted occupation measures of this law (Chapter 4); they are generally not its unweighted marginals. The player expansion therefore separates trajectory probability from the sign and scale of the backward contribution. It does not by itself identify Π with a given NRM measure.

This distinction matters for normalization and comparison. The NRM construction normalizes propagation columns under positive-sum assumptions; its identification with globally normalized original walk relevance additionally requires the constant-sum conditions of Xiong et al. (2026, Section 3.5). Our transition probabilities remain normalized while discounts retain the scale of the backward rule, and bias and stabilizer absorption are accounted for explicitly. Hellinger distance applies directly to the resulting probability laws; it does not apply directly to arbitrary signed relevance measures. Conditioning Π on a concept requires a positive-probability event, whereas signed relevance can have zero total through cancellation of nonzero terms.

The two perspectives suggest a useful extension: query the game trajectories through selected concepts, retaining the signs and discounts when computing their attributions, and compare the resulting trajectory laws across inputs or models. Establishing when those quantities coincide with a particular NRM/CRP construction requires matching the propagation rules, normalization, and absorption conventions. This is a proposed connection, not an equivalence proved in this thesis.

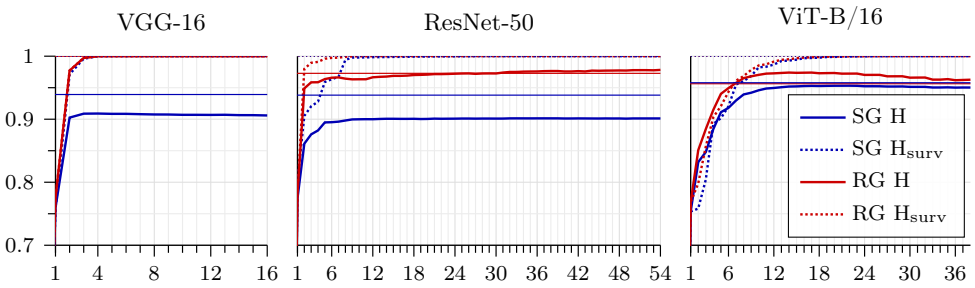


Fig. 8.1: Hellinger trajectory distance under cascading parameter randomization (Adebayo et al., 2018). Hellinger distance H (vertical axis, from 0.7) against the number of cascadingly re-initialized layers, output to input (horizontal axis; $H = 0$ before the first layer is randomized). Solid: plain H; dotted: H_{surv}, conditioned on survival to an ordinary input-layer state. Thin horizontal reference lines mark the fully-random-model values from Table 8.3 (solid: plain H; dotted: H_{surv}), in the color of the corresponding game.

8.2 Trajectory Sanity Check

The cascading-randomization sanity check of Adebayo et al. (2018) asks whether an explanation meaningfully depends on the learned parameters, and answers by comparing attribution maps in pixel space. But the object of interest is the backward calculation itself—a distribution Π over graph trajectories—and the attribution map is only its pixel projection. The **Hellinger distance** (Hellinger, 1909) $H(\Pi_A, \Pi_B) = (1 - BC)^{1/2}$, with Bhattacharyya coefficient (Bhattacharyya, 1943) $BC = \sum_\zeta \sqrt{\Pi_A(\zeta)\,\Pi_B(\zeta)}$, measures the routing logic directly rather than its visual projection. This section probes the representation claim: parameter randomization should move Π even when its pixel projection remains stable. We remark that $H = 0$ iff $\Pi_A = \Pi_B$, and the metric takes its maximal value $H = 1$ iff their supports are disjoint. Unlike KL-type divergences, H stays finite when randomization opens or closes routes, and since both laws are layered Markov chains on the same state graph, BC factorizes along layers: one backward pass computes H exactly, in time linear in the number of edges, and H^2 decomposes into per-input-state contributions (Section 8.3).

8.2.1 Trajectories change, maps need not

Under cascading randomization (layers cumulatively re-initialized output→input), both SG and RG diverge from the first randomized layer and plateau near the fully-random-model reference of Table 8.3 (Figure 8.1). The reference itself sits below the ceiling $H = 1$ because two fully randomized models still share some short trajectories: both may stop early at structurally dead neurons and route mass to the common cemetery state. Such short paths have a higher chance of coinciding across models than full trajectories reaching the input layer. Conditioning on survival removes this shared cemetery mass and gives the strict ceiling $H_{surv} \approx 1$. Trajectory- and map-level readouts are per-image orthogonal (Spearman $|\rho| \leq 0.19$ between H and pixel-level Gradient or $\alpha\beta$-LRP similarity): images with similar attribution maps need not have similar calculations. Per-game values, the conditioning, the floor mechanism and the per-image analysis are in Section 8.4.

What the map hides. Cascading randomization on the maps reproduces Adebayo et al. (2018) and Sixt et al. (2020): some methods retain map structure under full randomization. On ViT-B/16, the RG anchor itself retains Spearman $|\rho| = 0.454$, the highest of all methods, and HOG $r = 0.469$; on VGG-16, LRP-ε retains 0.341 and 0.457 (Table 8.2). The localization-selected RG modes are slightly more model sensitive than the anchor on all three models. Read against Figure 8.1, the two readouts

disagree: H sits at the fully random reference while the projection retains structure. The map-level failure stands; H locates the loss between calculation and projection.

8.3 Hellinger Distance and Disagreement Map

We compare two layered Markov processes on a common state graph and seek both a scalar divergence of the trajectory distributions and an exact per-pixel decomposition of this divergence at the input layer. We use the squared Hellinger distance because the geometric mean $\sqrt{PQ}$ preserves the multiplicative structure of Markov trajectory distributions and yields an exact linear-time dynamic program. Unlike Jeffreys divergence, it remains finite under support mismatch; unlike Jensen–Shannon, it admits an exact recursion on the layered graph. Later, we decompose the squared Hellinger distance as a sum of non-negative per-pixel contributions over all input-layer states, giving a disagreement map at the input layer (Theorem 8.4).

8.3.1 Layered Markov Process Foundations

We collect here the layered-Markov-process abstraction that underlies every Hellinger statement in this section: a finite layered graph augmented with a cemetery †, and the act-state *Markov Process*s (MPs) that the two games of this chapter instantiate on it. The natural RG carries an extra linear-routing state per neuron between consecutive activation states. We retain its gate, branch, and routing substeps for full-law diagnostics; only ordinary surviving paths admit exact act-only elimination, as detailed in Remark 8.1. For full routing-process dynamic programming below, the layer index includes these refined substeps; activation-boundary readouts form a subset of those layers.

Layered Markov Process with Cemetery

State graph. Fix an integer $L \geq 1$. For each $l \in \{0, 1, \ldots, L\}$ let $\mathscr{S}^{(l)}$ be a finite set of *ordinary states* at layer l. Adjoin a single absorbing *cemetery state* † shared across all layers and write $\mathscr{S}_+^{(l)} := \mathscr{S}^{(l)} \cup \{\dagger\}$. The full state graph is the disjoint union $\bigsqcup_l \mathscr{S}_+^{(l)}$, and transitions are only permitted from layer l to layer $l-1$ (or into the cemetery, which then absorbs the trajectory forever). The states at layer 0, the input layer, are referred to as *pixels* below; the per-pixel decomposition will sum exactly over $\mathscr{S}_+^{(0)}$.

Transition kernel. For each layer $l \geq 1$, a *backward transition kernel* $T^{(l)} : \mathscr{S}_+^{(l)} \to \Delta\big(\mathscr{S}_+^{(l-1)}\big)$ assigns to each state $s \in \mathscr{S}_+^{(l)}$ a probability distribution $T^{(l)}(s \to \cdot)$ over $\mathscr{S}_+^{(l-1)}$, with the constraint that the cemetery is absorbing:

$$T^{(l)}(\dagger \to \dagger) = 1, \qquad T^{(l)}(\dagger \to s') = 0 \ \text{ for } \ s' \in \mathscr{S}^{(l-1)}. \tag{8.1}$$

Ordinary states may transition either to the next layer's ordinary states or into the cemetery (early termination). We write $T^{(l)}(s \to \dagger)$ for the probability that an ordinary state at layer l jumps to the cemetery rather than to a layer-$(l-1)$ ordinary state.

Initial distribution and trajectories. Fix a deterministic starting state $s_0 \in \mathscr{S}^{(L)}$. A *trajectory* (indexed by layer, $\zeta_l \in \mathscr{S}_+^{(l)}$) is a sequence

$$\zeta = (\zeta_L, \zeta_{L-1}, \ldots, \zeta_0) \ \in \ \mathscr{S}_+^{(L)} \times \mathscr{S}_+^{(L-1)} \times \cdots \times \mathscr{S}_+^{(0)}, \tag{8.2}$$

and the kernel $T^{(\cdot)}$ together with δ_{s_0} at layer L induces a probability distribution on trajectories via

$$\Pi(\zeta_L, \zeta_{L-1}, \ldots, \zeta_0) \;=\; \mathbf{1}\{\zeta_L = s_0\} \prod_{k=1}^{L} T^{(k)}(\zeta_k \to \zeta_{k-1}). \tag{8.3}$$

For any $l \in \{0, \ldots, L\}$ the prefix distribution $\Pi^{(l)}$ over $(\zeta_L, \ldots, \zeta_l)$ is the marginal of Π over the trailing coordinates.

Two parallel processes. We compare two such processes that share the state graph and the start state s_0, but use different kernels: $M = A$ with $T_A^{(l)}, \Pi_A^{(l)}$, and $M = B$ with $T_B^{(l)}, \Pi_B^{(l)}$. The kernels may differ either because the underlying network weights differ (e.g. randomization), the activations differ (e.g. different inputs), or both.

How the Two Games Fit This Abstraction

We instantiate the layered MP of Section 8.3.1 for both games in this subsection. State labels $s_{i,p,\kappa}^{(l,\mathrm{type})}$ follow Appendices 4.6 and 4.7. We evaluate the SPNE policies of Theorems 4.5 (SG) and 4.9 (RG) at $\tau = 1$ (so activation-state decisions are deterministic, continue iff $z_j^{(l)} > 0$); for the RG we further fix $\varepsilon = 0$.

SG MP. Ordinary states at layers $l \in \{1, \ldots, L\}$ are activation states $s_{i,p}^{(l,\mathrm{act})}$, and layer-0 ordinary states are pixel-player terminal pairs $s_{j,q}^{(0)}$ (each pixel splits into two terminals, mirroring the player tag the trajectory carries to it). The deterministic start state is $s_0 = s_{u,+}^{(L,\mathrm{act})}$. The SPNE backward kernel keeps the player on positive weights and flips it on negative weights:

$$\begin{aligned} T_M^{(l)}\big(s_{j,p}^{(l,\mathrm{act})} \to s_{i,p}^{(l-1,\mathrm{act})}\big) &= \frac{W_M^{(l,+)}[i,j]}{\gamma_{M,j}^{(l)}}\, \mathbf{1}\{z_j^{(l)} > 0\}, \\ T_M^{(l)}\big(s_{j,p}^{(l,\mathrm{act})} \to s_{i,p'}^{(l-1,\mathrm{act})}\big) &= \frac{W_M^{(l,-)}[i,j]}{\gamma_{M,j}^{(l)}}\, \mathbf{1}\{z_j^{(l)} > 0\}, \qquad 2 \le l \le L, \end{aligned} \tag{8.4}$$

with $\gamma_{M,j}^{(l)} = \sum_i |W_M^{(l)}[i,j]|$, $p' = -p$, cemetery row $T_M^{(l)}(s_{j,p}^{(l,\mathrm{act})} \to \dagger) = 1 - \mathbf{1}\{z_j^{(l)} > 0\}\mathbf{1}\{\gamma_{M,j}^{(l)} > 0\}$, with $0/0 := 0$ in the ordinary transitions, and the analogous form at $l = 1$ landing on the pixel-player terminals $s_{i,p}^{(0)}, s_{i,p'}^{(0)}$. This is the natural SG MP of Section 4.6 verbatim.

RG MP. Ordinary states at layers $l \in \{1, \ldots, L\}$ are activation states $s_{i,p}^{(l,\mathrm{act})}$, and layer-0 ordinary states are pixel-stream terminal pairs $s_{j,\kappa}^{(0)}$ (each pixel splits into two terminals tagged by the layer-1 routing-edge stream). The deterministic start state is $s_0 = s_{u,+}^{(L,\mathrm{act})}$. The kernel composes the activation-state stop/continue decision and the subsequent κ-branched Gibbs routing into one act-to-act transition:

$$\begin{aligned} T_M^{(l)}\big(s_{j,p}^{(l,\mathrm{act})} \to s_{i,p}^{(l-1,\mathrm{act})}\big) &= \tfrac{1}{2}\, \pi_{M,+}^{(l)}(i \mid j)\, \mathbf{1}\{z_j^{(l)} > 0\}, \\ T_M^{(l)}\big(s_{j,p}^{(l,\mathrm{act})} \to s_{i,p'}^{(l-1,\mathrm{act})}\big) &= \tfrac{1}{2}\, \pi_{M,-}^{(l)}(i \mid j)\, \mathbf{1}\{z_j^{(l)} > 0\}, \qquad 2 \le l \le L, \end{aligned} \tag{8.5}$$

and at $l = 1$ the destinations are pixel-stream terminals carrying the routing-edge κ,

$$T_M^{(1)}\big(s_{j,p}^{(1,\mathrm{act})} \to s_{i,\kappa}^{(0)}\big) \;=\; \tfrac{1}{2}\,\pi_{M,\kappa}^{(1)}(i \mid j)\,\mathbf{1}\{z_j^{(1)} > 0\}, \qquad \kappa \in \{+,-\}, \tag{8.6}$$

For this act-only marginal, the cemetery probability is the residual $T_M^{(l)}(s_{j,p} \to \dagger) = 1 - \sum_{s' \in \mathscr{S}^{(l-1)}} T_M^{(l)}(s_{j,p} \to s')$. An empty routing stream has zero ordinary policy mass and its 1/2 branch share is absorbed, even when the source neuron is alive. The Gibbs policy on a nonempty stream is the temperature-τ distribution

$$\pi_{M,\kappa}^{(l)}(i \mid j) \;\propto\; \big[\, a_{M,i}^{(l-1)}\, W_{M,ij}^{(l,\kappa)} \,\big]^{1/\tau}, \qquad i \in \mathrm{Pred}_j^{(l)}, \tag{8.7}$$

(at $l = 1$, $a_{M,i}^{(0)} W_{M,ij}^{(1,\kappa)}$ is replaced by $[x_i\, W_{M,ij}^{(1)}]^{\kappa}$; when $\varepsilon > 0$, each continuing κ branch sends its $\frac{1}{2}\varepsilon^{1/\tau}/Z_{j,\kappa}^{(l)}$ share to the cemetery; as in the implementation, the bias node of Section 4.7 is omitted). If all scores in a stream are $-\infty$, its ordinary policy is zero and the whole branch is absorbed, in agreement with the outside-option convention. Equations (8.5)–(8.6) are the act-to-act marginal of the *diagnostic* routing process: it uses the original activation gates, omits the bias action, and normalizes each remaining routing branch. When the original bias mass is nonzero, this differs from the bias-inclusive game of Section 4.7. For example, $a = W = b = 1$, $\varepsilon = 0$, gives probability 1/4 on the ordinary positive route in that game, but 1/2 after omitting the bias action. Remark 8.1 concerns state elimination within the same chosen process; it does not identify these two different kernels. The reported full-trajectory diagnostics use the refined gate, branch, and routing substeps: absorption at different substeps is retained in the trajectory history. The ordinary surviving-path overlap can be computed from the displayed act-to-act kernels, but their collapsed cemetery row must not be substituted for the refined process when computing the full overlap.

Marginal Occupation Measures

For each model $M \in \{A,B\}$, the marginal probability that the chain visits state s at layer l is

$$m_M^{(l)}(s) \;=\; \sum_{\zeta:\zeta_l = s} \Pi_M^{(l)}(\zeta), \qquad s \in \mathscr{S}_+^{(l)}. \tag{8.8}$$

This satisfies the deterministic boundary condition

$$m_M^{(L)}(s_0) = 1, \qquad m_M^{(L)}(s) = 0 \;\text{ for }\; s \neq s_0, \tag{8.9}$$

and the one-step recursion

$$m_M^{(l-1)}(s') \;=\; \sum_{s \in \mathscr{S}_+^{(l)}} m_M^{(l)}(s)\, T_M^{(l)}(s \to s'), \qquad s' \in \mathscr{S}_+^{(l-1)}. \tag{8.10}$$

We refer to the single backward propagation (8.10) as the *Marginal Pass*. At the input layer, $m_M^{(0)}(s)$ is the probability that a backward trajectory from s_0 terminates at pixel s under model M, and $m_M^{(0)}(\dagger)$ is the total mass absorbed by the cemetery along the way.

8.3.2 Hellinger Distance from Bhattacharyya Coefficients

Properties. $\mathrm{BC}^{(l)}$ and $\mathrm{H}^{(l)}$ lie in [0,1] for every l, even under support mismatch. If a state is alive in one model and dead in the other, its geometric-mean row vanishes and its mass contributes directly

to the Hellinger increment. Since H is a metric and satisfies $\mathrm{H}^2 \le d_{\mathrm{TV}} \le \sqrt{2}\,\mathrm{H}$ and $\mathrm{H}^2 \le \frac{1}{2}\mathrm{KL}$, small Hellinger distance controls TV and small KL controls H as well. *Normalization conventions.* Here $d_{\mathrm{TV}}(\Pi_A, \Pi_B) = \frac{1}{2}\sum_\zeta |\Pi_A(\zeta) - \Pi_B(\zeta)| \in [0,1]$ is the probability-metric (half-L^1) total-variation distance, and KL is measured in nats; the displayed inequalities follow this convention. Readers using the unnormalized L^1 total variation $\|\Pi_A - \Pi_B\|_1 = 2d_{\mathrm{TV}}$ should absorb the factor of 2 accordingly.

Alternative divergences. KL and Jeffreys divergence have an exact Markov-chain chain rule but are unbounded and become infinite under support mismatch, e.g. when randomization opens or closes routes. Jensen–Shannon is bounded and finite, but $\frac{1}{2}(\Pi_A + \Pi_B)$ is not Markov on the layered graph, so no exact linear-time dynamic program is available. Wasserstein distances need a ground cost between neuron-labeled trajectories (across layers, signs and permuted neurons), for which there is no canonical choice, and the resulting transport problem has no exact local recursion. Hellinger is bounded, finite, and exactly tractable: the overlap factorizes along the layered law (Lemma 8.1), which gives a dynamic program linear in the number of edges (Algorithm 8.1) and the non-negative per-state decomposition of Theorem 8.4. Note that H compares the *undiscounted* law Π: the discount d_t does not enter the kernels, so H is blind to the path weights (α,β) and to the SG weight magnitudes $\gamma_j^{(l)}$, which enter only through d_t; it compares *where* trajectories go, not how much weight they carry.

Computation. Compute the marginal occupations $m_M^{(l)}$, run the Hellinger pass (Algorithm 8.1), and read off $\mathrm{H}^2(s)$ from (8.25); the total cost is linear in the number of edges.

Bhattacharyya Coefficient and the Geometric-Mean Kernel

Definitions. For two probability distributions P, Q on a common finite set, the *Bhattacharyya coefficient* is the geometric overlap

$$\mathrm{BC}(P,Q) \;:=\; \sum_y \sqrt{P(y)\,Q(y)} \;\in\; [0,1], \tag{8.11}$$

and the *squared Hellinger distance* is

$$\mathrm{H}^2(P,Q) \;:=\; \tfrac{1}{2}\sum_y \big(\sqrt{P(y)} - \sqrt{Q(y)}\big)^2 \;=\; 1 - \mathrm{BC}(P,Q) \;\in\; [0,1]. \tag{8.12}$$

The Hellinger distance $\mathrm{H}(P,Q) = \sqrt{1 - \mathrm{BC}(P,Q)}$ is a true metric: $0 \le \mathrm{H} \le 1$, with $\mathrm{H} = 0$ iff $P = Q$ and $\mathrm{H} = 1$ iff P and Q have disjoint supports. Crucially, $\sqrt{0 \cdot Q(y)} = 0$, so mismatched supports cause no infinity—only a smaller overlap.

Geometric-mean kernel. For each layer $l \ge 1$ and pair of states $(s, s') \in \mathscr{S}_+^{(l)} \times \mathscr{S}_+^{(l-1)}$, define the *geometric-mean kernel*

$$\widetilde{T}^{(l)}(s, s') \;:=\; \sqrt{T_A^{(l)}(s \to s')\; T_B^{(l)}(s \to s')}. \tag{8.13}$$

This kernel is non-negative but in general sub-stochastic. Its row sum

$$\widetilde{R}^{(l)}(s) \;:=\; \sum_{s' \in \mathscr{S}_+^{(l-1)}} \widetilde{T}^{(l)}(s, s') \tag{8.14}$$

satisfies $0 \le \widetilde{R}^{(l)}(s) \le 1$ by Cauchy–Schwarz, with equality iff $T_A^{(l)}(s \to \cdot) = T_B^{(l)}(s \to \cdot)$. Cemetery handling is automatic: $T_A^{(l)}(\dagger \to \dagger) = T_B^{(l)}(\dagger \to \dagger) = 1$, so $\widetilde{T}^{(l)}(\dagger, \dagger) = 1$ and $\widetilde{R}^{(l)}(\dagger) = 1$.

Multiplicative Chain Rule

Lemma 8.1 (Chain rule, `Lean_70`). *For every $l \in \{0, 1, \dots, L\}$, the Bhattacharyya coefficient of the prefix distributions factorizes along trajectories:*

$$\mathrm{BC}\big(\Pi_A^{(l)}, \Pi_B^{(l)}\big) \;=\; \sum_{(\zeta_L, \dots, \zeta_l)} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l+1}^{L} \widetilde{T}^{(k)}(\zeta_k, \zeta_{k-1}). \tag{8.15}$$

Proof. Substitute the Markov factorization $\Pi_M^{(l)}(\zeta) = \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l+1}^{L} T_M^{(k)}(\zeta_k, \zeta_{k-1})$ for $M \in \{A,B\}$ into the definition of BC:

$$\mathrm{BC}\big(\Pi_A^{(l)}, \Pi_B^{(l)}\big) \;=\; \sum_{(\zeta_L, \dots, \zeta_l)} \mathbf{1}\{\zeta_L = s_0\} \sqrt{\prod_{k=l+1}^{L} T_A^{(k)}(\zeta_k, \zeta_{k-1})\, T_B^{(k)}(\zeta_k, \zeta_{k-1})} \tag{8.16}$$

$$= \sum_{(\zeta_L, \dots, \zeta_l)} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l+1}^{L} \widetilde{T}^{(k)}(\zeta_k, \zeta_{k-1}), \tag{8.17}$$

using $\sqrt{\prod_k a_k b_k} = \prod_k \sqrt{a_k b_k}$ for non-negative factors. □

Lemma 8.2 (Monotonicity, `Lean_71`). *Let $\mathrm{BC}^{(l)} := \mathrm{BC}(\Pi_A^{(l)}, \Pi_B^{(l)})$. Then $\mathrm{BC}^{(l)}$ is monotonically non-increasing as l decreases from L to 0:*

$$1 \;=\; \mathrm{BC}^{(L)} \;\ge\; \mathrm{BC}^{(L-1)} \;\ge\; \cdots \;\ge\; \mathrm{BC}^{(0)} \;\ge\; 0. \tag{8.18}$$

Equivalently, the squared Hellinger distance $\mathrm{H}^2_{(l)} = 1 - \mathrm{BC}^{(l)}$ is monotonically non-decreasing.

Proof. Define

$$g^{(l)}(s) \;:=\; \sum_{(\zeta_L, \dots, \zeta_l)} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l+1}^{L} \widetilde{T}^{(k)}(\zeta_k, \zeta_{k-1})\, \mathbf{1}\{\zeta_l = s\}. \tag{8.19}$$

From Lemma 8.1, $\mathrm{BC}^{(l)} = \sum_{s \in \mathscr{S}_+^{(l)}} g^{(l)}(s)$. Grouping the layer-$(l-1)$ sum by $\zeta_l = s$,

$$\mathrm{BC}^{(l-1)} \;=\; \sum_{s \in \mathscr{S}_+^{(l)}} g^{(l)}(s)\; \widetilde{R}^{(l)}(s). \tag{8.20}$$

Since $\widetilde{R}^{(l)}(s) \in [0,1]$ and $g^{(l)}(s) \ge 0$, we get $\mathrm{BC}^{(l-1)} \le \mathrm{BC}^{(l)}$. The base case $\mathrm{BC}^{(L)} = 1$ holds because $\Pi_A^{(L)} = \Pi_B^{(L)} = \delta_{s_0}$ and $\sqrt{1 \cdot 1} = 1$. □

Hellinger backward pass

The chain rule (Lemma 8.1) yields a backward DP that propagates the per-state cumulative $\widetilde{T}$-mass $g^{(l)}(s)$ from the start state s_0 down through the layers. The prefix Bhattacharyya coefficient at each layer is the row sum $\mathrm{BC}^{(l)} = \sum_s g^{(l)}(s)$, the prefix Hellinger distance is $\mathrm{H}^{(l)} = \sqrt{1 - \mathrm{BC}^{(l)}}$, and the per-terminal-state disagreement map is read off at $l = 0$ from (8.25). The map is defined on the full terminal set $\mathscr{S}_+^{(0)} = \mathscr{S}^{(0)} \cup \{\dagger\}$; the visualized pixel attribution map is its restriction to ordinary pixels $\mathscr{S}^{(0)}$, and its sum is therefore $\mathrm{H}^2_{\mathrm{ord}} = \mathrm{H}^2 - \mathrm{H}^2(\dagger)$, not the full H^2 (Theorem 8.4(iii)).

Algorithm 8.1: Hellinger backward pass (with per-terminal-state disagreement map)

Initialize the occupation marginals $m_M^{(L)}$ and overlap mass $g^{(L)}$ at the target output.;
For $l = L, L-1, \ldots, 1$, propagate each model's marginals with its kernel and propagate overlap by $g^{(l-1)}(s') = \sum_s g^{(l)}(s) \sqrt{T_A^{(l)}(s,s') T_B^{(l)}(s,s')}$.;
At each layer compute $\mathrm{BC}^{(l)} = \sum_s g^{(l)}(s)$ and $\mathrm{H}^{(l)} = \sqrt{1 - \mathrm{BC}^{(l)}}$.;
At every terminal state return $\mathrm{H}^2(s) = \frac{1}{2}(m_A^{(0)}(s) + m_B^{(0)}(s)) - g^{(0)}(s)$. The pixel disagreement map restricts this output to ordinary input states.

Lemma 8.3 (Backward pass correctness, `Lean_72`). *Algorithm 8.1 computes $g^{(l)}$, hence $\mathrm{BC}^{(l)}$, $\mathrm{H}^{(l)}$, and $\mathrm{H}^2(s)$ correctly for every l and s. In particular,*

$$\sum_{s \in \mathscr{S}_+^{(0)}} \mathrm{H}^2(s) \;=\; \mathrm{H}^2_{(0)}. \tag{8.21}$$

Proof. Correctness of $g^{(l)}$ by downward induction on l. At $l = L$, the initialization $g^{(L)}(s_0) = 1$ and $g^{(L)}(s) = 0$ for $s \neq s_0$ matches the definition $g^{(L)}(s) = \mathbf{1}\{s = s_0\}$ in Lemma 8.2 (the empty product equals 1).

Assume $g^{(l)}$ stored by the algorithm equals the $g^{(l)}$ defined in Lemma 8.2. In iteration l, the algorithm computes

$$g^{(l-1)}(s') \;=\; \sum_{s \in \mathscr{S}_+^{(l)}} g^{(l)}(s)\, \widetilde{T}^{(l)}(s, s'), \tag{8.22}$$

which by direct expansion equals

$$\begin{aligned} &\sum_s \Big(\sum_{(\zeta_L, \ldots, \zeta_l)} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l+1}^{L} \widetilde{T}^{(k)}(\zeta_k, \zeta_{k-1})\, \mathbf{1}\{\zeta_l = s\} \Big) \widetilde{T}^{(l)}(s, s') \\ &\quad = \sum_{(\zeta_L, \ldots, \zeta_{l-1})} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l}^{L} \widetilde{T}^{(k)}(\zeta_k, \zeta_{k-1})\, \mathbf{1}\{\zeta_{l-1} = s'\}. \end{aligned} \tag{8.23}$$

This equals $g^{(l-1)}(s')$ from Lemma 8.2, closing the induction.

Correctness of $\mathrm{BC}^{(l)}$. After the loop, the algorithm sets $\mathrm{BC}^{(l)} = \sum_{s'} g^{(l)}(s')$, which by the proof of Lemma 8.2 equals $\mathrm{BC}\big(\Pi_A^{(l)}, \Pi_B^{(l)}\big)$. Then $\mathrm{H}^{(l)} = \sqrt{1 - \mathrm{BC}^{(l)}}$ by definition (8.12).

Correctness of $\mathrm{H}^2(\cdot)$. At $l = 0$, the stored value satisfies

$$g^{(0)}(s) = \sum_{\zeta:\zeta_0=s} \sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)}, \tag{8.24}$$

which is exactly the quantity used in the proof of Theorem 8.4. Substituting into (8.25) gives the per-pixel disagreement values from Theorem 8.4, and (8.26) guarantees they sum to $\mathrm{H}^2_{(0)}$. □

Remark 8.1 (Routing-state elimination and absorption histories, `Lean_73`*).* The refined diagnostic routing process just defined alternates activation and linear-routing states. A dead activation stops at the gate substep; an alive activation chooses its κ branch uniformly, after which an empty branch or outside option can absorb at the routing substep. The refined layered process retains these substeps and pads the cemetery state after absorption. Algorithm 8.1 applies to this expanded state graph; the diagnostics are read at the activation boundaries. This is the process implemented by the separate gate and branch absorption updates.

For a trajectory that remains ordinary, deleting each intermediate routing state is a bijection: its neuron and player are determined by the adjacent activation states, and its branch by whether the player is kept or switched (or by the pixel-stream tag at the input). The act-to-act formulas therefore preserve ordinary surviving-path probabilities and their Bhattacharyya overlap. The same applies after conditioning both laws on survival.

For absorbing trajectories, deletion need not be injective: a gate stop and a later empty-branch stop can become the same act-only trajectory. Thus full-law Bhattacharyya and Hellinger equality is asserted only when absorption histories are recoverable, for example when no alive routing branch leaks to the cemetery. In general the finite-fibre Cauchy–Schwarz inequality gives

$$\mathrm{BC}_{\text{refined}} \le \mathrm{BC}_{\text{act-only}}, \qquad \mathrm{H}_{\text{refined}} \ge \mathrm{H}_{\text{act-only}}.$$

For example, take one input $x = 1$ and weight $W = 1$. Model A has bias 0 and continues, with its empty negative branch absorbing with probability $1/2$; model B has bias -2 and stops at the gate. Their refined path supports are disjoint, so $\mathrm{BC}_{\text{refined}} = 0$. Collapsing the two absorption substeps instead gives $\mathrm{BC}_{\text{act-only}} = 1/\sqrt{2}$.

8.3.3 Per-Pixel Disagreement Map

The squared Hellinger distance H^2 admits an exact non-negative decomposition over input-layer states: each input pixel carries a calibrated share of the total trajectory disagreement (Theorem 8.4), giving a disagreement *map* that is itself an attribution of the trajectory mismatch back to the input. The conditioned-on-survival variant removes the cemetery contribution that otherwise bounds plain H from below on randomized models.

Per-Pixel Decomposition: The Disagreement Map

The squared Hellinger distance at layer 0 decomposes exactly over terminal pixels.

Theorem 8.4 (Per-pixel decomposition, `Lean_74`**).** *Let* $\mathrm{H}^2 := \mathrm{H}^2\big(\Pi_A^{(0)}, \Pi_B^{(0)}\big)$. *For* $s \in \mathscr{S}_+^{(0)}$, *define*

$$\mathrm{H}^2(s) \;:=\; \tfrac{1}{2}\big(m_A^{(0)}(s) + m_B^{(0)}(s)\big) \;-\; g^{(0)}(s), \tag{8.25}$$

where $g^{(0)}(s)$ is the cumulative $\widetilde{T}$-mass at s. Then:

(i) $\mathrm{H}^2(s) \geq 0$ *for every* $s \in \mathscr{S}_+^{(0)}$.
(ii)

$$\mathrm{H}^2 \;=\; \sum_{s \in \mathscr{S}_+^{(0)}} \mathrm{H}^2(s). \tag{8.26}$$

(iii)

$$\mathrm{H}^2_{\mathrm{ord}} \;\coloneqq\; \sum_{s \in \mathscr{S}^{(0)}} \mathrm{H}^2(s) \;=\; \mathrm{H}^2 - \mathrm{H}^2(\dagger) \tag{8.27}$$

Proof. Apply the algebraic identity $(\sqrt{a} - \sqrt{b})^2 = a + b - 2\sqrt{ab}$ to each trajectory:

$$\mathrm{H}^2 \;=\; \tfrac{1}{2}\sum_{\zeta}\big(\sqrt{\Pi_A^{(0)}(\zeta)} - \sqrt{\Pi_B^{(0)}(\zeta)}\big)^2 \;=\; \tfrac{1}{2}\sum_{\zeta}\big[\Pi_A^{(0)}(\zeta) + \Pi_B^{(0)}(\zeta) - 2\sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)}\big]. \tag{8.28}$$

Group the trajectory sum by the terminal pixel $\zeta_0 = s$:

$$\mathrm{H}^2 \;=\; \tfrac{1}{2}\sum_{s \in \mathscr{S}_+^{(0)}}\Big[\sum_{\zeta:\zeta_0 = s}\Pi_A^{(0)}(\zeta) + \sum_{\zeta:\zeta_0 = s}\Pi_B^{(0)}(\zeta) - 2\sum_{\zeta:\zeta_0 = s}\sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)}\Big]. \tag{8.29}$$

By definition, $\sum_{\zeta:\zeta_0=s}\Pi_M^{(0)}(\zeta) = m_M^{(0)}(s)$ for $M \in \{A,B\}$. By Lemma 8.1 restricted to trajectories terminating at s (i.e. tracing through Lemma 8.2's definition of $g^{(l)}$), $\sum_{\zeta:\zeta_0=s}\sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)} = g^{(0)}(s)$. Substituting,

$$\mathrm{H}^2 \;=\; \tfrac{1}{2}\sum_{s}\big[m_A^{(0)}(s) + m_B^{(0)}(s) - 2g^{(0)}(s)\big] \;=\; \sum_{s \in \mathscr{S}_+^{(0)}}\mathrm{H}^2(s), \tag{8.30}$$

proving (ii).

For (i), apply Cauchy–Schwarz to the sum defining $g^{(0)}(s)$:

$$g^{(0)}(s) = \sum_{\zeta:\zeta_0=s}\sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)} \leq \sqrt{\Big(\sum_{\zeta:\zeta_0=s}\Pi_A^{(0)}(\zeta)\Big)\Big(\sum_{\zeta:\zeta_0=s}\Pi_B^{(0)}(\zeta)\Big)} = \sqrt{m_A^{(0)}(s)\,m_B^{(0)}(s)}. \tag{8.31}$$

Combined with the AM–GM inequality $\sqrt{m_A^{(0)}(s)m_B^{(0)}(s)} \leq \tfrac{1}{2}(m_A^{(0)}(s) + m_B^{(0)}(s))$, this gives $g^{(0)}(s) \leq \tfrac{1}{2}(m_A^{(0)}(s) + m_B^{(0)}(s))$, so $\mathrm{H}^2(s) \geq 0$. Equality holds iff $\Pi_A^{(0)}(\cdot \mid \zeta_0 = s) = \Pi_B^{(0)}(\cdot \mid \zeta_0 = s)$ *and* $m_A^{(0)}(s) = m_B^{(0)}(s)$.

Item (iii) is immediate from (ii) by separating the cemetery from the ordinary pixels. □

The disagreement map. Theorem 8.4 defines $\mathrm{H}^2(\cdot)$ on the full terminal set $\mathscr{S}_+^{(0)} = \mathscr{S}^{(0)} \cup \{\dagger\}$: this is the *full terminal-state decomposition*, with $\sum_{s \in \mathscr{S}_+^{(0)}}\mathrm{H}^2(s) = \mathrm{H}^2_{(0)}$, and the cemetery contribution $\mathrm{H}^2(\dagger)$ is part of it. The *Hellinger disagreement map* is the ordinary-pixel restriction $s \mapsto \mathrm{H}^2(s)$ on $\mathscr{S}^{(0)}$, which

is the quantity we visualize. It is non-negative, sums to $\mathrm{H}^2_{\mathrm{ord}}$ over ordinary pixels, and is computed from $g^{(0)}, m_A^{(0)}, m_B^{(0)}$. The quantity $\mathrm{H}^2_{\mathrm{ord}}$ is an unnormalized ordinary-state subsum; the conditioned-on-survival Hellinger distance that normalizes it is introduced below. Recall from Section 8.3.1 that layer-0 ordinary states of the two games are *tagged* pixel pairs $s^{(0)}_{j,\pm}$, tagged by the player q for the SG and by the stream sign κ for the RG; the per-pixel attribution displayed in figures aggregates the two tags,

$$\mathrm{H}^2_{\mathrm{pixel}}(j) \;=\; \mathrm{H}^2\big(s^{(0)}_{j,+}\big) + \mathrm{H}^2\big(s^{(0)}_{j,-}\big), \tag{8.32}$$

which remains non-negative and sums to $\mathrm{H}^2_{\mathrm{ord}}$ over pixels. By the Cauchy–Schwarz step in the proof of Theorem 8.4(i), $g^{(0)}(s) \le \sqrt{m_A^{(0)}(s)\, m_B^{(0)}(s)}$, so $\mathrm{H}^2(s)$ is bounded below by the marginal surrogate $\frac{1}{2}\big(m_A^{(0)}(s) + m_B^{(0)}(s)\big) - \sqrt{m_A^{(0)}(s)\, m_B^{(0)}(s)}$, the per-pixel squared Hellinger distance between the layer-0 marginal occupations; the trajectory map is strictly larger whenever the conditional distributions of trajectories reaching pixel s differ between models.

Conditioning on Survival to the Ordinary Input Layer

When the cemetery absorbs a large fraction of trajectory mass, the total Hellinger distance mixes two effects: disagreement about *whether* a trajectory survives and disagreement about *where surviving trajectories go*. To isolate the second effect, we condition each model's trajectory law on the event that the trajectory reaches an ordinary input-layer state. The resulting scalar compares the two *surviving* trajectory distributions directly and removes the common cemetery floor. Because survival to the input layer factorizes along the layered graph, this conditioning remains Markov and is implemented by a per-state Doob h-transform (with $h = \psi_M$).

Theorem 8.5 (Conditioned-survival trajectory law and conditioned Hellinger distance, `Lean_75`). *Let* $\mathsf{Surv} := \{(\zeta_L, \dots, \zeta_0) : \zeta_0 \in \mathscr{S}^{(0)}\}$ *be the event, in the trajectory space* $\prod_{l=0}^{L} \mathscr{S}_+^{(l)}$, *that the backward trajectory ends in an ordinary input-layer state — equivalently, the event that the trajectory never enters the cemetery* †. *For each model* $M \in \{A,B\}$ *define the survival function*

$$\psi_M^{(l)}(s) \;:=\; \mathbb{P}_M(\mathsf{Surv} \mid \zeta_l = s), \qquad s \in \mathscr{S}_+^{(l)},\;\; l \in \{0, \dots, L\}. \tag{8.33}$$

Then:

(i) The boundary and recursion are

$$\psi_M^{(0)}(s) = 1 \;\; \text{for } s \in \mathscr{S}^{(0)}, \qquad \psi_M^{(0)}(\dagger) = 0, \tag{8.34}$$

and for every $l \ge 1$,

$$\psi_M^{(l)}(s) \;=\; \sum_{s' \in \mathscr{S}_+^{(l-1)}} T_M^{(l)}(s \to s')\, \psi_M^{(l-1)}(s') \;=\; \sum_{s' \in \mathscr{S}^{(l-1)}} T_M^{(l)}(s \to s')\, \psi_M^{(l-1)}(s'). \tag{8.35}$$

In particular,

$$Z_M \;:=\; \mathbb{P}_M(\mathsf{Surv}) \;=\; \psi_M^{(L)}(s_0) \;=\; \sum_{s \in \mathscr{S}^{(0)}} m_M^{(0)}(s) \;=\; 1 - m_M^{(0)}(\dagger). \tag{8.36}$$

(ii) *Assume $Z_M > 0$. For every ordinary state $s \in \mathscr{S}^{(l)}$ with $\psi_M^{(l)}(s) > 0$, define the conditioned kernel on ordinary states by*

$$T_{M,\mathrm{surv}}^{(l)}(s \to s') \;\coloneqq\; \frac{T_M^{(l)}(s \to s')\, \psi_M^{(l-1)}(s')}{\psi_M^{(l)}(s)}, \qquad s' \in \mathscr{S}^{(l-1)}. \tag{8.37}$$

Then $T_{M,\mathrm{surv}}^{(l)}(s \to \cdot)$ is row-stochastic on $\mathscr{S}^{(l-1)}$, and the conditional trajectory distribution $\Pi_{M,\mathrm{surv}} \coloneqq \Pi_M(\,\cdot \mid \mathsf{Surv}\,)$ factorizes as the layered Markov law

$$\Pi_{M,\mathrm{surv}}(\zeta_L, \dots, \zeta_0) = \mathbf{1}\{\zeta_L = s_0\} \prod_{k=1}^{L} T_{M,\mathrm{surv}}^{(k)}(\zeta_k \to \zeta_{k-1}), \qquad (\zeta_L, \dots, \zeta_0) \in \mathsf{Surv}. \tag{8.38}$$

Its layer-l marginals satisfy

$$m_{M,\mathrm{surv}}^{(l)}(s) \;=\; \frac{m_M^{(l)}(s)\, \psi_M^{(l)}(s)}{Z_M}, \qquad s \in \mathscr{S}^{(l)}. \tag{8.39}$$

(iii) *Define the conditioned geometric-mean kernel*

$$\widetilde{T}_{\mathrm{surv}}^{(l)}(s,s') \coloneqq \sqrt{T_{A,\mathrm{surv}}^{(l)}(s \to s')\, T_{B,\mathrm{surv}}^{(l)}(s \to s')} = \frac{\widetilde{T}^{(l)}(s,s') \sqrt{\psi_A^{(l-1)}(s')\, \psi_B^{(l-1)}(s')}}{\sqrt{\psi_A^{(l)}(s)\, \psi_B^{(l)}(s)}}. \tag{8.40}$$

The Bhattacharyya coefficient of the conditioned trajectory laws is

$$\mathrm{BC}\big(\Pi_{A,\mathrm{surv}}, \Pi_{B,\mathrm{surv}}\big) \;=\; \sum_{(\zeta_L,\dots,\zeta_0) \in \mathsf{Surv}} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=1}^{L} \widetilde{T}_{\mathrm{surv}}^{(k)}(\zeta_k, \zeta_{k-1}), \tag{8.41}$$

so it is computed by the same Hellinger pass (Algorithm 8.1), but on the ordinary-state graph with the conditioned kernel.

(iv) *Let*

$$\mathrm{BC}_{\mathrm{ord}}^{(0)} \;\coloneqq\; \sum_{s \in \mathscr{S}^{(0)}} g^{(0)}(s) \;=\; \sum_{(\zeta_L,\dots,\zeta_0) \in \mathsf{Surv}} \sqrt{\Pi_A(\zeta_L, \dots, \zeta_0)\, \Pi_B(\zeta_L, \dots, \zeta_0)}. \tag{8.42}$$

Then

$$\mathrm{BC}\big(\Pi_{A,\mathrm{surv}}, \Pi_{B,\mathrm{surv}}\big) \;=\; \frac{\mathrm{BC}_{\mathrm{ord}}^{(0)}}{\sqrt{Z_A Z_B}}, \tag{8.43}$$

and the conditioned-survival Hellinger distance is

$$\mathrm{H}_{\mathrm{surv}}^2 \;\coloneqq\; \mathrm{H}^2\big(\Pi_{A,\mathrm{surv}}, \Pi_{B,\mathrm{surv}}\big) \;=\; 1 - \frac{\mathrm{BC}_{\mathrm{ord}}^{(0)}}{\sqrt{Z_A Z_B}}. \tag{8.44}$$

Proof. For (i), (8.34) is immediate from the definition of Surv: at layer 0 every ordinary state survives and the cemetery does not. For $l \geq 1$, conditioning on the next step gives

$$\psi_M^{(l)}(s) = \sum_{s' \in \mathscr{S}_+^{(l-1)}} T_M^{(l)}(s \to s')\, \mathbb{P}_M(\mathsf{Surv} \mid \zeta_{l-1} = s') = \sum_{s' \in \mathscr{S}_+^{(l-1)}} T_M^{(l)}(s \to s')\, \psi_M^{(l-1)}(s'). \tag{8.45}$$

Since $\psi_M^{(l-1)}(\dagger) = 0$, the cemetery term drops out and (8.35) follows. Evaluating at s_0 gives $Z_M = \psi_M^{(L)}(s_0)$. The identities with $m_M^{(0)}$ are just the layer-0 marginal of the event Surv.

For (ii), row-stochasticity follows from (8.35):

$$\sum_{s' \in \mathscr{S}^{(l-1)}} T_{M,\mathrm{surv}}^{(l)}(s \to s') = \frac{1}{\psi_M^{(l)}(s)} \sum_{s' \in \mathscr{S}^{(l-1)}} T_M^{(l)}(s \to s')\, \psi_M^{(l-1)}(s') = 1. \tag{8.46}$$

Now fix $(\zeta_L, \dots, \zeta_0) \in \mathsf{Surv}$. By definition of conditional probability,

$$\Pi_{M,\mathrm{surv}}(\zeta_L, \dots, \zeta_0) = \frac{\Pi_M(\zeta_L, \dots, \zeta_0)}{Z_M} = \frac{\mathbf{1}\{\zeta_L = s_0\}}{Z_M} \prod_{k=1}^{L} T_M^{(k)}(\zeta_k \to \zeta_{k-1}). \tag{8.47}$$

Multiplying the conditioned kernels and telescoping the survival factors gives

$$\prod_{k=1}^{L} T_{M,\mathrm{surv}}^{(k)}(\zeta_k \to \zeta_{k-1}) = \prod_{k=1}^{L} T_M^{(k)}(\zeta_k \to \zeta_{k-1}) \frac{\psi_M^{(k-1)}(\zeta_{k-1})}{\psi_M^{(k)}(\zeta_k)} \tag{8.48}$$

$$= \frac{\psi_M^{(0)}(\zeta_0)}{\psi_M^{(L)}(s_0)} \prod_{k=1}^{L} T_M^{(k)}(\zeta_k \to \zeta_{k-1}) \tag{8.49}$$

$$= \frac{1}{Z_M} \prod_{k=1}^{L} T_M^{(k)}(\zeta_k \to \zeta_{k-1}), \tag{8.50}$$

because $\zeta_0 \in \mathscr{S}^{(0)}$ implies $\psi_M^{(0)}(\zeta_0) = 1$. This is (8.38). For the marginals,

$$m_{M,\mathrm{surv}}^{(l)}(s) = \mathbb{P}_M(\zeta_l = s \mid \mathsf{Surv}) = \frac{\mathbb{P}_M(\zeta_l = s, \mathsf{Surv})}{Z_M} = \frac{m_M^{(l)}(s)\, \psi_M^{(l)}(s)}{Z_M}, \tag{8.51}$$

which is (8.39).

For (iii), substitute (8.37) into the square root to obtain (8.40). Then apply Lemma 8.1 to the conditioned trajectory laws (8.38) on the ordinary-state graph. This gives (8.41) directly.

For (iv), expand the Bhattacharyya coefficient of the conditioned laws:

$$\mathrm{BC}\big(\Pi_{A,\mathrm{surv}}, \Pi_{B,\mathrm{surv}}\big) = \sum_{(\zeta_L,\dots,\zeta_0) \in \mathsf{Surv}} \sqrt{\frac{\Pi_A(\zeta_L, \dots, \zeta_0)}{Z_A} \frac{\Pi_B(\zeta_L, \dots, \zeta_0)}{Z_B}} \tag{8.52}$$

$$= \frac{1}{\sqrt{Z_A Z_B}} \sum_{(\zeta_L,\dots,\zeta_0) \in \mathsf{Surv}} \sqrt{\Pi_A(\zeta_L, \dots, \zeta_0)\, \Pi_B(\zeta_L, \dots, \zeta_0)}. \tag{8.53}$$

Grouping the last sum by the ordinary terminal state $\zeta_0 = s$ and using the definition of $g^{(0)}(s)$ exactly as in the proof of Theorem 8.4 yields (8.42), hence (8.43). Equation (8.44) is then the identity $\mathrm{H}^2 = 1 - \mathrm{BC}$ applied to the conditioned trajectory laws. □

Implementation consequence. Theorem 8.5 gives two exactly equivalent computations of the conditioned scalar:

(a) *Kernel-side conditioning:* replace each row by the reweighted kernel (8.37), form the geometric mean (8.40), and run the ordinary Hellinger backward pass on $\mathscr{S}^{(L)} \times \cdots \times \mathscr{S}^{(0)}$.
(b) *Post-hoc normalization:* run the original pass once, extract the ordinary overlap $\mathrm{BC}^{(0)}_{\mathrm{ord}} = \sum_{s \in \mathscr{S}^{(0)}} g^{(0)}(s)$ and the survival masses Z_A, Z_B, and apply (8.44).

The first route makes the conditioned trajectory law explicit; the second is cheaper when only the scalar is needed, but for deep networks it becomes numerically infeasible when the survival probability Z_M of either model $M \in \{A,B\}$ gets too small.

Conditioned disagreement map. Applying Theorem 8.4 to the conditioned kernels $T^{(l)}_{M,\mathrm{surv}}$ yields the conditioned disagreement map

$$\mathrm{H}^2_{\mathrm{surv}}(s) \;\coloneqq\; \tfrac{1}{2}\big(m^{(0)}_{A,\mathrm{surv}}(s) + m^{(0)}_{B,\mathrm{surv}}(s)\big) - g^{(0)}_{\mathrm{surv}}(s), \qquad s \in \mathscr{S}^{(0)}, \tag{8.54}$$

which is non-negative and satisfies $\sum_{s \in \mathscr{S}^{(0)}} \mathrm{H}^2_{\mathrm{surv}}(s) = \mathrm{H}^2_{\mathrm{surv}}$.

8.3.4 Experiments

This subsection puts the trajectory-space machinery to two empirical tests, holding the model fixed: how the Hellinger trajectory distance reacts to input perturbations, and whether the resulting distance separates inputs from different ImageNet-S classes from one another and from random noise. Here SG is the Stopping-Game kernel (8.4) without risk aversion ($\lambda = 0$; named "ADF" in the code) and RG is the Routing-Game kernel (8.5) at $\tau = 1$.

Input-Noise Sensitivity

For each architecture and each noise standard deviation $\sigma \in \{0.005, 0.01, 0.02, 0.03, 0.05, 0.1, 0.2, 0.3, 0.5\}$ we draw one standard-Gaussian tensor η per image and compute $\mathrm{H}(\Pi_A(x), \Pi_A(x + \sigma\eta))$ under both game kernels with the argmax target of the clean input. Figure 8.2 reports the mean H and $\mathrm{H}_{\mathrm{surv}}$ curves over the swept σ; both increase with σ for both games on all three architectures. On VGG-16 the RG H and $\mathrm{H}_{\mathrm{surv}}$ are equal, as the kernel routes no trajectories to states belonging to dead neurons. On ResNet-50 and ViT-B/16, skip connections route trajectories to dead-neuron states, and on ViT-B/16 the GELU gate also sends some trajectories to the cemetery at every state.

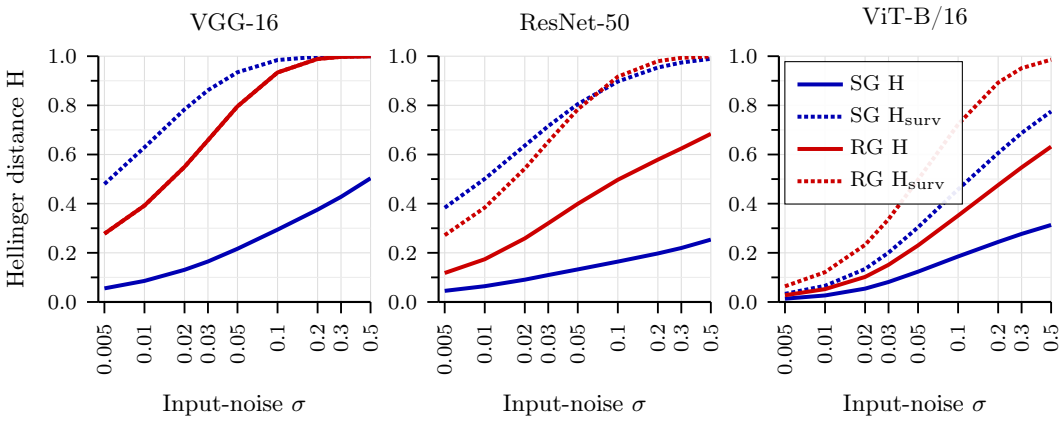


Fig. 8.2: Input-noise Hellinger trajectory distance. Solid: plain H; dotted: H_{surv}, conditioned on survival to an ordinary input-layer state. x-axis is log-scale input-noise σ; same (A,A) model on both sides, only the input differs.

Class Separability and Reference Baselines

Can the Hellinger distance differentiate between inputs of different classes? Partly; the reference baselines in Table 8.1 make the answer architecture- and kernel-specific:

- On **VGG-16**, the SG H is the kernel that does the work: clean-intraclass H = 0.513 sits below clean-interclass H = 0.607. The RG kernel saturates at H = 1 on every row on VGG-16, so it cannot resolve class structure on this architecture.
- On **ResNet-50**, the RG kernel is the discriminating one: clean-intraclass H = 0.786 vs. clean-interclass H = 0.902. The SG kernel does not separate classes on ResNet-50 (intraclass 0.272 vs. interclass 0.268).
- On **ViT-B/16**, the RG kernel gives intraclass 0.789 vs. interclass 0.882; the SG gap (0.422 ± 0.023 vs. 0.435 ± 0.017) is smaller than the spread over pairs, so the SG does not separate classes there.

Random-vs-random is the lowest row in every column whose rows differ, and clean-vs-random is the highest row of every plain-H column except the SG on ResNet-50 (0.269 vs. intraclass 0.272) and the saturated RG on VGG-16, consistent with the model routing Gaussian inputs differently from in-distribution images. The conditioned variant H_{surv} is at least 0.99 on every row of VGG-16 and ResNet-50 and for the RG on ViT-B/16, so it carries no class-separability signal there. For the SG on ViT-B/16 it separates random-vs-random (0.852) from the clean rows (0.957–0.968), but neither intraclass from interclass pairs (0.963 vs. 0.968) nor clean-vs-random (0.957) from the clean pairs.

Architecture	Baseline	Stopping Game		Routing Game	
		H	H_{surv}	H	H_{surv}
VGG-16	clean-intraclass	0.513 ± 0.071	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
	clean-interclass	0.607 ± 0.032	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
	clean-vs-random	0.663 ± 0.020	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
	random-vs-random	0.320 ± 0.019	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
ResNet-50	clean-intraclass	0.272 ± 0.056	0.999 ± 0.000	0.786 ± 0.044	1.000 ± 0.000
	clean-interclass	0.268 ± 0.045	1.000 ± 0.000	0.902 ± 0.033	1.000 ± 0.000
	clean-vs-random	0.269 ± 0.035	1.000 ± 0.000	0.943 ± 0.010	1.000 ± 0.000
	random-vs-random	0.152 ± 0.008	0.992 ± 0.000	0.739 ± 0.011	0.999 ± 0.000
ViT-B/16	clean-intraclass	0.422 ± 0.023	0.969 ± 0.008	0.789 ± 0.067	1.000 ± 0.000
	clean-interclass	0.435 ± 0.017	0.974 ± 0.005	0.882 ± 0.010	1.000 ± 0.000
	clean-vs-random	0.439 ± 0.014	0.963 ± 0.006	0.891 ± 0.007	1.000 ± 0.000
	random-vs-random	0.377 ± 0.013	0.867 ± 0.013	0.476 ± 0.018	0.997 ± 0.000

Tab. 8.1: Reference baselines for the input-noise sweep, averaged over R pairs drawn from the 565-image ImageNet-S test split (clean rows: $R = 200$ sampled pairs; clean-vs-random: $R = 565$ images paired with a fresh Gaussian each; random-vs-random: $R = 200$ Gaussian pairs); entries are mean $\pm$ standard deviation over pairs. **clean-intraclass**: $\mathrm{H}(\Pi_A(x_1), \Pi_A(x_2))$ with x_1, x_2 from the same ground-truth class; **clean-interclass**: same, with distinct classes; **clean-vs-random**: one side clean, other Gaussian; **random-vs-random**: both sides Gaussian.

8.4 Randomization Sanity Checks

This section collects the theoretical and empirical material behind the cascading-randomization experiments of Section 8.2. §8.4.1 carries the protocol of Adebayo et al. (2018) over to the trajectory-space Hellinger diagnostic of Section 8.3; §8.4.2 is the pixel-level heatmap-similarity sanity check on the main-paper attribution methods; §8.4.3 gives the fully random reference; §8.4.4 compares trajectory and map diagnostics per image.

8.4.1 Cascading Randomization in Trajectory Space

We apply the Hellinger framework of Section 8.3 to the parameter randomization test of Adebayo et al. (2018). Replacing one weight matrix yields two kernels on the same layered graph, and the Hellinger pass quantifies the resulting trajectory-space change. Notation follows Section 8.3: $\widetilde{T}^{(l)}$ is the geometric-mean kernel and $g^{(l)}(s)$ the overlap mass reaching state s, so $\mathrm{BC}^{(l)} = \sum_{s \in \mathscr{S}_+^{(l)}} g^{(l)}(s)$; $\mathrm{BC}^{(l)}_{\mathrm{ord}}$ denotes the same sum over ordinary states $s \in \mathscr{S}^{(l)}$ only.

Lemma 8.6 (Alive-set mismatch, `Lean_76`). *Suppose a dead state sends all its mass to the cemetery. Every state in the alive symmetric difference then has zero geometric-mean mass to ordinary successors: $\sum_{s' \in \mathscr{S}^{(l-1)}} \widetilde{T}^{(l)}(s_i, s') = 0$. If alive states have no cemetery leakage, as for the stated nondegenerate Stopping-Game kernel, its full geometric-mean row is also zero.*

Proof. If i is alive in A and dead in B, then $T_B^{(l)}(s_i, s') = 0$ for every ordinary successor, so each ordinary geometric mean is zero. The same argument applies with A and B exchanged. If the alive row also has zero cemetery mass, the cemetery geometric mean vanishes. With alive-state leakage this last conclusion does not follow; the ordinary-state conclusion still holds. □

Fix a ReLU MLP with hidden widths $n_1, \dots, n_L$, an input x, and start state s_0 at the target output. Let B replace $W^{(l^*)}$ by an i.i.d. matrix from a continuous distribution symmetric about zero, and assume that for each $l \geq l^*$ and every neuron i, $\mathbb{P}_B\big(i \in \text{alive}_B^{(l)} \mid \text{kernels above layer } l\big) \leq 1 - \delta$ for some $\delta \in (0, \frac{1}{2}]$. Then $T_A^{(l)} = T_B^{(l)}$ for $l < l^*$; at $l = l^*$ the row weights differ and the alive sets may differ; for $l > l^*$ the perturbation propagates through the activations. The same reduction applies to convolutional and transformer blocks interpreted as single *Layered Markov Process* (LMP) layers. Write $\eta := 1 - \delta \in \big[\frac{1}{2}, 1\big)$ and $N := L - l^* + 1$.

Theorem 8.7 (Randomization in the Stopping Game, `Lean_77`). $\mathbb{E}_B\big[\mathrm{BC}_{\mathrm{ord}}^{(0)}(\Pi_A, \Pi_B)\big] \leq \eta^N$. *Hence, for every $z > 0$, the conditioned-survival distance of Theorem 8.5 satisfies* $\mathbb{E}_B\big[\mathrm{H}_{\mathrm{surv}}^2\, \mathbf{1}\{Z_A Z_B \geq z\}\big] \geq \mathbb{P}_B(Z_A Z_B \geq z) - \eta^N/\sqrt{z}$, *while the plain distance is* $\mathrm{H}^2 = 1 - \mathrm{BC}_{\mathrm{ord}}^{(0)} - g^{(0)}(\dagger)$.

Proof. At a layer $l \geq l^*$, ordinary overlap mass at a state dead in both models moves into the common cemetery, mass on the alive symmetric difference vanishes (Lemma 8.6), and on states alive in both models the row mass is at most 1 by Cauchy–Schwarz. Hence $\mathrm{BC}_{\mathrm{ord}}^{(l-1)} \leq \sum_{s \in \mathscr{S}^{(l)}} g^{(l)}(s)\, \mathbf{1}\{s \in \text{alive}_B^{(l)}\}$, and, since $g^{(l)}$ is fixed by the kernels above layer l, the conditional bound gives $\mathbb{E}_B[\mathrm{BC}_{\mathrm{ord}}^{(l-1)}] \leq \eta\, \mathbb{E}_B[\mathrm{BC}_{\mathrm{ord}}^{(l)}]$. Iterating from $\mathrm{BC}_{\mathrm{ord}}^{(L)} = 1$ over $l = L, \dots, l^*$ and using that $\mathrm{BC}_{\mathrm{ord}}^{(l)}$ is non-increasing below l^* yields the bound. The $\mathrm{H}_{\mathrm{surv}}$ statement follows from $\mathrm{H}_{\mathrm{surv}}^2 = 1 - \mathrm{BC}_{\mathrm{ord}}^{(0)}/\sqrt{Z_A Z_B} \geq 1 - \mathrm{BC}_{\mathrm{ord}}^{(0)}/\sqrt{z}$ on $\{Z_A Z_B \geq z\}$ (Theorem 8.5), the plain one from $\mathrm{BC}^{(0)} = \mathrm{BC}_{\mathrm{ord}}^{(0)} + g^{(0)}(\dagger)$. □

Remark 8.2 (Cemetery floor). Theorem 8.7 bounds only the ordinary-state overlap. Overlap mass that reaches a neuron dead in both models enters the cemetery in both, $g^{(l-1)}(\dagger) = g^{(l)}(\dagger) + \sum_{s \in \text{dead}_A^{(l)} \cap \text{dead}_B^{(l)}} g^{(l)}(s)$, and is never thinned again, so plain H saturates below one. In a mean-field approximation with uniform dead fraction q under A and alive probability p under B, the ordinary overlap shrinks by $(1-q)p$ per layer while a fraction $q(1-p)$ is absorbed, so $g^{(0)}(\dagger) \to q(1-p)/(1-(1-q)p)$ as $N \to \infty$; for illustrative values $q = 0.2$, $p = \frac{1}{2}$ this gives $\mathrm{H} \approx 0.91$, the same order as the observed 0.94–0.97 (Table 8.3). The conditioned-survival distance removes the cemetery floor exactly, and $\mathbb{E}_B[\mathrm{H}_{\mathrm{surv}}^2 \mid Z_A Z_B \geq z] \to 1$ for every fixed $z > 0$ with $\mathbb{P}_B(Z_A Z_B \geq z)$ bounded away from zero. For the Routing Game the activation-weighted kernel concentrates $g^{(l)}$ on neurons with largest $|aW|$, which rarely coincide with $\text{dead}_A \cap \text{dead}_B$, so H is nearly identical to $\mathrm{H}_{\mathrm{surv}}$ in practice.

Theorem 8.8 (Randomization in the Routing Game, `Lean_78`). *Under the same setup without the alive-set assumption, take the RG kernel at temperature $\tau > 0$ and assume that for every $l \geq l^*$ each state alive in both models has row mass $\sum_{s'} \widetilde{T}^{(l)}(s, s') \leq \bar{b}$ for some $0 \leq \bar{b} < 1$. Then* $\mathrm{BC}_{\mathrm{ord}}^{(0)}(\Pi_A, \Pi_B) \leq \bar{b}^N$.

Proof. Since $a_{M,i}^{(l-1)} = 0$ for $i \notin \text{alive}_M^{(l-1)}$, the Gibbs policy (8.7) sends no mass to dead neurons, so for $1 \leq l < L$ the overlap $g^{(l)}$ vanishes outside $\text{alive}_A^{(l)} \cap \text{alive}_B^{(l)}$, and the start state is alive in both, in the symmetric difference or dead in both. In the latter two cases its ordinary overlap contribution is zero by the ordinary-state part of Lemma 8.6. Hence $\mathrm{BC}_{\mathrm{ord}}^{(l-1)} \leq \bar{b}\, \mathrm{BC}_{\mathrm{ord}}^{(l)}$ for every $l \geq l^*$, with no alive-set factor (the alive mismatch at layer $l-1$ is already part of the row masses at layer l); iterate as in Theorem 8.7. The assumption reflects the structure of the RG: each act-to-act row factors as the source-deadness indicator $\mathbf{1}\{z_j^{(l)} > 0\}$, the constant $\frac{1}{2}$ uniform κ-branch, and the Gibbs policy $\pi_{M,\kappa}^{(l)}(\cdot \mid j) \propto [a_M^{(l-1)}[\cdot]\, W_M^{(l,\kappa)}[\cdot, j]]^{1/\tau}$. At $l = l^*$ the two policies share $a^{(l^*-1)}$, while their weights may differ; above that layer their activation factors may also differ. These changes can reduce overlap, but a continuous randomization law alone does not imply distinct normalized rows almost surely. For

example, if each sign branch has just one positive-mass ordinary route, changing its weight without changing its sign leaves that branch's normalized policy unchanged. The strict conditional row-overlap bound $\bar{b} < 1$ is therefore an explicit hypothesis of this theorem; its applicability requires a separate argument for the architecture and randomization law, or an empirical assessment. The equality case of Cauchy–Schwarz explains strict contraction when the completed normalized rows actually differ. □

The discounted occupation measures at the input layer recover gradient and $\alpha\beta$-LRP maps (Theorems 4.8 and 4.10). The undiscounted endpoint marginal $m_M^{(0)}$ is generally different: Hellinger compares the trajectory routing laws and does not include the path discounts. Theorems 8.7–8.8 show that Π_M changes drastically under randomization, while Table 8.2 shows that the resulting attribution maps can remain similar. The two diagnostics therefore detect different aspects of parameter sensitivity: the map-level test of Adebayo et al. (2018) asks whether the explanation delivered to the user depends on the learned parameters, the trajectory-level test whether the calculation behind it does. A map can fail the first while its calculation passes the second; the trajectory-level result locates where the sensitivity is lost (between the calculation and its pixel projection), but it does not make such a map adequate for model debugging (Adebayo et al., 2018; Sixt et al., 2020).

8.4.2 Heatmap similarity at full randomization

We apply the cascading parameter randomization test of Adebayo et al. (2018) to the attribution methods of Table 8.2. Starting from a pretrained network, we cumulatively re-initialize the convolutional and linear layers from the output toward the input (weights Kaiming normal (He et al., 2015), biases $\mathcal{N}(0, 0.01^2)$); BatchNorm (ResNet-50) and LayerNorm, the packed query/key/value projection of the attention, the positional embedding and the class token (ViT-B/16) keep their pretrained values, here and in the Hellinger cascade of Section 8.2.1. At each randomization depth we compute the attribution for every test image under both the original and the (partially) randomized model, then measure the similarity between the two attribution maps. A method that genuinely reflects the learned parameters should produce increasingly dissimilar maps as more layers are randomized; a method that merely responds to low-level input structure (edges, textures) will show high similarity even at full randomization.

Metrics. We report two complementary similarity measures between attribution maps:

- **Spearman** $\rho(|a|)$: The Spearman rank correlation of the absolute attribution values, measuring whether the two maps rank pixels in the same order of importance. We use absolute values because sign conventions may differ between original and randomized attributions; the correlation itself is signed.
- **HOG** r: The Pearson correlation between Histograms of Oriented Gradients (HOG) descriptors (Dalal and Triggs, 2005) extracted from the attribution maps, capturing structural similarity at the level of local gradient orientations.

We intentionally do *not* report SSIM here: Binder et al. (2023) show that randomization-based similarity scores can remain deceptively high because input-dependent structure and activation scales survive top-down randomization. All values are averaged over the 565-image test split of ImageNet-S (50 classes); a map that is constant over the image (LIME and KernelSHAP on the fully randomized ResNet-50, whose output no longer depends on the superpixels) has similarity 0 by convention. Table 8.2 reports the similarity at *full randomization* (all convolutional and linear layers re-initialized).

Discussion. Several patterns emerge. First, Gradient and Grad-CAM stay closest to zero on the CNNs (and LayerCAM on VGG-16), and Gradient, SmoothGrad and GAE on ViT-B/16: they are sensitive to the parameters, though this does not imply that they are faithful. Second, LRP-ε keeps the most rank structure on VGG-16 (0.341); on ResNet-50 the LRP rules (LRP-γ 0.254, LRP-ε 0.250) are close to Integrated Gradients and LayerCAM, which keep the most there. Third, LIME and KernelSHAP keep a HOG correlation of about 0.86 on VGG-16 and ViT-B/16 at low Spearman correlation: their superpixels are computed from the input image, so the maps inherit its edge structure; on ResNet-50 the fully randomized model's output no longer depends on the superpixels and their maps are constant (‡). Fourth, the RG rows lie between Gradient and LRP-ε on VGG-16, slightly below the LRP rules on ResNet-50, and keep the highest Spearman correlation of all methods on ViT-B/16 (0.454 for the anchor, 0.450 for the selected mode). Two maps become anti-correlated rather than unrelated (Grad-CAM++ on ResNet-50, TIBAV on ViT-B/16). The complementary trajectory-space analysis of §8.4.1 measures divergence of the underlying trajectory distributions rather than pixel-level map similarity.

Tab. 8.2: Parameter randomization sanity check (Adebayo et al., 2018). Each cell reports the average similarity between the attribution of the original pretrained model and the attribution of a fully randomized copy (every convolutional and linear layer re-initialized; normalization layers, the attention's query/key/value projection and the ViT embeddings stay pretrained). Values near zero indicate that the method detects the parameter change; the Spearman correlation is signed, so negative values occur. Bold marks the value closest to zero per metric within each architecture, among maps that are not constant; ‡: the map is constant at full randomization (similarity 0 by convention). Rows: the reported configuration of every method (Section 7.5).

Method	Spear. $\rho(\lvert a\rvert)$	HOG r
VGG-16		
Gradient	0.013	**0.009**
SmGrad	0.084	0.101
IntGrad	0.207	0.348
DeepLIFT	0.216	0.340
LRP-ε	0.341	0.457
LRP-γ	0.092	0.370
GCAM	**-0.007**	0.069
GCAM++	0.180	0.129
LCAM	-0.011	0.052
LIME	0.257	0.864
KernelSHAP	0.075	0.868
SG (λ=-1,σ^2=5, gate on)	0.289	0.456
RG anchor	0.299	0.535
RG (τ=0.3,ε=0.1)	0.284	0.287
ResNet-50		
Gradient	0.148	0.134
SmGrad	0.233	0.214
IntGrad	0.277	0.424
DeepLIFT	0.224	0.351
LRP-ε	0.250	0.393
LRP-γ	0.254	0.130
GCAM	**0.020**	**0.072**
GCAM++	-0.502	0.151
LCAM	0.266	0.145
LIME‡	0.000	0.000
KernelSHAP‡	0.000	0.000
RG anchor	0.217	0.344
RG (τ=0.9,ε=1)	0.180	0.316
ViT-B/16		
Gradient	**-0.003**	0.041
SmGrad	0.010	**0.038**
IntGrad	0.272	0.362
LRP-ε	0.165	0.300
LRP-γ	0.027	0.234
AttnLRP	0.124	0.176
DAVE	0.178	0.323
TIBAV	-0.175	0.150
GAE	-0.009	0.130
AR	0.120	0.153
LIME	0.248	0.861
KernelSHAP	0.131	0.867
RG anchor	0.454	0.469
RG (τ=0.4,λ_{sm}=-10,σ^2=5,ε=0.1)	0.450	0.310

Spear. $\rho(\lvert a\rvert)$ = Spearman rank correlation of absolute attribution values; HOG r = Pearson correlation of HOG descriptors. RG anchor = the $\alpha\beta$-LRP representation (τ=1, ε=0.5); the second RG row and the SG row are the localization-selected configurations and list the modes that differ (τ = temperature; λ, λ_{sm} = risk aversion; σ^2 = ADF input variance; ε = LRP stabilizer; $\tilde{\Phi}$ = noisy-observation gate).

8.4.3 Double-Random Empirical Baseline

To calibrate the asymptotic value that Theorems 8.7–8.8 approach under *full* randomization, we compare two *independently random-initialized* copies of each architecture. This endpoint matches the deepest step of the cascading test of Adebayo et al. (2018) while eliminating any residual correlation with the pretrained weights.

Protocol. For each architecture we run $R = 500$ independent rounds. Each round: *(i)* instantiate two copies A_r, B_r of the pretrained checkpoint and re-initialize every `Conv2d`/`Linear` layer of each copy independently as in the cascade, matching its endpoint; the remaining parameters (normalization layers, the attention's packed query/key/value projection, positional embedding and class token) keep their pretrained values in both copies (starting from the checkpoint rather than from `weights=None`, whose ViT classification head is identically zero and would make the two trajectory laws trivially indistinguishable; the same construction is used for all three architectures). *(ii)* Draw one image index with replacement from the 565-image ImageNet-S test split (the 500 draws hit 336 distinct images under the fixed seed we use). *(iii)* Compute the plain Hellinger $\mathrm{H}(\Pi_{A_r}, \Pi_{B_r})$ and conditioned-survival $\mathrm{H}_{\mathrm{surv}}$ for both the Stopping Game and Routing Game kernels using the ground-truth label as target class. Table 8.3 reports the round-to-round mean and standard deviation; the round-to-round variance mixes two sources (image choice and the random-pair realization).

	Stopping Game		Routing Game	
Architecture	H	$\mathrm{H}_{\mathrm{surv}}$	H	$\mathrm{H}_{\mathrm{surv}}$
VGG-16	0.939 ± 0.003	1.000 ± 0.000	1.000 ± 0.000	1.000 ± 0.000
ResNet-50	0.938 ± 0.003	1.000 ± 0.000	0.973 ± 0.002	1.000 ± 0.000
ViT-B/16	0.958 ± 0.002	1.000 ± 0.000	0.956 ± 0.001	1.000 ± 0.000

Tab. 8.3: Hellinger trajectory distance between two independently random-initialized copies of the same architecture, averaged over $R = 500$ rounds (one fresh ImageNet-S test image per round). Plain H concentrates at an architecture-dependent cemetery floor of 0.94–0.97; conditioned-survival $\mathrm{H}_{\mathrm{surv}}$ saturates at unity. The VGG-16 Routing Game cell is the one case in which the plain distance also saturates at 1.000.

Interpretation. Plain H concentrates at a model-dependent floor of 0.94–0.97, matching the cemetery-floor prediction of Remark 8.2: once both random kernels produce uncorrelated dead masks, the shared dead-in-both coincidence rate bounds $\mathrm{BC}^{(0)}$ below by a strictly positive constant and plain H cannot reach unity. The conditioned-survival distance removes the cemetery floor by renormalizing on survival and correctly reports the two independent random trajectory laws as essentially support-disjoint on ordinary input states ($\mathrm{H}_{\mathrm{surv}} \approx 1$ throughout). The pretrained-vs-randomized Hellinger distance that the cascading sanity check reports should therefore be read against this architecture-specific asymptote rather than against unity: values close to the double-random baseline already indicate that the game-theoretic trajectory law has become fully decoupled from the pretrained one.

8.4.4 Per-Image Trajectory-vs-Map Independence

Theorems 8.7–8.8 and the double-random baseline of Table 8.3 bound the *aggregate* behavior of the Hellinger trajectory distance and the pixel-level attribution-map similarity. A sharper version of the argument of §8.4.1 is the claim that the two diagnostics disagree *per image*: at fixed randomization

depth, whether a given image's trajectory distribution is strongly displaced by the randomization need not predict whether its attribution map becomes more dissimilar. Table 8.4 tests this directly.

Protocol. For each architecture we pick a cascading step at which the Hellinger run and the attribution-sanity run have randomized the same part of the network, so per-image values are directly comparable: the layers down to `features.26` on VGG-16 and down to `layer4.1.conv3` on ResNet-50 (identical sets of re-initialized layers), and the last encoder block on ViT-B/16, where the Hellinger cascade re-initializes the MLP and the output projection of the block and the attribution cascade additionally its Q, K and V projections. The mean H / $\mathrm{H}_{\mathrm{surv}}$ at these steps can be read off Figure 8.1. For each of the 565 ImageNet-S test images we then record four Hellinger scalars (H plain/surv for the Stopping and Routing Games) and four attribution-map scalars (Spearman $\rho(|a|)$ and HOG, each computed between the pretrained and partially-randomized model, for Gradient and for the $\alpha\beta$-RG anchor at τ=1; these are the attribution methods whose marginal laws line up with the Hellinger game kernels). Table 8.4 reports Spearman rank correlations across the 565 images.

		Within Hellinger				Gradient		RG anchor	
Architecture	Hellinger variant	$\mathrm{H}_{\mathrm{RG,pl}}$	$\mathrm{H}_{\mathrm{RG,surv}}$	$\mathrm{H}_{\mathrm{SG,pl}}$	$\mathrm{H}_{\mathrm{SG,surv}}$	Spear. $\rho(\lvert a\rvert)$	HOG r	Spear. $\rho(\lvert a\rvert)$	HOG r
VGG-16	$\mathrm{H}_{\mathrm{RG,plain}}$	1.00	+1.00	−0.33	+0.85	+0.07	+0.10	−0.01	+0.04
	$\mathrm{H}_{\mathrm{RG,surv}}$	+1.00	1.00	−0.33	+0.85	+0.07	+0.10	−0.01	+0.04
	$\mathrm{H}_{\mathrm{SG,plain}}$	−0.33	−0.33	1.00	−0.30	+0.12	+0.11	+0.00	−0.02
	$\mathrm{H}_{\mathrm{SG,surv}}$	+0.85	+0.85	−0.30	1.00	+0.01	+0.05	−0.10	−0.10
ResNet-50	$\mathrm{H}_{\mathrm{RG,plain}}$	1.00	+0.58	−0.08	+0.70	+0.05	−0.01	−0.08	−0.19
	$\mathrm{H}_{\mathrm{RG,surv}}$	+0.58	1.00	+0.16	+0.89	+0.15	+0.08	+0.05	−0.05
	$\mathrm{H}_{\mathrm{SG,plain}}$	−0.08	+0.16	1.00	+0.16	−0.04	+0.07	−0.01	+0.02
	$\mathrm{H}_{\mathrm{SG,surv}}$	+0.70	+0.89	+0.16	1.00	+0.16	+0.07	−0.01	−0.10
ViT-B/16	$\mathrm{H}_{\mathrm{RG,plain}}$	1.00	+0.96	+0.29	+0.30	+0.02	+0.00	−0.04	−0.08
	$\mathrm{H}_{\mathrm{RG,surv}}$	+0.96	1.00	+0.25	+0.28	+0.02	+0.02	−0.07	−0.07
	$\mathrm{H}_{\mathrm{SG,plain}}$	+0.29	+0.25	1.00	+0.99	−0.12	−0.06	−0.03	−0.05
	$\mathrm{H}_{\mathrm{SG,surv}}$	+0.30	+0.28	+0.99	1.00	−0.12	−0.06	−0.04	−0.05

Tab. 8.4: Per-image Spearman rank correlations across the 565-image ImageNet-S test split at the aligned cascading steps of the Hellinger and attribution-sanity pipelines. Left block: within-Hellinger cross-correlations of the plain ($\mathrm{H}_{\cdot,\mathrm{pl}}$) and survival-conditioned ($\mathrm{H}_{\cdot,\mathrm{surv}}$) distances under the RG and SG kernels (self-entries shown as −; trivially +1). Right block: Hellinger-vs-map cross-correlations. The Hellinger-vs-map block is uniformly $|\rho| \leq 0.19$: per-image Hellinger and per-image attribution-map similarity rank images essentially independently.

Readings. The Hellinger-vs-map cross-block in Table 8.4 is uniformly near zero ($|\rho| \leq 0.19$ on all three architectures): per-image trajectory displacement and per-image attribution-map drift rank images essentially independently.

The within-Hellinger block shows that per-image H carries a stable ordering of the images. Where little trajectory mass reaches the cemetery, the plain and conditioned distances of the same game rank images almost identically: $\rho = +0.96$ (RG) and +0.99 (SG) on ViT-B/16, and identical values for the RG on VGG-16, whose kernel routes no trajectories to states of dead neurons. Under the SG on the ReLU backbones the two normalizations decouple ($\rho = -0.30$ on VGG-16, +0.16 on ResNet-50): the cemetery share of plain H varies with the image's dead masks, and the renormalization by $1/\sqrt{Z_A Z_B}$ removes exactly this share. The conditioned distances of the two games agree strongly on the ReLU backbones ($\rho = +0.85$ on VGG-16, +0.89 on ResNet-50) and only weakly on ViT-B/16 (+0.28), where the two kernels read partly different per-image structure.

A secondary observation from the within-method pattern: Spearman $\rho(|a|)$ and HOG within a single attribution method agree well ($\rho = 0.65$–0.82, computed from the same per-image joins), but Gradient and $\alpha\beta$-RG similarities barely agree across images (ρ between -0.10 and $+0.30$ depending on architecture and similarity metric). Together with the near-zero Hellinger-vs-map block, this tightens the conclusion of §8.4.1: trajectory-space and pixel-level diagnostics look at orthogonal per-image structure, and within pixel-level diagnostics the choice of attribution method changes *which* images a randomization sanity check flags.

9 Conclusion and Future Work

Section 9.1 retraces the preceding chapters; Section 9.2 records the limitations of the theory and evidence; and Section 9.4 collects the directions this work opens.

9.1 Summary of Contributions

The starting point was the growing role of interpretability in improving and controlling models. When attribution guides a correction or an intervention, both the internal representation it reads and the backward calculation it performs matter. This thesis connects those two choices through *network decomposition*: expose structure a trained ReLU network already contains, preserve its predictions, and study how attribution propagates through that structure.

The common mathematical foundation is the decomposition theory of Chapter 3. A *valid decomposition* replaces each layer map by a pair of coordinated split-streams whose difference reproduces the network exactly, and the chapter populated the definition with instances of increasing structure. The mixing family f^{η}_{mix} interpolates between trivial and fully separated splits; its endpoint $\eta = 1$ recovers the classical DC decomposition of a ReLU network into a difference $f = g - h$ of two convex, monotone split-streams, routing the weight parts $W^{(l,+)}, W^{(l,-)}$ into separate channels (Zimmermann and Loho, 2026). Beyond the piecewise-linear case, Theorem 2.15 established a DC–monotone splitting for every activation with integrable curvature — GELU (Hendrycks and Gimpel, 2016), Swish (Ramachandran et al., 2017), Mish, tanh, sigmoid, softplus — with a canonicity statement singling out the split of minimal curvature, and a bounded-variation extension recovering ReLU as the Dirac limit. The stopping decomposition f_{stop} added an early-termination channel, and the regularized family f^{Ω}_{reg} showed that a convex regularizer on the stop/continue decision generates smooth decompositions systematically, the softplus decomposition f^{θ}_{soft} arising as the binary-entropy subcase — placing the softplus replacement of Dombrowski et al. (2022) inside the framework rather than beside it. The chapter closed with the stabilization that makes split passes usable at depth and a full CNN extension: convolutions, gated pooling, batch normalization, and residual connections.

The game semantics of Chapter 4 gives the stopping instance a second, complementary reading. Building on the ReLU-network games of Gaubert & Vlassopoulos (2025), the stopping decomposition was reinterpreted as a two-player zero-sum SG on a doubled state graph: weight signs route between the players P^{+} and P^{-}, gradients and $\alpha\beta$-LRP attributions emerge as discounted occupation measures under equilibrium policies, and the deformation parameters of Chapter 3 — temperature, risk aversion, stabilizers — become principled modifications of the game. Chapter 8 takes the decisive step: the equilibrium policies induce a genuine *probability distribution* Π over complete backward trajectories, and this distribution — not any pixel-space projection — is the backward computation. The comparison with the NRM framework of Xiong et al. (2026) in Section 8.1 identifies their shared path perspective: NRM assigns normalized signed relevance to neuron walks, while the game separates a probability law on expanded states from the signs and discounts used to recover attribution. A direct identification requires matching the propagation and normalization conventions. The probability law makes trajectory-level statistical comparison possible: the Hellinger distance between two backward computations is computable in one linear-time pass, and the parameter-randomization sanity check of Adebayo et al. (2018) lifts from attribution maps to trajectory space, where Theorem 8.7 shows Bhattacharyya mass decays geometrically in the number of randomized layers — detecting collapse even when heatmaps remain deceptively similar (Sixt et al., 2020).

These representations lead to the attribution methods of Chapter 6. SplitCAM, SplitLRP, and SplitGrad apply established attribution principles to the decomposed streams, separating factual (positive) from counterfactual (negative) evidence at every layer; the difference-of-convex input neural networks (DCINNs) of Section 6.1.4 added a self-interpretable counterpart trained directly in split form. The tropical foundations (Section 2.4) supply the geometric picture: through the correspondence between ReLU networks and tropical rational maps (Zhang et al., 2018b), the split-streams g and h acquire Newton polytopes, and the *imaginative polytope* of Section 2.4.3 interprets what the decomposition adds to the geometry. Chapter 5 developed how gated splits of matrix and Hadamard products extend the framework to multiplicative architectures (Vaswani et al., 2017), attention entering the game view as an oracle routing policy realized on ViT-B/16 (Dosovitskiy et al., 2021).

The shared evaluation questions in Chapter 7 concern localization, faithfulness, robustness, and the cost of obtaining these signals. The split methods were evaluated on VGG-16 and ResNet-18 on ImageNet-S; the revised game study covers VGG-16, ResNet-50, and ViT-B/16 on ImageNet-S, with a ViT-B/16 transfer study on Pascal VOC. No single method dominates every metric. On ViT-B/16, one adapted $\alpha\beta$-LRP configuration outperforms all evaluated baselines in attribution localization, and this lead transfers to Pascal VOC without re-tuning. The LRP baselines remain stronger on faithfulness, while the anchor configurations are particularly robust. Independent validation selections for localization and faithfulness, bootstrap confidence intervals, and Holm-corrected paired tests make these trade-offs explicit. Hellinger diagnostics complement pixel-level comparisons: they distinguish trained from randomized calculations even where heatmaps remain similar, increase with Gaussian input noise, and separate intra- from inter-class pairs for at least one game on each tested architecture.

Taken together, the thesis substantiates its central claim: monotone, convex, game-theoretic, and polyhedral structure need not be trained into a network — it is hidden in every trained ReLU network, and a single decomposition framework makes all four readings available at once.

9.2 Limitations

The results above come with qualifications that should be stated plainly.

Computational overhead. A split pass propagates two coordinated streams per channel and, currently, materializes the split network with duplicated weight matrices. Memory and compute grow by a small constant factor — harmless for the CNNs here, but an obstacle for very large models. The overhead is an implementation artifact, not a mathematical necessity: the split can be realized through masked views of the original weights, though this thesis has not delivered such an implementation.

Stabilization hyperparameters. The stabilization of Section 3.6 is essential at depth and introduces choices — thresholds, shifting schemes, and in the game view the temperatures τ, θ and the risk-aversion parameter λ — that the theory constrains but does not determine. The revised experiments use declared validation grids and independent localization and faithfulness selections. Their differing winners show that there is no universally optimal setting; transfer to Pascal VOC is demonstrated for configurations fixed on ImageNet-S. The same holds for layer selection in SplitCAM and SplitLRP: different layers excel at different metrics.

Rigor of the transformer extension. Chapter 5 was deliberately framed as outlook. The attention representation is conditional on the realized attention pattern, LayerNorm statistics, and GELU gates. It describes relevance routing through Value paths with a detached QK-softmax reference; it does not recover the full unconditioned derivative through Q and K. These choices give a representation of a specified backward rule and do not establish that the architecture determines a unique explanatory game. The ViT-B/16 results are therefore evidence for the *usefulness* of the extension, not its uniqueness or optimality.

Benchmark scope. The empirical evidence is specific. All numbers are inherited verbatim from the two underlying papers, so architectures (VGG-16, ResNet-18, ResNet-50, ViT-B/16), datasets (ImageNet-S, Pascal VOC), and metric selections are those of the original studies; nothing was re-run. Localization against human masks is only one axis, and sharper attributions can discard network-relevant evidence outside the annotated object. All experiments concern vision models; whether the gains transfer to other modalities or into downstream task gains is untested. Finally, the trajectory-level diagnostics were validated on sanity checks and controlled perturbations — dedicated calculation-level benchmarks do not yet exist.

9.3 Broader Impact

The work is methodological: it studies attribution methods for existing image classifiers and does not introduce a deployed decision system, a new dataset of people, or a high-risk generative model. Its intended positive impact is to make interpretability methods more diagnostically accountable. In particular, trajectory-level comparisons can reveal cases where visually similar attribution maps arise from different backward calculations, and can therefore support more careful auditing of interpretability pipelines used in scientific or engineering workflows.

The same capability can also have negative or misleading uses. Sharper or more stable attribution maps may create unwarranted confidence in a model if users treat attributions as causal guarantees rather than as post hoc diagnostics. In safety-critical or fairness-sensitive settings, improved-looking attributions could be used to justify a classifier whose predictions remain brittle, biased, or dataset-dependent. The Hellinger trajectory distance also compares internal calculation paths rather than social validity; small trajectory distance does not imply that the underlying model is fair, robust, or appropriate for deployment.

We therefore view the framework as an analysis tool rather than a deployment certificate. The experiments use public benchmark classifiers and ImageNet-derived evaluation data, and the results should be interpreted as evidence about attribution behavior under these benchmarks, not as evidence that the underlying models are suitable for consequential use.

9.4 Future Research Directions

Four directions appear most promising.

Concept queries on the expanded state graph. Section 8.1 identifies a possible connection with the NRM framework (Xiong et al., 2026). Its queries over sets of neurons suggest corresponding events for game trajectories. Their probabilities and, separately, their signed discounted occupation measures could support concept-level analysis across layers. Establishing a correspondence with a chosen NRM or CRP rule requires matching the propagation factors, normalization, and absorption conventions; discounts do not replace probability normalization. For positive-probability concept events, the conditioned game trajectories could then be compared using the Hellinger machinery. This remains a proposed extension and could supply calculation-level benchmarks of the kind discussed in Section 9.2.

Regularized decompositions for smooth activations. Theorem 2.15 guarantees canonical DC–monotone splits for GELU, Swish, and their relatives, and the regularized family $f^{\Omega}_{\mathrm{reg}}$ of Section 3.5 explains softplus as an entropy-regularized gate — but the two are not yet joined. A direct identification is impossible for GELU and Swish: a conjugate activation $\Omega^*(z) = \sup_{\pi\in[0,1]}\{\pi z - \Omega(\pi)\}$ is always convex, non-decreasing, and 1-Lipschitz, whereas these activations do not have those global properties. The open question is how to combine their exact DC–monotone decompositions with games on the non-negative streams, or to formulate local game interpretations conditional on the realized activation pattern. Such a construction would need its own recovery and backward-occupation theorems before trajectory diagnostics could be transferred to it.

DCINNs at scale. The MNIST experiments of Section 6.1.4 showed that networks trained directly in split form separate factual from counterfactual attributes with striking clarity, but the evidence stops at small models. Scaling DCINNs raises optimization and theory questions: whether the difference-of-monotone parametrization trains stably at ImageNet scale, whether DC-programming techniques (Thi and Dinh, 2018) or ideas from ICNNs (Amos et al., 2017) accelerate training, and how the accuracy–interpretability trade-off behaves with capacity. A positive answer would upgrade the framework from a post hoc lens into a design principle for self-interpretable models.

Transformers, LLMs, and efficient split passes. Three engineering-flavored directions complete the picture. Extending the multiplicative splits of Chapter 5 from encoder-style ViTs to autoregressive LLMs requires handling causal attention and the key–value cache inside the split pass, making trajectory-level diagnostics available where explanation quality is hardest to assess. Implementing split passes through masked views rather than materialized duplicates would remove the memory overhead of Section 9.2, and the nonnegativity of the split weights invites hardware-level exploitation. The observed layer dependence of attribution quality calls for principled layer selection or multi-layer aggregation, for which the trajectory distribution offers the right vocabulary: layers are intermediate marginals of one measure Π.

The unifying lesson is that a trained network is a richer object than the function it computes. One set of weights carries a difference of convex monotone parts, a zero-sum game between cooperating adversaries, a probability distribution over backward trajectories, and a pair of Newton polytopes — none of which requires changing the network. Interpretability, on this view, provides structured access to computations that can be inspected, compared, and modified. The decomposition and game perspectives supply a mathematical basis for that access; evaluating how these tools improve a model through downstream correction, training, or steering is the next step.

Appendix A

Extended Experimental Results

This appendix collects extended results for both experimental campaigns. The split-method sections retain the hidden-monotonicity evaluation; the game-method sections use the revised AISTATS protocol, confidence intervals, validation selections, ablations and cross-dataset results.

A.1 Full Evaluation Results for the Split Methods

This section provides the comprehensive evaluation results for all interpretability methods tested in the split-method setup of Section 7.1.

A.1.1 VGG-16 Qualitative Saliency Map Comparison

Complementing Figure 7.1, Figure A.1 gives more saliency examples. Again, SplitCAM in layer 26 focuses more on the whole classified objects compared to LayerCAM in the same layer. SplitGrad in layer 0 detects too many edges of the original image to be a useful explanation of the model's decision; especially in images with many background details, e.g. the rubble in the wood rabbit image, it fails to focus on the important image parts. This aligns with the observed performance peaks of SplitGrad and SplitCAM in deeper layers.

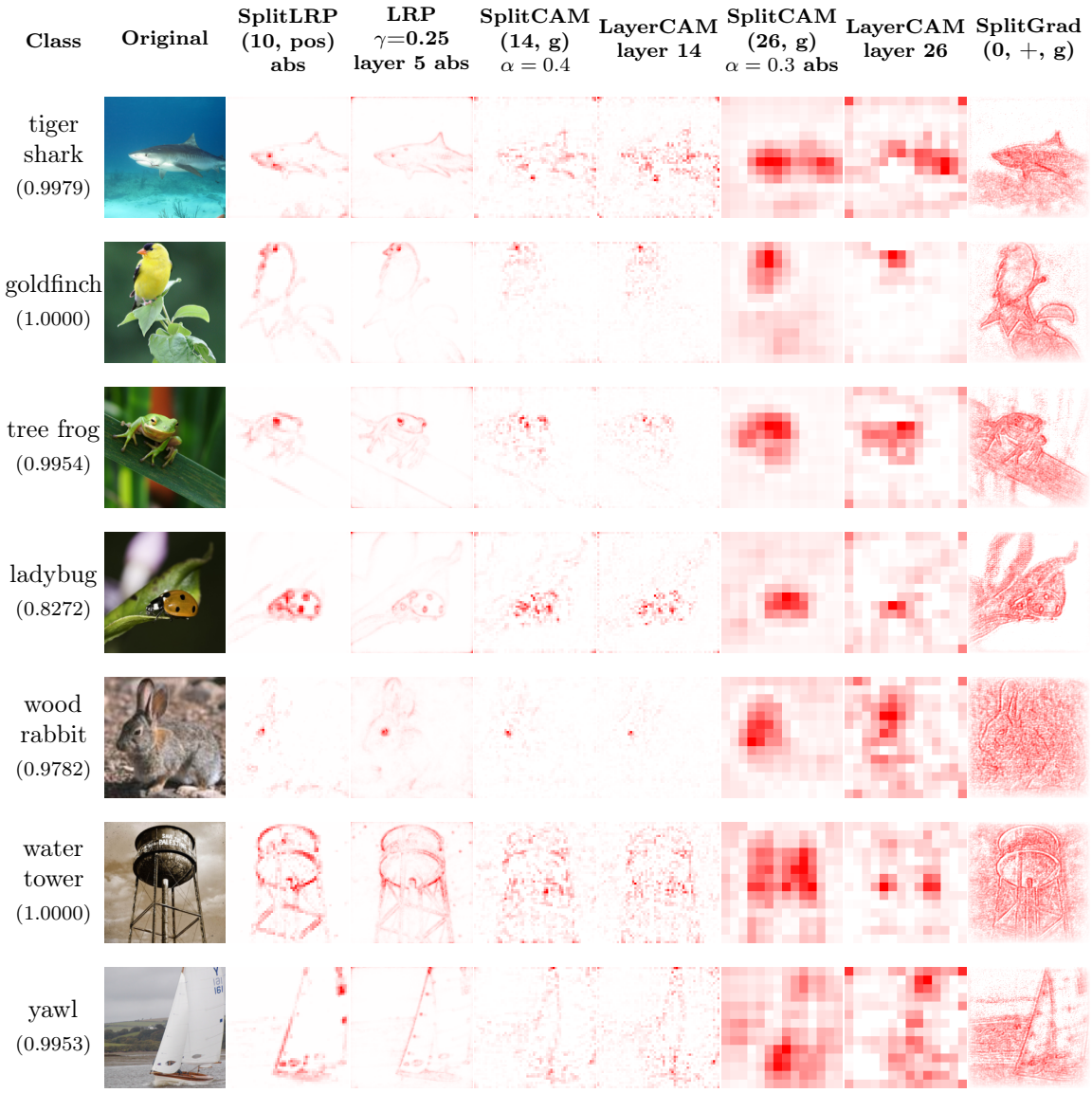


Fig. A.1: Saliency map comparison for VGG-16 on images from the custom ImageNet-S test subset. Each row shows the predicted class (with confidence), original image, and saliency maps from different methods. Methods include SplitLRP, classical LRP, SplitCAM variants, LayerCAM, and SplitGrad.

A.1.2 Quantitative Analysis on VGG-16

Table A.1 presents the detailed quantitative metrics across all split configurations, baselines, and classical methods for VGG-16. There is a consistent boost in performance for SplitCAM with $\alpha = 0.4$ compared to $\alpha = 0.5$ and compared to classical LayerCAM, respectively, indicating that the LayerCAM formulas indeed yield more powerful explanations on the split-pair than on the original model; this is even more pronounced in the ablation studies of Section A.2.

Tab. A.1: Comprehensive interpretability evaluation results (VGG-16; representative configurations, best per method). Abbreviations as in Table 7.1. For each method configuration the best metric values across the considered layer configurations are highlighted in bold with dark blue shading. PF^{drop} is score-drop AUC; n/r denotes a result not reported in the paper, and n/a an inapplicable entry.

Method	Layer	Abs	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
SplitCAM Methods								
SplitCAM (sc, α 0.4)	(12, g)	n	4.720	**0.651**	**0.912**	0.213	1.794	**0.389**
SplitCAM (sc, α 0.4)	(14, g)	n	**2.727**	0.563	0.836	**0.300**	**2.404**	1.012
SplitCAM (sc, α 0.4, wta)	(14, g)	n	**2.761**	0.543	0.827	**0.303**	**2.414**	1.056
SplitCAM (sc, α 0.4, wta)	(26, +, g)	y	4.711	**0.635**	**0.938**	0.165	1.590	**0.456**
SplitCAM (sc, α 0.5, wta)	(14, g)	n	**2.766**	0.536	0.823	**0.303**	**2.416**	1.056
SplitCAM (sc, α 0.5, wta)	(26, +, g)	n	4.608	**0.607**	**0.903**	0.185	1.684	**0.566**
SplitGrad Methods								
SplitGrad (α 0.4)	(12, +, g)	n	5.633	**0.613**	**0.873**	0.148	1.461	**0.364**
SplitGrad (α 0.4)	(14, +, g)	y	**3.429**	0.533	0.820	**0.235**	**2.119**	0.851
SplitGrad (α 0.4, wta)	(12, +, g)	n	4.735	**0.635**	**0.883**	0.186	1.756	0.651
SplitGrad (α 0.4, wta)	(14, +, g)	y	**3.287**	0.540	0.818	**0.249**	**2.186**	0.857
SplitGrad (α 0.5, wta)	(12, +, g)	y	**3.379**	**0.551**	0.859	**0.261**	**2.220**	0.869
SplitGrad (α 0.5, wta)	(26, +, g)	y	4.948	0.543	**0.899**	0.162	1.527	**0.617**
SplitLRP Methods								
SplitLRP (sc)	(7, pos)	n	**3.062**	0.092	**0.857**	**0.296**	**2.290**	0.865
SplitLRP (sc)	(19, pos)	n	4.393	**0.600**	0.823	0.166	1.656	**0.291**
SplitLRP (sc, wta)	(7, pos)	n	**3.235**	-0.036	0.836	**0.296**	**2.291**	0.866
SplitLRP (sc, wta)	(26, pos)	y	5.168	**0.588**	**0.871**	0.151	1.473	**0.282**
Classical Methods (No Layer Optimization)								
Deconvolution	0	y	5.435	0.468	0.629	0.204	1.715	0.956
DeepLIFT	0	y	4.447	0.551	0.726	0.281	1.945	1.053
Feature Ablation	0	y	6.029	0.479	0.624	0.123	1.250	0.859
GradientSHAP	0	y	4.808	0.552	0.783	0.261	1.862	1.142
IntGrad	0	y	4.838	0.554	0.797	0.262	1.851	1.069
Occlusion	0	y	5.214	0.499	0.716	0.130	1.217	0.699
GCAM++	30	n	5.955	0.564	0.827	0.130	1.245	0.401
Classical Methods (Layer Optimization)								
Guided Backprop	2	n	**3.276**	**0.632**	0.883	**0.281**	**2.302**	0.571
Guided Backprop	5	n	3.448	0.614	**0.887**	0.240	2.091	**0.375**
Guided GCAM	0	n	**4.301**	**-0.053**	**0.857**	0.161	**1.459**	1.047
Guided GCAM	26	n	5.130	-2.635	0.843	**0.179**	1.415	**0.947**
GCAM	0	n	6.006	**0.356**	**0.638**	0.060	0.899	1.123
GCAM	14	n	**5.555**	0.306	0.580	**0.136**	**1.525**	**0.903**
LCAM	14	n/a	2.792	0.576	0.798	**0.317**	**2.474**	**2.656**
LCAM	2	n/a	**3.128**	**0.580**	**0.855**	0.300	2.191	5.508
LRP $\gamma = 0.25$	0	n	**3.189**	**0.615**	**0.680**	0.218	1.918	0.519
LRP $\gamma = 0.25$	14	n	3.593	0.577	0.611	**0.279**	**2.255**	**0.491**
Baseline Methods								
Baseline: Binary Mask	0	y	6.011	1.000	1.000	0.091	1.054	n/r
Baseline: Original Image	0	y	8.770	-1.699	0.572	0.061	0.619	n/r
Random Baseline	0	y	3.570	0.425	0.458	0.203	1.585	n/r

A.1.3 Quantitative Analysis on ResNet-18

Table A.2 presents the detailed quantitative metrics across all split configurations, baselines, and classical methods for ResNet-18.

Tab. A.2: Comprehensive interpretability evaluation results (ResNet-18; representative configurations, best per method). Abbreviations as in Table 7.2. For each method configuration the best metric values across the considered layer configurations are highlighted in bold with dark blue shading. PF^{drop} is score-drop AUC; n/r denotes a result not reported in the paper, and n/a an inapplicable entry.

Method	Layer	Abs	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
SplitCAM Methods								
SplitCAM (sc, α 0.3, wta)	(1.0.2, g)	n	**3.503**	0.487	0.747	**0.168**	**1.395**	1.277
SplitCAM (sc, α 0.3, wta)	(4.1.2, g)	n	5.654	**0.673**	**0.972**	0.081	0.880	**0.233**
SplitCAM (sc, α 0.5, wta)	(2.0.1, g)	n	**3.682**	-1.370	0.887	**0.158**	**1.447**	1.078
SplitCAM (sc, α 0.5, wta)	(4.1.1, +, g)	y	6.526	**0.611**	**0.956**	0.066	0.737	**0.187**
SplitGrad Methods								
SplitGrad (α 0.3, wta)	(1.0.2, g)	y	**4.449**	0.513	0.777	**0.149**	**1.266**	0.813
SplitGrad (α 0.3, wta)	(4.1.2, g)	y	5.663	**0.649**	**0.972**	0.082	0.882	**0.219**
SplitGrad (α 0.5, wta)	(3.1.1, g)	y	**3.588**	0.559	0.880	**0.130**	**1.366**	0.859
SplitGrad (α 0.5, wta)	(4.1.2, g)	y	5.690	**0.654**	**0.972**	0.083	0.886	**0.230**
SplitLRP Methods								
SplitLRP (sc, wta)	(conv1, pos)	y	**6.313**	**0.506**	**0.587**	**0.056**	**0.697**	0.755
SplitLRP (sc, wta)	(1.0.2, pos)	y	6.785	0.483	0.567	0.050	0.686	**0.696**
Classical Methods (No Layer Optimization)								
Deconvolution	conv1	y	5.474	0.478	0.666	0.151	1.177	0.618
GradientSHAP	conv1	y	5.269	0.527	0.779	0.166	1.227	1.074
GCAM++	4.1.2	n	6.054	0.556	0.945	0.077	0.838	0.140
IntGrad	conv1	y	5.265	0.528	0.777	0.167	1.226	0.994
Feature Ablation	conv1	n	4.655	0.136	0.684	0.188	1.388	1.077
Occlusion	conv1	n	5.300	0.179	0.726	0.130	1.030	0.785
Deconvolution	conv1	n	3.934	0.484	0.666	0.147	1.203	0.751
Classical Methods (Layer Optimization)								
GCAM	2.0.1	y	**4.003**	0.450	0.678	0.052	**1.024**	0.738
GCAM	4.1.2	n	6.070	**0.554**	**0.945**	**0.076**	0.829	**0.131**
Guided GCAM	4.0.2	n	4.929	**0.742**	0.866	0.119	1.008	0.687
Guided GCAM	4.1.2	n	**3.843**	0.717	**0.883**	**0.129**	**1.096**	**0.612**
LCAM	2.0.2	n	**3.690**	0.549	0.865	**0.144**	**1.428**	0.913
LCAM	4.1.1	n	5.661	**0.664**	**0.971**	0.083	0.890	**0.227**
LRP $\gamma = 0.25$	conv1	y	**3.550**	**0.656**	0.852	**0.213**	**1.737**	0.521
LRP $\gamma = 0.25$	4.1.1	n	5.649	0.630	**0.965**	0.081	0.877	**0.200**
Guided Backprop	conv1	n	**3.517**	**0.621**	0.867	**0.212**	**1.757**	0.459
Guided Backprop	3.1.2	n	3.874	0.582	**0.938**	0.121	1.307	**0.213**
Baseline Methods								
Baseline: Original Image	conv1	y	8.351	-1.699	0.572	0.049	0.435	n/r
Baseline: Binary Mask	conv1	y	5.964	1.000	1.000	0.067	0.739	n/r
Random Baseline	n/a	y	4.071	0.425	0.440	0.147	1.067	n/r

A.1.4 Randomization Sanity Check

The model-randomization sanity check (Adebayo et al., 2018) tests whether a saliency method truly depends on the model's learned parameters or merely reflects input-dependent patterns that survive when the model is corrupted. For each attribution method, saliency maps on the trained VGG-16 are compared with maps obtained after randomly reinitializing the weights of the last three feedforward layers with Gaussian noise; a method genuinely sensitive to the trained parameters should show a substantial similarity drop. While Binder et al. (2023) discuss important shortcomings of top-down randomization-based sanity checks, the test is applied here as a complementary diagnostic (the trajectory-space randomization diagnostic of Section 8.4 is the thesis' primary tool). Similarity is quantified by (i) HOG similarity (Pearson correlation of the histograms of oriented gradients), (ii) signed Spearman rank correlation, and (iii) SSIM; lower similarity indicates stronger parameter dependence.

The results in Table A.3 highlight that the backward-pass stabilization used in SplitCAM and SplitGrad effectively mitigates the gradient-explosion issues caused by the non-negative split-stream weights: both method families show substantially lower similarities than all considered baselines, particularly in Spearman and SSIM, demonstrating exceptionally strong dependence on the learned weights. SplitLRP, in contrast, remains fairly insensitive (HOG ≈ 0.99, Spearman ≈ 1.00; only SSIM registers the change at ≈ 0.45–0.56). This insensitivity is attributable to the diminishing-gradient effect of the ϵ-LRP rule on the non-negative split stream: changes in the final-layer weights become largely insignificant for the propagated relevance, and indeed the layer-26 SplitLRP configuration shows the lowest similarity across all three metrics among the SplitLRP configurations.

A.1.5 Computational Efficiency

Mean computation times across the custom 50-image validation subset of ImageNet-S are reported in Table A.4 for both ResNet-18 and VGG-16. The model-splitting operation itself is performed once per model (averaged over 20 split procedures) and requires only 0.033 s for ResNet-18 and 0.030 s for VGG-16 — negligible overhead. SplitCAM and SplitGrad achieve competitive per-image times (0.33–0.44 s) compared to classical CAM-based methods while being orders of magnitude faster than the perturbation-based approaches Feature Ablation and Occlusion. The shifting parameter α does not affect runtime, as it only modifies the forward-pass computations. The measured implementation duplicates all weight matrices and does not use PyTorch hooks; since the split stream can in principle operate directly on the original model weights and both passes can be parallelized, the reported overhead is an upper bound.

Tab. A.3: Randomization sanity check for the split methods on VGG-16: mean similarity between attribution maps of the trained model and of a copy whose last three feedforward layers are randomly reinitialized, across the custom test subset of ImageNet-S. Lower similarity indicates stronger dependence on the learned parameters. Abbreviations as in Table 7.1.

Method	HOG ↓	Spearman ↓	SSIM ↓
Baselines			
Deconvolution	0.985	0.999	0.481
DeepLIFT	0.365	0.274	0.716
GradientSHAP	0.286	0.081	0.571
Input × Gradient	0.313	0.082	0.621
IntGrad	0.303	0.102	0.584
LCAM (layer 14)	0.688	0.123	0.417
LCAM (layer 2)	0.542	0.114	0.475
Saliency	0.188	0.416	0.353
SplitCAM			
SplitCAM (sc, α 0.4) (12, g)	0.736	0.729	0.202
SplitCAM (sc, α 0.4) (14, g)	0.724	0.194	0.174
SplitCAM (sc, α 0.4, wta) (14, g)	0.724	0.389	0.303
SplitCAM (sc, α 0.4, wta) (26, +, g) abs	0.728	0.342	0.319
SplitCAM (sc, α 0.5, wta) (14, g)	0.728	0.096	0.292
SplitCAM (sc, α 0.5, wta) (26, +, g)	0.804	0.186	0.148
SplitGrad			
SplitGrad (α 0.4) (12, +, g)	0.677	0.711	0.174
SplitGrad (α 0.4) (14, +, g) abs	0.704	0.197	0.091
SplitGrad (α 0.4, wta) (12, +, g)	0.706	0.454	0.174
SplitGrad (α 0.4, wta) (14, +, g) abs	0.721	0.211	0.199
SplitGrad (α 0.5, wta) (12, +, g) abs	0.725	0.238	0.206
SplitGrad (α 0.5, wta) (26, +, g) abs	0.795	0.149	0.306
SplitLRP			
SplitLRP (7, pos)	0.997	1.000	0.471
SplitLRP (19, pos)	0.998	1.000	0.562
SplitLRP (wta) (7, pos)	0.998	1.000	0.456
SplitLRP (wta) (26, pos) abs	0.994	0.999	0.446

Tab. A.4: Mean saliency computation time comparison (seconds per image), averaged over the custom 50-image validation subset of ImageNet-S..

Method	**ResNet-18**	**VGG-16**
Model Splitting Time	0.0331	0.0295
Split Methods		
SplitCAM	0.4368	0.3348
SplitGrad	0.4284	0.3309
SplitLRP	0.5711	0.3204
Classical Methods		
GCAM	0.0216	0.0232
GCAM++	0.0157	0.0155
LCAM	0.0149	0.0124
Guided GCAM	0.3102	0.2808
LRP $\gamma = 0.25$	0.0599	0.0507
DeepLIFT	0.0203	0.0210
IntGrad	0.0070	0.0121
GradientSHAP	0.0084	0.0082
Guided Backprop	0.0253	0.0258
Deconvolution	0.0087	0.0074
Saliency	0.0060	0.0047
Input $\times$ Gradient	0.0055	0.0046
Feature Ablation	231.6631	192.2602
Occlusion	238.2302	201.0051

A.2 Stabilization Ablations for the Split Methods

This section presents the ablation studies examining the effect of the proposed stabilization techniques for the forward and backward pass, as well as the influence of the selected max-pooling implementation (Section 3.6).

A.2.1 Backward-Pass Stabilization

Table A.5 reports the full α-ablation for SplitCAM and SplitGrad across the selected configurations, extending the main-text Table 7.3. Performance generally improves as α decreases below 0.5, while all metrics except Maximum Sensitivity exhibit a sharp drop at $\alpha = 0.0$: the backward-pass stabilization plays a crucial role in both methods. The improvements in Pixel Flipping are modest, but the gains in the two localization metrics and the robustness metric are substantial. The table also compares max-pooling in convex mode versus the winner-takes-it-all mode: for α close to 0.5 the differences are minimal (for $\alpha = 0.5$, split-pair gradients coincide with scaled original gradients, so the max-pooling choice has no influence on the gradients), whereas for $\alpha \in \{0.30, 0.35\}$ the winner-takes-it-all mode shows mostly stronger performance across all methods and metrics.

A.2.2 Forward-Pass Stabilization

This subsection investigates the influence of the forward stabilization on SplitLRP for the framework-selected layer configurations, and on SplitCAM for manually selected layer configurations. As a baseline, a shifting mode is introduced that shifts the activations of positive neurons to half of the respective original activation and those of negative neurons to the negative half. A SplitCAM variant without any forward stabilization exhibits an activation explosion in late VGG-16 layers on the order of 10^{18}, resulting in numerical errors on the order of 10^9 despite per-layer correction; for SplitLRP, the absence of forward stabilization produced nearly entirely white saliency maps with only small, random, semantically meaningless artifacts, so no quantitative metrics were computed for that setting.

Table A.7 summarizes all SplitCAM configurations *without* forward-pass stabilization, and Table A.8 the results *with* the shifting-based stabilization; both mirror the configurations of Table A.5. Table A.6 directly compares SplitLRP with shifting versus scaling stabilization: for both layer-7 configurations, replacing scaling with shifting leads to substantial performance drops across all metrics; for the later-layer configurations the picture is mixed — shifting yields slightly higher Attribution Localization, but scaling achieves markedly better Pointing Game performance and significantly improved robustness. These findings highlight the clear advantage of using scaled split-pair activations, and not the original activations, for LRP within the DC-decomposition framework.

Tab. A.5: Alpha ablation as performed in Section 7.4 across four SplitGrad and SplitCAM configurations of layers 14 and 26, respectively. The notation is similar to Table 7.1.

Method	Metric	α 0.50	α 0.45	α 0.40	α 0.35	α 0.30	α 0.00
	PG↑	0.8975	0.9064	0.9470	**0.9558**	0.8887	0.4011
SplitGrad (wta)	AL↑	0.5434	0.5476	0.6051	**0.6169**	0.5346	0.4486
(26, g), abs	MaxS↓	0.6160	0.6071	0.4399	0.2382	0.3344	**0.0283**
	PF_{20}^{drop} ↑	1.5270	1.5337	**1.5808**	1.4476	1.3171	1.2471
	PG↑	0.8975	0.9064	**0.9470**	0.9099	0.2827	0.6413
SplitGrad	AL↑	0.5434	0.5476	**0.6037**	0.5216	0.4245	0.4563
(26, g), abs	MaxS↓	0.6171	0.6102	0.4426	0.3864	0.0622	**0.0079**
	PF_{20}^{drop} ↑	1.5272	1.5333	**1.5876**	1.3702	0.2431	0.5755
	PG↑	0.8198	0.8198	0.8198	**0.8322**	0.8127	0.7898
SplitGrad (wta)	AL↑	**0.5403**	0.5403	0.5402	0.5006	0.4936	0.4857
(14, g), abs	MaxS↓	0.8579	0.8597	0.8584	0.0875	0.0677	**0.0481**
	PF_{20}^{drop} ↑	2.1887	**2.1894**	2.1869	1.8740	1.8304	1.7684
	PG↑	0.8198	**0.8198**	0.8180	0.7951	0.8004	0.7968
SplitGrad	AL↑	0.5403	**0.5403**	0.5330	0.5054	0.4926	0.4847
(14, g), abs	MaxS↓	0.8618	0.8584	0.8478	0.1103	0.0662	**0.0301**
	PF_{20}^{drop} ↑	**2.1894**	2.1891	2.1185	1.3425	1.2959	1.1927
	PG↑	0.8675	0.8675	**0.9382**	0.9329	0.6131	0.8551
SplitCAM (sc, wta)	AL↑	0.5501	0.5535	0.6347	**0.6627**	0.4033	0.5133
(26, +, g), abs	MaxS↓	0.6906	0.6820	0.4573	0.2645	0.2511	**0.1196**
	PF_{20}^{drop} ↑	1.6067	**1.6085**	1.5849	1.4989	0.8534	1.5071
	PG↑	0.8940	0.8905	**0.9346**	0.6466	0.8251	0.8604
SplitCAM (sc)	AL↑	0.5595	0.5636	**0.6363**	0.4015	0.4977	0.5114
(26, +, g), abs	MaxS↓	0.6738	0.6640	0.4520	0.3729	0.1102	**0.1062**
	PF_{20}^{drop} ↑	1.5902	**1.5978**	1.5906	0.9076	1.2784	1.3605
	PG↑	0.8251	0.8251	**0.8269**	0.7880	0.7809	0.7703
SplitCAM (sc, wta)	AL↑	0.5363	0.5363	**0.5435**	0.5338	0.5286	0.5225
(14, g)	MaxS↓	1.0566	1.0569	1.0489	**0.1620**	0.1625	0.1630
	PF_{20}^{drop} ↑	2.4128	2.4130	**2.4135**	1.9014	1.8613	1.8731
	PG↑	0.8251	0.8251	**0.8428**	0.7756	0.7686	0.7756
SplitCAM (sc)	AL↑	0.5363	0.5363	**0.5626**	0.5363	0.5271	0.5208
(14, g)	MaxS↓	1.0552	1.0593	1.0136	0.1629	0.1585	**0.1555**
	PF_{20}^{drop} ↑	2.4171	**2.4172**	2.4065	1.5032	1.5082	1.5139

Tab. A.6: Scale (sc) vs. shift (sh) forward-pass stabilization for SplitLRP methods on VGG-16.

Method	Type	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
SplitLRP (wta), 7	sc	**3.235**	**-0.036**	**0.836**	**0.296**	**2.291**	**0.866**
	sh	3.619	-0.610	0.698	0.260	2.070	0.914
SplitLRP (wta), 26, abs	sc	**5.168**	0.588	0.871	0.151	1.473	**0.282**
	sh	5.168	**0.628**	**0.877**	**0.161**	**1.478**	0.468
SplitLRP, 7	sc	**3.235**	**-0.036**	**0.836**	**0.296**	**2.291**	**0.866**
	sh	3.619	-0.695	0.698	0.260	2.070	0.914
SplitLRP, 19	sc	5.168	0.588	**0.871**	0.151	1.473	**0.282**
	sh	**4.955**	**0.629**	0.853	**0.197**	**1.692**	0.507

Tab. A.7: Alpha ablation for scale normalization across four SplitCAM configurations of layers 14 and 26. Here, in the forward pass *no* stabilization is applied. Notation and structure are similar to Table A.5. Light blue entries marked with * indicate better performance of the method compared to the default variant with scale stabilization in the forward pass.

Method	**Metric**	α 0.50	α 0.45	α 0.40	α 0.35	α 0.30	α 0.00
SplitCAM (sc, wta) (26, +, g), abs	PG↑	0.7686	0.7686	0.8632	**0.8712**	0.3441	0.5453
	AL↑	0.5374	0.5412	0.5943	**0.6118**	0.3595	0.4569
	MaxS↓	0.6578*	0.6507*	0.4841	0.2873	0.1918*	**0.0135***
	PF_{20}^{drop} ↑	1.2744	1.2859	**1.3209**	1.1792	0.4430	0.6527
SplitCAM (sc) (26, +, g), abs	PG↑	0.7565	0.7565	**0.8451**	0.3783	0.4507	0.4970
	AL↑	0.5346	0.5384	**0.5897**	0.3663	0.4420	0.4545
	MaxS↓	0.6604*	0.6512*	0.4985	0.3077*	0.0279*	**0.0043***
	PF_{20}^{drop} ↑	1.2680	1.2856	**1.3295**	0.5912	0.4906	0.5563
SplitCAM (sc, wta) (14, g)	PG↑	**0.8491***	0.8491*	0.8491*	0.7867	0.7847*	0.7686
	AL↑	0.5324	0.5321	**0.5406**	0.5306	0.5261	0.5200
	MaxS↓	1.0834	1.0814	1.0885	**0.2271**	0.2309	0.2384
	PF_{20}^{drop} ↑	2.4454*	2.4482*	**2.4486***	1.9588*	1.9193*	1.9349*
SplitCAM (sc) (14, g)	PG↑	0.7746	0.7767	**0.7847**	0.5855	0.5594	0.5573
	AL↑	0.4003	0.4144	**0.5517**	0.5300	0.5209	0.5137
	MaxS↓	1.5733	1.5627	1.5443	**1.2661**	1.3939	1.4272
	PF_{20}^{drop} ↑	2.1298	**2.1384**	2.1371	1.6669*	1.6793*	1.6740*

Tab. A.8: Alpha ablation for scaling normalization across four SplitCAM configurations of layers 14 and 26. Here, in the forward pass shift normalization is applied. Notation and structure are similar to Table A.5. Light blue entries marked with * indicate better performance of the method compared to the default variant with scale stabilization in the forward pass.

Method	Metric	α 0.50	α 0.45	α 0.40	α 0.35	α 0.30	α 0.00
	PG↑	0.8330	0.8149	0.7968	**0.9235**	0.6036	0.8592*
SplitCAM (sc, wta)	AL↑	0.5242	0.5203	0.5147	**0.6549**	0.3968	0.5093
(26, +, g), abs	MaxS↓	0.6180*	0.6182*	0.6509	0.3104	0.2791	**0.1665**
	PF_{20}^{drop} ↑	**1.6804***	1.6646*	1.5065	1.4962	0.8896*	1.5405*
	PG↑	0.8330	0.8149	0.7988	0.5654	0.8431*	**0.8652***
SplitCAM (sc)	AL↑	**0.5242**	0.5203	0.5076	0.3736	0.4949	0.5094
(26, +, g), abs	MaxS↓	0.6129*	0.6168*	0.6494	0.3820	0.1686	**0.1664**
	PF_{20}^{drop} ↑	**1.6804***	1.6646*	1.5105	0.8714	1.4289*	1.4765*
	PG↑	**0.8310***	0.8310*	0.8310*	0.7887*	0.7827*	0.7726*
SplitCAM (sc, wta)	AL↑	0.5312	0.5312	**0.5376**	0.5315	0.5264	0.5203
(14, g)	MaxS↓	1.0562*	1.0521*	1.0649	**0.1633**	0.1641	0.1642
	PF_{20}^{drop} ↑	**2.4412***	2.4412*	2.4400*	1.9306*	1.8953*	1.9059*
	PG↑	0.8310*	0.8310*	**0.8431***	0.7746	0.7706*	0.7807*
SplitCAM (sc)	AL↑	0.5311	0.5312	**0.5580**	0.5337	0.5245	0.5183
(14, g)	MaxS↓	1.0554	1.0568*	1.0198	0.1641	0.1587	**0.1570**
	PF_{20}^{drop} ↑	**2.4412***	2.4411*	2.4318*	1.5187*	1.5222*	1.5329*

A.3 Evaluation Protocol

A.3.1 Data and splits

We evaluate on a *custom* 50-class subset of ImageNet-S (Gao et al., 2023), sampled once at the start of the project and held fixed across all experiments. The subset has two partitions: a **custom validation split of 50 images** (one image per class, seed 42) used only for configuration selection, and the **test split** of the remaining images, used for every reported test number. Images whose object mask lies entirely outside the 224×224 center crop are excluded after the split, because the mask-based metrics are undefined for them; this removes one test image and leaves **565 test images**. The Pascal VOC 2012 transfer split is described in Section A.4.4.

Backbone and dataset provenance. All game-based experiments use the pretrained torchvision checkpoints shipped with the library release used for evaluation. Specifically, VGG-16 uses `VGG16_Weights.IMAGENET1K_V1` (https://download.pytorch.org/models/vgg16-397923af.pth; recipe https://github.com/pytorch/vision/tree/main/references/classification#alexnet-and-vgg), ResNet-50 uses `ResNet50_Weights.IMAGENET1K_V2` (https://download.pytorch.org/models/resnet50-11ad3fa6.pth; recipe https://github.com/pytorch/vision/issues/3995#issuecomment-1013906621), and ViT-B/16 uses `ViT_B_16_Weights.IMAGENET1K_V1` (https://download.pytorch.org/models/vit_b_16-c867db91.pth; recipe https://github.com/pytorch/vision/tree/main/references/classification#vit_b_16). These are ImageNet-1K classifiers trained on the ImageNet / ILSVRC benchmark (Deng et al., 2009; Russakovsky et al., 2015). Quantitative scoring is performed with Quantus (Hedström et al., 2023).

Licenses. ImageNet images are used under the ImageNet terms of access (non-commercial research and educational use) (Deng et al., 2009). The ImageNet-S masks (Gao et al., 2023) are released by their authors without an explicit license; their images are ImageNet images and fall under the same terms. The Pascal VOC 2012 images (Everingham et al., 2009) are subject to the Flickr terms of use, as stated by the dataset providers. The checkpoints are distributed with torchvision (BSD-3-Clause) and were trained on ImageNet-1K. Libraries: PyTorch and torchvision (BSD-3-Clause), Quantus (GPL-3.0), zennit (GPL-3.0/LGPL-3.0), lxt, captum and scikit-image (BSD-3-Clause), pytorch-grad-cam (MIT); the reference code of TIBAV and GAE (MIT) is included with its license notice.

A.3.2 Methods

Gradient-based block. **Gradient** is the raw input-gradient saliency map (Simonyan et al., 2014), the maximum over color channels of the absolute gradient. **SmGrad** is SmoothGrad, i.e. an average over noisy gradient maps (Smilkov et al., 2017). **IntGrad** is Integrated Gradients (Sundararajan et al., 2017) with a black-image baseline. **DeepLIFT** propagates activation differences relative to a reference input with the Rescale rule (Shrikumar et al., 2017) (CNNs only). **DAVE** is the distribution-aware ViT gradient decomposition of Wróbel et al. (2026). **SG** is our SG, included in this block because it recovers the gradient as a Stopping-Game occupation measure (Theorem 4.8).

LRP / relevance-propagation block. **LRP-ε** denotes the stabilized Layer-wise Relevance Propagation rule (Bach et al., 2015). **LRP-γ** is the positive-weight-amplified LRP variant used in best-practice comparisons (Kohlbrenner et al., 2020). **AttnLRP** is transformer LRP that propagates relevance through the softmax and the bilinear attention product as well as the Value path (Achtibat et al.,

2024). **RG** is our RG. **RG+Eq** is the same method with equivariant Reynolds averaging, used mainly on ViT to suppress tokenization-grid artifacts. **TIBAV** (Chefer et al., 2021b), **GAE** (Chefer et al., 2021a) and Attention Rollout (**AR**) are the transformer-specific attention-map baselines.

CAM block. **GCAM** is Grad-CAM (Selvaraju et al., 2017), **GCAM++** is Grad-CAM++ (Chattopadhay et al., 2018), and **LCAM** is LayerCAM, which weights class-activation evidence per location (Jiang et al., 2021). All three are computed at the last convolutional block of the CNNs and upsampled bilinearly.

Perturbation block. **LIME** is Local Interpretable Model-agnostic Explanations (Ribeiro et al., 2016). **KernelSHAP** is the model-agnostic weighted-regression estimator of SHAP values (Lundberg and Lee, 2017). Both perturb SLIC superpixels (Achanta et al., 2012). These methods explain predictions by perturbing the input and fitting a local surrogate, so they are separated from the backward-propagation methods above.

A.3.3 Metric definitions

Let $x \in \mathbb{R}^{C\times H\times W}$ be an image, let y be its target class, and let $a \in \mathbb{R}^{H\times W}$ be the scalar attribution map produced by an explanation method after channel reduction. Write $d \coloneqq HW$ and let $\mathrm{vec}(a) \in \mathbb{R}^d$ denote the flattened map. For any binary segmentation mask $m \in \{0,1\}^{H\times W}$, write $\mathrm{vec}(m) \in \{0,1\}^d$. When a metric is configured with `abs=true` and `normalise=true`, the evaluator applies the max-abs normalization (not min–max)

$$\psi(a) = \frac{|\mathrm{vec}(a)|}{\max_i |\mathrm{vec}(a)_i|}, \tag{A.1}$$

with the convention that the all-zero map remains zero. We denote the preprocessed attribution vector by $\bar{a}$. All metrics below use $\bar{a}$; faithfulness metrics use raw logits (no softmax) to avoid saturation effects, and the perturbation baseline is the channel-wise mean.

Attribution localization (AL). The inside-mass ratio of Kohlbrenner et al. (2020), who compute it with ground-truth bounding boxes; we use the ImageNet-S pixel masks:

$$\mathrm{AL}(a,m) = \frac{\sum_{i=1}^{d} \bar{a}_i m_i}{\sum_{i=1}^{d} \bar{a}_i + 10^{-9}}. \tag{A.2}$$

Pointing game (PG). Following the Excitation Backprop protocol (Zhang et al., 2018a), the pointing-game config uses `abs=true` and no normalization. Let $M(a) \coloneqq \{i \in \{1,\dots,d\} : \bar{a}_i = \max_j \bar{a}_j\}$ be the entire set of maximizers. Then

$$\mathrm{PG}(a,m) = \mathbf{1}\big\{M(a) \cap \{i : m_i = 1\} \neq \varnothing\big\}. \tag{A.3}$$

Ties are scored as a hit whenever at least one maximizer lies inside the object mask. PG is binary per image, so its test mean is a multiple of $1/565$.

Top-k intersection (TK). Following Theiner et al. (2021), with $k = 1000$ and the permutation π that sorts $\bar{a}$ in descending order:

$$\mathrm{TK}(a, m) = \frac{1}{1000} \sum_{r=1}^{1000} m_{\pi_r}. \tag{A.4}$$

Pixel-flipping AUC (PF_5, PF_{20}). Pixel flipping (Bach et al., 2015; Samek et al., 2017) removes the most relevant features first and tracks the class score. The evaluator uses `pixel_flipping_abs` with `features_in_step = 1568` and `perturb_baseline = mean`. For $T \coloneqq \lceil d/1568 \rceil$, define perturbed inputs $x^{(t)}$ by replacing features $\pi_{(t-1)\cdot 1568+1}, \ldots, \pi_{\min\{t\cdot 1568, d\}}$ (and all earlier ones) with the per-sample mean baseline. The evaluator computes

$$f_t \coloneqq F(x^{(t)})_y, \qquad t = 1, \ldots, T, \tag{A.5}$$

and PF_p is the trapezoidal area over the first $p\%$ of this flipping curve, i.e. over its first K_p points with the abscissa rescaled to [0,1]: $\mathrm{PF}_p = \mathrm{trapz}\big(\{(t-1)/(K_p-1), f_t\}_{t=1}^{K_p}\big)$, $K_p = \max(2, \lceil pT/100 \rceil)$; with $T = 96$ steps of 1568 features (the attribution is broadcast over the three color channels), $K_5 = 5$ and $K_{20} = 20$. Lower is better.

Selectivity (Sel). Following Montavon et al. (2018) and its Quantus implementation (Hedström et al., 2023), $\bar{a}$ is summed over square patches of 39×39 pixels (side $\lfloor \sqrt{1568} \rfloor$); non-overlapping patches are removed (replaced by the per-sample mean) in decreasing order of their summed attribution, and the target logit is recorded after each removal. Sel is the trapezoidal area under this curve, with the fraction of removed patches on the abscissa. Lower is better.

Max and average sensitivity (MaxS, AvgS). Following Yeh et al. (2019), the evaluator regenerates attributions on the same $N_s = 50$ inputs for both metrics, $x + \xi_k$ with uniform noise $\xi_k \sim \mathscr{U}[-r, r]^{C\times H\times W}$, $r = 0.2$, and reports the maximum (MaxS) or mean (AvgS) over k of $\|\psi(a(x+\xi_k)) - \psi(a(x))\|_F / \|\psi(a(x))\|_F$. Lower is better, as it indicates stability of the explanation to input noise. The noise is seeded per image, so every row sees the same perturbed inputs. MaxS and AvgS are computed on a fixed random subset of 200 test images (seed 0) that is identical for all rows and models; all other metrics use the full test split.

A.3.4 Declared validation grids

Table A.9 lists the grid of every method. Functionally duplicate rows (e.g. different σ^2 when $\lambda = \lambda_{\mathrm{sm}} = 0$) are evaluated once; in total 2,021 validation configurations are evaluated. On ViT-B/16 the activation-side λ is fixed to 0, which keeps GELU gates conditioned (Section 5.3). At attention–Value products, RG applies $\alpha\beta$-LRP-ε to the composite map of the attention product and the Value projection (Theorem 5.6; Section 5.4.4). Baselines without a declared grid are run at the settings of their reference implementations.

Tab. A.9: Per-method validation grids (all architectures unless stated). Symbols: Section A.4.

Method	Grid
Gradient	single config
SmGrad	N=50, noise level $\sigma/(x_{\max}-x_{\min}) \in \{0.1, 0.15, 0.2\}$, mean aggregation (3 configs)
IntGrad	black-image baseline, $N_{\text{steps}} \in \{20, 50, 100, 300\}$ (4 configs)
DeepLIFT	Rescale rule, single config (CNNs only)
DAVE	$N \in \{25, 50\} \times \theta_{\max} \in \{0, 10, 20\} \times s_{\max} \in \{0, 0.05, 0.1\} \times t_{\max} \in \{0, 0.25, 0.5\}$, horizontal flips with probability 0.5, averaged effective map multiplied by the input (54 configs; ViT-B/16 only)
SG	$\lambda \in \{-5, -2, -1, -0.3, 0\} \times \sigma^2 \in \{1, 2, 5, 10\}\times$ noisy gate $\tilde{\Phi} \in \{\text{off}, \text{on}\}$ (40 configs; VGG-16 only)
LRP-ε	$\varepsilon \in \{0.01, 0.05, 0.1, 0.25, 0.5, 1, 2\}$ (7 configs)
LRP-γ	$(\varepsilon, \gamma) \in \{0.1, 0.25, 0.5\} \times \{0.1, 0.25, 0.5, 1, 2\}$; ViT-B/16: γ on all Linear layers (15 configs); CNNs: both zennit's EpsilonGammaBox composite ($z^{\mathscr{B}}$ input layer, γ on convolutions, ε on dense layers) and γ on all layers (30 configs)
AttnLRP	γ-composite of Achtibat et al. (2024): reference setting $(\gamma_{\text{conv}}, \gamma_{\text{lin}}, \varepsilon_{\text{attn}})=(0.25, 0.05, 10^{-6})$ + univariate sweeps $\gamma_{\text{lin}} \in \{0.01, 0.025, 0.1, 0.25\}$, $\gamma_{\text{conv}} \in \{0.1, 0.5, 1\}$, $\varepsilon_{\text{attn}} \in \{0.01, 0.05, 0.25\}$ (11 configs; ViT-B/16 only)
TIBAV	single config (ViT-B/16 only)
GAE	single config (ViT-B/16 only)
AR	residual weight $w_r \in \{0, 0.25, 0.5, 0.75\}$ (4 configs; ViT-B/16 only)
RG	$\tau \in \{0.3, 0.4, \ldots, 1.0\}$, $(\alpha,\beta)=(2,1)$, $\sigma^2 \in \{1, 2, 5, 10\}$, $\varepsilon \in \{0.1, 0.25, 0.5, 1\}$; ViT-B/16: λ=0, $\lambda_{\text{sm}} \in \{-50, -10, -5, -2, -1, 0\}$, composite Value rule of Theorem 5.6 (672 configs); CNNs: $\lambda \in \{-5, -2, -1, -0.3, 0\}$ (544 configs)
RG+Eq	$\tau \in \{0.3, 0.4, \ldots, 1.0\} \times \lambda_{\text{sm}} \in \{0, -10\}$, σ^2=2, ε=0.5, N=50, composite Value rule (16 configs; ViT-B/16 only)
GCAM, GCAM++, LCAM	last convolutional block (VGG-16: `features.29`, ResNet-50: `layer4`), single config (CNNs only)
LIME	80 SLIC superpixels, N=1000, ridge surrogate, exponential kernel of width 0.25 on the cosine distance of the binary superpixel vectors, removed superpixels filled with their mean color; fixed reference row
KernelSHAP	80 SLIC superpixels, N=2M+2048 for M superpixels, removed superpixels filled with the image mean; fixed reference row
Anchor	$\alpha\beta$-LRP anchor $(\alpha,\beta,\varepsilon,\tau)$=(2,1,0.5,1) of RG and RG+Eq, reported as the theorem-driven reference row (not selected)

A.3.5 Selection

For every tunable method, baselines and ours alike, two configurations are selected on the validation split: `loc` minimizes the rank-sum over $\{\text{AL}, \text{PG}, \text{TK}\}$ and `faith` the rank-sum over $\{\text{PF}_5, \text{PF}_{20}, \text{Sel}\}$. Ranks are computed within each method's grid, respecting metric directions; ties are broken in favor of the default (for RG, the anchor), then of fewer non-default parameters, then of grid order. A configuration is admissible only if its map is non-zero and every metric is defined on every validation image; inadmissible configurations are excluded before ranking (5 of the 40 SG configurations on VGG-16, all with strong risk aversion and the deterministic gate, whose maps vanish on some images; no other configuration). MaxS and AvgS are not used for selection because each evaluation re-explains

every image $2N_s$ times. The `loc` row is the one reported in the main tables, and the `faith` row of every method is reported alongside (Section A.4): localization is a natural target because the analytical modes are designed to control routing focus, because mask localization has an external ground truth whereas perturbation metrics evaluate off-distribution inputs, and because comparing LRP rule choices by object localization has precedent, there with bounding boxes (Kohlbrenner et al., 2020); localization is not faithfulness, so both rows are reported and the trade-off stays visible.

A.3.6 Statistics

Every test metric is reported with a 95% percentile bootstrap confidence interval over test images (Efron and Tibshirani, 1993) (10,000 resamples of the images; the same resampled image indices are used for all rows, so comparisons are paired). MaxS and AvgS are computed on the fixed 200-image subset of Section A.3.3, and their intervals resample the images of that subset. For each metric, every row is tested against the best row for that metric: continuous metrics with a two-sided paired bootstrap test of the mean per-image difference (the bootstrap distribution of the mean difference is centered at zero, and p is the fraction of centered resampled differences at least as large in absolute value as the observed one), and the binary PG with McNemar's test (McNemar, 1947) (χ^2 statistic with continuity correction). The p-values are Holm-corrected (Holm, 1979) within each metric: over the rows shown in the main table, over all rows of a column in the appendix tables, and, for the statement that a row is significantly ahead of every baseline row, over its comparisons with the baseline rows. We call a difference significant at level 0.05 after correction and describe all other differences as comparable.

A.3.7 Compute resources

The attribution evaluations are inference-only. The main cost is the repeated backward attribution pass over the validation and test images across the method grids, including the equivariant-Reynolds and noise-averaged configurations. Runtime measurements were taken on a single NVIDIA A100-SXM4-40GB with batch size 1; evaluation sweeps were distributed over several GPUs of the same type, with up to two evaluation processes per GPU. The full pipeline (validation grids, test evaluation, robustness, Pascal VOC, ablations and randomization diagnostics) required about 450 GPU-hours.

A.3.8 Code and correctness tests

The code (https://github.com/JayPiZimmermann/Playing_The_Network_Backward_Official_Repository) contains unit tests that pin the implemented backward rules to the statements of the appendix. Two of them concern the attention block. `tests/test_attention_ab_recovery.py` runs the RG hooks at $\tau = 1$ on a small attention block with signed inputs, a Value bias and $\varepsilon > 0$, in float64, and compares the result with an independent implementation of $\alpha\beta$-LRP-ε on the composite map $X \mapsto A(XW_v^\top + b_v)$, i.e. the target rule (5.9) of Theorem 5.6, for a single head and for a multi-head block in the layout of `nn.MultiheadAttention`; it also asserts that the result differs from the sequential variant of §5.4.1, which the released code does not offer (`av_rule` accepts only `'merged'`). `tests/test_hellinger_merged_attention.py` enumerates every backward trajectory of a small attention block under two models, at $\tau \in \{0.5, 1, 2\}$, and checks the per-input masses, the Bhattacharyya mass and the cemetery masses of the Hellinger pass (Section 8.3) against this brute-force reference; a pre-norm residual block built from `nn.MultiheadAttention` must give the same distance as the same block with its attention matrix supplied as a constant.

A.4 Additional Results

The validation, test and VOC tables of this section are generated by one pipeline from the declared grids and the selection rules of Section A.3.

Configuration symbols. The configuration columns of all tables use the following parameters. RG, RG+Eq and SG (Table 4.1): temperature τ, activation-side risk aversion λ, attention-logit risk aversion λ_{sm}, ADF input variance σ^2, LRP stabilizer ε, stream weights (α,β) (default (2,1)), noisy-observation gate $\tilde{\Phi}$ ("gate on/off"), and the number N of transformed inputs averaged by RG+Eq. SmoothGrad: N noisy samples with noise standard deviation σ relative to the input range (σ/range). Integrated Gradients: N_{steps} integration steps from a black-image baseline. DAVE: N transformed inputs with rotations up to $\theta_{\max}$ degrees, shifts up to $s_{\max}$ of the image extent, and variance-preserving Gaussian noise of interpolation weight up to $t_{\max}$. LRP-ε and LRP-γ: stabilizer ε and positive-weight boost γ; on the CNNs, LRP-γ is zennit's composite with the $z^{\mathscr{B}}$ rule at the input layer. AttnLRP: γ-rule parameters γ_{conv} and γ_{lin} of the convolutional and linear layers and the stabilizer $\varepsilon_{\text{attn}}$ of the attention layers. AR (attention rollout): residual weight w_r. LIME and KernelSHAP: N perturbation samples ($2M$+2048 for M superpixels) on 80 SLIC superpixels; LIME fits a ridge surrogate. DeepLIFT: Rescale rule. Grad-CAM, Grad-CAM++ and LayerCAM: the target layer (torchvision module name). Row labels in parentheses give the selection criterion (`loc`, `faith`); single-configuration methods have no criterion.

A.4.1 Validation selections

Tables A.10–A.12 list, for every method, the configurations selected on the 50-image validation split under the `loc` and `faith` criteria of Section A.3.5, together with both validation rank-sums. For fixed configurations there is no selection comparison, so the rank-sums are marked n/a.

Tab. A.10: ViT-B/16: validation-selected configurations per method and criterion (`loc`: minimal rank-sum over AL, PG, TK; `faith`: over PF_5, PF_{20}, Sel; rank-sums over the configurations of one method, lower is better; the number of configurations is given after the method name). Symbols: Section A.4.

Method	Criterion	Selected configuration	Loc. rank-sum	Faith. rank-sum
Gradient (1)	fixed	fixed	n/a	n/a
SmGrad (3)	loc	N=50,σ/range=0.2	3.5	3
	faith	N=50,σ/range=0.2	3.5	3
IntGrad (4)	loc	N_{steps}=100, black baseline	3.5	4
	faith	N_{steps}=100, black baseline	3.5	4
LRP-ε (7)	loc	ε=0.25	5.5	7
	faith	ε=0.1	6.5	3
LRP-γ (15)	loc	ε=0.1,γ=2	6	25
	faith	ε=0.1,γ=0.1	32	12
AttnLRP (11)	loc	γ_{conv}=0.25,γ_{lin}=0.05,$\varepsilon_{\text{attn}}$=0.25	3	29
	faith	γ_{conv}=0.25,γ_{lin}=0.05,$\varepsilon_{\text{attn}}$=0.01	14	7
DAVE (54)	loc	N=50,$\theta_{\max}$=0,$s_{\max}$=0,$t_{\max}$=0.5	14	6
	faith	N=50,$\theta_{\max}$=0,$s_{\max}$=0,$t_{\max}$=0.5	14	6
TIBAV (1)	fixed	fixed	n/a	n/a
GAE (1)	fixed	fixed	n/a	n/a
AR (4)	loc	w_r=0.75	3.5	3
	faith	w_r=0.75	3.5	3
LIME (1)	fixed	N=1000, 80 superpixels, ridge	n/a	n/a
KernelSHAP (1)	fixed	N=2M+2048, 80 superpixels	n/a	n/a
RG (672)	loc	τ=0.4,λ_{sm}=$-$10,σ^2=5,ε=0.1	47	788
	faith	τ=0.7,λ_{sm}=$-$5,σ^2=5,ε=0.1	1156	22
	anchor	τ=1	1420	344
RG+Eq (16)	loc	τ=0.5,λ_{sm}=$-$10,σ^2=2,N=50	6.5	29
	faith	τ=0.7,λ_{sm}=$-$10,σ^2=2,N=50	14	11
	anchor	τ=1,N=50	47	31

Tab. A.11: VGG-16: validation-selected configurations per method and criterion, as in Table A.10.

Method	Criterion	Selected configuration	Loc. rank-sum	Faith. rank-sum
Gradient (1)	fixed	fixed	n/a	n/a
SmGrad (3)	loc	N=50,σ/range=0.1	3	3
	faith	N=50,σ/range=0.1	3	3
IntGrad (4)	loc	N_{steps}=20, black baseline	6	3
	faith	N_{steps}=20, black baseline	6	3
DeepLIFT (1)	fixed	Rescale	n/a	n/a
LRP-ε (7)	loc	ε=0.5	6	9
	faith	ε=1	6	4
LRP-γ (30)	loc	ε=0.5,γ=0.25, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	13	43
	faith	ε=0.25,γ=0.1	36	5
GCAM (1)	fixed	`features.29`	n/a	n/a
GCAM++ (1)	fixed	`features.29`	n/a	n/a
LCAM (1)	fixed	`features.29`	n/a	n/a
LIME (1)	fixed	N=1000, 80 superpixels, ridge	n/a	n/a
KernelSHAP (1)	fixed	N=2M+2048, 80 superpixels	n/a	n/a
SG (40)	loc	λ=-1,σ^2=5, gate on	37	10
	faith	λ=-0.3,σ^2=1, gate off	49.5	6
RG (544)	loc	τ=0.3,ε=0.1	78	194
	faith	τ=0.8,ε=0.1	173.5	5
	anchor	τ=1	339	60

Tab. A.12: ResNet-50: validation-selected configurations per method and criterion, as in Table A.10.

Method	Criterion	Selected configuration	Loc. rank-sum	Faith. rank-sum
Gradient (1)	fixed	fixed	n/a	n/a
SmGrad (3)	loc	N=50,σ/range=0.15	4	5
	faith	N=50,σ/range=0.1	8	4
IntGrad (4)	loc	N_{steps}=100, black baseline	3	6
	faith	N_{steps}=50, black baseline	7.5	6
DeepLIFT (1)	fixed	Rescale	n/a	n/a
LRP-ε (7)	loc	ε=0.5	5.5	9
	faith	ε=2	8	4
LRP-γ (30)	loc	ε=0.5,γ=2, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	12.5	33
	faith	ε=0.5,γ=0.5, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	36	22.5
GCAM (1)	fixed	`layer4`	n/a	n/a
GCAM++ (1)	fixed	`layer4`	n/a	n/a
LCAM (1)	fixed	`layer4`	n/a	n/a
LIME (1)	fixed	N=1000, 80 superpixels, ridge	n/a	n/a
KernelSHAP (1)	fixed	N=2M+2048, 80 superpixels	n/a	n/a
RG (544)	loc	τ=0.9,ε=1	19	224
	faith	τ=1,λ=−1,σ^2=1,ε=0.25	466	38
	anchor	τ=1	212.5	140

A.4.2 Test results with confidence intervals

Tables A.13–A.15 list every row evaluated on the 565-image test split (the `loc`- and `faith`-selected configuration of every method, the anchors and the single-configuration methods) with 95% bootstrap confidence intervals and the tests of Section A.3.6 (bold and dark shading: best row of a column; ≈ and light shading: not significantly different from it). Per-image runtimes come from the single-GPU benchmark of Section A.3.7.

Tab. A.13: ViT-B/16, ImageNet-S test split: all rows with 95% bootstrap confidence intervals. Metrics as in Table 7.4; configuration symbols and row labels: Section A.4.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
Gradient	fixed	0.411 [0.389,0.432]	0.361 [0.322,0.402]	0.386 [0.359,0.413]	8.45 [8.33,8.56]	8.08 [7.94,8.22]	6.033 [5.905,6.154]	1.290 [1.233,1.351]	0.922 [0.893,0.953]
SmGrad (loc/faith)	N=50,σ/range=0.2	0.517 [0.498,0.536]	0.706 [0.667,0.743]	0.675 [0.654,0.694]	8.10 [7.96,8.24]	7.02 [6.83,7.20]	5.029 [4.894,5.163]	0.597 [0.569,0.630]	0.468 [0.451,0.485]
IntGrad (loc/faith)	N_{steps}=100,black baseline	0.458 [0.437,0.479]	0.513 [0.473,0.554]	0.498 [0.472,0.524]	8.37 [8.24,8.48]	7.93 [7.78,8.08]	5.712 [5.577,5.842]	1.090 [1.041,1.140]	0.825 [0.796,0.855]
LRP-ε (loc)	ε=0.25	0.592 [0.575,0.610]	0.834 [0.804,0.864]	0.746 [0.728,0.764]	7.97 [7.81,8.11]	6.88 [6.69,7.07]	4.859 [4.733,4.983]	0.486 [0.468,0.505]	0.384 [0.372,0.395]
LRP-ε (faith)	ε=0.1	0.582 [0.564,0.599]	0.834 [0.804,0.864]	0.748 [0.730,0.765]	7.84 [7.68,7.99]	6.76 [6.57,6.96]	4.890 [4.763,5.015]	0.623 [0.597,0.650]	0.498 [0.482,0.515]
LRP-γ (loc)	ε=0.1,γ=2	0.644 [0.628,0.660]	0.873 [0.844,0.899]	0.816 [0.800,0.832]	7.93 [7.78,8.08]	6.64 [6.44,6.83]	4.451≈ [4.324,4.572]	0.300 [0.287,0.314]	0.218 [0.210,0.226]

continued on next page

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
LRP-γ (faith)	ε=0.1,γ=0.1	0.614 [0.597,0.630]	0.844 [0.814,0.873]	0.779 [0.763,0.795]	7.73 [7.56,7.89]	6.40 [6.20,6.60]	4.603 [4.478,4.724]	0.394 [0.378,0.410]	0.309 [0.299,0.319]
AttnLRP (loc)	γ_{conv}=0.25, γ_{lin}=0.05, $\varepsilon_{\text{attn}}$=0.25	0.644 [0.628,0.660]	0.855 [0.825,0.883]	0.809 [0.794,0.825]	7.77 [7.61,7.93]	6.43 [6.23,6.62]	4.582 [4.452,4.706]	0.456 [0.430,0.484]	0.342 [0.324,0.364]
AttnLRP (faith)	γ_{conv}=0.25, γ_{lin}=0.05, $\varepsilon_{\text{attn}}$=0.01	0.629 [0.613,0.645]	0.850 [0.819,0.878]	0.798 [0.783,0.814]	**7.64** [7.47,7.81]	**6.12** [5.92,6.32]	**4.441** [4.314,4.563]	0.416 [0.395,0.438]	0.317 [0.304,0.331]
DAVE (loc/faith)	N=50,$\theta_{\max}$=0, $s_{\max}$=0,$t_{\max}$=0.5	0.491 [0.472,0.510]	0.717 [0.678,0.752]	0.612 [0.591,0.633]	8.12 [7.98,8.25]	7.37 [7.19,7.54]	5.386 [5.256,5.514]	0.781 [0.755,0.810]	0.643 [0.626,0.661]
TIBAV	fixed	0.615 [0.597,0.634]	0.878 [0.851,0.904]	0.782 [0.759,0.805]	8.13 [8.01,8.24]	7.60 [7.46,7.74]	4.906 [4.774,5.033]	1.062 [1.001,1.126]	0.512 [0.486,0.540]
GAE	fixed	0.533 [0.514,0.551]	0.743 [0.708,0.779]	0.649 [0.623,0.677]	8.13 [8.01,8.25]	7.62 [7.47,7.76]	5.098 [4.961,5.231]	0.525 [0.494,0.558]	0.368 [0.346,0.391]
AR (loc/faith)	w_r=0.75	0.447 [0.428,0.467]	0.565 [0.522,0.605]	0.537 [0.510,0.564]	8.44 [8.34,8.53]	8.06 [7.94,8.17]	5.254 [5.128,5.374]	0.208 [0.195,0.222]	0.122 [0.116,0.128]
LIME	N=1000,80 superpixels,ridge	0.535 [0.515,0.553]	0.897 [0.871,0.922]	0.686 [0.656,0.716]	8.36 [8.25,8.46]	7.68 [7.54,7.82]	5.376 [5.242,5.505]	0.937 [0.905,0.972]	0.668 [0.647,0.689]
KernelSHAP	N=2M+2048,80 superpixels	0.565 [0.548,0.583]	**0.961** [0.945,0.977]	0.823$^{\approx}$ [0.799,0.848]	8.29 [8.18,8.40]	7.50 [7.35,7.65]	5.140 [5.012,5.261]	0.897 [0.868,0.928]	0.650 [0.632,0.667]
RG anchor	τ=1	0.509 [0.490,0.528]	0.724 [0.685,0.759]	0.699 [0.678,0.720]	8.19 [8.05,8.31]	6.97 [6.78,7.15]	4.854 [4.732,4.972]	**0.166** [0.161,0.172]	0.120 [0.117,0.124]
RG (loc)	τ=0.4,λ_{sm}=−10, σ^2=5,ε=0.1	**0.711** [0.692,0.730]	0.825 [0.793,0.855]	0.788 [0.768,0.808]	8.27 [8.14,8.39]	7.48 [7.31,7.64]	5.109 [4.987,5.226]	0.729 [0.685,0.777]	0.494 [0.471,0.518]
RG (faith)	τ=0.7,λ_{sm}=−5, σ^2=5,ε=0.1	0.549 [0.531,0.567]	0.770 [0.736,0.804]	0.741 [0.721,0.760]	8.11 [7.97,8.24]	6.78 [6.59,6.97]	4.808 [4.686,4.925]	0.257 [0.245,0.268]	0.174 [0.168,0.179]
RG+Eq anchor	τ=1,N=50	0.520 [0.501,0.539]	0.761 [0.726,0.796]	0.752 [0.729,0.775]	8.41 [8.29,8.52]	7.72 [7.56,7.87]	4.864 [4.742,4.981]	0.170$^{\approx}$ [0.165,0.175]	**0.115** [0.112,0.118]
RG+Eq (loc)	τ=0.5,λ_{sm}=−10, σ^2=2,N=50	0.698 [0.680,0.717]	0.855 [0.827,0.883]	**0.836** [0.815,0.857]	8.39 [8.28,8.50]	7.80 [7.65,7.94]	5.010 [4.889,5.127]	0.441 [0.421,0.463]	0.301 [0.290,0.313]
RG+Eq (faith)	τ=0.7,λ_{sm}=−10, σ^2=2,N=50	0.654 [0.636,0.672]	0.812 [0.781,0.844]	0.820 [0.799,0.841]	8.39 [8.28,8.51]	7.71 [7.56,7.86]	4.856 [4.735,4.974]	0.310 [0.297,0.325]	0.199 [0.194,0.206]

Tab. A.14: VGG-16, ImageNet-S test split: all rows with 95% bootstrap confidence intervals, as in Table A.13.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
Gradient	fixed	0.522 [0.501,0.542]	0.738 [0.701,0.773]	0.688 [0.665,0.711]	14.47 [13.95,14.99]	10.96 [10.55,11.37]	6.470 [6.212,6.731]	1.212 [1.162,1.269]	0.947 [0.917,0.980]
SmGrad (loc/faith)	N=50,σ/range=0.1	0.533 [0.515,0.552]	0.782 [0.747,0.816]	0.723 [0.705,0.741]	14.12 [13.60,14.65]	9.24 [8.85,9.63]	5.768 [5.526,6.007]	0.641 [0.625,0.659]	0.523 [0.515,0.531]
IntGrad (loc/faith)	N_{steps}=20,black baseline	0.550 [0.530,0.571]	0.770 [0.735,0.804]	0.706 [0.683,0.730]	15.39 [14.86,15.95]	11.89 [11.46,12.32]	6.438 [6.174,6.694]	1.132 [1.089,1.179]	0.917 [0.890,0.945]
DeepLIFT	Rescale	0.542 [0.523,0.562]	0.701 [0.664,0.738]	0.654 [0.631,0.677]	13.05 [12.59,13.52]	9.32 [8.96,9.69]	6.049 [5.806,6.291]	1.193 [1.145,1.249]	0.948 [0.919,0.985]
LRP-ε (loc)	ε=0.5	0.613 [0.595,0.632]	0.832 [0.800,0.862]	0.745 [0.725,0.764]	12.63 [12.18,13.10]	8.61 [8.26,8.96]	5.685 [5.458,5.908]	1.173 [1.127,1.223]	0.918 [0.893,0.947]
LRP-ε (faith)	ε=1	0.639 [0.621,0.657]	0.809 [0.777,0.841]	0.744 [0.725,0.762]	**12.43** [11.98,12.89]	8.17 [7.83,8.50]	5.489 [5.270,5.704]	1.100 [1.059,1.143]	0.840 [0.820,0.861]
LRP-γ (loc)	ε=0.5,γ=0.25, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	0.601 [0.584,0.617]	0.692 [0.655,0.729]	0.704 [0.687,0.721]	14.83 [14.28,15.39]	10.07 [9.68,10.46]	**5.010** [4.799,5.219]	0.515 [0.489,0.542]	0.368 [0.353,0.384]
LRP-γ (faith)	ε=0.25,γ=0.1	0.615 [0.598,0.633]	0.793 [0.759,0.827]	0.698 [0.681,0.716]	12.66 [12.18,13.13]	**7.89** [7.56,8.23]	5.078 [4.867,5.285]	0.702 [0.675,0.733]	0.561 [0.543,0.580]
GCAM	`features.29`	0.557 [0.538,0.576]	0.830 [0.798,0.860]	**0.821** [0.798,0.844]	16.36 [15.79,16.93]	12.47 [12.01,12.93]	5.640 [5.407,5.870]	0.430 [0.412,0.449]	0.334 [0.320,0.349]
GCAM++	`features.29`	0.551 [0.533,0.569]	0.823 [0.791,0.855]	0.814$^{\approx}$ [0.792,0.835]	16.68 [16.09,17.27]	12.87 [12.40,13.35]	5.700 [5.462,5.938]	0.314 [0.301,0.327]	0.243 [0.233,0.253]

continued on next page

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
LCAM	`features.29`	0.544 [0.526,0.562]	0.720 [0.683,0.756]	0.747 [0.725,0.768]	15.98 [15.42,16.54]	11.48 [11.05,11.91]	5.341 [5.113,5.565]	0.434 [0.416,0.452]	0.330 [0.319,0.341]
LIME	N=1000,80 superpixels,ridge	0.603 [0.585,0.622]	**0.972** [0.958,0.984]	0.814≈ [0.790,0.837]	16.66 [16.09,17.25]	12.62 [12.17,13.06]	5.785 [5.551,6.015]	0.848 [0.816,0.882]	0.621 [0.599,0.644]
KernelSHAP	N=2M+2048,80 superpixels	0.559 [0.541,0.577]	0.954≈ [0.936,0.970]	0.820≈ [0.795,0.844]	16.68 [16.10,17.27]	12.82 [12.37,13.26]	6.140 [5.912,6.366]	0.994 [0.955,1.035]	0.736 [0.712,0.761]
SG (loc)	λ=−1,σ^2=5,gate on	0.597 [0.577,0.616]	0.770 [0.735,0.804]	0.770 [0.747,0.793]	17.28 [16.69,17.87]	13.32 [12.85,13.79]	6.217 [5.969,6.459]	**0.278** [0.267,0.290]	**0.196** [0.188,0.204]
SG (faith)	λ=−0.3,σ^2=1,gate off	0.598 [0.577,0.618]	0.821 [0.789,0.851]	0.788≈ [0.764,0.812]	17.08 [16.50,17.67]	12.98 [12.53,13.44]	6.278 [6.025,6.524]	0.543 [0.516,0.571]	0.423 [0.401,0.446]
RG anchor	τ=1	0.619 [0.601,0.637]	0.759 [0.724,0.795]	0.691 [0.673,0.709]	14.27 [13.74,14.80]	9.49 [9.10,9.89]	5.179 [4.968,5.388]	0.551 [0.526,0.578]	0.398 [0.385,0.412]
RG (loc)	τ=0.3,ε=0.1	**0.685** [0.665,0.705]	0.736 [0.699,0.772]	0.673 [0.654,0.692]	14.48 [13.95,15.02]	9.64 [9.25,10.03]	5.632 [5.411,5.845]	1.349 [1.273,1.430]	0.879 [0.842,0.918]
RG (faith)	τ=0.8,ε=0.1	0.640 [0.623,0.658]	0.712 [0.673,0.749]	0.688 [0.670,0.706]	14.05 [13.53,14.58]	9.07 [8.70,9.46]	5.117 [4.906,5.325]	0.655 [0.621,0.690]	0.466 [0.448,0.485]

Tab. A.15: ResNet-50, ImageNet-S test split: all rows with 95% bootstrap confidence intervals, as in Table A.13.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
Gradient	fixed	0.481 [0.459,0.504]	0.556 [0.515,0.596]	0.568 [0.540,0.595]	6.33 [6.22,6.44]	5.91 [5.78,6.04]	5.027 [4.900,5.151]	1.448 [1.365,1.535]	1.072 [1.019,1.130]
SmGrad (loc)	N=50,σ/range=0.15	0.549 [0.530,0.569]	0.789 [0.756,0.821]	0.738 [0.719,0.755]	6.05 [5.93,6.17]	5.17 [5.02,5.31]	4.429 [4.303,4.554]	0.735 [0.718,0.753]	0.600 [0.593,0.609]
SmGrad (faith)	N=50,σ/range=0.1	0.544 [0.525,0.564]	0.773 [0.740,0.807]	0.735 [0.715,0.753]	6.06 [5.94,6.19]	5.19 [5.04,5.34]	4.489 [4.361,4.612]	0.726 [0.706,0.748]	0.586 [0.577,0.597]
IntGrad (loc)	N_{steps}=100,black baseline	0.510 [0.488,0.532]	0.635 [0.596,0.676]	0.600 [0.574,0.626]	6.27 [6.16,6.38]	5.83 [5.69,5.96]	4.949 [4.820,5.072]	1.203 [1.152,1.257]	0.947 [0.917,0.979]
IntGrad (faith)	N_{steps}=50,black baseline	0.507 [0.484,0.529]	0.604 [0.563,0.642]	0.595 [0.568,0.622]	6.28 [6.16,6.39]	5.82 [5.68,5.95]	4.952 [4.822,5.075]	1.207 [1.157,1.261]	0.951 [0.922,0.983]
DeepLIFT	Rescale	0.543 [0.523,0.564]	0.727 [0.690,0.765]	0.653 [0.629,0.677]	5.97 [5.85,6.10]	5.29 [5.15,5.43]	4.515 [4.390,4.637]	1.273 [1.203,1.351]	1.000 [0.955,1.052]
LRP-ε (loc)	ε=0.5	0.603 [0.584,0.622]	0.775 [0.740,0.809]	0.714 [0.694,0.734]	5.94 [5.81,6.06]	5.14 [4.99,5.28]	4.318 [4.200,4.433]	1.246 [1.203,1.292]	0.982 [0.960,1.006]
LRP-ε (faith)	ε=2	0.637 [0.618,0.656]	0.699 [0.662,0.736]	0.683 [0.663,0.702]	5.94 [5.81,6.06]	4.91 [4.76,5.05]	4.280 [4.164,4.394]	1.262 [1.224,1.301]	0.967 [0.948,0.986]
LRP-γ (loc)	ε=0.5,γ=2,$z^{\mathscr{B}}/\gamma/\varepsilon$ composite	0.711 [0.695,0.726]	0.842 [0.812,0.873]	0.822 [0.807,0.836]	5.69 [5.55,5.83]	4.53 [4.38,4.68]	3.935≈ [3.814,4.053]	0.633 [0.601,0.666]	0.486 [0.464,0.507]
LRP-γ (faith)	ε=0.5,γ=0.5, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	0.700 [0.684,0.716]	0.834 [0.802,0.864]	0.796 [0.781,0.812]	**5.65** [5.51,5.79]	**4.43** [4.28,4.59]	**3.931** [3.809,4.050]	0.745 [0.714,0.777]	0.584 [0.562,0.605]
GCAM	`layer4`	0.664 [0.645,0.683]	0.873 [0.844,0.899]	0.850≈ [0.830,0.870]	6.42 [6.32,6.52]	6.15 [6.03,6.26]	4.344 [4.216,4.469]	0.292 [0.274,0.312]	0.207 [0.195,0.220]
GCAM++	`layer4`	0.627 [0.607,0.646]	0.834 [0.804,0.864]	0.836 [0.815,0.857]	6.43 [6.34,6.52]	6.18 [6.07,6.29]	4.483 [4.356,4.609]	0.294 [0.278,0.310]	0.207 [0.197,0.219]
LCAM	`layer4`	0.649 [0.630,0.668]	0.869 [0.841,0.897]	**0.851** [0.831,0.871]	6.44 [6.34,6.53]	6.18 [6.07,6.29]	4.437 [4.311,4.559]	**0.274** [0.260,0.288]	**0.194** [0.184,0.204]
LIME	N=1000,80 superpixels,ridge	0.553 [0.533,0.572]	0.926 [0.903,0.947]	0.716 [0.687,0.744]	6.37 [6.26,6.46]	5.99 [5.88,6.11]	4.759 [4.630,4.881]	0.916 [0.891,0.941]	0.672 [0.655,0.690]
KernelSHAP	N=2M+2048,80 superpixels	0.558 [0.540,0.575]	**0.965** [0.949,0.979]	0.818 [0.793,0.841]	6.26 [6.16,6.36]	5.81 [5.69,5.92]	4.727 [4.601,4.849]	0.924 [0.896,0.954]	0.688 [0.670,0.708]
RG anchor	τ=1	0.693 [0.676,0.710]	0.752 [0.717,0.788]	0.756 [0.738,0.774]	5.84 [5.70,5.97]	4.71 [4.56,4.86]	4.017 [3.898,4.133]	0.708 [0.651,0.775]	0.508 [0.481,0.541]
RG (loc)	τ=0.9,ε=1	**0.719** [0.701,0.737]	0.763 [0.727,0.796]	0.760 [0.741,0.778]	5.85 [5.72,5.98]	4.70 [4.55,4.85]	4.047 [3.929,4.163]	0.798 [0.749,0.855]	0.560 [0.535,0.586]

continued on next page

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
RG (faith)	τ=1,λ=−1,σ^2=1, ε=0.25	0.674 [0.656,0.691]	0.743 [0.708,0.779]	0.738 [0.719,0.756]	5.88 [5.74,6.01]	4.74 [4.59,4.89]	4.030 [3.911,4.144]	0.685 [0.629,0.755]	0.489 [0.460,0.525]

A.4.3 Trade-off sweeps

Table A.17 includes one-at-a-time sweeps of τ and λ_{sm} around the `loc`-selected ViT-B/16 RG configuration. These sweep rows are evaluated on the test split only after selection and never feed back into it.

A.4.4 Cross-Dataset Evaluation: Pascal VOC 2012

This part details the protocol behind the ViT-B/16 cross-dataset row of the summary table in Section 7.5 (all rows in Table A.16). The goal is twofold: (i) test whether the RG / RG+Eq configurations selected on ImageNet-S transfer to a different dataset *without* re-tuning, and (ii) put baselines on equal footing by re-running the same per-method validation selection on the new pool.

Dataset. We use Pascal VOC 2012 (Everingham et al., 2009) segmentation annotations: the trainval pool of 2,913 images with pixel-accurate `SegmentationClass` masks across 20 foreground classes plus background. Each image is expanded to one (image, foreground-class) sample per distinct VOC class label present in its mask, yielding 3,422 total samples after dropping the `person` class (no faithful ImageNet-1K target exists for it) and discarding sliver masks whose pixel area on the *original* (pre-resize) annotation is below 64 pixels. Each (image, class) sample carries the same input image and an explanation target equal to the most semantically central ImageNet-1K index for that VOC class (e.g. `horse` → N02389026 SORREL; `tvmonitor` → N03782006 MONITOR); the curated mapping is part of the released code.

Per-image disjoint val/test split. The trainval pool is partitioned randomly with seed 42 at the *image* level (not the (image, class) level) into a 1:5 val/test split: the trainval stems are shuffled deterministically and the first 1/6 is assigned to val. All (image, class) samples for a given image stay in the same partition, so leakage between val tuning and test evaluation is structurally impossible. As on ImageNet-S, samples whose object mask lies entirely outside the 224×224 center crop are excluded after the split (18 val and 105 test samples). The resulting partitions:

	Images	(image, class) samples
val (tuning)	460	556
test (eval)	2,293	2,743

Pixel masks for localization. AL, PG, and Top-k Intersection are computed against the per-(image, class) *binary VOC2012 segmentation pixel mask* for that class, not bounding boxes. The mask is resized via NEAREST interpolation and center-cropped to 224×224 to match the input pipeline (`Resize(256)` → `CenterCrop(224)`), then passed verbatim into Quantus (Hedström et al., 2023) alongside the attribution map. This avoids the well-known bbox over-credit issue (Selvaraju et al., 2017) where a method that puts mass anywhere inside the box wins the metric; pixel masks reward only mass on the actual foreground object.

Tuning protocol. The protocol mirrors the ImageNet-S protocol of Section A.3, with one deliberate asymmetry: RG and RG+Eq do *not* get a fresh grid search on VOC.

- **Baselines.** Every row of the per-method ViT-B/16 grid of Table A.9 is evaluated on the 556-sample val pool, and the `loc` and `faith` configurations are selected per method with the criteria of Section A.3.5. The selected rows are re-evaluated on the 2,743-sample test pool.
- **RG / RG+Eq (transfer test).** The anchor and the `loc`- and `faith`-selected ImageNet-S configurations are evaluated on the VOC test pool unchanged, without any VOC-side re-tuning.

To save compute, VOC is evaluated on localization and faithfulness only (no MaxS/AvgS and no runtime column). Any tuning advantage flows to the baselines, not to us. Wins, ties and losses are counted per metric with the tests of Section A.3.6.

Tab. A.16: ViT-B/16 on the Pascal VOC 2012 test pool (2,743 samples): all rows with 95% bootstrap confidence intervals. RG / RG+Eq use the ImageNet-S-selected configurations without re-tuning; baselines are selected on the VOC val pool. Only localization and faithfulness metrics were evaluated; the unmeasured MaxS and AvgS columns are omitted. Metrics as in Table 7.4; configuration symbols and row labels: Section A.4.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
Gradient	fixed	0.295 [0.286,0.304]	0.360 [0.343,0.378]	0.355 [0.344,0.367]	3.45 [3.32,3.57]	3.17 [3.05,3.28]	2.316 [2.234,2.398]
SmGrad (loc)	N=50,σ/range=0.1	0.344 [0.335,0.353]	0.557 [0.539,0.576]	0.511 [0.500,0.521]	3.25 [3.13,3.37]	2.70 [2.60,2.81]	1.940 [1.870,2.010]
SmGrad (faith)	N=50,σ/range=0.15	0.347 [0.338,0.355]	0.555 [0.537,0.574]	0.511 [0.500,0.522]	3.24 [3.12,3.35]	2.67 [2.57,2.77]	1.907 [1.841,1.975]
IntGrad (loc)	N_{steps}=300,black baseline	0.292 [0.283,0.301]	0.388 [0.369,0.406]	0.358 [0.346,0.369]	3.43 [3.31,3.56]	3.21 [3.10,3.32]	2.278 [2.197,2.356]
IntGrad (faith)	N_{steps}=100,black baseline	0.292 [0.283,0.301]	0.389 [0.370,0.407]	0.358 [0.346,0.369]	3.44 [3.31,3.56]	3.21 [3.10,3.32]	2.279 [2.198,2.357]
LRP-ε (loc/faith)	ε=0.1	0.378 [0.368,0.387]	0.604 [0.586,0.622]	0.518 [0.507,0.529]	3.07 [2.96,3.19]	2.63 [2.54,2.73]	1.923 [1.857,1.990]
LRP-γ (loc/faith)	ε=0.1,γ=1	0.403 [0.394,0.412]	0.591 [0.572,0.609]	0.545 [0.533,0.556]	2.99 [2.88,3.10]	2.37 [2.27,2.46]	1.694≈ [1.634,1.756]
AttnLRP (loc/faith)	γ_{conv}=0.25, γ_{lin}=0.05, $\varepsilon_{\text{attn}}$=0.05	0.404 [0.394,0.413]	0.602 [0.583,0.620]	0.542 [0.530,0.553]	**2.91** [2.80,3.01]	**2.31** [2.22,2.40]	**1.690** [1.631,1.750]
DAVE (loc)	N=50,$\theta_{\max}$=0, $s_{\max}$=0,$t_{\max}$=0.25	0.334 [0.325,0.342]	0.547 [0.529,0.565]	0.456 [0.445,0.467]	3.37 [3.25,3.49]	3.01 [2.90,3.12]	2.172 [2.097,2.247]
DAVE (faith)	N=50,$\theta_{\max}$=0, $s_{\max}$=0,$t_{\max}$=0.5	0.332 [0.323,0.341]	0.535 [0.517,0.553]	0.454 [0.444,0.465]	3.37 [3.25,3.49]	2.98 [2.88,3.09]	2.153 [2.080,2.226]
TIBAV	fixed	0.413 [0.403,0.423]	0.729 [0.713,0.746]	**0.575** [0.561,0.589]	3.04 [2.92,3.15]	2.67 [2.56,2.78]	1.731 [1.666,1.798]
GAE	fixed	0.355 [0.346,0.364]	0.712 [0.695,0.729]	0.548 [0.535,0.561]	3.00 [2.88,3.11]	2.63 [2.52,2.74]	1.754 [1.685,1.823]
AR (loc/faith)	w_r=0.75	0.272 [0.264,0.281]	0.471 [0.452,0.489]	0.411 [0.398,0.424]	3.57 [3.45,3.68]	3.41 [3.30,3.51]	2.146 [2.075,2.216]
LIME	N=1000,80 superpixels,ridge	0.351 [0.342,0.360]	0.785≈ [0.769,0.799]	0.523 [0.508,0.537]	3.45 [3.33,3.56]	3.21 [3.11,3.32]	2.121 [2.051,2.191]
KernelSHAP	N=2M+2048,80 superpixels	0.341 [0.332,0.350]	**0.793** [0.778,0.808]	0.561≈ [0.546,0.576]	3.36 [3.25,3.48]	3.03 [2.93,3.13]	2.077 [2.009,2.147]
RG anchor	τ=1	0.323 [0.314,0.332]	0.517 [0.498,0.536]	0.489 [0.477,0.501]	3.30 [3.18,3.41]	2.67 [2.57,2.77]	1.980 [1.914,2.046]
RG (loc)	τ=0.4,λ_{sm}=−10, σ^2=5,ε=0.1	**0.459** [0.448,0.471]	0.562 [0.544,0.580]	0.512 [0.500,0.526]	3.44 [3.33,3.56]	3.06 [2.96,3.17]	2.152 [2.082,2.222]

continued on next page

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	τ=0.7,λ_{sm}=−5, σ^2=5,ε=0.1	0.351 [0.342,0.360]	0.538 [0.520,0.557]	0.508 [0.496,0.520]	3.27 [3.15,3.38]	2.63 [2.53,2.73]	1.965 [1.900,2.032]
RG+Eq anchor	τ=1,N=50	0.329 [0.320,0.338]	0.574 [0.556,0.592]	0.534 [0.521,0.548]	3.43 [3.31,3.55]	3.06 [2.95,3.17]	1.988 [1.922,2.055]
RG+Eq (loc)	τ=0.5,λ_{sm}=−10, σ^2=2,N=50	0.456 [0.445,0.467]	0.599 [0.581,0.617]	0.571≈ [0.556,0.585]	3.45 [3.33,3.57]	3.14 [3.03,3.25]	2.085 [2.017,2.154]
RG+Eq (faith)	τ=0.7,λ_{sm}=−10, σ^2=2,N=50	0.424 [0.414,0.434]	0.591 [0.573,0.609]	0.562≈ [0.548,0.576]	3.44 [3.32,3.55]	3.08 [2.98,3.19]	2.004 [1.937,2.071]

A.4.5 One-factor ablations

Tables A.17–A.19 vary one parameter at a time around each validation-selected configuration of RG, RG+Eq and SG (bold: the selected value), over the values of the declared grid; all other parameters stay at their selected values. The rows are evaluated on the 565-image test split with the localization and faithfulness metrics and never feed back into the selection. On ViT-B/16, "+Eq" evaluates RG+Eq at the parameters of the selected RG row. Configurations that are inadmissible on the validation split (Section A.3.5) are not included.

Tab. A.17: ViT-B/16, ImageNet-S test split: one-factor ablations around the selected configurations. Center: the selected configuration (criterion in parentheses) around which the parameter in the Factor column is varied over the Value column, all other parameters fixed; bold: the center's own value. Symbols: Section A.4.

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	τ	0.3	0.705	0.802	0.731	8.34	7.60	5.195
		0.4	0.688	0.827	0.773	8.26	7.41	5.095
		0.5	0.665	0.821	0.793	8.22	7.20	4.956
		0.6	0.613	0.814	0.774	8.13	6.88	4.820
		0.7	0.549	0.770	0.741	8.11	6.78	4.808
		0.8	0.507	0.742	0.712	8.14	6.85	4.881
		0.9	0.482	0.697	0.683	8.21	7.02	4.983
		1	0.467	0.648	0.648	8.29	7.25	5.115
RG (faith)	λ_{sm}	-50	0.467	0.556	0.541	8.51	8.13	5.928
		-10	0.556	0.777	0.763	8.16	6.97	4.876
		-5	0.549	0.770	0.741	8.11	6.78	4.808
		-2	0.547	0.743	0.720	8.11	6.81	4.793
		-1	0.546	0.733	0.713	8.11	6.83	4.794
		0	0.546	0.727	0.706	8.13	6.87	4.804
RG (faith)	σ^2	1	0.558	0.738	0.703	8.16	6.90	4.790
		2	0.555	0.743	0.719	8.13	6.81	4.783
		5	0.549	0.770	0.741	8.11	6.78	4.808
		10	0.545	0.765	0.749	8.12	6.82	4.833
RG (faith)	ε	**0.1**	0.549	0.770	0.741	8.11	6.78	4.808
		0.25	0.595	0.804	0.778	8.16	6.95	4.846
		0.5	0.621	0.809	0.792	8.24	7.21	4.944
		1	0.630	0.812	0.794	8.29	7.43	5.051
RG (faith)	+Eq (N=50)	on	0.557	0.800	0.791	8.38	7.69	4.816

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (loc)	τ	0.3	0.723	0.816	0.746	8.33	7.64	5.201
		0.4	0.711	0.825	0.788	8.27	7.48	5.109
		0.5	0.691	0.830	0.812	8.23	7.31	4.969
		0.6	0.632	0.827	0.801	8.16	7.00	4.847
		0.7	0.556	0.777	0.763	8.16	6.97	4.876
		0.8	0.506	0.743	0.715	8.22	7.13	5.013
		0.9	0.477	0.683	0.673	8.29	7.28	5.143
		1	0.461	0.628	0.629	8.36	7.42	5.247
RG (loc)	λ_{sm}	-50	0.520	0.543	0.528	8.52	7.99	5.745
		-10	0.711	0.825	0.788	8.27	7.48	5.109
		-5	0.688	0.827	0.773	8.26	7.41	5.095
		-2	0.676	0.827	0.764	8.26	7.39	5.086
		-1	0.673	0.818	0.761	8.26	7.38	5.080
		0	0.671	0.818	0.758	8.26	7.38	5.076
RG (loc)	σ^2	1	0.707	0.823	0.769	8.23	7.35	4.986
		2	0.712	0.823	0.780	8.24	7.39	5.032
		5	0.711	0.825	0.788	8.27	7.48	5.109
		10	0.705	0.837	0.787	8.31	7.58	5.168
RG (loc)	ε	**0.1**	0.711	0.825	0.788	8.27	7.48	5.109
		0.25	0.709	0.842	0.780	8.29	7.55	5.155
		0.5	0.708	0.828	0.776	8.30	7.57	5.171
		1	0.708	0.823	0.773	8.31	7.59	5.181
RG (loc)	+Eq (N=50)	on	0.711	0.846	0.842	8.38	7.82	5.077
RG+Eq (faith)	τ	0.3	0.721	0.805	0.786	8.37	7.75	5.129
		0.4	0.712	0.832	0.822	8.39	7.81	5.049
		0.5	0.698	0.855	0.836	8.39	7.80	5.010
		0.6	0.681	0.846	0.833	8.39	7.77	4.943
		0.7	0.654	0.812	0.820	8.39	7.71	4.856
		0.8	0.610	0.805	0.803	8.38	7.70	4.814
		0.9	0.567	0.796	0.787	8.41	7.71	4.825
		1	0.534	0.777	0.772	8.42	7.74	4.876
RG+Eq (faith)	λ_{sm}	**-10**	0.654	0.812	0.820	8.39	7.71	4.856
		0	0.613	0.814	0.815	8.37	7.73	4.914
RG+Eq (loc)	τ	0.3	0.721	0.805	0.786	8.37	7.75	5.129
		0.4	0.712	0.832	0.822	8.39	7.81	5.049
		0.5	0.698	0.855	0.836	8.39	7.80	5.010
		0.6	0.681	0.846	0.833	8.39	7.77	4.943
		0.7	0.654	0.812	0.820	8.39	7.71	4.856
		0.8	0.610	0.805	0.803	8.38	7.70	4.814
		0.9	0.567	0.796	0.787	8.41	7.71	4.825
		1	0.534	0.777	0.772	8.42	7.74	4.876
RG+Eq (loc)	λ_{sm}	**-10**	0.698	0.855	0.836	8.39	7.80	5.010
		0	0.652	0.848	0.828	8.40	7.85	5.073

Tab. A.18: VGG-16, ImageNet-S test split: one-factor ablations around the selected configurations, as in Table A.17.

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	τ	0.3	0.685	0.736	0.673	14.48	9.64	5.632
		0.4	0.645	0.657	0.641	14.40	9.36	5.574
		0.5	0.626	0.653	0.628	14.27	9.14	5.451
		0.6	0.633	0.651	0.646	14.07	9.00	5.258
		0.7	0.643	0.671	0.674	14.03	8.98	5.142
		0.8	0.640	0.712	0.688	14.05	9.07	5.117
		0.9	0.630	0.742	0.693	14.05	9.17	5.126
		1	0.618	0.749	0.695	14.06	9.25	5.137
RG (faith)	λ	-5	0.518	0.458	0.536	17.60	13.82	7.362
		-2	0.623	0.604	0.631	15.90	11.58	5.852
		-1	0.636	0.676	0.661	15.03	10.35	5.404
		-0.3	0.640	0.694	0.676	14.54	9.70	5.231
		0	0.640	0.712	0.688	14.05	9.07	5.117
RG (faith)	ε	**0.1**	0.640	0.712	0.688	14.05	9.07	5.117
		0.25	0.639	0.692	0.684	14.18	9.23	5.146
		0.5	0.638	0.690	0.680	14.34	9.46	5.183
		1	0.639	0.690	0.676	14.63	9.79	5.249
RG (loc)	τ	**0.3**	0.685	0.736	0.673	14.48	9.64	5.632
		0.4	0.645	0.657	0.641	14.40	9.36	5.574
		0.5	0.626	0.653	0.628	14.27	9.14	5.451
		0.6	0.633	0.651	0.646	14.07	9.00	5.258
		0.7	0.643	0.671	0.674	14.03	8.98	5.142
		0.8	0.640	0.712	0.688	14.05	9.07	5.117
		0.9	0.630	0.742	0.693	14.05	9.17	5.126
		1	0.618	0.749	0.695	14.06	9.25	5.137
RG (loc)	λ	-5	0.561	0.543	0.555	17.05	13.21	6.898
		-2	0.657	0.696	0.634	15.31	10.80	5.885
		-1	0.674	0.735	0.656	14.85	10.13	5.731
		-0.3	0.680	0.727	0.666	14.66	9.87	5.675
		0	0.685	0.736	0.673	14.48	9.64	5.632
RG (loc)	ε	**0.1**	0.685	0.736	0.673	14.48	9.64	5.632
		0.25	0.691	0.747	0.673	14.66	9.91	5.659
		0.5	0.697	0.742	0.672	14.87	10.19	5.705
		1	0.702	0.742	0.671	15.13	10.55	5.767
SG (faith)	λ	-2	0.614	0.710	0.636	18.17	14.83	6.941
		-1	0.609	0.750	0.697	17.91	13.87	6.517
		-0.3	0.598	0.821	0.788	17.08	12.98	6.278
		0	0.547	0.781	0.764	17.63	14.35	6.789
SG (faith)	σ^2	**1**	0.598	0.821	0.788	17.08	12.98	6.278
		2	0.601	0.811	0.765	17.30	13.07	6.323
		5	0.604	0.798	0.731	17.66	13.50	6.417
		10	0.607	0.763	0.706	17.86	13.77	6.500
SG (faith)	gate $\tilde{\Phi}$	**off**	0.598	0.821	0.788	17.08	12.98	6.278

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
		on	0.568	0.823	0.805	17.30	13.74	6.428
SG (loc)	λ	-5	0.568	0.550	0.558	18.17	14.31	7.509
		-2	0.620	0.703	0.681	17.84	13.63	6.404
		-1	0.597	0.770	0.770	17.28	13.32	6.217
		-0.3	0.566	0.830	0.809	17.33	13.75	6.452
		0	0.554	0.827	0.802	17.55	14.16	6.618
SG (loc)	σ^2	1	0.596	0.793	0.766	17.32	13.35	6.246
		2	0.597	0.786	0.766	17.31	13.34	6.232
		5	0.597	0.770	0.770	17.28	13.32	6.217
		10	0.595	0.772	0.776	17.25	13.30	6.206
SG (loc)	gate $\tilde{\Phi}$	off	0.614	0.696	0.628	18.19	15.00	7.050
		on	0.597	0.770	0.770	17.28	13.32	6.217

Tab. A.19: ResNet-50, ImageNet-S test split: one-factor ablations around the selected configurations, as in Table A.17.

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	τ	0.3	0.688	0.727	0.684	6.10	5.10	4.357
		0.4	0.676	0.712	0.692	6.08	5.03	4.340
		0.5	0.677	0.720	0.706	6.06	5.00	4.308
		0.6	0.673	0.704	0.711	5.99	4.96	4.270
		0.7	0.667	0.680	0.707	5.95	4.89	4.219
		0.8	0.665	0.688	0.711	5.89	4.85	4.134
		0.9	0.674	0.720	0.728	5.88	4.79	4.075
		1	0.674	0.743	0.738	5.88	4.74	4.030
RG (faith)	λ	-5	0.667	0.740	0.730	5.90	4.81	4.034
		-2	0.672	0.740	0.736	5.89	4.75	4.034
		-1	0.674	0.743	0.738	5.88	4.74	4.030
		-0.3	0.674	0.747	0.739	5.87	4.75	4.026
		0	0.675	0.747	0.740	5.86	4.74	4.023
RG (faith)	σ^2	**1**	0.674	0.743	0.738	5.88	4.74	4.030
		2	0.673	0.742	0.737	5.87	4.74	4.032
		5	0.671	0.736	0.736	5.88	4.75	4.031
		10	0.670	0.747	0.734	5.89	4.76	4.031
RG (faith)	ε	0.1	0.658	0.745	0.722	5.90	4.79	4.049
		0.25	0.674	0.743	0.738	5.88	4.74	4.030
		0.5	0.692	0.747	0.754	5.86	4.72	4.023
		1	0.712	0.766	0.761	5.85	4.72	4.038
RG (loc)	τ	0.3	0.733	0.761	0.719	6.03	5.04	4.272
		0.4	0.717	0.758	0.718	6.02	4.96	4.243
		0.5	0.709	0.740	0.720	5.98	4.92	4.216
		0.6	0.710	0.733	0.728	5.93	4.87	4.172
		0.7	0.716	0.745	0.741	5.91	4.80	4.125
		0.8	0.720	0.761	0.752	5.87	4.72	4.078
		0.9	0.719	0.763	0.760	5.85	4.70	4.047

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
		1	0.713	0.770	0.763	5.85	4.70	4.035
RG (loc)	λ	-5	0.708	0.765	0.745	5.89	4.80	4.069
		-2	0.716	0.773	0.755	5.87	4.74	4.055
		-1	0.718	0.777	0.757	5.86	4.72	4.054
		-0.3	0.719	0.768	0.758	5.86	4.72	4.050
		0	0.719	0.763	0.760	5.85	4.70	4.047
RG (loc)	ε	0.1	0.656	0.712	0.712	5.89	4.84	4.106
		0.25	0.674	0.717	0.730	5.87	4.77	4.067
		0.5	0.696	0.736	0.749	5.84	4.70	4.042
		1	0.719	0.763	0.760	5.85	4.70	4.047

A.4.6 Pointing game at matched granularity

The pointing game of Quantus counts a hit whenever any pixel attaining the map maximum lies in the mask. For piecewise-constant maps (superpixel methods; attention maps on the 14×14 patch grid) the maximum is a region of hundreds to thousands of pixels, for pixel-level maps usually a single pixel. Table A.20 reports, for the reported rows, the median size of the maximum set, the expected hit rate of a single maximum pixel drawn uniformly from that set (PG_{1px}), and the pointing game after average-pooling every map to the same 16×16-pixel grid (PG_{16px}). Drawing a single maximum pixel removes most of the advantage of the superpixel methods LIME and KernelSHAP. At the common 16×16 grid the leading methods move closer together: on ViT-B/16 LRP-γ and AttnLRP lead, on VGG-16 the localization-selected SG row and the faithfulness-selected RG row are not significantly below the best baseline (Grad-CAM++), and on ResNet-50 Grad-CAM, LayerCAM and LRP-γ lead (McNemar tests against the best baseline row).

Tab. A.20: Pointing game under the Quantus tie rule (PG, as in the main tables) and at matched granularity, ImageNet-S test split. max. set: median number of pixels attaining the map maximum; PG_{1px}: expected hit rate of one pixel drawn uniformly from that set; PG_{16px}: PG after average-pooling the map to 16×16-pixel cells. Rows: the `loc`-selected or fixed configuration of every method, plus the `faith` rows of our games. **Bold**: best per model and column.

Model	Method	PG	max. set (px)	PG_{1px}	PG_{16px}
ViT-B/16	Gradient	0.361	1	0.361	0.382
	SmGrad	0.706	1	0.706	0.835
	IntGrad	0.513	1	0.513	0.543
	LRP-ε	0.834	1	0.834	0.904
	LRP-γ	0.873	1	**0.873**	**0.968**
	AttnLRP	0.855	1	0.855	0.926
	DAVE	0.717	1	0.717	0.781
	TIBAV	0.878	256	0.784	0.878
	GAE	0.743	256	0.670	0.743
	AR	0.565	256	0.516	0.565
	LIME	0.897	1104	0.693	0.786
	KernelSHAP	**0.961**	1031	0.833	0.903
	RG (faith)	0.770	1	0.770	0.899
	RG	0.825	1	0.825	0.888
	RG+Eq (faith)	0.812	1	0.812	0.919
	RG+Eq	0.855	1	0.855	0.919
VGG-16	Gradient	0.738	1	0.738	0.812
	SmGrad	0.782	1	0.782	0.915
	IntGrad	0.770	1	0.770	0.860
	DeepLIFT	0.701	1	0.701	0.784
	LRP-ε	0.832	1	**0.832**	0.906
	LRP-γ	0.692	1	0.692	0.908
	GCAM	0.830	1	0.823	0.920
	GCAM++	0.823	1	0.815	**0.924**
	LCAM	0.720	1	0.715	0.834
	LIME	**0.972**	1103	0.826	0.899
	KernelSHAP	0.954	1011	0.826	0.894
	SG (faith)	0.821	1	0.821	0.883
	SG	0.770	1	0.770	0.908
	RG (faith)	0.712	1	0.712	0.915
	RG	0.736	1	0.736	0.873
ResNet-50	Gradient	0.556	1	0.556	0.630
	SmGrad	0.789	1	0.789	0.919
	IntGrad	0.635	1	0.635	0.743
	DeepLIFT	0.727	1	0.727	0.793
	LRP-ε	0.775	1	0.775	0.880
	LRP-γ	0.842	1	0.842	0.958
	GCAM	0.873	1	**0.862**	**0.959**
	GCAM++	0.834	1	0.827	0.945
	LCAM	0.869	1	0.860	0.958
	LIME	0.926	1058	0.729	0.844
	KernelSHAP	**0.965**	1014	0.825	0.912
	RG (faith)	0.743	1	0.743	0.903
	RG	0.763	1	0.763	0.912

A.4.7 Gradient-fallback attention

The RG backward operates by default through a routing rule that holds the attention policy fixed and routes relevance only through the Value path, with zero gradient on Q, K (Value-only routing conditioned on the realized attention matrix, as in CP-LRP (Ali et al., 2022); see Section 5.4.4). The natural alternative is a gradient fallback: pure autograd through the attention block, recovering the zennit-style ε-LRP attention treatment. Table A.21 swaps the attention rule at the $\alpha\beta$-LRP anchor and at the two validation-selected RG configurations and evaluates both rules on the 565-image test split. The softmax risk λ_{sm} acts on the Value-path routing and is therefore inactive under the gradient fallback. The gradient fallback lowers all three localization metrics at every configuration, AL and TK significantly throughout, and it does not improve faithfulness at any configuration: PF_5, PF_{20} and Sel are worse in every cell, significantly so in four of the nine.

Tab. A.21: ViT-B/16, RG: attention rule ablation on the ImageNet-S test split (565 images), at the anchor and at the loc- and faith-selected configurations. Each cell shows the gradient fallback on top and the composite Value rule of Theorem 5.6 (gray) below; configuration symbols: Section A.4; †: paired difference significant (paired bootstrap / McNemar for PG, Holm-corrected over the table, $p < 0.05$).

Center	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
anchor (τ=1)	0.504† 0.509	0.722 0.724	0.694† 0.699	8.19 8.19	6.98† 6.97	4.877† 4.854
loc (τ=0.4,λ_{sm}=−10,σ^2=5,ε=0.1)	0.624† 0.711	0.743† 0.825	0.672† 0.788	8.31 8.27	7.50 7.48	5.364† 5.109
faith (τ=0.7,λ_{sm}=−5,σ^2=5,ε=0.1)	0.543† 0.549	0.729† 0.770	0.706† 0.741	8.13 8.11	6.87† 6.78	4.816 4.808

A.4.8 Qualitative comparison

Figure A.2 compares attribution maps for six ImageNet-S examples on ViT-B/16. Each column shows a method at its `loc`-selected configuration (Section A.3.5), or at its fixed configuration if the method has no tunable parameters (Gradient, TIBAV): RG with τ=0.4,λ_{sm}=−10,σ^2=5,ε=0.1 and RG+Eq with τ=0.5,λ_{sm}=−10,σ^2=2,N=50, both with the merged attention–Value rule, i.e. the composite rule of Theorem 5.6, which routes through the conditioned attention block as one map $X \mapsto O$; SmoothGrad with N=50,σ/range=0.2; Integrated Gradients with N_{steps}=100 and a black baseline; DAVE with N=50,$\theta_{\max}$=0,$s_{\max}$=0,$t_{\max}$=0.5; LRP-γ with ε=0.1,γ=2; and AttnLRP with γ_{conv}=0.25,γ_{lin}=0.05,$\varepsilon_{\text{attn}}$=0.25. As in the evaluation, every map explains the ImageNet-S label, which is the top-1 prediction for all images except the kuvasz (predicted: English setter). The first image is from the test split, the other five are from the 50-image validation split used for the selection. Heatmaps show the raw evaluator-side maps, before the absolute value and max-abs normalization of Section A.3.3 (the 14×14 TIBAV map is upsampled by nearest neighbor, as for the metrics): red marks positive and blue negative attribution; each sign is divided by its 99th percentile over the image and clipped, so the top 1% of pixels saturate and concentrated maps stay legible in print; the same display transform is used for every method.

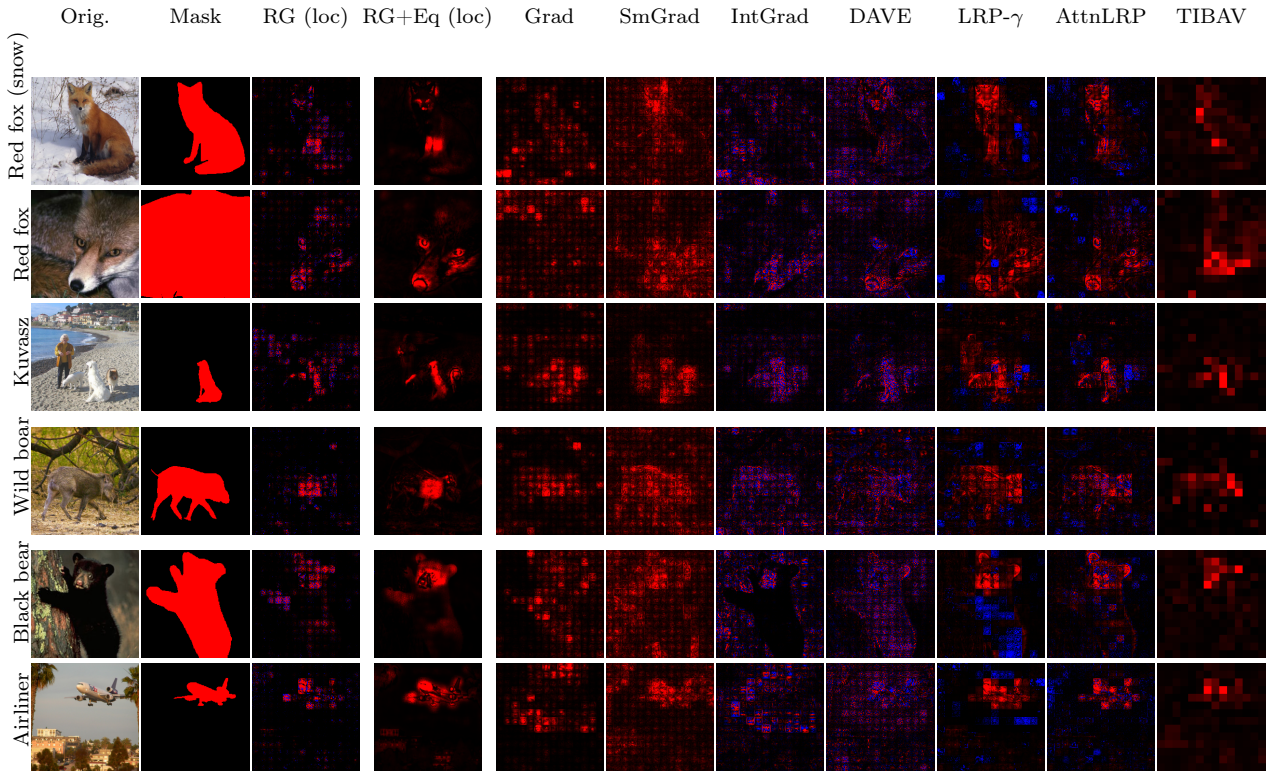


Fig. A.2: Dense qualitative comparison on six ImageNet-S examples (ViT-B/16). Columns: Original; ImageNet-S pixel Mask; RG and RG+Eq (merged attention–Value rule); Gradient, SmoothGrad, Integrated Gradients, DAVE, LRP-γ, AttnLRP and TIBAV. Every method is shown at its `loc`-selected configuration, Gradient and TIBAV at their fixed configurations (values in the text). Red: positive, blue: negative attribution, each sign divided by its 99th percentile over the image and clipped.

A.5 Formal Verification and Mathematical Audit

The accompanying thesis repository contains the TeX sources, pinned links to both official code repositories, and a Lean 4 project with mathlib pinned to version 4.35.0. The formalization makes the assumptions of each checked statement explicit. It remains partial: a related lemma does not certify an entire theorem or either paper.

Numbered `Lean_1`, `Lean_2`, ... references appear in the brackets of theorem and lemma headings, following the convention of the planar-features paper. Each links to the checked declaration or components of the corresponding statement; the coverage map distinguishes these cases. The exact assumptions and remaining parts are recorded by source label in `formalization/coverage_map.json`. Selected equation numbers also link to their checked identities or definitions. All links identify a fixed Git commit and the actual declaration line, following the TeX-linking approach of arXiv:2610.01728; subsequent edits to the default branch cannot move these targets.

Both papers use the same finite, varying-width MLP parameter definition in Lean. A network is either an identity output or an affine layer followed by a smaller network; its evaluators specify the ordinary activation or a pair activation. Local preservation of the difference is the only algebraic requirement for a valid pair activation, and induction proves recovery of the complete network. The same definition

supports the ReLU, mixing, and Softplus developments. A separate affine readout accounts for an ungated target logit.

A forward trace records the actual gate decisions. A sensitivity row is a linear function taking an input perturbation to the corresponding scalar output change; a pair of rows gives the partial sensitivities with respect to the two input streams. Their half-difference is the derivative along the balanced lift $(x/2, -x/2)$. At strict gate comparisons, the recursively constructed Jacobian is proved to be the actual Fréchet derivative of the complete network.

The shifted backward algorithm is defined layer by layer and proved equal to its closed formula for arbitrary widths and shift parameters. Its source terms are actual derivatives of explicitly defined Q-networks, with the original prefix, one minimum, and a linear absolute-weight tail without biases. The suffix products of shift coefficients, sensitivity-mass law, separate cross-head partial identities, and original-gradient reconstruction are checked. Shifted algorithmic sensitivities are distinguished from derivatives through input-dependent forward re-centering; classical derivatives at gate ties are not asserted.

The game developments define finite sign-labeled paths, their probabilities, discounts, and occupation sums. For the stated policies, the discounted stopping occupations recover the actual MLP gradient; routing occupations recover the complete $\alpha\beta$ recurrence from network-derived contributions. The smooth case includes the actual full Softplus derivative. The attention result treats a fixed attention matrix and the complete composite Value projection, including its bias. It does not assert Query/Key differentiation or a global equilibrium merely from the backward recovery identity. Generic finite layered-game saddle bounds and Bellman uniqueness are checked separately with actual randomized policies.

The trajectory module checks finite path normalization, dynamic-programming identities, geometric-mean overlap, per-terminal-state disagreement, and positive survival conditioning. Ordinary probability laws used for Hellinger comparison are kept distinct from discounted occupation measures. The regularization module checks the finite Gibbs variational maximum, its attaining policy, the binary stop/continue optimizer, and actual Softplus derivatives.

The audit exposed a missing factor 1/2 in the four-partial backward invariant, a shift-product index error, an omitted weight matrix, and overly broad claims about differentiability, multiplicative convexity, re-centering, and nonzero combined gradients. The corrections and counterexamples are documented in `reports/mathematical-corrections.md`. In particular, nonzero raw sensitivity mass does not force a nonzero balanced head gradient.

The source inventory records mathematical statements still outside the checked scope. Neither these proofs nor the corrections certify the Python implementations or empirical experiments. The table adapter independently regenerates the thirteen outputs supported by the revised game's published summaries and checks them against the thesis. Diagnostic and hidden-monotonicity tables whose raw summaries are unavailable retain their published source provenance. To reproduce the mathematical checks, build `formalization/` with `lake build` and run its explicit axiom audit; the README documents the complete commands.

Appendix B

Publication Provenance and Tool Use

The results and experiments integrated in this thesis originate in *Hidden Monotonicity* (Zimmermann and Loho, 2026) and the revised *Playing the Network Backward* manuscript (arXiv:2605.06212v2). Their technical content is organized by the shared decomposition framework rather than reproduced as two separate papers. Original experimental settings and table values are retained; the thesis adds the common definitions, connections between the constructions, and Lean formalization.

B.1 Disclosure in the Revised Game Paper

The revised game paper contains the following AI-use statement, reproduced here as a statement about the preparation of that publication:

> In this work, we used generative AI tools for implementing standard baseline methods and evaluation code, for producing plots and tables of results, for drafting parts of the text, and for editing (grammar, spelling, word choice, and reformatting). We have not used generative AI tools to generate research ideas, to formulate the games, or to produce the theorems and their proofs. All AI-assisted code was reviewed and tested by the authors, including unit tests that check the game-theoretic occupation measures against reference gradient and $\alpha\beta$-LRP implementations; all AI-assisted text was checked by the authors for correctness, and every reference was verified manually. We take responsibility for the final content of this work, including text, claims, and artifacts produced with the aid of generative AI.

The additional Lean development and the editing and integration of this thesis used AI coding assistance. Those subsequent activities are distinct from the preparation of the original paper described above. The formalization repository records source-level corrections, reviewed theorem types, proof-checking receipts, and the remaining coverage limits; successful Lean checking certifies the stated formal propositions and does not certify the empirical experiments.

List of References

Abnar, S., W. Zuidema (2020): Quantifying Attention Flow in Transformers. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 4190–4197. DOI: 10.18653/v1/2020.acl-main.385. URL: http://dx.doi.org/10.18653/v1/2020.acl-main.385 (cit. on pp. 5, 48, 80, 81).

Achanta, R., A. Shaji, K. Smith, A. Lucchi, P. Fua, S. Süsstrunk (2012): *SLIC superpixels compared to state-of-the-art superpixel methods*. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 34(11):2274–2282, 2012 (cit. on p. 152).

Achtibat, R., M. Dreyer, I. Eisenbraun, S. Bosse, T. Wiegand, W. Samek, S. Lapuschkin (2023): From Attribution Maps to Human-Understandable Explanations through Concept Relevance Propagation. *Nature Machine Intelligence* 5 (9), 1006–1019. DOI: 10.1038/s42256-023-00711-8 (cit. on pp. 80, 113).

Achtibat, R., S. M. Vakilzadeh Hatefi, M. Dreyer, A. Jain, T. Wiegand, S. Lapuschkin, W. Samek (2024): AttnLRP: Attention-Aware Layer-Wise Relevance Propagation for Transformers. *Proceedings of the 41st International Conference on Machine Learning (ICML)*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 135–168 (cit. on pp. 5, 48, 49, 80, 83, 84, 92, 110, 111, 151, 154).

Adebayo, J., J. Gilmer, M. Muelly, I. J. Goodfellow, M. Hardt, B. Kim (2018): Sanity Checks for Saliency Maps. *NeurIPS*, 9525–9536 (cit. on pp. 2, 3, 5, 99, 114, 128, 130, 132, 133, 136, 144).

Ali, A., T. Schnake, O. Eberle, G. Montavon, K.-R. Müller, L. Wolf (2022): XAI for Transformers: Better Explanations through Conservative Propagation. *Proceedings of the 39th International Conference on Machine Learning (ICML)*. Vol. 162. Proceedings of Machine Learning Research. PMLR, 435–451 (cit. on pp. 5, 48, 49, 80, 81, 83, 84, 91, 92, 170).

Alvarez-Melis, D., T. S. Jaakkola (2018): *On the Robustness of Interpretability Methods*. DOI: 10.48550/ARXIV.1806.08049. URL: https://arxiv.org/abs/1806.08049 (cit. on p. 5).

Amos, B., L. Xu, J. Z. Kolter (2017): Input Convex Neural Networks. *Proceedings of the 34th International Conference on Machine Learning*. Ed. by D. Precup, Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 146–155. URL: https://proceedings.mlr.press/v70/amos17b.html (cit. on pp. 1, 4, 10, 32, 33, 46, 101, 139).

Ancona, M., E. Ceolini, C. Öztireli, M. Gross (2018): *Towards better understanding of gradient-based attribution methods for deep neural networks*. In *International Conference on Learning Representations (ICLR)*, 2018. URL: https://openreview.net/forum?id=Sy21R9JAW (cit. on p. 47).

Anders, C. J., D. Neumann, W. Samek, K.-R. Müller, S. Lapuschkin (2023): *Software for Dataset-wide XAI: From Local Explanations to Global Insights with Zennit, CoRelAy, and ViRelAy*. arXiv: 2106.13200 [cs.LG]. URL: https://arxiv.org/abs/2106.13200 (cit. on pp. 48, 72, 110).

Anders, C. J., L. Weber, D. Neumann, W. Samek, K.-R. Müller, S. Lapuschkin (2022): Finding and removing Clever Hans: Using explanation methods to debug and improve deep models. *Information Fusion* 77, 261–295. ISSN: 1566-2535. DOI: https://doi.org/10.1016/j.inffus.2021.07.015. URL: https://www.sciencedirect.com/science/article/pii/S1566253521001573 (cit. on p. 1).

Auer, P., N. Cesa-Bianchi, P. Fischer (2002): Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47 (2-3), 235–256. ISSN: 1573-0565. DOI: 10.1023/a:1013689704352. URL: http://dx.doi.org/10.1023/A:1013689704352 (cit. on pp. 51, 75).

Awasthi, P., A. Mao, M. Mohri, Y. Zhong (2023): DC-programming for neural network optimizations. *Journal of Global Optimization*. DOI: 10.1007/s10898-023-01344-2 (cit. on pp. 4, 31).

Ayinde, B. O., J. M. Zurada (2018): Deep Learning of Constrained Autoencoders for Enhanced Understanding of Data. *IEEE Transactions on Neural Networks and Learning Systems* 29 (9), 3969–3979. DOI: 10.1109/TNNLS.2017.2747861 (cit. on pp. 1, 97).

Bach, F. (2017): Breaking the Curse of Dimensionality with Convex Neural Networks. *Journal of Machine Learning Research* 18 (19), 1–53. URL: http://jmlr.org/papers/v18/14-546.html (cit. on p. 4).

Bach, S., A. Binder, G. Montavon, F. Klauschen, K.-R. Müller, W. Samek (2015): On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation. *PLOS ONE* 10 (7), 1–46. DOI: 10.1371/journal.pone.0130140. URL: https://doi.org/10.1371/journal.pone.0130140 (cit. on pp. 3, 5, 47, 48, 71, 72, 80, 93, 98, 151, 153).

Balduzzi, D., M. Frean, L. Leary, J. P. Lewis, K. W.-D. Ma, B. McWilliams (2017): The Shattered Gradients Problem: If resnets are the answer, then what is the question?. *Proceedings of the 34th International*

Conference on Machine Learning. Ed. by D. Precup, Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 342–350. URL: https://proceedings.mlr.press/v70/balduzzi17b.html (cit. on p. 100).

Balestriero, R., R. G. Baraniuk (2021): Mad Max: Affine Spline Insights Into Deep Learning. *Proceedings of the IEEE* 109 (5), 704–727. DOI: 10.1109/JPROC.2020.3042100 (cit. on p. 4).

Bhatt, U., A. Xiang, S. Sharma, A. Weller, A. Taly, Y. Jia, J. Ghosh, R. Puri, J. M. F. Moura, P. Eckersley (2020): Explainable machine learning in deployment. *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*. FAT* '20. Barcelona, Spain: Association for Computing Machinery, 648–657. ISBN: 9781450369367. DOI: 10.1145/3351095.3375624. URL: https://doi.org/10.1145/3351095.3375624 (cit. on p. 5).

Bhattacharyya, A. (1943): *On a measure of divergence between two statistical populations defined by their probability distributions. Bulletin of the Calcutta Mathematical Society*, 35:99–109, 1943 (cit. on p. 114).

Binder, A., G. Montavon, S. Lapuschkin, K.-R. Müller, W. Samek (2016): Layer-wise Relevance Propagation for Neural Networks with Local Renormalization Layers. *International Conference on Artificial Neural Networks (ICANN)*. Vol. 9887. Lecture Notes in Computer Science. Springer, 63–71. DOI: 10.1007/978-3-319-44781-0_8 (cit. on p. 47).

Binder, A., L. Weber, S. Lapuschkin, G. Montavon, K.-R. Müller, W. Samek (2023): Shortcomings of Top-Down Randomization-Based Sanity Checks for Evaluations of Deep Neural Network Explanations. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 16143–16152. DOI: 10.1109/cvpr52729.2023.01549. URL: http://dx.doi.org/10.1109/CVPR52729.2023.01549 (cit. on pp. 99, 130, 144).

Boyd, S., L. Vandenberghe (2004): *Convex Optimization*. Cambridge University Press. ISBN: 9780511804441. DOI: 10.1017/cbo9780511804441. URL: http://dx.doi.org/10.1017/CBO9780511804441 (cit. on pp. 15, 18, 65, 77, 87).

Brandenburg, M.-C., M. Grillo, C. Hertrich (2025): Decomposition Polyhedra of Piecewise Linear Functions. *International Conference on Representation Learning*. Ed. by Y. Yue, A. Garg, N. Peng, F. Sha, R. Yu. Vol. 2025, 62808–62837. URL: https://proceedings.iclr.cc/paper_files/paper/2025/file/9d66f74820f11ce037fb5f711ab9acd4-Paper-Conference.pdf (cit. on pp. 4, 24).

Brandenburg, M.-C., G. Loho, G. Montufar (2024): The Real Tropical Geometry of Neural Networks for Binary Classification. *Transactions on Machine Learning Research*. ISSN: 2835-8856. URL: https://openreview.net/forum?id=I7JWf8XA2w (cit. on pp. 17, 19).

Calafiore, G. C., S. Gaubert, C. Possieri (2020): A Universal Approximation Result for Difference of Log-Sum-Exp Neural Networks. *IEEE Trans. Neural Networks Learn. Syst.* 31 (12), 5603–5612. DOI: 10.1109/TNNLS.2020.2975051. URL: https://doi.org/10.1109/TNNLS.2020.2975051 (cit. on p. 4).

Chattopadhay, A., A. Sarkar, P. Howlader, V. N. Balasubramanian (2018): Grad-CAM++: Generalized Gradient-Based Visual Explanations for Deep Convolutional Networks. *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 839–847. DOI: 10.1109/WACV.2018.00097 (cit. on pp. 5, 97, 152).

Chefer, H., S. Gur, L. Wolf (2021a): Generic Attention-Model Explainability for Interpreting Bi-Modal and Encoder-Decoder Transformers. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 397–406 (cit. on pp. 5, 80, 110, 111, 152).

Chefer, H., S. Gur, L. Wolf (2021b): Transformer Interpretability Beyond Attention Visualization. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 782–791 (cit. on pp. 5, 48, 80, 110, 111, 152).

Chen, Z., Y. Wang, et al. (2025): Recurrent Convex Difference Neural Networks for Safety-Critical Model Predictive Control. *IEEE Robotics and Automation Letters*. Early access; bibliographic details may update. URL: https://www.researchgate.net/publication/391585816_Recurrent_Convex_Difference_Neural_Networks_for_Safety-Critical_Model_Predictive_Control (cit. on pp. 4, 101).

Clevert, D., T. Unterthiner, S. Hochreiter (2016): Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*. Ed. by Y. Bengio, Y. LeCun. URL: http://arxiv.org/abs/1511.07289 (cit. on p. 45).

Dalal, N., B. Triggs (2005): *Histograms of oriented gradients for human detection*. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 886–893, 2005 (cit. on p. 130).

Daniels, H., M. Velikova (2010): Monotone and partially monotone neural networks. *Trans. Neur. Netw.* 21 (6), 906–917. ISSN: 1045-9227. DOI: 10.1109/TNN.2010.2044803. URL: https://doi.org/10.1109/TNN.2010.2044803 (cit. on pp. 4, 9).

Darcet, T., M. Oquab, J. Mairal, P. Bojanowski (2024): Vision Transformers Need Registers. *ICLR*. OpenReview.net (cit. on pp. 5, 80).

Dayan, P., L. F. Abbott (2001): *Theoretical Neuroscience: Computational and Mathematical Modeling of Neural Systems*. Cambridge, MA: MIT Press. ISBN: 9780262041997 (cit. on p. 97).

Deng, J., W. Dong, R. Socher,
bibinitperiod Li, K. Li, L. Fei-Fei (2009): ImageNet: A Large-Scale Hierarchical Image Database. English (US). *2009 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2009*. 2009 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2009. Publisher Copyright: © 2009 IEEE.; 2009 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2009 ; Conference date: 20-06-2009 Through 25-06-2009. United States: IEEE Computer Society, 248–255. ISBN: 9781424439911. DOI: 10.1109/CVPR.2009.5206848 (cit. on pp. 23, 110, 151).

Dombrowski, A.-K., M. Alber, C. J. Anders, M. Ackermann, K.-R. Müller, P. Kessel (2019): Explanations Can Be Manipulated and Geometry Is to Blame. *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32 (cit. on p. 5).

Dombrowski, A.-K., C. J. Anders, K.-R. Müller, P. Kessel (2022): Towards robust explanations for deep neural networks. *Pattern Recognition* 121, 108194. ISSN: 0031-3203. DOI: 10.1016/j.patcog.2021.108194. URL: http://dx.doi.org/10.1016/j.patcog.2021.108194 (cit. on pp. 5, 48, 79, 136).

Doshi-Velez, F., B. Kim (2017): *Towards A Rigorous Science of Interpretable Machine Learning*. DOI: 10.48550/arXiv.1702.08608. arXiv: 1702.08608. URL: https://arxiv.org/abs/1702.08608 (cit. on p. 1).

Dosovitskiy, A., L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby (2021): An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR*. OpenReview.net (cit. on pp. 4, 80, 104, 109, 137).

Efron, B., R. J. Tibshirani (1993): An Introduction to the Bootstrap. Chapman & Hall, 1993 (cit. on pp. 110, 155).

Everingham, M., L. Van Gool, C. K. I. Williams, J. Winn, A. Zisserman (2009): The Pascal Visual Object Classes (VOC) Challenge. *International Journal of Computer Vision* 88 (2), 303–338. ISSN: 1573-1405. DOI: 10.1007/s11263-009-0275-4. URL: http://dx.doi.org/10.1007/s11263-009-0275-4 (cit. on pp. 110, 151, 162).

Fei, Y., Z. Yang, Y. Chen, Z. Wang, Q. Xie (2020): Risk-Sensitive Reinforcement Learning: Near-Optimal Risk-Sample Tradeoff in Regret. *NeurIPS* (cit. on pp. 5, 51, 75).

Fel, T., M. Ducoffe, D. Vigouroux, R. Cadène, M. Capelle, C. Nicodème, T. Serre (2023): Don't Lie to Me! Robust and Efficient Explainability With Verified Perturbation Analysis. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) — Poster*. Method: EVA. URL: https://cvpr.thecvf.com/virtual/2023/poster/22072 (cit. on pp. 5, 93).

Fong, R., A. Vedaldi (2017): Interpretable Explanations of Black Boxes by Meaningful Perturbation. *IEEE International Conference on Computer Vision (ICCV)*, 3429–3437 (cit. on pp. 5, 93).

Gao, S., Z.-Y. Li, M.-H. Yang, M.-M. Cheng, J. Han, P. Torr (2023): Large-Scale Unsupervised Semantic Segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45 (6), 7457–7476. ISSN: 1939-3539. DOI: 10.1109/tpami.2022.3218275. URL: http://dx.doi.org/10.1109/TPAMI.2022.3218275 (cit. on pp. 110, 151).

Gast, J., S. Roth (2018): Lightweight Probabilistic Deep Networks. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, 3369–3378. DOI: 10.1109/cvpr.2018.00355. URL: http://dx.doi.org/10.1109/cvpr.2018.00355 (cit. on pp. 51, 75).

Gaubert, S., Y. Vlassopoulos (2025): *Relu and softplus neural nets as zero-sum turn-based games*. arXiv: 2512.20582 [cs.LG]. URL: https://arxiv.org/abs/2512.20582 (cit. on pp. 2, 3, 5, 13–15, 47, 56, 77, 136).

Geirhos, R., J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, F. A. Wichmann (2020): Shortcut Learning in Deep Neural Networks. *Nature Machine Intelligence* 2. DOI: 10.1038/s42256-020-00257-z. URL: https://arxiv.org/abs/2004.07780 (cit. on p. 1).

Geist, M., B. Scherrer, O. Pietquin (2019): *A theory of regularized Markov decision processes*. In *International Conference on Machine Learning (ICML)*, pages 2160–2169, 2019 (cit. on p. 50).

Ghorbani, A., A. Abid, J. Zou (2019): Interpretation of Neural Networks Is Fragile. *Proceedings of the AAAI Conference on Artificial Intelligence* 33 (01), 3681–3688. ISSN: 2159-5399. DOI: 10.1609/aaai.v33i01.33013681. URL: http://dx.doi.org/10.1609/aaai.v33i01.33013681 (cit. on pp. 2, 5).

Gildenblat, J., contributors (2021): *PyTorch library for CAM methods.* https://github.com/jacobgil/pytorch-grad-cam, 2021. URL: https://github.com/jacobgil/pytorch-grad-cam (cit. on p. 110).

Goodfellow, I., D. Warde-Farley, M. Mirza, A. Courville, Y. Bengio (2013): Maxout Networks. *Proceedings of the 30th International Conference on Machine Learning.* Ed. by S. Dasgupta, D. McAllester. Vol. 28. Proceedings of Machine Learning Research 3. Atlanta, Georgia, USA: PMLR, 1319–1327. URL: https://proceedings.mlr.press/v28/goodfellow13.html (cit. on pp. 10, 19, 31, 81, 101).

Griewank, A., A. Walther (2020): Polyhedral DC Decomposition and DCA Optimization of Piecewise Linear Functions. *Algorithms* 13 (7), 166. DOI: 10.3390/a13070166. URL: https://www.mdpi.com/1999-4893/13/7/166 (cit. on pp. 4, 24).

Gu, J., Y. Yang, V. Tresp (2019): Understanding Individual Decisions of CNNs via Contrastive Backpropagation. *Computer Vision – ACCV 2018, Revised Selected Papers, Part III.* Vol. 11363. Lecture Notes in Computer Science. Springer, 119–134 (cit. on p. 47).

Guillemot, M., C. Heuselé (2020): Breaking Batch Normalization for Better Explainability of Deep Neural Networks through Layer-Wise Relevance Propagation. *arXiv preprint arXiv:2002.11018* (cit. on p. 44).

Gupta, M., A. Cotter, J. Pfeifer, K. Voevodski, K. Canini, A. Mangylov, W. Moczydlowski, A. van Esbroeck (2016): Monotonic Calibrated Interpolated Look-Up Tables. *Journal of Machine Learning Research* 17 (109), 1–47. URL: http://jmlr.org/papers/v17/15-243.html (cit. on p. 4).

Haase, C. A., C. Hertrich, G. Loho (2023): Lower Bounds on the Depth of Integral ReLU Neural Networks via Lattice Polytopes. *The Eleventh International Conference on Learning Representations* (cit. on p. 19).

Hanna, M., S. Pezzelle, Y. Belinkov (2024): Have Faith in Faithfulness: Going Beyond Circuit Overlap When Finding Model Mechanisms. *CoRR* abs/2403.17806 (cit. on pp. 1, 5, 80).

He, K., X. Zhang, S. Ren, J. Sun (2015): *Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification.* In *IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015 (cit. on p. 130).

He, K., X. Zhang, S. Ren, J. Sun (2016): Deep Residual Learning for Image Recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR).* IEEE, 770–778. DOI: 10.1109/cvpr.2016.90. URL: http://dx.doi.org/10.1109/CVPR.2016.90 (cit. on pp. 3, 43, 98, 109).

Hedström, A., L. Weber, D. Krakowczyk, D. Bareeva, F. Motzkus, W. Samek, S. Lapuschkin, M. M.-C. Müller (2023): Quantus: An Explainable AI Toolkit for Responsible Evaluation of Neural Network Explanations and Beyond. *Journal of Machine Learning Research* 24 (34), 1–11. URL: http://jmlr.org/papers/v24/22-0142.html (cit. on pp. 3, 5, 98, 110, 151, 153, 162).

Hellinger, E. (1909): *Neue Begründung der Theorie quadratischer Formen von unendlichvielen Veränderlichen. Journal für die reine und angewandte Mathematik*, 136:210–271, 1909 (cit. on p. 114).

Hendrycks, D., K. Gimpel (2016): Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415.* URL: https://arxiv.org/abs/1606.08415 (cit. on pp. 38, 76, 83, 136).

Hertrich, C., G. Loho (2025): *Neural Networks and (Virtual) Extended Formulations.* arXiv: 2411.03006 [math.CO]. URL: https://arxiv.org/abs/2411.03006 (cit. on pp. 19, 82).

Holm, S. (1979): *A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics*, 6(2):65–70, 1979 (cit. on pp. 110, 155).

Huchette, J., G. Muñoz, T. Serra, C. Tsay (2025): *When Deep Learning Meets Polyhedral Theory: A Survey.* arXiv: 2305.00241 [math.OC]. URL: https://arxiv.org/abs/2305.00241 (cit. on pp. 4, 9, 19).

Hui, L. Y. W., A. Binder (2019): BatchNorm Decomposition for Deep Neural Network Interpretation. *Advances in Computational Intelligence – IWANN 2019.* Vol. 11507. Lecture Notes in Computer Science. Springer, 280–291. DOI: 10.1007/978-3-030-20518-8_24 (cit. on p. 44).

Jiang, P.-T., C.-B. Zhang, Q. Hou, M.-M. Cheng, Y. Wei (2021): LayerCAM: Exploring Hierarchical Class Activation Maps for Localization. *IEEE Transactions on Image Processing* 30, 5875–5888. DOI: 10.1109/TIP.2021.3089943 (cit. on pp. 3, 5, 97, 152).

Kahardipraja, P., R. Achtibat, T. Wiegand, W. Samek, S. Lapuschkin (2025): The Atlas of In-Context Learning: How Attention Heads Shape In-Context Retrieval Augmentation. *NeurIPS* (cit. on pp. 1, 5, 80).

Kauffmann, J., K.-R. Müller, G. Montavon (2020a): Towards explaining anomalies: A deep Taylor decomposition of one-class models. *Pattern Recognition* 101, 107198. ISSN: 0031-3203. DOI: https://doi.org/10.1016

/j.patcog.2020.107198. URL: https://www.sciencedirect.com/science/article/pii/S0031320320300054 (cit. on p. 1).

Kauffmann, J., L. Ruff, G. Montavon, K.-R. Müller (2020b): *The Clever Hans Effect in Anomaly Detection*. arXiv: 2006.10609 [cs.LG]. URL: https://arxiv.org/abs/2006.10609 (cit. on p. 1).

Kazda, K., X. Li (2024): A linear programming approach to difference-of-convex piecewise linear approximation. *European Journal of Operational Research* 312 (2), 493–511. ISSN: 0377-2217. DOI: https://doi.org/10.1016/j.ejor.2023.07.026. URL: https://www.sciencedirect.com/science/article/pii/S0377221723005647 (cit. on p. 4).

Kindermans, P.-J., K. T. Schütt, M. Alber, K.-R. Müller, D. Erhan, B. Kim, S. Dähne (2018): Learning How to Explain Neural Networks: PatternNet and PatternAttribution. *International Conference on Learning Representations (ICLR)*. URL: https://openreview.net/forum?id=Hkn7CBaTW (cit. on pp. 2, 5).

Koehler, F., A. Risteski (2018): Representational Power of ReLU Networks and Polynomial Kernels: Beyond Worst-Case Analysis. *CoRR* abs/1805.11405. arXiv: 1805.11405. URL: http://arxiv.org/abs/1805.11405 (cit. on p. 4).

Kohlbrenner, M., A. Bauer, S. Nakajima, A. Binder, W. Samek, S. Lapuschkin (2020): Towards Best Practice in Explaining Neural Network Decisions with LRP. *IJCNN*. IEEE, 1–7 (cit. on pp. 47, 98, 110, 151, 152, 155).

Kokhlikyan, N., V. Miglani, M. Martin, E. Wang, B. Alsallakh, J. Reynolds, A. Melnikov, N. Kliushkina, C. Araya, S. Yan, O. Reblitz-Richardson (2020): *Captum: A unified and generic model interpretability library for PyTorch*. arXiv: 2009.07896 [cs.LG] (cit. on p. 110).

Kolek, S., R. Windesheim, H. Andrade-Loarca, G. Kutyniok, R. Levie (2023): Explaining Image Classifiers With Multiscale Directional Image Representation. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) — Poster*. Method: ShearletX. URL: https://cvpr.thecvf.com/virtual/2023/poster/21819 (cit. on pp. 5, 93).

Komorowski, P., E. Golimblevskaia, R. Achtibat, T. Wiegand, S. Lapuschkin, W. Samek (2025): Attribution-Guided Decoding. *CoRR* abs/2509.26307 (cit. on pp. 1, 5, 80).

Koutschan, C., B. Moser, A. Ponomarchuk, J. Schicho (2023): Representing piecewise linear functions by functions with small arity: Representing piecewise linear functions by functions with.... *Appl. Algebra Eng., Commun. Comput.* 36 (4), 595–610. ISSN: 0938-1279. DOI: 10.1007/s00200-023-00627-1. URL: https://doi.org/10.1007/s00200-023-00627-1 (cit. on pp. 4, 24).

Kramár, J., T. Lieberum, R. Shah, N. Nanda (2024): *AtP*: An efficient and scalable method for localizing LLM behaviour to components*. arXiv: 2403.00745 [cs.LG]. URL: https://arxiv.org/abs/2403.00745 (cit. on pp. 1, 5, 80).

Kripfganz, A., R. Schulze (1987): Piecewise affine functions as a difference of two convex functions. *Optimization* 18 (1), 23–29. DOI: 10.1080/02331938708843210. URL: https://www.tandfonline.com/doi/abs/10.1080/02331938708843210 (cit. on pp. 4, 9, 23, 29).

Lapuschkin, S., S. Wäldchen, A. Binder, G. Montavon, W. Samek, K.-R. Müller (2019): Unmasking Clever Hans predictors and assessing what machines really learn. *Nature Communications* 10 (1), 1096. DOI: 10.1038/s41467-019-08987-4. URL: https://doi.org/10.1038/s41467-019-08987-4 (cit. on p. 1).

Lee, D. D., H. S. Seung (1999): Learning the Parts of Objects by Non-Negative Matrix Factorization. *Nature* 401 (6755), 788–791. DOI: 10.1038/44565. URL: https://doi.org/10.1038/44565 (cit. on pp. 1, 97).

Lipton, Z. C. (2018): The Mythos of Model Interpretability. *Communications of the ACM* 61 (10), 36–43. DOI: 10.1145/3233231. URL: https://arxiv.org/abs/1606.03490 (cit. on p. 1).

Liu, X., S. Coogan (2020): Certified Monotonic Neural Networks. *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://proceedings.neurips.cc/paper/2020/file/b139aeda1c2914e3b579aafd3ceeb1bd-Paper.pdf (cit. on pp. 4, 9, 103).

Lundberg, S. M., S.-I. Lee (2017): A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*. Ed. by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett. Vol. 30. Curran Associates, Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf (cit. on pp. 5, 47, 93, 152).

Maclagan, D., B. Sturmfels (2015): *Introduction to Tropical Geometry*. Vol. 161. Graduate Studies in Mathematics. American Mathematical Society. DOI: 10.1090/gsm/161 (cit. on p. 16).

Magron, V., J. Wang (2022): *Sparse Polynomial Optimization: Theory and Practice*. arXiv: 2208.11158 [math.OC]. URL: https://arxiv.org/abs/2208.11158 (cit. on p. 4).

Mao, X., G. Qi, Y. Chen, X. Li, R. Duan, S. Ye, Y. He, H. Xue (2022): Towards Robust Vision Transformer. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 12032–12041. DOI: 10.1109/CVPR52688.2022.01173 (cit. on p. 3).

McNemar, Q. (1947): *Note on the sampling error of the difference between correlated proportions or percentages*. *Psychometrika*, 12(2):153–157, 1947 (cit. on p. 155).

Melzer, D. (1986): On the expressibility of piecewise-linear continuous functions as the difference of two piecewise-linear convex functions. In: *Quasidifferential Calculus*. Vol. 29. Mathematical Programming Studies. Berlin, Heidelberg: Springer, 118–134. DOI: 10.1007/BFb0121142. URL: https://link.springer.com/chapter/10.1007/BFb0121142 (cit. on pp. 4, 9, 23, 29).

Mersha, M., K. Lam, J. Wood, A. K. AlShami, J. Kalita (2024): Explainable artificial intelligence: A survey of needs, techniques, applications, and future direction. *Neurocomputing* 599, 128111. DOI: 10.1016/j.neucom.2024.128111. URL: https://doi.org/10.1016/j.neucom.2024.128111 (cit. on p. 1).

Misiakos, D., A. Weissenbacher, G. Loho (2022): Neural Network Expressivity and Complexity of Tropical Polynomial Functions. *arXiv preprint arXiv:2202.05093* (cit. on pp. 17, 19).

Misiakos, K., C. Thrampoulidis (2022): On the geometry of ReLU networks with one hidden layer. *arXiv preprint arXiv:2201.09816* (cit. on p. 4).

Montavon, G., A. Binder, S. Lapuschkin, W. Samek, K.-R. Müller (2019): Layer-Wise Relevance Propagation: An Overview. In: *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. Ed. by W. Samek, G. Montavon, A. Vedaldi, L. K. Hansen, K.-R. Müller. Cham: Springer International Publishing, 193–209. ISBN: 978-3-030-28954-6. DOI: 10.1007/978-3-030-28954-6_10. URL: https://doi.org/10.1007/978-3-030-28954-6_10 (cit. on p. 98).

Montavon, G., S. Lapuschkin, A. Binder, W. Samek, K.-R. Müller (2017): Explaining nonlinear classification decisions with deep Taylor decomposition. *Pattern Recognition* 65, 211–222. ISSN: 0031-3203. DOI: https://doi.org/10.1016/j.patcog.2016.11.008. URL: https://www.sciencedirect.com/science/article/pii/S0031320316303582 (cit. on pp. 5, 47, 49, 93).

Montavon, G., W. Samek, K.-R. Müller (2018): Methods for interpreting and understanding deep neural networks. *Digital Signal Processing* 73, 1–15. ISSN: 1051-2004. DOI: https://doi.org/10.1016/j.dsp.2017.10.011. URL: https://www.sciencedirect.com/science/article/pii/S1051200417302385 (cit. on pp. 5, 98, 153).

Mukherjee, A., A. Basu, R. Arora, P. Mianjy (2018): Understanding Deep Neural Networks with Rectified Linear Units. English. International Conference on Learning Representations , ICLR ; Conference date: 30-04-2018 Through 03-05-2018. URL: https://iclr.cc/Conferences/2018 (cit. on pp. 4, 8, 9, 19, 29, 30, 54).

Murdoch, W. J., C. Singh, K. Kumbier, R. Abbasi-Asl, B. Yu (2019): Definitions, methods, and applications in interpretable machine learning. *Proceedings of the National Academy of Sciences* 116 (44), 22071–22080. DOI: 10.1073/pnas.1900654116. URL: https://doi.org/10.1073/pnas.1900654116 (cit. on p. 1).

Nam, W., S. Gur, J. Choi, L. Wolf, S. Lee (2020): Relative Attributing Propagation: Interpreting the Comparative Contributions of Individual Units in Deep Neural Networks. *AAAI*. AAAI Press, 2501–2508 (cit. on p. 47).

Nguyen, A.-P., D. L. Moreno, N. Le-Bel, M. Rodríguez Martínez (2023): MonoNet: enhancing interpretability in neural networks via monotonic features. *Bioinformatics Advances* 3 (1), vbad016. ISSN: 2635-0041. DOI: 10.1093/bioadv/vbad016. eprint: https://academic.oup.com/bioinformaticsadvances/article-pdf/3/1/vbad016/50156829/vbad016.pdf. URL: https://doi.org/10.1093/bioadv/vbad016 (cit. on p. 4).

Nie, W., Y. Zhang, A. Patel (2018): A Theoretical Explanation for Perplexing Behaviors of Backpropagation-based Visualizations. *ICML*. Vol. 80. Proceedings of Machine Learning Research. PMLR, 3806–3815 (cit. on pp. 2, 5).

Osborne, M. J., A. Rubinstein (1994): A Course in Game Theory. MIT Press, 1994 (cit. on p. 49).

Pahde, F., M. Dreyer, W. Samek, S. Lapuschkin (2023): Reveal to Revise: An Explainable AI Life Cycle for Iterative Bias Correction of Deep Models. In: *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023*. Springer Nature Switzerland, 596–606. ISBN: 9783031438950. DOI: 10.1007/978-3-031-43895-0_56. URL: http://dx.doi.org/10.1007/978-3-031-43895-0_56 (cit. on pp. 1, 5).

Paszke, A., S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala (2019): PyTorch: an imperative style, high-performance deep learning library. *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc. (cit. on pp. 43, 110).

Phuong, M., C. H. Lampert (2020): Functional vs. parametric equivalence of ReLU networks. *ICLR*. OpenReview.net (cit. on p. 4).

Puterman, M. L. (1994): *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons. DOI: 10.1002/9780470316887 (cit. on pp. 11, 12, 49).

Ramachandran, P., B. Zoph, Q. V. Le (2017): Searching for Activation Functions. *CoRR* abs/1710.05941. arXiv: 1710.05941. URL: http://arxiv.org/abs/1710.05941 (cit. on pp. 45, 136).

Ribeiro, M. T., S. Singh, C. Guestrin (2016): "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. San Francisco, California, USA: Association for Computing Machinery, 1135–1144. ISBN: 9781450342322. DOI: 10.1145/2939672.2939778. URL: https://doi.org/10.1145/2939672.2939778 (cit. on pp. 1, 5, 93, 152).

Ross, A. S., F. Doshi-Velez (2018): Improving the Adversarial Robustness and Interpretability of Deep Neural Networks by Regularizing Their Input Gradients. *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. DOI: 10.1609/aaai.v32i1.11504. URL: https://arxiv.org/abs/1711.09404 (cit. on p. 1).

Ross, A. S., M. C. Hughes, F. Doshi-Velez (2017): Right for the Right Reasons: Training Differentiable Models by Constraining their Explanations. *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*, 2662–2670. DOI: 10.24963/ijcai.2017/371. URL: https://ijcai.org/proceedings/2017/371 (cit. on p. 1).

Rudin, C. (2019): Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence* 1 (5), 206–215. DOI: 10.1038/s42256-019-0048-x. URL: https://doi.org/10.1038/s42256-019-0048-x (cit. on p. 2).

Runje, D., S. M. Shankaranarayana (2023): Constrained monotonic neural networks. *Proceedings of the 40th International Conference on Machine Learning*. ICML'23. Honolulu, Hawaii, USA: JMLR.org (cit. on pp. 4, 9, 10).

Russakovsky, O., J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, L. Fei-Fei (2015): ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision* 115 (3), 211–252. ISSN: 1573-1405. DOI: 10.1007/s11263-015-0816-y. URL: http://dx.doi.org/10.1007/s11263-015-0816-y (cit. on pp. 110, 151).

Samek, W., A. Binder, G. Montavon, S. Lapuschkin, K.-R. Müller (2017): *Evaluating the visualization of what a deep neural network has learned. IEEE Transactions on Neural Networks and Learning Systems*, 28(11):2660–2673, 2017 (cit. on p. 153).

Samek, W., G. Montavon, S. Lapuschkin, C. J. Anders, K.-R. Müller (2021): Explaining Deep Neural Networks and Beyond: A Review of Methods and Applications. *Proceedings of the IEEE* 109 (3), 247–278. DOI: 10.1109/JPROC.2021.3060483 (cit. on p. 1).

Sankaranarayanan, P., R. Rengaswamy (2022): CDiNN – Convex difference neural networks. *Neurocomput.* 495 (C), 153–168. ISSN: 0925-2312. DOI: 10.1016/j.neucom.2022.01.024. URL: https://doi.org/10.1016/j.neucom.2022.01.024 (cit. on pp. 4, 101).

Schlüter, N., M. S. Darup (2021): Novel Convex Decomposition of Piecewise Affine Functions. *arXiv preprint arXiv:2108.03950*. URL: https://arxiv.org/abs/2108.03950 (cit. on p. 4).

Selvaraju, R. R., M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra (2017): Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *2017 IEEE International Conference on Computer Vision (ICCV)*, 618–626. DOI: 10.1109/ICCV.2017.74 (cit. on pp. 5, 93, 97, 152, 162).

Shapley, L. S. (1953): Stochastic Games. *Proceedings of the National Academy of Sciences* 39 (10), 1095–1100. DOI: 10.1073/pnas.39.10.1095 (cit. on p. 13).

Shrikumar, A., P. Greenside, A. Kundaje (2017): Learning Important Features Through Propagating Activation Differences. *Proceedings of the 34th International Conference on Machine Learning*. Ed. by D. Precup, Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 3145–3153. URL: https://proceedings.mlr.press/v70/shrikumar17a.html (cit. on pp. 5, 93, 151).

Siahkamari, A., A. Gangrade, B. Kulis, V. Saligrama (2020): Piecewise Linear Regression via a Difference of Convex Functions. *Proceedings of the 37th International Conference on Machine Learning*. Ed. by H. D. III, A. Singh. Vol. 119. Proceedings of Machine Learning Research. PMLR, 8895–8904. URL: https://proceedings.mlr.press/v119/siahkamari20a.html (cit. on pp. 4, 31).

Sill, J. (1997): Monotonic Networks. *Advances in Neural Information Processing Systems 10 (NIPS 1997)*. URL: https://papers.nips.cc/paper_files/paper/1997/hash/83adc9225e4deb67d7ce42d58fe5157c-Paper.pdf (cit. on pp. 1, 4, 9, 103).

Simonyan, K., A. Vedaldi, A. Zisserman (2014): Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps. *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Workshop Track Proceedings*. Ed. by Y. Bengio, Y. LeCun. URL: http://arxiv.org/abs/1312.6034 (cit. on pp. 5, 93, 151).

Simonyan, K., A. Zisserman (2015): Very Deep Convolutional Networks for Large-Scale Image Recognition. *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*. Ed. by Y. Bengio, Y. LeCun. URL: http://arxiv.org/abs/1409.1556 (cit. on pp. 3, 23, 43, 98, 109).

Sivaprasad, S., A. Singh, N. Manwani, V. Gandhi (2021): The Curious Case of Convex Neural Networks. *Machine Learning and Knowledge Discovery in Databases. Research Track*. Ed. by N. Oliver, F. Pérez-Cruz, S. Kramer, J. Read, J. A. Lozano. Cham: Springer International Publishing, 738–754. ISBN: 978-3-030-86486-6 (cit. on p. 4).

Sivaraman, A., G. Farnadi, T. Millstein, G. V. den Broeck (2020): Counterexample-Guided Learning of Monotonic Neural Networks. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. Method/tool: COMET. URL: https://proceedings.neurips.cc/paper/2020/hash/8ab70731b1553f17c11a3bbc87e0b605-Abstract.html (cit. on p. 4).

Sixt, L., M. Granz, T. Landgraf (2020): When Explanations Lie: Why Many Modified BP Attributions Fail. *ICML*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 9046–9057 (cit. on pp. 5, 99, 114, 130, 136).

Smilkov, D., N. Thorat, B. Kim, F. Viégas, M. Wattenberg (2017): *SmoothGrad: removing noise by adding noise*. arXiv: 1706.03825 [cs.LG]. URL: https://arxiv.org/abs/1706.03825 (cit. on pp. 5, 93, 151).

Springenberg, J. T., A. Dosovitskiy, T. Brox, M. Riedmiller (2015): Striving for Simplicity: The All Convolutional Net. *International Conference on Learning Representations (Workshop)* (cit. on pp. 5, 93).

Sriperumbudur, B. K., G. R. G. Lanckriet (2009): On the Convergence of the Concave-Convex Procedure. *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://papers.nips.cc/paper/3646-on-the-convergence-of-the-concave-convex-procedure (cit. on p. 4).

Sundararajan, M., A. Taly, Q. Yan (2017): Axiomatic Attribution for Deep Networks. *Proceedings of the 34th International Conference on Machine Learning*. Ed. by D. Precup, Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 3319–3328. URL: https://proceedings.mlr.press/v70/sundararajan17a.html (cit. on pp. 5, 93, 94, 151).

Sutton, R. S., A. G. Barto (2018): *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press (cit. on pp. 11, 12).

Syed, A., C. Rager, A. Conmy (2024): Attribution Patching Outperforms Automated Circuit Discovery. *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*. Association for Computational Linguistics, 407–416. DOI: 10.18653/v1/2024.blackboxnlp-1.25. URL: http://dx.doi.org/10.18653/v1/2024.blackboxnlp-1.25 (cit. on pp. 1, 5, 80).

Tao, P. D., L. T. H. An (1997): Convex Analysis Approach to D.C. Programming: Theory, Algorithms and Applications. *Acta Mathematica Vietnamica* 22 (1), 289–355 (cit. on p. 4).

Theiner, J., E. Müller-Budack, R. Ewerth (2021): Interpretable Semantic Photo Geolocation. *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 1474–1484 (cit. on p. 153).

Thi, H. A. L., T. P. Dinh (2018): DC programming and DCA: thirty years of developments. *Mathematical Programming* 169 (1), 5–68. DOI: 10.1007/s10107-018-1235-y (cit. on pp. 4, 139).

Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin (2017): Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30, 5998–6008 (cit. on pp. 80, 137).

Voita, E., D. Talbot, F. Moiseev, R. Sennrich, I. Titov (2019): Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 5797–5808 (cit. on pp. 5, 80).

Vukadin, D., P. Afrić, M. Šilić, G. Delač (2024): Advancing Attribution-Based Neural Network Explainability through Relative Absolute Magnitude Layer-Wise Relevance Propagation and Multi-Component Evaluation.

ACM Transactions on Intelligent Systems and Technology 15 (3), 1–30. ISSN: 2157-6912. DOI: 10.1145/3649458. URL: http://dx.doi.org/10.1145/3649458 (cit. on pp. 47, 80).

Wang, S., M. Gupta (2020): Deontological Ethics By Monotonicity Shape Constraints. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*. Ed. by S. Chiappa, R. Calandra. Vol. 108. Proceedings of Machine Learning Research. PMLR, 2043–2054. URL: https://proceedings.mlr.press/v108/wang20e.html (cit. on p. 1).

Wróbel, A., S. Gairola, J. Tabor, B. Schiele, B. Zieliński, D. Rymarczyk (2026): *DAVE: Distribution-aware Attribution via ViT Gradient Decomposition*. arXiv: 2602.06613 [cs.CV]. URL: https://arxiv.org/abs/2602.06613 (cit. on pp. 5, 48, 49, 80, 83, 84, 91, 92, 104, 105, 111, 151).

Xiong, P., T. Schnake, G. Montavon, K.-R. Müller, S. Nakajima (2026): Normalized Relevance Measure as a Unifying Framework to Explain Neural Network Latent Structures. *arXiv preprint arXiv:2606.00557*. DOI: 10.48550/arXiv.2606.00557. arXiv: 2606.00557 [cs.LG]. URL: https://arxiv.org/abs/2606.00557 (cit. on pp. 5, 47, 113, 136, 139).

Yeh, C.-K., C.-Y. Hsieh, A. Suggala, D. I. Inouye, P. K. Ravikumar (2019): On the (In)fidelity and Sensitivity of Explanations. *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32, 10965–10976 (cit. on pp. 5, 153).

Zalgaller, V. A. (2000): Representation of functions of several variables by differences of convex functions. *Journal of Mathematical Sciences* 100, 2209–2227. DOI: 10.1007/s10958-000-0006-4. URL: https://link.springer.com/article/10.1007/s10958-000-0006-4 (cit. on pp. 4, 23).

Zeiler, M. D., R. Fergus (2014): Visualizing and Understanding Convolutional Networks. *Computer Vision – ECCV 2014*. Ed. by D. Fleet, T. Pajdla, B. Schiele, T. Tuytelaars. Cham: Springer International Publishing, 818–833. ISBN: 978-3-319-10590-1 (cit. on pp. 5, 93).

Zhang, J., S. A. Bargal, Z. Lin, J. Brandt, X. Shen, S. Sclaroff (2018a): Top-down Neural Attention by Excitation Backprop. *European Conference on Computer Vision (ECCV)*. Springer, 543–559. DOI: https://doi.org/10.1007/s11263-017-1059-x (cit. on pp. 5, 47, 49, 152).

Zhang, L., G. Naitzat, L.-H. Lim (2018b): Tropical Geometry of Deep Neural Networks. *Proceedings of the 35th International Conference on Machine Learning*. Ed. by J. Dy, A. Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, 5824–5832. URL: https://proceedings.mlr.press/v80/zhang18i.html (cit. on pp. 4, 9, 17–19, 21, 31, 137).

Zhou, B., A. Khosla, A. Lapedriza, A. Oliva, A. Torralba (2016): Learning Deep Features for Discriminative Localization. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2921–2929 (cit. on pp. 93, 97).

Ziegler, G. M. (1995): *Lectures on Polytopes*. Vol. 152. Graduate Texts in Mathematics. New York: Springer (cit. on p. 17).

Zimmermann, J. P., G. Loho (2026): *Hidden Monotonicity: Explaining Deep Neural Networks via their DC Decomposition*. arXiv: 2601.07700 [cs.CV]. URL: https://arxiv.org/abs/2601.07700 (cit. on pp. 2, 19, 23, 29, 30, 39, 47, 54, 81, 93, 136, 173).